FEM Practices

Jan Tomek

August 7, 2026

Contents

Chapter 1

Coordinate Transformations

1.1 Constructing Transformation Matrix

1.1.1 Rotation Matrix

From: wolfram

1. 2-D (in $\mathbb{R}^2$):

   Rotating an object to a new coordinate system:

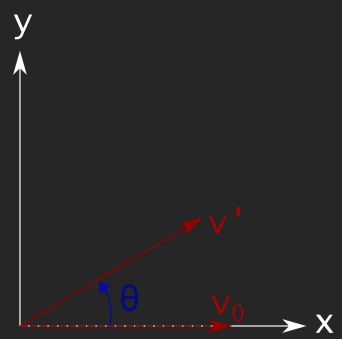
   

   Figure 1.1: Rotating object

   In $\mathbb{R}^2$, consider the matrix that rotates a given vector $\mathbf{v}_0$ by a counterclockwise angle θ in a fixed coordinate system. Then

   $$\mathbf{T}_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}, \tag{1.1}$$

   so

   $$\mathbf{v}^{'} = \mathbf{T}_\theta \mathbf{v}_0. \tag{1.2}$$

   On the other hand, consider the matrix that rotates the *coordinate system* through a counterclockwise angle θ. The coordinates of the fixed vector $\mathbf{v}$ in the rotated coordinate system are now given by a rotation matrix which is the *transpose* of the fixed-axis matrix, as can be seen on the second figure, is equivalent to rotating the vector by a counterclockwise angle $-\theta$ relative to a fixed set of axes, giving:

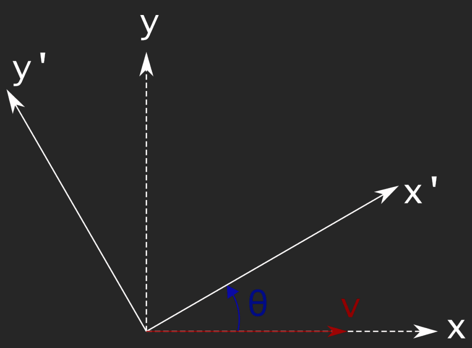


Figure 1.2: Rotating Axes

$$\mathbf{T}_\theta' = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}. \tag{1.3}$$

This is the convention commonly used in textbooks.

2. 3-D ($\mathbb{R}^3$):

   In $\mathbb{R}^3$, coordinate system rotations of the x-, y- and z-*axis* in a counter-clockwise direction when looking towards the origin give the matrices:

$$\mathbf{R}_x(\varphi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\varphi & \sin\varphi \\ 0 & -\sin\varphi & \cos\varphi \end{bmatrix}. \tag{1.4}$$

$$\mathbf{R}_y(\theta) = \begin{bmatrix} \cos\theta & 0 & -\sin\theta \\ 0 & 1 & 0 \\ \sin\theta & 0 & \cos\theta \end{bmatrix}. \tag{1.5}$$

$$\mathbf{R}_z(\psi) = \begin{bmatrix} \cos\psi & \sin\psi & 0 \\ -\sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{1.6}$$

It is best to think about a rotation around axis as about a **rotation in a plane**. Therefore rotation around $\mathbf{x}$ is rotation in $\mathbf{yz}$ plane, rotaton around $\mathbf{y}$ is a rotation in $\mathbf{zx}$ plane and a rotaton around $\mathbf{z}$ is a rotation in $\mathbf{xy}$ plane.
The defining the index of the axes such that $\mathbf{x} = 0$, $\mathbf{y} = 1$ and $\mathbf{z} = 2$ we can define the rotation matrix so that:

$$\mathbf{R} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{1.7}$$

then

$$\begin{aligned} \mathbf{R}_{i,i} &= \cos\theta \\ \mathbf{R}_{i,j} &= \sin\theta \\ \mathbf{R}_{j,i} &= -\sin\theta \\ \mathbf{R}_{j,j} &= \cos\theta \end{aligned} \tag{1.8}$$

where i is index of first plane axis and j is the index of the second plane axis.
The plane axes always cycle in the following manner: $xy, yz, zx, xy, ...$

Any **rotation** can be given as a composition of rotations about three axes (**Euler's rotation theorem**), and thus be represented by a 3×3 matrix operating on a vector.

$$\begin{bmatrix} x_1^{'} \\ x_2^{'} \\ x_3^{'} \end{bmatrix} = \begin{bmatrix} t_{11} & t_{12} & t_{13} \\ t_{21} & t_{22} & t_{23} \\ t_{31} & t_{32} & t_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}. \tag{1.9}$$

An **arbitrary rotation** around a unit vector $\mathbf{U} = (U_x, U_y, U_z)$ by an angle θ is given by:

$$R_U(\theta) = \mathbf{I} + (sin\theta)S + (1 - cos\theta)S^2 \tag{1.10}$$

where $\mathbf{I}$ is an identity matrix,

$$\mathbf{S} = \begin{bmatrix} 0 & -U_x & U_y \\ U_z & 0 & -U_x \\ -U_y & U_x & 0 \end{bmatrix} \tag{1.11}$$

and $\theta > 0$ indicates a counterclockwise rotation in the plane normal to $\mathbf{U}$. The observer is positioned on the side of the plane to which the vector points and looking back at the origin.

3. Alternate way of constructing **Rotation matrix** without specifying angles:

   The rotation matrix to a new coordinate system can be constructed from its axes, where the rotation matrix can be defined as:

$$\mathbf{T}_R = \begin{bmatrix} x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \\ z_1 & z_2 & z_3 \end{bmatrix}, \tag{1.12}$$

   where $\mathbf{x} = [x_1, x_2, x_3]^T$, $\mathbf{y} = [y_1, y_2, y_3]^T$ and $\mathbf{z} = [z_1, z_2, z_3]^T$ are unit vectors in the direction of x-, y- and z-*axis* of the new coordinate system.

The **transformation matrix** can be defined by any two vectors $\mathbf{u}$ and $\mathbf{v}$, where $\mathbf{u}$ defines the direction of *x-axis* and $\mathbf{v}$ defines the *xy-plane*. To construct the **rotation matrix** use the following steps:

(a) create x-axis of unit lenght

$$\mathbf{x} = \frac{\mathbf{u}}{||\mathbf{u}||}. \tag{1.13}$$

(b) create z-axis of unit lenght

$$\mathbf{z} = \frac{\mathbf{u} \times \mathbf{v}}{||\mathbf{u} \times \mathbf{v}||}. \tag{1.14}$$

(c) create y-axis of unit lenght

$$\mathbf{y} = \frac{\mathbf{z} \times \mathbf{x}}{||\mathbf{z} \times \mathbf{x}||}. \tag{1.15}$$

(d) compose **rotation matrix**

$$\mathbf{T}_R = \begin{bmatrix} \mathbf{x}^T \\ \mathbf{y}^T \\ \mathbf{z}^T \end{bmatrix}. \tag{1.16}$$

The creation of the **rotation matrix** by any two other vectors is analogic, as **orthogonal** base must be constructed first, then the matrix is composed of its vectors.

To test, whether the transformation matrix is correct, one can use the orthogonality criterion:

$$\mathbf{T}^T = \mathbf{T}^{-1} \tag{1.17}$$

$$\mathbf{T}\mathbf{T}^T = \mathbf{T}^T\mathbf{T} = \mathbf{I} \tag{1.18}$$

or

$$\det(\mathbf{T}) = 1 \tag{1.19}$$

1.1.2 Euler's angles of XYZ rotation

With the order of xyz rotation the rotation matrix is composed as:

$$\begin{aligned}
\mathbf{T} &= \mathbf{R}_z(\psi)\mathbf{R}_y(\theta)\mathbf{R}_x(\varphi) \\
&= \begin{bmatrix} \cos\psi & \sin\psi & 0 \\ -\sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & 0 & -\sin\theta \\ 0 & 1 & 0 \\ \sin\theta & 0 & \cos\theta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\varphi & \sin\varphi \\ 0 & -\sin\varphi & \cos\varphi \end{bmatrix} \\
&= \begin{bmatrix} \cos\theta\cos\psi & \cos\psi\sin\theta\sin\varphi - \cos\varphi\sin\psi & \cos\varphi\cos\psi\sin\theta + \sin\varphi\sin\psi \\ \cos\theta\sin\psi & \cos\varphi\cos\psi + \sin\theta\sin\varphi\sin\psi & \cos\varphi\sin\theta\sin\psi - \cos\psi\sin\varphi \\ -\sin\theta & \cos\theta\sin\varphi & \cos\theta\cos\varphi \end{bmatrix}
\end{aligned} \tag{1.20}$$

Then the three *Euler angles* for a xyz rotation can be obtained:

$$\varphi = \text{atan2}(t_{32}, t_{33}), \tag{1.21}$$

$$\theta = \text{atan2}\left(-t_{31}, \sqrt{t_{32}^2 + t_{33}^2}\right), \tag{1.22}$$

$$\psi = \text{atan2}(t_{21}, t_{11}), \tag{1.23}$$

where $\text{atan2}(y, x)$ definition is:

$$\text{atan2}(y,x) = \left\{ \begin{array}{lll} \arctan\left(\frac{y}{x}\right) & if & (x > 0), \\ \arctan\left(\frac{y}{x}\right) + \pi & if & (x < 0) \wedge (y \geq 0), \\ \arctan\left(\frac{y}{x}\right) - \pi & if & (x < 0) \wedge (y < 0), \\ \frac{\pi}{2} & if & (x = 0) \wedge (y > 0), \\ -\frac{\pi}{2} & if & (x = 0) \wedge (y < 0), \\ undefined & if & (x = 0) \wedge (y = 0) \end{array} \right\}. \tag{1.24}$$

Note the angle limitation for values returned while doing an Euler angle decomposition of a **rotation** matrix:

$$\begin{array}{l} \varphi \in (-\pi, \pi) \\ \theta \in (-\frac{\pi}{2}, \frac{\pi}{2}) \\ \psi \in (-\pi, \pi) \end{array} . \tag{1.25}$$

Alternative way to obtain the angles:

$$\left\{ \begin{array}{l} \varphi = \text{atan2}(-t_{21}, t_{22}) \\ \theta = \arcsin(t_{20}) \\ \psi = \text{atan2}(-t_{10}, t_{00}) \end{array} \right. \tag{1.26}$$

A python implementation with alternative components used to obtain the angles

```python
import numpy as np

def EulerXYZ(R: np.ndarray) -> tuple:
    if R[2,0] < 1.:
        if R[2,0] > -1.:
            b = np.arcsin(R[2,0])
            a = np.arctan2(-R[2,1], R[2,2])
            c = np.arctan2(-R[1,0], R[0,0])
        else:
            b = -np.pi / 2.
            a = -np.arctan2(R[0,1], R[1,1])
            c = 0.
```

```
    else:
        b = np.pi / 2.
        a = np.arctan2(R[0,1], R[1,1])
        c = 0.

    return a, b, c
```

1.1.3 Euler's angles of ZYX rotation

For a rotation defined in *zyx* order the rotation matrix is:

$$\begin{aligned}\mathbf{T} &= \mathbf{R}_x(\varphi)\mathbf{R}_y(\theta)\mathbf{R}_z(\psi)\\ &= \begin{bmatrix}1 & 0 & 0\\ 0 & \cos\varphi & \sin\varphi\\ 0 & -\sin\varphi & \cos\varphi\end{bmatrix}\begin{bmatrix}\cos\theta & 0 & -\sin\theta\\ 0 & 1 & 0\\ \sin\theta & 0 & \cos\theta\end{bmatrix}\begin{bmatrix}\cos\psi & \sin\psi & 0\\ -\sin\psi & \cos\psi & 0\\ 0 & 0 & 1\end{bmatrix}\\ &= \begin{bmatrix}\cos\theta\cos\varphi & -\cos\theta\sin\varphi & \sin\varphi\\ \cos\psi\sin\varphi+\cos\varphi\sin\theta\sin\psi & \cos\varphi\cos\psi-\sin\theta\sin\varphi\sin\psi & -\cos\theta\sin\psi\\ \sin\varphi\sin\psi-\cos\varphi\cos\psi\sin\theta & \cos\psi\sin\theta\sin\varphi+\cos\varphi\sin\psi & \cos\theta\cos\psi\end{bmatrix}\end{aligned} \quad . \tag{1.27}$$

Note that the angles φ, θ and ψ defined here are different from the *XYZ* rotation defined above! The rotation is performed by first rotating around *z-axis* counterclockwise by an angle ψ, then around *y-axis* counterclockwise by an angle θ and finaly around *x-axis*, counterclockwise, by an angle φ.

Then the *Euler's angles* can be obtained as:

$$\begin{cases}\varphi = \arctan\left(\frac{-t_{12}}{t_{11}}\right)\\ \theta = \arcsin\left(t_{13}\right)\\ \psi = \arctan\left(\frac{-t_{23}}{t_{33}}\right)\end{cases} \quad . \tag{1.28}$$

Alternative way to obtain the angles:

$$\begin{cases}\varphi = \mathrm{atan2}(t_{12}, t_{22})\\ \theta = \arcsin(-t_{02})\\ \psi = \mathrm{atan2}(t_{01}, t_{00})\end{cases} \tag{1.29}$$

A python implementation with alternative components used to obtain the angles

```
import numpy as np

def EulerZYX(R: np.ndarray) -> tuple:
    if R[0,2] < 1.:
        if R[0,2] > -1.:
            b = np.arcsin(-R[0,2])
            a = np.arctan2(R[0,1], R[0,0])
            c = np.arctan2(R[1,2], R[2,2])
        else:
            b = np.pi / 2.
            a = -np.arctan2(-R[2,1], R[1,1])
            c = 0.
    else:
        b = -np.pi / 2.
        a = np.arctan2(-R[2,1], R[1,1])
        c = 0.

    return a, b, c
```

1.1.4 Euler's angles of XZY rotation

For a rotation defined in xzy order the rotation matrix is:

$$\begin{aligned}\mathbf{T} &= \mathbf{R}_x(\varphi)\mathbf{R}_z(\psi)\mathbf{R}_y(\theta)\\ &= \begin{bmatrix}1 & 0 & 0\\ 0 & \cos\varphi & \sin\varphi\\ 0 & -\sin\varphi & \cos\varphi\end{bmatrix}\begin{bmatrix}\cos\psi & \sin\psi & 0\\ -\sin\psi & \cos\psi & 0\\ 0 & 0 & 1\end{bmatrix}\begin{bmatrix}\cos\theta & 0 & -\sin\theta\\ 0 & 1 & 0\\ \sin\theta & 0 & \cos\theta\end{bmatrix}\\ &= \begin{bmatrix}\cos\theta\cos\psi & \sin\varphi\sin\theta+\cos\varphi\cos\theta\sin\psi & -\cos\varphi\sin\theta+\cos\theta\sin\varphi\sin\psi\\ -\sin\psi & \cos\varphi\cos\psi & \cos\psi\sin\varphi\\ \cos\psi\sin\theta & -\cos\theta\sin\varphi+\cos\varphi\sin\theta\sin\psi & \cos\varphi\cos\theta+\sin\varphi\sin\theta\sin\psi\end{bmatrix}\end{aligned}. \tag{1.30}$$

The the angles are:

$$\begin{cases} \varphi = \text{atan2}(t_{12}, t_{11}) \\ \theta = \text{atan2}(t_{20}, t_{00}) \\ \psi = \arcsin(-t_{10}) \end{cases} \tag{1.31}$$

A python implementation used to obtain the angles

```python
import numpy as np

def EulerXZY(R: np.ndarray) -> tuple:
    if R[1,0] < 1.:
        if R[1,0] > -1.:
            b = np.arcsin(-R[1,0])
            a = np.arctan2(R[1,2], R[1,1])
            c = np.arctan2(R[2,0], R[0,0])
        else:
            b = np.pi / 2.
            a = -np.arctan2(-R[0,2], R[2,2])
            c = 0.
    else:
        b = -np.pi / 2.
        a = np.arctan2(-R[0,2], R[2,2])
        c = 0.

    return a, b, c
```

1.1.5 Euler's angles of ZXY rotation

For a rotation defined in *zxy* order the rotation matrix is:

$$
\begin{aligned}
\mathbf{T} &= \mathbf{R}_z(\psi)\mathbf{R}_x(\varphi)\mathbf{R}_y(\theta) \\
&= \begin{bmatrix} \cos\psi & \sin\psi & 0 \\ -\sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\varphi & \sin\varphi \\ 0 & -\sin\varphi & \cos\varphi \end{bmatrix} \begin{bmatrix} \cos\theta & 0 & -\sin\theta \\ 0 & 1 & 0 \\ \sin\theta & 0 & \cos\theta \end{bmatrix} \\
&= \begin{bmatrix} \cos\theta\cos\psi - \sin\varphi\sin\theta\sin\psi & \cos\psi\sin\varphi\sin\theta + \cos\theta\sin\psi & -\cos\varphi\sin\theta \\ -\cos\varphi\sin\psi & \cos\varphi\cos\psi & \sin\varphi \\ \cos\psi\sin\theta + \cos\theta\sin\varphi\sin\psi & -\cos\theta\cos\psi\sin\theta + \sin\theta\sin\psi & \cos\varphi\cos\theta \end{bmatrix}
\end{aligned} . \tag{1.32}
$$

The the angles are:

$$
\begin{cases} \varphi = \operatorname{atan2}(t_{12}, t_{11}) \\ \theta = \operatorname{atan2}(t_{20}, t_{00}) \\ \psi = \arcsin(-t_{10}) \end{cases} \tag{1.33}
$$

Python example of algorithm:

```python
import numpy as np

def EulerZXY(R: np.ndarray) -> tuple:
    if R[1,2] < 1.:
        if R[1,2] > -1.:
            b = np.arcsin(R[1,2])
            a = np.arctan2(-R[1,0], R[1,1])
            c = np.arctan2(-R[0,2], R[2,2])
        else:
            b = -np.pi / 2.
            a = -np.arctan2(R[2,0], R[0,0])
            c = 0.
    else:
        b = np.pi / 2.
        a = np.arctan2(R[2,0], R[0,0])
        c = 0.

    return a, b, c
```

1.1.6 Euler's angles of YXZ rotation

For a rotation defined in yxz order the rotation matrix is:

$$\begin{aligned}
\mathbf{T} &= \mathbf{R}_y(\theta)\mathbf{R}_x(\varphi)\mathbf{R}_z(\psi) \\
&= \begin{bmatrix} \cos\theta & 0 & -\sin\theta \\ 0 & 1 & 0 \\ \sin\theta & 0 & \cos\theta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\varphi & \sin\varphi \\ 0 & -\sin\varphi & \cos\varphi \end{bmatrix} \begin{bmatrix} \cos\psi & \sin\psi & 0 \\ -\sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \\
&= \begin{bmatrix} \cos\theta\cos\psi + \sin\varphi\sin\theta\sin\psi & \cos\varphi\sin\psi & -\cos\psi\sin\theta + \cos\theta\sin\varphi\sin\psi \\ \cos\psi\sin\varphi\sin\theta - \cos\theta\sin\psi & \cos\varphi\cos\psi & \cos\theta\cos\psi\sin\varphi + \sin\theta\sin\psi \\ \cos\varphi\sin\theta & -\sin\varphi & \cos\varphi\cos\theta \end{bmatrix}
\end{aligned}. \tag{1.34}$$

The the angles are:

$$\begin{cases} \varphi = \arcsin(-t_{21}) \\ \theta = \operatorname{atan2}(t_{20}, t_{22}) \\ \psi = \operatorname{atan2}(t_{01}, t_{11}) \end{cases} \tag{1.35}$$

Python example of algorithm:

```

import numpy as np

def EulerYXZ(R: np.ndarray) -> tuple:
    if R[2,1] < 1.:
        if R[2,1] > -1.:
            b = np.arcsin(-R[2,1])
            a = np.arctan2(R[2,0], R[2,2])
            c = np.arctan2(R[0,1], R[1,1])
        else:
            b = np.pi / 2.
            a = -np.arctan2(-R[1,0], R[0,0])
            c = 0.
    else:
        b = -np.pi / 2.
        a = np.arctan2(-R[1,0], R[0,0])
```

```
        c = 0.

    return a, b, c
```

1.1.7 Euler's angles of YZX rotation

For a rotation defined in yzx order the rotation matrix is:

$$
\begin{aligned}
\mathbf{T} &= \mathbf{R}_y(\theta)\mathbf{R}_x(\varphi)\mathbf{R}_z(\psi) \\
&= \begin{bmatrix} \cos\theta & 0 & -\sin\theta \\ 0 & 1 & 0 \\ \sin\theta & 0 & \cos\theta \end{bmatrix} \begin{bmatrix} \cos\psi & \sin\psi & 0 \\ -\sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\varphi & \sin\varphi \\ 0 & -\sin\varphi & \cos\varphi \end{bmatrix} \\
&= \begin{bmatrix} \cos\theta\cos\psi & \sin\psi & -\cos\psi\sin\theta \\ \sin\varphi\sin\theta - \cos\varphi\cos\theta\sin\psi & \cos\varphi\cos\psi & \cos\theta\sin\varphi + \cos\varphi\sin\theta\sin\psi \\ \cos\varphi\sin\theta + \cos\theta\sin\varphi\sin\psi & -\cos\psi\sin\varphi & \cos\varphi\cos\theta - \sin\varphi\sin\theta\sin\psi \end{bmatrix}
\end{aligned} \tag{1.36}
$$

The the angles are:

$$
\begin{cases} \varphi = \operatorname{atan2}(-t_{12}, t_{11}) \\ \theta = \operatorname{atan2}(-t_{02}, t_{00}) \\ \psi = \arcsin(t_{01}) \end{cases} \tag{1.37}
$$

Python example of algorithm:

```python
import numpy as np

def EulerYZX(R: np.ndarray) -> tuple:
    if R[0,1] < 1.:
        if R[0,1] > -1.:
            b = np.arcsin(R[0,1])
            a = np.arctan2(-R[0,2], R[0,0])
            c = np.arctan2(-R[2,1], R[1,1])
        else:
            b = -np.pi / 2.
```

```
            a = -np.arctan2(R[1,2], R[2,2])
            c = 0.
    else:
        b = np.pi / 2.
        a = np.arctan2(R[1,2], R[2,2])
        c = 0.

    return a, b, c
```

1.1.8 Euler's angles of XYX rotation

For a rotation defined in xyx order the rotation matrix is:

$$\begin{aligned}\mathbf{T} &= \mathbf{R}_x(\varphi_1)\mathbf{R}_y(\theta)\mathbf{R}_x(\varphi_0)\\ &= \begin{bmatrix}1 & 0 & 0\\ 0 & \cos\varphi_1 & \sin\varphi_1\\ 0 & -\sin\varphi_1 & \cos\varphi_1\end{bmatrix}\begin{bmatrix}\cos\psi & \sin\psi & 0\\ -\sin\psi & \cos\psi & 0\\ 0 & 0 & 1\end{bmatrix}\begin{bmatrix}1 & 0 & 0\\ 0 & \cos\varphi_0 & \sin\varphi_0\\ 0 & -\sin\varphi_0 & \cos\varphi_0\end{bmatrix}\\ &= \begin{bmatrix}\cos\psi & \sin\psi\cos\varphi_1 & \sin\psi\sin\varphi_1\\ -\sin\psi\cos\varphi_0 & \cos\psi\cos\varphi_0\cos\varphi_1-\sin\varphi_0\sin\varphi_1 & \cos\psi\cos\varphi_0\sin\varphi_1+\cos\varphi_1\sin\varphi_0\\ \sin\psi\sin\varphi_0 & -\cos\varphi_0\sin\varphi_1-\cos\psi\cos\varphi_1\sin\varphi_0 & \cos\varphi_0\cos\varphi_1-\cos\psi\sin\varphi_0\sin\varphi_1\end{bmatrix}\end{aligned} \quad (1.38)$$

The the angles are:

$$\begin{cases}\varphi_0 = \operatorname{atan2}(t_{02}, -t_{10})\\ \psi = \arccos(t_{00})\\ \varphi_1 = \operatorname{atan2}(t_{02}, t_{01})\end{cases} \quad (1.39)$$

Python example of algorithm:

```
import numpy as np

def EulerXYX(R: np.ndarray) -> tuple:
    if R[0,0] < 1.:
        if R[0,0] > -1.:
```

```
            b = np.arccos(R[0,0])
            a = np.arctan2(R[0,1], -R[0,2])
            c = np.arctan2(R[1,0], R[2,0])
        else:
            b = np.pi
            a = -np.arctan2(-R[2,1], R[1,1])
            c = 0.
    else:
        b = 0.
        a = np.arctan2(-R[2,1], R[1,1])
        c = 0.

    return a, b, c
```

1.1.9 Euler's angles of XZX rotation

For a rotation defined in xzx order the rotation matrix is:

$$
\begin{aligned}
\mathbf{T} &= \mathbf{R}_x(\varphi_1)\mathbf{R}_z(\psi)\mathbf{R}_x(\varphi_0) \\
&= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\varphi_1 & \sin\varphi_1 \\ 0 & -\sin\varphi_1 & \cos\varphi_1 \end{bmatrix} \begin{bmatrix} \cos\theta & 0 & -\sin\theta \\ 0 & 1 & 0 \\ \sin\theta & 0 & \cos\theta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\varphi_0 & \sin\varphi_0 \\ 0 & -\sin\varphi_0 & \cos\varphi_0 \end{bmatrix} \\
&= \begin{bmatrix} \cos\theta & \sin\theta\sin\varphi_1 & -\sin\theta\cos\varphi_1 \\ \sin\theta\sin\varphi_0 & \cos\varphi_0\cos\varphi_1 - \cos\theta\sin\varphi_0\sin\varphi_1 & \cos\varphi_0\sin\varphi_1 + \cos\theta\cos\varphi_1\sin\varphi_0 \\ \sin\theta\cos\varphi_0 & -\cos\theta\cos\varphi_0\sin\varphi_1 - \cos\varphi_1\sin\varphi_0 & \cos\theta\cos\varphi_0\cos\varphi_1 - \sin\varphi_0\sin\varphi_1 \end{bmatrix}.
\end{aligned}
\tag{1.40}
$$

The the angles are:

$$
\begin{cases} \varphi_0 = \operatorname{atan2}(t_{10}, t_{20}) \\ \theta = \arccos(t_{00}) \\ \varphi_1 = \operatorname{atan2}(t_{01}, -t_{02}) \end{cases} \tag{1.41}
$$

Python example of algorithm:

```
import numpy as np
```

```python

def EulerXZX(R: np.ndarray) -> tuple:
    if R[0,0] < 1.:
        if R[0,0] > -1.:
            b = np.arccos(R[0,0])
            a = np.arctan2(R[0,2], R[0,1])
            c = np.arctan2(R[2,0], -R[1,0])
        else:
            b = np.pi
            a = -np.arctan2(R[1,2], R[2,2])
            c = 0.
    else:
        b = 0.
        a = np.arctan2(R[1,2], R[2,2])
        c = 0.

    return a, b, c
```

1.1.10 Euler's angles of YXY rotation

```python
import numpy as np

def EulerYXY(R: np.ndarray) -> tuple:
    if R[1,1] < 1.:
        if R[1,1] > -1.:
            b = np.arccos(R[1,1])
            a = np.arctan2(R[1,0], R[1,2])
            c = np.arctan2(R[0,1], -R[2,1])
        else:
            b = np.pi
            a = -np.arctan2(R[2,0], R[0,0])
            c = 0.
    else:
        b = 0.
        a = np.arctan2(R[2,0], R[0,0])
        c = 0.

```

```python
    return a, b, c
```

1.1.11 Euler's angles of YZY rotation

```python
import numpy as np

def EulerYZY(R: np.ndarray) -> tuple:
    if R[1,1] < 1.:
        if R[1,1] > -1.:
            b = np.arccos(R[1,1])
            a = np.arctan2(R[1,2], -R[1,0])
            c = np.arctan2(R[2,1], R[0,1])
        else:
            b = np.pi
            a = -np.arctan2(-R[0,2], R[2,2])
            c = 0.
    else:
        b = 0.
        a = np.arctan2(-R[0,2], R[2,2])
        c = 0.

    return a, b, c
```

1.1.12 Euler's angles of ZXZ rotation

Note: One of the most used forms of the **Euler** angles.

```python
import numpy as np

def EulerZXZ(R: np.ndarray) -> tuple:
    if R[2,2] < 1.:
        if R[2,2] > -1.:
            b = np.arccos(R[2,2])
```

```
            a = np.arctan2(R[2,0], -R[2,1])
            c = np.arctan2(R[0,2], R[1,2])
        else:
            b = np.pi
            a = -np.arctan2(-R[1,0], R[0,0])
            c = 0.
    else:
        b = 0.
        a = np.arctan2(-R[1,0], R[0,0])
        c = 0.

    return a, b, c
```

1.1.13 Euler's angles of ZYZ rotation

```
import numpy as np

def EulerZYZ(R: np.ndarray) -> tuple:
    if R[2,2] < 1.:
        if R[2,2] > -1.:
            b = np.arccos(R[2,2])
            a = np.arctan2(R[2,1], R[2,0])
            c = np.arctan2(R[1,2], -R[0,2])
        else:
            b = np.pi
            a = -np.arctan2(R[0,1], R[1,1])
            c = 0.
    else:
        b = 0.
        a = np.arctan2(R[0,1], R[1,1])
        c = 0.

    return a, b, c
```

1.2 Coordinate Transformation

- **Rotation:**

  Let $\mathbf{A}^T = [a_x, a_y, a_z]$ be the original coordinates, $\mathbf{B}^T = [b_x, b_y, b_z]$ be the transformed coordinates, and $\mathbf{T}$ a tranformation matrix in the form:

$$\mathbf{T} = \begin{bmatrix} x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \\ z_1 & z_2 & z_3 \end{bmatrix} = \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \\ \mathbf{z} \end{bmatrix}, \tag{1.42}$$

  where $\mathbf{x} = [x_1, x_2, x_3]$, $\mathbf{y} = [y_1, y_2, y_3]$ and $\mathbf{z} = [z_1, z_2, z_3]$ are the *x-axis*, *y-axis* and *z-axis* **unit vectors**, respectively, of the new coordinate system defined in the original one, while sharing their origin.

  The following applies:

$$\mathbf{T} \times \mathbf{A} = \mathbf{B}, \tag{1.43}$$

  and

$$\mathbf{T}^T \times \mathbf{B} = \mathbf{A}. \tag{1.44}$$

  Note: Perpendicular vector is created using *cross-product.*

1.3 Tensor Transformation

From: source 1, source 2, source 3

Let real symmetric matrix $\mathbf{A}$ be the original tensor, real symmetric matrix $\mathbf{B}$ the transformed tensor, and orthogonal matrix $\mathbf{T}$ a tranformation matrix in the form:

$$\mathbf{A} = \begin{bmatrix} \sigma_{11}^A & \sigma_{12}^A & \sigma_{13}^A \\ \sigma_{21}^A & \sigma_{22}^A & \sigma_{23}^A \\ \sigma_{31}^A & \sigma_{32}^A & \sigma_{33}^A \end{bmatrix}, \tag{1.45}$$

$$\mathbf{B} = \begin{bmatrix} \sigma_{11}^B & \sigma_{12}^B & \sigma_{13}^B \\ \sigma_{21}^B & \sigma_{22}^B & \sigma_{23}^B \\ \sigma_{31}^B & \sigma_{32}^B & \sigma_{33}^B \end{bmatrix}, \tag{1.46}$$

$$\mathbf{T} = \begin{bmatrix} x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \\ z_1 & z_2 & z_3 \end{bmatrix} = \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \\ \mathbf{z} \end{bmatrix}, \tag{1.47}$$

where $\mathbf{x} = [x_1, x_2, x_3]$, $\mathbf{y} = [y_1, y_2, y_3]$ and $\mathbf{z} = [z_1, z_2, z_3]$ are the *x-axis*, *y-axis* and *z-axis* **unit vectors**, respectively, of the new coordinate system defined in the original one, while sharing their origin.

The transformation matrix $\mathbf{T}$ must satisfy $\mathbf{T}\mathbf{T}^T = \mathbf{T}^T\mathbf{T} = \mathbf{I}$, where $\mathbf{I}$ is an identity matrix. This means that transformation matrix must be orthogonal, therefore satisfying $\mathbf{T}^T = \mathbf{T}^{-1}$.

Then **tensor transformation** is performed as such:

$$\mathbf{T} \times \mathbf{A} \times \mathbf{T}^T = \mathbf{B}. \tag{1.48}$$

$$\mathbf{T}^T \times \mathbf{B} \times \mathbf{T} = \mathbf{A}. \tag{1.49}$$

- **Principal values:**

  The principal values can be found as the **eigenvalues** of the tensor matrix. Characteristic equation of tensor $\mathbf{\Sigma}$:

  $$\mathbf{\Sigma}\mathbf{V} = \mathbf{\Lambda}\mathbf{V}, \tag{1.50}$$

  therefore:

  $$(\mathbf{\Sigma} - \mathbf{\Lambda}\mathbf{I})\mathbf{V} = 0, \tag{1.51}$$

  and:

$$det(\mathbf{\Sigma} - \mathbf{\Lambda I}) = 0, \tag{1.52}$$

where $\mathbf{\Lambda}$ is a diagonal matrix of eigenvalues (principal values) and $\mathbf{V}$ is the eigenvector matrix.

The characteristic cubic equation can be also written as:

$$\lambda^3 - I_1\lambda^2 + I_2\lambda - I_3 = 0, \tag{1.53}$$

giving three roots equal to tensor eigenvalues.

First compute **Invariants** $\mathbf{I}_n$:

$$\begin{array}{ll} I_1 & = \sigma_{11} + \sigma_{22} + \sigma_{33} \\ I_2 & = \sigma_{11}\sigma_{22} + \sigma_{22}\sigma_{33} + \sigma_{33}\sigma_{11} - \left|\sigma_{12}\right|^2 - \left|\sigma_{13}\right|^2 - \left|\sigma_{23}\right|^2 \\ I_3 & = -\sigma_{11}\left|\sigma_{23}\right|^2 - \sigma_{22}\left|\sigma_{13}\right|^2 - \sigma_{33}\left|\sigma_{12}\right|^2 + \sigma_{11}\sigma_{22}\sigma_{33} + 2Re\left(\sigma_{12}\sigma_{13}\sigma_{23}\right) \end{array} . \tag{1.54}$$

where $Re()$ means **real part** in case some of the values are complex.

In matrix form:

$$\begin{array}{ll} I_1 & = tr[\mathbf{\Sigma}] \\ I_2 & = \begin{vmatrix} \sigma_{11} & \sigma_{12} \\ \sigma_{12} & \sigma_{22} \end{vmatrix} + \begin{vmatrix} \sigma_{11} & \sigma_{13} \\ \sigma_{13} & \sigma_{33} \end{vmatrix} + \begin{vmatrix} \sigma_{22} & \sigma_{23} \\ \sigma_{23} & \sigma_{33} \end{vmatrix} \\ I_3 & = det(\mathbf{\Sigma}) \end{array} . \tag{1.55}$$

Then compute *help values*:

$$Q = \frac{3I_2 - I_1^2}{9}, \tag{1.56}$$

$$R = \frac{2I_1^3 - 9I_1I_2 + 27I_3}{54}, \tag{1.57}$$

$$\theta = cos^{-1}\left(\frac{R}{\sqrt{-Q^3}}\right). \tag{1.58}$$

Lastly to get **principal values**:

$$\overline{\sigma_1} = 2\sqrt{-Q}\cos\left(\frac{\theta}{3}\right) + \frac{1}{3}I_1. \tag{1.59}$$

$$\overline{\sigma_2} = 2\sqrt{-Q}\cos\left(\frac{\theta + 2\pi}{3}\right) + \frac{1}{3}I_1. \tag{1.60}$$

$$\overline{\sigma_3} = 2\sqrt{-Q}\cos\left(\frac{\theta + 4\pi}{3}\right) + \frac{1}{3}I_1. \tag{1.61}$$

The **principal values are not sorted**. The principal tensor is then:

$$\mathbf{\Sigma} = \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & \sigma_3 \end{bmatrix}, \tag{1.62}$$

where $\overline{\sigma}_1$, $\overline{\sigma}_2$ and $\overline{\sigma}_3 \implies \sigma_1 > \sigma_2 > \sigma_3$.

Afterwards the **principal axes** are obtained as:

$$\begin{matrix} \mathbf{\Gamma}_1 = \mathbf{\Sigma} - \sigma_1 \mathbf{I} \\ \mathbf{\Gamma}_2 = \mathbf{\Sigma} - \sigma_2 \mathbf{I} \\ \mathbf{\Gamma}_3 = \mathbf{\Sigma} - \sigma_3 \mathbf{I} \end{matrix}, \tag{1.63}$$

where $\mathbf{\Gamma}$ is a coordinate system corresponding to the principal value i and $\mathbf{I}$ is a 3×3 identity matrix.

The vectors corresponding to each **principal value** are obtained:

$$\mathbf{\Gamma}_1 = \begin{bmatrix} \overline{x}_{11} & \overline{x}_{12} & \overline{x}_{13} \\ \overline{y}_{11} & \overline{y}_{12} & \overline{y}_{13} \\ \overline{z}_{11} & \overline{z}_{12} & \overline{z}_{13} \end{bmatrix} = \begin{bmatrix} \overline{\mathbf{x}}_1 \\ \overline{\mathbf{y}}_1 \\ \overline{\mathbf{z}}_1 \end{bmatrix}, \tag{1.64}$$

$$\mathbf{\Gamma}_2 = \begin{bmatrix} \overline{x}_{21} & \overline{x}_{22} & \overline{x}_{23} \\ \overline{y}_{21} & \overline{y}_{22} & \overline{y}_{23} \\ \overline{z}_{21} & \overline{z}_{22} & \overline{z}_{23} \end{bmatrix} = \begin{bmatrix} \overline{\mathbf{x}}_2 \\ \overline{\mathbf{y}}_2 \\ \overline{\mathbf{z}}_2 \end{bmatrix}, \tag{1.65}$$

$$\mathbf{\Gamma}_3 = \begin{bmatrix} \overline{x}_{31} & \overline{x}_{32} & \overline{x}_{33} \\ \overline{y}_{31} & \overline{y}_{32} & \overline{y}_{33} \\ \overline{z}_{31} & \overline{z}_{32} & \overline{z}_{33} \end{bmatrix} = \begin{bmatrix} \overline{\mathbf{x}}_3 \\ \overline{\mathbf{y}}_3 \\ \overline{\mathbf{z}}_3 \end{bmatrix}, \tag{1.66}$$

where $\overline{\mathbf{x}}$, $\overline{\mathbf{y}}$ and $\overline{\mathbf{z}}$ are vectors of size 3.

$$\mathbf{x} = \frac{\overline{\mathbf{y}}_1 \times \overline{\mathbf{z}}_1}{|\overline{\mathbf{y}}_1 \times \overline{\mathbf{z}}_1|}, \tag{1.67}$$

$$\mathbf{y} = \frac{\overline{\mathbf{z}}_2 \times \overline{\mathbf{x}}_2}{|\overline{\mathbf{z}}_2 \times \overline{\mathbf{x}}_2|}, \tag{1.68}$$

$$\mathbf{z} = \frac{\overline{\mathbf{x}}_3 \times \overline{\mathbf{y}}_3}{|\overline{\mathbf{x}}_3 \times \overline{\mathbf{y}}_3|}. \tag{1.69}$$

The principal axes are also the **eigenvectors** of tensor $\mathbf{\Sigma}$:

$$\mathbf{V} = \begin{bmatrix} \mathbf{x} & \mathbf{y} & \mathbf{z} \end{bmatrix} = \begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \end{bmatrix}. \tag{1.70}$$

- **Principal values, Cardano's formula derivation and python implementation:**

  An efficient way to compute the **roots of the cubic equation** is the **Cardano's formula** or its variations. **Cardano's equation** works only for **depressed** 3rd degree polynomials.

  Once again we start with a **real, symmetric** tensor Σ:

$$\mathbf{\Sigma} = \begin{bmatrix} \sigma_{xx} & \tau_{xy} & \tau_{xz} \\ \tau_{yx} & \sigma_{yy} & \tau_{yz} \\ \tau_{zx} & \tau_{zy} & \sigma_{zz} \end{bmatrix}, \tag{1.71}$$

  Then from the definition of eigenvalues and eignevectors:

$$\mathbf{\Sigma V} = \mathbf{\Lambda V}, \tag{1.72}$$

  therefore:

$$(\mathbf{\Sigma} - \mathbf{\Lambda I})\mathbf{V} = 0, \tag{1.73}$$

  and:

$$det(\mathbf{\Sigma} - \mathbf{\Lambda I}) = 0, \tag{1.74}$$

where $\mathbf{\Lambda}$ is a diagonal matrix of eigenvalues (principal values) and $\mathbf{V}$ is the eigenvector matrix.

$$\mathbf{\Lambda_i} = \begin{bmatrix} \lambda_i & 0 & 0 \\ 0 & \lambda_i & 0 \\ 0 & 0 & \lambda_i \end{bmatrix}, \tag{1.75}$$

$$\mathbf{V}_i = \begin{bmatrix} \overline{x}_{i1} & \overline{x}_{i2} & \overline{x}_{i3} \\ \overline{y}_{i1} & \overline{y}_{i2} & \overline{y}_{i3} \\ \overline{z}_{i1} & \overline{z}_{i2} & \overline{z}_{i3} \end{bmatrix} = \begin{bmatrix} \overline{\mathbf{x}}_i \\ \overline{\mathbf{y}}_i \\ \overline{\mathbf{z}}_i \end{bmatrix}. \tag{1.76}$$

It needs to be noted that each **principal value** of a **tensor matrix** corresponds to three vectors that define a coordinate system where first vector is perpendicular to the principal plane.

Rewriting the characteristic cubic equation:

$$\begin{aligned} 0 =& \lambda^3 \\ &+ \lambda^2 \left(-\sigma_{xx} - \sigma_{yy} - \sigma_{zz}\right) \\ &+ \lambda \left(\sigma_{xx}\sigma_{yy} + \sigma_{xx}\sigma_{zz} + \sigma_{yy}\sigma_{zz} - \tau_{xy}^2 - \tau_{xz}^2 - \tau_{yz}^2\right) \\ &- \sigma_{xx}\sigma_{yy}\sigma_{zz} + \sigma_{xx}\tau_{yz}^2 + \sigma_{yy}\tau_{xz}^2 + \sigma_{zz}\tau_{xy}^2 - 2\tau_{xy}\tau_{xz}\tau_{yz} \end{aligned} \tag{1.77}$$

and setting I_1, I_2, I_3 as invariants:

$$\begin{aligned} I_1 &= -\sigma_{xx} - \sigma_{yy} - \sigma_{zz} \\ I_2 &= \sigma_{xx}\sigma_{yy} + \sigma_{xx}\sigma_{zz} + \sigma_{yy}\sigma_{zz} - \tau_{xy}^2 - \tau_{xz}^2 - \tau_{yz}^2 \\ I_3 &= \sigma_{xx}\tau_{yz}^2 + \sigma_{yy}\tau_{xz}^2 + \sigma_{zz}\tau_{xy}^2 - \sigma_{xx}\sigma_{yy}\sigma_{zz} - 2Re\left(\tau_{xy}\tau_{xz}\tau_{yz}\right) \end{aligned} \tag{1.78}$$

where $Re()$ means **real part** (contrary to $Im()$ which would mean imaginarz part).

in **matrix** form:

$$
\begin{aligned}
I_1 &= -tr[\mathbf{\Sigma}] \\
I_2 &= \begin{vmatrix} \sigma_{11} & \sigma_{12} \\ \sigma_{12} & \sigma_{22} \end{vmatrix} + \begin{vmatrix} \sigma_{11} & \sigma_{13} \\ \sigma_{13} & \sigma_{33} \end{vmatrix} + \begin{vmatrix} \sigma_{22} & \sigma_{23} \\ \sigma_{23} & \sigma_{33} \end{vmatrix} \\
I_3 &= -det(\mathbf{\Sigma})
\end{aligned} \tag{1.79}
$$

substituting 1.78 into 1.77 we get:

$$\lambda^3 + I_1\lambda^2 + I_2\lambda + I_3 = 0, \tag{1.80}$$

where the inveriants are:

Cardano's method requires first transforming 1.77 to the form:

$$x^3 - 3x = t \tag{1.81}$$

by defining:

$$
\begin{aligned}
p &= {I_1}^2 - 3I_2 \\
q &= -\frac{27}{2}I_3 - I_1^3 + \frac{9}{2}I_1 I_2 \\
t &= 2p^{-3/2}q \\
x &= \frac{3}{\sqrt{p}}\left(\lambda + \frac{1}{3}I_1\right)
\end{aligned} \tag{1.82}
$$

the the solution to 1.81 is given by:

$$x = \frac{1}{u} + u \tag{1.83}$$

with

$$u = \sqrt[3]{\frac{t}{2} \pm \sqrt{\frac{t^2}{4} - 1}} \tag{1.84}$$

We can then write $u = e^{i\phi}$, with:

$$
\begin{aligned}
\phi &= \frac{1}{3}\arctan\frac{\sqrt{|t^2/4-1|}}{t/2} \\
\phi &= \frac{1}{3}\arctan\frac{\sqrt{p^3-q^2}}{q} \\
\phi &= \frac{1}{3}\arctan\frac{\sqrt{27\left[\frac{1}{4}I_2^2\left(p-I_2\right)+I_3*\left(q+\frac{27}{4}I_3\right)\right]}}{q}
\end{aligned}
\tag{1.85}
$$

The last step of 1.85 is necessary for numerical stability and precision as the term p^3-q^2 is highly sensitive.

The 1.85 equation can be for better stability and to enforce the angle to the domain $\phi \in \langle -\pi, \pi \rangle$:

$$
\phi = \frac{1}{3}arctan2\left(\sqrt{27\left[\frac{1}{4}I_2^2\left(p-I_2\right)+I_3*\left(q+\frac{27}{4}I_3\right)\right]}, q\right) \tag{1.86}
$$

When evaluating either 1.85 or 1.86, care must be taken to correctly resolve the ambiguity in the arctan by taking account of the sign of q. For $q>0$, the solution must lie in the first quadrant, for $q<0$, it must be located in the second. In constrast to this, solutions differing by myltiples of 2π are equivalent, so x can take three different values:

$$
\begin{aligned}
x_1 &= 2\cos\phi \\
x_2 &= 2\cos\left(\phi+\frac{2\pi}{3}\right) = -\cos\phi-\sqrt{3}\sin\phi \\
x_3 &= 2\cos\left(\phi-\frac{2\pi}{3}\right) = -\cos\phi+\sqrt{3}\sin\phi
\end{aligned}
\tag{1.87}
$$

These values correspond to the three eigenvalues of $\mathbf{\Sigma}$:

$$
\lambda_i = \frac{\sqrt{p}}{3}x_i - \frac{1}{3}I_1 \tag{1.88}
$$

A python implementation:

```
import numpy as np

def principal_cardano_equation(flat_tensor_matrix: np.ndarray) -> np.ndarray:
    """Eigenvalues of a Hermitian 3x3 matrix using Cardano equations

    source: https://www.mpi-hd.mpg.de/personalhomes/globes/3x3/index.html
            https://en.wikipedia.org/wiki/Cubic_equation

    [ sx, txy, txz]    [a, d, f]
    [txy,  sy, tyz] = [d, b, e]
    [txz, tyz,  sz]    [f, e, c]

    Args:
        flat_tensor_matrix (np.ndarray): a symmetric, positive definite tensor components
                                          [[sx, sy, sz, txy, tyz, txz],
                                           [sx, sy, sz, txy, tyz, txz]
                                                       .
                                                       .
                                                       .
                                           [sx, sy, sz, txy, tyz, txz]]

    Returns:
        np.ndarray: prinicpal stresses sorted from largest to smallest [s1, s2, s3]
    """
    # store each component of the tensor into standalone vector
    a, b, c, d, e, f = flat_tensor_matrix.T

    # prepare values that are used multiple times
    M_SQRT3 = np.sqrt(3)
    de = d  *  e
    dd = d  *  d
    ee = e  *  e
    ff = f  *  f

    # tr(A)
```

```python
    m = a + b + c
    # det(A23) + det(13) + det(12)
    c1 = (a * b + a * c + b * c) - (dd + ee + ff)
    # det(A)
    c0 = c * dd + a * ee + b * ff - a * b * c - 2 * f * d * e

    p = m * m - 3.0 * c1
    q = m * (p - (3.0 / 2.0) * c1) - (27.0 / 2.0) * c0
    sqrt_p = np.sqrt(np.abs(p))

    phi = 27.0 * ( 0.25 * c1 * c1 * (p - c1) + c0 * (q + 27.0 / 4.0 * c0))
    phi = (1.0 / 3.0) * np.arctan2(np.sqrt(np.abs(phi)), q)

    c = sqrt_p * np.cos(phi)
    s = (1.0 / M_SQRT3) * sqrt_p * np.sin(phi)

    l2 = (1.0 / 3.0) * (m - c)
    l3  = l2 + s
    l1  = l2 + c
    l2 -= s

    # Sort the eigenvalues using a fast sorting network
    l1, l2 = np.minimum(l1, l2), np.maximum(l1, l2)
    l2, l3 = np.minimum(l2, l3), np.maximum(l2, l3)
    l1, l2 = np.minimum(l1, l2), np.maximum(l1, l2)

    return np.array([l3, l2, l1], dtype=float).T

```

- **Maximum Shear value:**

  Maximum **shear stress** occurs at an angle of 45 degrees to principal axes. If principal stresses are aligned such that $\sigma_1 > \sigma_2 > \sigma_3$ and the stress tensor is:

$$\mathbf{\Sigma} = \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & \sigma_3 \end{bmatrix}, \tag{1.89}$$

  then maximum shear stress τ_{max} can be obtained as follows:

$$\tau_{max} = \frac{\sigma_1 - \sigma_3}{2} \tag{1.90}$$

and is at 45 degrees angle in the 1-3 plane of the principal axes to the principal coordinate system.

More generally (in 2D):

$$\tau_{max} = \sqrt{\left(\frac{\sigma_x - \sigma_y}{2}\right)^2 + \tau_{xy}^2}. \tag{1.91}$$

When the principal tensor is rotated by 45 degrees to obtain **maximum shear stress**, the axial stress components corresponding to this shear stress are equal.
The value of maximal shear stress is obtained:

$$\tau_{max} = \frac{\sigma_{max} - \sigma_{min}}{2}. \tag{1.92}$$

The value of axial stresses corresponding to maximal shear stress is the average of maximal and minimal axial stresses:

$$\sigma_{\tau_{max}} = \frac{\sigma_{max} + \sigma_{min}}{2}. \tag{1.93}$$

- **Example:**

  Let a stress tensor in principal coordinate system be:

$$\mathbf{\Sigma} = \begin{bmatrix} 433 & 0 & 0 \\ 0 & 125 & 0 \\ 0 & 0 & 24 \end{bmatrix}, \tag{1.94}$$

  then $\sigma_1 = \sigma_{max} = 433$ MPa and $\sigma_3 = \sigma_{min} = 24$ MPa.

  Transformation matrix for rotating by 45 degrees in 3-1 plane is:

$$\mathbf{T} = \begin{bmatrix} \frac{\sqrt{2}}{2} & 0 & -\frac{\sqrt{2}}{2} \\ 0 & 1 & 0 \\ \frac{\sqrt{2}}{2} & 0 & \frac{\sqrt{2}}{2} \end{bmatrix}. \tag{1.95}$$

Rotating tensor $\mathbf{\Sigma}$ by transformation matrix $\mathbf{T}$ we get:

$$\mathbf{\Sigma}_\tau = \mathbf{T\Sigma T}^T = \begin{bmatrix} 228.5 & 0 & 204.5 \\ 0 & 125 & 0 \\ 204.5 & 0 & 228.5 \end{bmatrix}, \tag{1.96}$$

where:

$$\begin{array}{l} \tau_{max} = \sigma_{13} = \sigma_{31} = 204.5 \\ \sigma_{11} = \sigma_{33} = 228.5 \\ \sigma_{22} = \sigma_2 \end{array} . \tag{1.97}$$

Check for correct values:

$$\tau_{max} = \frac{\sigma_1 - \sigma_3}{2} = \frac{433 - 24}{2} = \frac{409}{2} = 204.5 \text{ MPa} \implies \text{OK!}, \tag{1.98}$$

$$\sigma_{11} = \sigma_{33} = \frac{\sigma_1 + \sigma_3}{2} = \frac{433 + 24}{2} = 228.5 \text{ MPa} \implies \text{OK!}. \tag{1.99}$$

σ_2 has not changed as it is colinear with the axis of rotation.

The relationship between principal normal streses and maximum shear stresses can be better understood by examining a plot of the stresses as a function of the rotation angle.

Notice that there are multiple θ_p and $\theta_{\tau-max}$ angles because of the periodical nature of the equations. However, they will give the same absolute values.

At the principal stress angle, θ_p, the shear stress will always be zero, as shown on the diagram. And the maximum shear stress will occur when the two principal normal stresses, σ_1 and σ_2, are equal.

When σ_x or σ_y are either max or min, the shear stress τ_{xy} is equal to 0. When shear stress τ_{xy} is max or min, the stresses σ_x and σ_y are equal to their average.

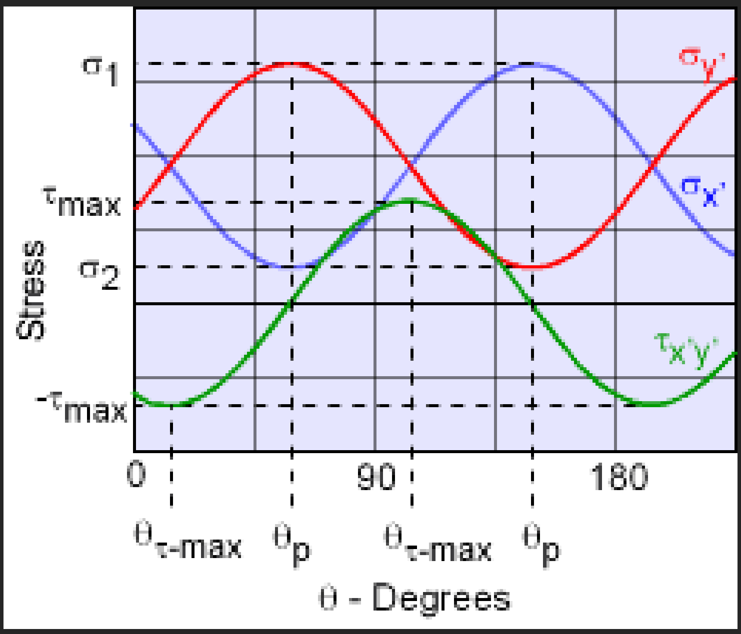


Figure 1.3: Stresses as a function of angle

- **Principal axes, obtaining principal axes with python implementation:**

  The principal axes can be obtained from the following property

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v} \tag{1.100}$$

As the **principal axis** (eigenvector) of $\mathbf{A}$ is not changed by $\mathbf{A}$, only scaled, then it lies in the **nullspace** of the matrix $\mathbf{B}$ from the characteristic equation:

$$\mathbf{B} = (\mathbf{A} - \lambda\mathbf{I}) \tag{1.101}$$

The matrix $\mathbf{B}$ is **singular** with 2 independent rows and a third one being a linear combination of others. The vector that lies in the **nullspace** can be obtained by using a cross product of any two rows. This can be performed for the first two principal values (eigenvalues λ_i and the third principal axis can be again computed using a cross product of the two previously extracted axes.

Python example:

```python
import numpy as np

def gram_schmidt(vectors: np.ndarray) -> np.ndarray:
    """Perform Gram-Schmidt orthogonalisation of vectors

    Args:
        vectors (np.ndarray) an array of row vectors to orthogonalise against
                                 each other, if the array is 3 dimensional then
                                 preform orthogonalisation for each array of
                                 row vectors separately
    Returns:
        (np.ndarray): an array of orthogonal vectors
    """
    ortho = np.copy(vectors)

    if len(ortho.shape) == 3:
        for i in range(1, ortho.shape[1]):
            for j in range(i):
                # np.sum(A * B, axis=1) is equal to row by row dot product of two sets of vectors
                # written as a 2D matrix
                vec = ortho[:,j,:] / np.linalg.norm(ortho[:,j,:], axis=1)[:,None]
                ortho[:,i,:] -= vec * np.sum(ortho[:,i,:] * vec, axis=1)[:,None]
    elif len(ortho.shape) == 2:
        for i in range(1, ortho.shape[0]):
            for j in range(i):
                # np.sum(A * B, axis=1) is equal to row by row dot product of two sets of vectors
                # written as a 2D matrix
                vec = ortho[j,:] / np.linalg.norm(ortho[j,:])
                ortho[i,:] -= vec * np.dot(ortho[i,:], vec)
```

```python

    return ortho


def normalise(vectors: np.ndarray) -> np.ndarray:
    """Normalise supplied vectors to a length of 1"""
    normalised = np.copy(vectors)
    if len(normalised.shape) == 3:
        for i in range(vectors.shape[1]):
            normalised[:,i,:] /= np.linalg.norm(normalised[:,i,:],
            ↪  axis=1)[:,None]

    elif len(normalised.shape) == 2:
        normalised /= np.linalg.norm(normalised, axis=1)[:,None]

    elif len(normalised.shape) == 1:
        normalised /= np.linalg.norm(normalised)

    return normalised


def orthonormalise(vectors: np.ndarray) -> np.ndarray:
    """Orhtonormalise sets of supplied vectors using GramSchmidt method
    first and then normalisation to a length of 1
    """
    return normalise(gram_schmidt(vectors))


def principal_axes(tensors: np.ndarray, principal_values: np.ndarray) ->
↪  np.ndarray:
    """Get the principal axes corrsponding to the principal values by
    getting the vector that is in the nullspace of the rank 2 homogeneous
    equation:
         (A  - l I) = 0
    by setting l to l1, l2 and l3, we can obtain three vectors that are
    in the nullspace of the first, second and third eigenvalue by using
    cross product of any two rows of the characteristic equation.

    Args:
        tensors (np.ndarray):          an array of tensors, one tensor
                                       on each row
```

```
                                       [[sx, sy, sz, txy, tyz, tzx], ... ]
        principal_values (np.ndarray): an array of the same number of rows
                                       as the tensors and 3 columns with
                                       principal values

    Returns:
        (np.ndarray): the principal axes belonging to the supplied
                      principal values
    """
    principal = np.copy(principal_values)
    if len(principal.shape) == 1 and principal.shape[0] == 3:
        principal = principal.reshape(1, 3)

    if len(principal.shape) != len(tensor.shape):
        raise ValueError(f"A mismatch in tensor and principal values dimensions " +
            f"{str(tensor.shape):s} != {str(principal.shape):s}")
    elif principal.shape[0] != tensor.shape[0]:
        raise ValueError(f"The number of principal values does not match the number of tensors " +
            f"{str(tensor.shape):s} != {str(principal.shape):s}")
    elif principal.shape[1] != 3:
        raise ValueError(f"The number of principal values is not 3: {str(principal.shape):s}")

    # sort principal stresses by amount from largest to smallest
    amount_idx = np.argsort(np.abs(principal), axis=1)[:,::-1]
    principal  = np.take_along_axis(principal, amount_idx, axis=1)

    # prepare reverse sort from largest to smallest
    amount_idx = np.argsort(principal, axis=1)[:,::-1]

    a, b, c, d, e, f = tensor.T
    l1, l2, l3 = principal.T

    # prepare the array to store the 3x3 transformation matrix composed of
    # rows of principal axes
    T = np.zeros((principal.shape[0], 3, 3), dtype=tensor.dtype)

    # prepare a mask where True is if the stress tensor is 0
    mask_zero = np.isclose(a ** 2 + b ** 2 + c ** 2, 0.)
```

```python

# prepare a mask where True is if the prinicpal stresses are equal
mask_equal = np.equal(l1, l2, l3)

# other values - a general case
mask_nonz  = np.logical_not(mask_zero) & np.logical_not(mask_equal)

# a zero tensor
if np.any(mask_zero):
    T[mask_zero] = np.array([[1., 0., 0.], [0., 1., 0.], [0., 0., 1.]], dtype=tensor.dtype)

# triaxial tensrion/compression
if np.any(mask_equal):
    T[mask_equal] = np.array([[1., 0., 0.], [0., 1., 0.], [0., 0., 1.]], dtype=tensor.dtype)

# process each eigenvalue (principal stress) individually (only first two needed)
for i, l in enumerate([l1, l2]): # , l3]):
    # get rows of the tensor (homogeneous equation)
    v1 = np.array([a - l,     d,     f], dtype=tensor.dtype).T
    v2 = np.array([    d, b - l,     e], dtype=tensor.dtype).T
    v3 = np.array([    f,     e, c - l], dtype=tensor.dtype).T

    # get the vector in the tensor nullspace for all combinations of rows
    v0 = np.array([np.cross(v1, v2),
                   np.cross(v1, v3),
                   np.cross(v2, v3)], dtype=tensor.dtype).transpose(1,0,2)

    # get the vector length
    len_v0 = np.array([
        np.linalg.norm(v0[:,0], axis=1),
        np.linalg.norm(v0[:,1], axis=1),
        np.linalg.norm(v0[:,2], axis=1)], dtype=tensor.dtype).T

    # find the index of largest column value for each row
    idx_v0 = np.argmax(len_v0, axis=1)

```

```
        # select the longest vector in the nullspace of the homogeneous equation
        v0     = np.take_along_axis(    v0, idx_v0[:,None,None], axis=1).reshape(-1,3)

        # and its respective length
        len_v0 = np.take_along_axis(len_v0,      idx_v0[:,None], axis=1).flatten()

        # prepare an array masking vectors with 0 or close to 0 length
        mask_zero_len = np.isclose(len_v0, 0.)
        mask_some_len = np.logical_not(mask_zero_len)

        # norm the vectors that have nonzero length
        v0[mask_some_len] /= len_v0[mask_some_len,None]

        # now to deal with vectors that have zero length
        # 1. case where we are dealing with 1st eigenvalue
        # should not happen due to the presorting of eigenvalues by
        # magniture from largest to smallest. If the largest (by magnitude)
        # eigenvalue has close to 0 length, it is a degenerative case where
        # there are infinite possibilities for axs direction, so we can
        # choose one aligned with the global x axis
        if i == 0:
            # fallback axis value - global x axis because it does not matter
            v0[mask_zero_len] = np.array([1., 0., 0.], dtype=float)

        # 2. case where we are dealing with 2nd eigenvalue
        elif i == 1:
            # get first principal axis
            v = T[mask_zero_len,0,:]
            # check whether it is the fallback axis
            mask_occupied = np.all(np.isclose(v, np.array([1., 0., 0.])), axis=1)
            # where it is a fallback axis, use global z axis
            v[mask_occupied] = np.array([0., 0., 1.])
            # obtain an arbitrary axis that is perpendicular to 1st and fallback
            v = np.cross(v, np.array([1., 0., 0.]))
            # norm it
```

```
            v /= np.linalg.norm(v, axis=1)[:,None]
            # store it
            v0[mask_zero_len] = v

        # 3. case where we are dealing with 3rd eigenvalue
        else:
            # create an axis perpendicular to the first 2
            v = np.cross(T[mask_zero_len,0,:], T[mask_zero_len,1,:])
            # norm it
            v /= np.linalg.norm(v, axis=1)[:,None]
            # store it
            v0[mask_zero_len] = v

        # store the axis
        T[mask_nonz,i,:] = v0[mask_nonz,:]

    # create the 3rd axis using a cross product of the two previous ones
    T[:,2,:] = np.cross(T[:,0,:], T[:,1,:])
    T[:,2,:] /= np.linalg.norm(T[:,2,:], axis=1)[:,None]


    # reorder the principal values and axes back to s1 > s2 > s3
    principal = np.take_along_axis(principal, amount_idx, axis=1) # for checking
    T = np.take_along_axis(T, amount_idx[:,:,None], axis=1)

    return orthonormalise(T)
```

1.4 Moment of Inertia

From: Parallel axis theorem

Parallel axis theorem, also known as *Huygens-Steiner's theorem* or just *Steiner's theorem* can be used to determine the **moment of inertia** or **second moment of area** of a rigid body about any axis, given the body's moment of inertia about a parallel axis through the objects center of gravity and the perpendicular distance between the axes.

Moment of Inertia = **tensor**.

- **Steiner's share:**

$$\mathbf{I}_{ss} = m \times \begin{bmatrix} y^2+z^2 & -xy & -xz \\ -yz & x^2+z^2 & -yz \\ -zx & -zy & x^2+y^2 \end{bmatrix}. \tag{1.102}$$

Identities for a skew-symmetric matrix

In order to compare formulations of the parallel axis theorem using skew-symmetric matrices and the tensor formulation, the following identities are useful.

Let $[\mathbf{R}]$ be the skew-symmetric matrix associated with the position vector $\mathbf{R} = (x, y, z)$, then the product in the inertia matrix becomes:

$$-[\mathbf{R}][\mathbf{R}] = \begin{bmatrix} 0 & -z & y \\ z & 0 & -x \\ -y & x & 0 \end{bmatrix}^2 = \begin{bmatrix} y^2+z^2 & -xy & -xz \\ -yz & x^2+z^2 & -yz \\ -zx & -zy & x^2+y^2 \end{bmatrix}. \tag{1.103}$$

This product can be computed using the matrix formed by the outer product $[\mathbf{R}\mathbf{R}^T]$ using the identity

$$\begin{aligned} -[\mathbf{R}]^2 &= |\mathbf{R}|^2 |\mathbf{E}_3| - [\mathbf{R}\mathbf{R}^T] \\ &= \begin{bmatrix} x^2+y^2+z^2 & 0 & 0 \\ 0 & x^2+y^2+z^2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \begin{bmatrix} x^2 & xy & xz \\ yx & y^2 & yz \\ zx & zy & z^2 \end{bmatrix}, \end{aligned} \tag{1.104}$$

where $[\mathbf{E}_3$ is the $n \times n$ identity matrix.

Also notice, that

$$|\mathbf{R}|^2 = \mathbf{R} \cdot \mathbf{R} = tr[\mathbf{R}\mathbf{R}^T]. \tag{1.105}$$

where tr denotes the sum of the diagonal elements of the outer product matrix, known as its *trace*.

To obtain matrix of moments of inertia by rotating about any random point A while having the moment of inertia in the center of gravity of the object $\mathbf{I}_{COG}$ and the coordinate distance $d = [x, y, z]$ between the point A and **COG** we:

1. Compute the moment of inertia relative to point A

$$\mathbf{I}_A = \mathbf{I}_{COG} + m \times \mathbf{I}_{SS}^{A \to COG}. \tag{1.106}$$

2. Transform the tensor $\mathbf{I}_A$ to new rotated coordinate system using transformation matrix $\mathbf{T}$:

$$\mathbf{I}_A^{'} = \mathbf{T} \times \mathbf{I}_A \times \mathbf{T}^T. \tag{1.107}$$

3. Subtract the **Steiner's share** relative to new **COG**

$$\mathbf{I}_{COG}^{'} = \mathbf{I}_A^{'} - \mathbf{I}_{SS}^{A' \to COG'}. \tag{1.108}$$

1.5 Quaternions

In mathematics, the quaternion number system extends the complex numbers. Quaternions were first described by the Irish mathematician William Rowan Hamilton in 1843 and applied to mechanics in three-dimensional space. The algebra of quaternions is often denoted by **H** or $\mathbb{H}$ (for Hamilton).

Although multiplication of quaternions is noncommutative, it gives a definition of the quotient of two vectors in a three-dimensional space. Quaternions are generally represented in the form:

$$a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k} \tag{1.109}$$

where the coefficients a, b, c, d are *real numbers* and $\mathbf{1}, \mathbf{i}, \mathbf{j}, \mathbf{k}$ are *basis vectors* or *basis elements.*

The Quaterninon multiplication table (left column shows pre-multiplier, top row shows post-multiplier):

r × c	**1**	**i**	**j**	**k**
1	**1**	**i**	**j**	**k**
i	**i**	−**1**	**k**	−**j**
j	**j**	−**k**	−**1**	**i**
k	**k**	**j**	−**i**	−**1**

Table 1.1: Quaternion multiplication table

also, $a\mathbf{b} = \mathbf{b}a$ and $-\mathbf{b} = (-1)\mathbf{b}$ for $a \in \mathbb{R}, \quad b = \mathbf{i}, \mathbf{j}, \mathbf{k}$.

1.5.1 Quaternion properties

The following properties apply:

1. Addition:

$$\begin{aligned}(a_1 + b_1\mathbf{i} + c_1\mathbf{j} + d_1\mathbf{k}) + (a_2 + b_2\mathbf{i} + c_2\mathbf{j} + d_2\mathbf{k}) = \\ = (a_1 + a_2) + (b_1 + b_2)\mathbf{i} + (c_1 + c_2)\mathbf{j} + (d_1 + d_2)\mathbf{k}\end{aligned} \tag{1.110}$$

2. Component-wise scalar multiplication:

$$\lambda(a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}) = \lambda a + (\lambda b)\mathbf{i} + (\lambda c)\mathbf{j} + (\lambda d)\mathbf{k} \tag{1.111}$$

3. Multiplication of basis elements (the revelation of Hamilton in 1943):

$$\begin{aligned}\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 &= -1 \\ \mathbf{ijk} &= -1\end{aligned} \tag{1.112}$$

4. Multiplicative identity:

$$\begin{aligned}\mathbf{i}1 = \quad 1\mathbf{i} = \quad \mathbf{i} \\ \mathbf{j}1 = \quad 1\mathbf{j} = \quad \mathbf{j} \\ \mathbf{k}1 = \quad 1\mathbf{k} = \quad \mathbf{k}\end{aligned} \tag{1.113}$$

5. Products of basis elements:

$$\begin{aligned}\mathbf{ij} = -\mathbf{ji} = \quad \mathbf{k} \\ \mathbf{jk} = -\mathbf{kj} = \quad \mathbf{i} \\ \mathbf{ki} = -\mathbf{ik} = \quad \mathbf{j}\end{aligned} \tag{1.114}$$

6. Inverse:

$$(a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k})^{-1} = \frac{1}{a^2 + b^2 + c^3 + d^2}(a - b\mathbf{i} - c\mathbf{j} - d\mathbf{k}) \tag{1.115}$$

7. Product of two **quaternions** (also called the **Hamilton product**) is determined by the products of the basis elements and the **distributive** law:

$$(a_1 + b_1\mathbf{i} + c_1\mathbf{j} + d_1\mathbf{k}) \times (a_2 + b_2\mathbf{i} + c_2\mathbf{j} + d_2\mathbf{k}) = \tag{1.116}$$

$$\begin{aligned}
=&a_1a_2 && +a_1b_2\mathbf{i} && +a_1c_2\mathbf{j} && +a_1d_2\mathbf{k}\\
+&b_1a_2\mathbf{i} && +b_1b_2\mathbf{i}^2 && +b_1c_2\mathbf{ij} && +b_1d_2\mathbf{ik}\\
+&c_1a_2\mathbf{j} && +c_1b_2\mathbf{ij} && +c_1c_2\mathbf{j}^2 && +c_1d_2\mathbf{jk}\\
+&d_1a_2\mathbf{k} && +d_1b_2\mathbf{ik} && +d_1c_2\mathbf{jk} && +d_1d_2\mathbf{k}^2
\end{aligned} \tag{1.117}$$

$$\begin{aligned}
=&a_1a_2 && -b_1b_2 && -c_1c_2 && -d_1d_2\\
+&(a_1b_2 && +b_1a_2 && +c_1d_2 && -d_1c_2) \times \mathbf{i}\\
+&(a_1c_2 && -b_1d_2 && +c_1a_2 && +d_1b_2) \times \mathbf{j}\\
+&(a_1d_2 && +b_1c_2 && -c_1b_2 && +d_1a_2) \times \mathbf{k}
\end{aligned} \tag{1.118}$$

1.5.2 Scalar and vector parts

A quaternion of the form $a + 0\mathbf{i} + 0\mathbf{j} + 0\mathbf{k}$, where a is a real number, is called **scalar**, and a quaternion of the form $0 + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$ where b, c and d are real numbers, and at least on of b, c or d is nonzero, is called a **vector quaternion**. If $a+b\mathbf{i}+c\mathbf{j}+d\mathbf{k}$ is any quaternion, then a is called its **scalar part** and $b\mathbf{i}+c\mathbf{j}+d\mathbf{k}$ is called its **vector part**. Although every quaternion can be viewed as a vector in a four-dimensional vector space, it is common to refer to the **vector** part as vectors in three-dimensional space.

Hamilton also called vector quaternions **right quaternions** and real numbers (considered as quaternions with zero vector part) **scalar quaternions**.

If a quaternion is divided up into a scalar and a vector part, that is,

$$\mathbf{q} = (r, \vec{v}), \mathbf{q} \in \mathbb{H}, r \in \mathbb{R}, \vec{v} \in \mathbb{R}^3 \tag{1.119}$$

then the formulas for **addition**, **multiplication** and **multiplicative inverse** are:

$$(r_1, \vec{v}_1) + (r_2, \vec{v}_2) = (r_1 + r_2, \vec{v}_1 + \vec{v}_2) \tag{1.120}$$

$$(r_1, \vec{v}_1) \times (r_2, \vec{v}_2) = (r_1 r_2 - \vec{v}_1 \cdot \vec{v}_2, r_1\vec{v}_2 + r_2\vec{v}_1 + \vec{v}_1 \times \vec{v}_2) \tag{1.121}$$

$$(r, \vec{v})^{-1} = \left(\frac{r}{r^2 + \vec{v} \cdot \vec{v}}, \frac{-\vec{v}}{r^2 + \vec{v} \cdot \vec{v}}\right) \tag{1.122}$$

1.5.3 Conjugation, the norm and reciprocal

Conjugation of quaternions is analogous to conjugation of complex numbers. To define it, let $\mathbf{q} = a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$ be a quaternion. The **conjugate** of $\mathbf{q}$ is the quaternion $\mathbf{q}^* = a - b\mathbf{i} - c\mathbf{j} - d\mathbf{k}$. It is denoted by $\mathbf{q}^*$, $\mathbf{q}^t$ or $\bar{\mathbf{q}}$.

Conjugation is **involution**, meaning that it is its own inverse, so **conjugating** an element **twice** returns the original element. The **conjugate** of a **product** of two quaternions is the product of the conjugates in the **reverse order**. That is, if $\mathbf{p}$ and $\mathbf{q}$ are quaterninos, then:

$$(\mathbf{pq})^* = \mathbf{q}^*\mathbf{p}^* \tag{1.123}$$

The conjugation of a quaternion, in stark contrast to the complex setting, can be expressed with multiplication and addition of quaternions:

$$\mathbf{q}^* = -\frac{1}{2}(\mathbf{q} + \mathbf{iqi} + \mathbf{jqj} + \mathbf{kqi}) \tag{1.124}$$

Conjugation can be used to extract the **scalar** and **vector** part of a quaternion. The **scalar** part of $\mathbf{q}$ is:

$$r_q = \frac{1}{2}(\mathbf{q} + \mathbf{q}^*) \tag{1.125}$$

The **vector** part of $\mathbf{q}$ is:

$$\vec{v}_q = \frac{1}{2}(\mathbf{q} - \mathbf{q}^*) \tag{1.126}$$

The **square root** of the product of a quaternion with its conjugate is called its **norm** and is denoted $\|q\|$ (Hamilton called this quantity the **tensor** of $\mathbf{q}$).

$$\|\mathbf{q}\| = \sqrt{\mathbf{q}^*\mathbf{q}} = \sqrt{\mathbf{q}\mathbf{q}^*} = \sqrt{a^2 + b^2 + c^2 + d^2} \in \mathbb{R} \geq 0 \tag{1.127}$$

This is always a non-negative, real number and is the same as the Euclidean norm on $\mathbb{H}$ considered as the vector space $\mathbb{R}^4$. Multiplying a quaternion by a real number scales its norm by the absolute value of the number. That is, if $\alpha \in \mathbb{R}$ then:

$$\|\alpha\mathbf{q}\| = |\alpha|\|\mathbf{q}\| \tag{1.128}$$

The norm is **multiplicative**, meaning that:

$$\|\mathbf{p}\mathbf{q}\| = \|\mathbf{p}\|\|\mathbf{q}\| \tag{1.129}$$

for any two quaternion $\mathbf{p}$ and $\mathbf{q}$. Multiplicativity is a consequence of the formula for the conjugate of a product. Alternatively it follows fom the identity

$$det \begin{pmatrix} a + ib & id + c \\ id - c & a - ib \end{pmatrix} = a^2 + b^2 + c^2 + d^2 \tag{1.130}$$

(where i denotes the usual imaginary unit) and hence from the multiplicative property of determinants of square matrices.

This norm makes it possible to define the **distance** $d(\mathbf{p}, \mathbf{q})$ between $\mathbf{p}$ and $\mathbf{q}$ as the norm of their difference:

$$d(\mathbf{p}, \mathbf{q}) = \|\mathbf{p} - \mathbf{q}\| \tag{1.131}$$

This makes $\mathbb{H}$ a **metric space**. Addition and multiplication are **continuous** in regard to the associated metric topology. This follows exactly the same proof as for the real numbers $\mathbb{R}$ from the fact that $\mathbb{H}$ is a **normed algebra**.

1.5.4 Unit quaternion (or versor)

A **unit quaternion** is a quaternion of norm one. Dividing a nonzero quaternion $\mathbf{q}$ by its norm produces a unit quaternion $\mathbf{Uq}$ called the **versor** of $\mathbf{q}$:

$$\mathbf{Uq} = \frac{\mathbf{q}}{\|\mathbf{q}\|} \tag{1.132}$$

Every nonzero quaternion has a unique **polar decomposition** $\mathbf{q} = \|\mathbf{q}\| \cdot \mathbf{Uq}$, while the zero quaternion can be formed from any unit quaternion.

Using conjugation and the norm makes it possible to define the **reciprocal** of a nonzero quaternion. The product of a quaternion with its reciprocal should be equal 1, and the considerations above imply that the product of $\mathbf{q}$ and $\mathbf{q}^*/\|\mathbf{q}\|^2$ is 1 (for any order fo multiplication). So the **reciprocal** of $\mathbf{q}$ is defined to be:

$$\mathbf{q}^{-1} = \frac{\mathbf{q}^*}{\|\mathbf{q}\|^2} \tag{1.133}$$

Since the multiplication is non-commutative, the quotient quantities $\mathbf{pq}^{-1}$ or $\mathbf{q}^{-1}\mathbf{p}$ are different (except if $\mathbf{p}$ and $\mathbf{q}$ are scalar multiples of each other or if one is a scalar). The notation $\frac{p}{q}$ is ambiguous and **should not be used**.

1.5.5 Quaternions and three-dimensional geometry

The vector part of a quaternion can be interpreted as a coordinate vector in $\mathbb{R}^3$, therefore the algebraic operations fo the quaternions reflect the geometry of $\mathbb{R}^3$. Operations such as the vector dot and cross products can be defined in terms of quaternions and thei makes it possible to apply quaternion techniques wherever spatial vectors arise. A useful application of quaternions has been to interpolate the orientations of key-frames in computer graphics.

For the remainder of this section, $\mathbf{i}$, $\mathbf{j}$ and $\mathbf{k}$ wil denote both the three imaginary basis vectors of $\mathbb{H}$ and a basis for $\mathbb{R}^3$. Replacing $\mathbf{i}$ by $-\mathbf{i}$, $\mathbf{j}$ by $-\mathbf{j}$ and $\mathbf{k}$ by $-\mathbf{k}$ sends a vector to its **additive inverse**, so the additive inverse of a vector is the same as its conjugate as a quaternion. For this reason, conjugation is sometimes called the **spatial inverse**.

For two vector quaternions $\mathbf{p} = b_1\mathbf{i} + c_1\mathbf{j} + d_1\mathbf{k}$ and $\mathbf{q} = b_2\mathbf{i} + c_2\mathbf{j} + d_2\mathbf{k}$ their **dot product**, by analogy to vectors in $\mathbb{R}^3$, is:

$$\mathbf{p} \cdot \mathbf{q} = b_1 b_2 + c_1 c_2 + d_1 d_2 \tag{1.134}$$

It can also be expressed ina a component-free manner as:

$$\mathbf{p} \cdot \mathbf{q} = \frac{1}{2}(\mathbf{p}^*\mathbf{q} + \mathbf{q}^*\mathbf{p}) = \frac{1}{2}(\mathbf{p}\mathbf{q}^* + \mathbf{q}\mathbf{p}^*) \tag{1.135}$$

This is equal to the scalar parts of the products $\mathbf{pq}^*$, $\mathbf{qp}^*$, $\mathbf{p}^*\mathbf{q}$ and $\mathbf{q}^*\mathbf{p}$. Note that their vector parts are different.

The **cross product** of $\mathbf{p}$ and $\mathbf{q}$ relative to the orientation determined by the ordered basis $\mathbf{i}$, $\mathbf{j}$ and $\mathbf{k}$ is:

$$\mathbf{p} \times \mathbf{q} = (c_1 d_2 - d_1 c_2)\mathbf{i} + (d_1 b_2 - b_1 d_2)\mathbf{j} + (b_1 c_2 - c_1 b_2)\mathbf{k} \tag{1.136}$$

(The orientation is necessary to determine the sign.) This is equal to the vector part of the product $\mathbf{pq}$ (as quaternions), as well as the vector part of $-\mathbf{q}^*\mathbf{p}^*$. It also has the formula:

$$\mathbf{p} \times \mathbf{q} = \frac{1}{2}(\mathbf{pq} - \mathbf{qp}) \tag{1.137}$$

For the **commutator**, $[\mathbf{p}, \mathbf{q}] = \mathbf{pq} - \mathbf{qp}$ of two vector quaternions one obtains:

$$[\mathbf{p}, \mathbf{q}] = 2\mathbf{p} \times \mathbf{q} \tag{1.138}$$

In general, let $\mathbf{p}$ and $\mathbf{q}$ be quaternions and write:

$$\begin{aligned}\mathbf{p} &= \mathbf{p}_s + \mathbf{p}_v \\ \mathbf{q} &= \mathbf{q}_s + \mathbf{q}_v\end{aligned} \tag{1.139}$$

where $\mathbf{p}_s$ and $\mathbf{q}_s$ are the scalar parts, and $\mathbf{p}_v$ and $\mathbf{q}_v$. Then we have the formula:

$$\mathbf{pq} = (\mathbf{pq})_s + (\mathbf{pq})_v = (\mathbf{p}_s\mathbf{q}_s - \mathbf{p}_v \cdot \mathbf{q}_v) + (\mathbf{p}_s\mathbf{q}_v + \mathbf{q}_s\mathbf{p}_v + \mathbf{p}_v \times \mathbf{q}_v) \tag{1.140}$$

This shows that the noncommutativity of quaternion multiplication comes from the multiplication of vector quaternions. It also shows that two quaternions commute if and only if their vector parts are collinear. **Hamilton** showed that this product computes the third vertex of spherical triangle from two given vertices and their associated arc-lengths, which is also an algebra of points in **Elliptic geometry**.

Unit quaternions can be identified with rotations in $\mathbb{R}^3$ and were called **versors** by **Hamilton**.

1.5.6 Matrix representations

Just as complex numbers can be **represented as matrices**, co can be quaternions. There are *at least* two ways to represent quaternions as matrices in such a way that quaternion **addition** and **multiplication** correspond to the matrix addition and multiplication.

The quaternion $a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$ can be represented:

1. as a 2×2 complex matrix:

$$\begin{bmatrix} a+bi & c+di \\ -c+di & a-bi \end{bmatrix} \tag{1.141}$$

   This representation has the following properties:

- Constraining any two of b, c and d to zero produces a representation fo complex numbers. For example, setting $c = d = 0$ produces a diagonal complex matrix representation of complex numbers, and setting $b = d = 0$ produces a real matrix representation.
- The **norm** of a quaternion (the square root of the product with its conjugate, as with complex numbers) is the square root of the **determinant** of the corresponding matrix.
- The **conjugate** of a quaternion corresponds to the **conjugate transpose** of the matrix.
- There is a strong relation between quaternion units and **Pauli matrices** (these are needed to describe the *spin in quantum mechanics*). Obtain the eight quaternion unit matrices by taking a, b, c and d, set three of them to zero and the fourth to 1 or -1. Multiplying any two Pauli matrices always yields a quaternion unit matrix, all of them except for -1. One obtains -1 via the last equality:

$$\begin{aligned} \mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} = -1 \\ ijk = \sigma_1\sigma_2\sigma_3\sigma_1\sigma_2\sigma_3 = -1 \end{aligned} \tag{1.142}$$

2. Using 4×4 real matrix the same quaternion can be written as:

$$\begin{aligned} \begin{bmatrix} a & -b & -c & -d \\ b & a & -d & c \\ c & d & a & -b \\ d & -c & b & a \end{bmatrix} &= a \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ &+ b \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \\ &+ c \begin{bmatrix} 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \end{bmatrix} \\ &+ d \begin{bmatrix} 0 & 0 & 0 & -1 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \end{aligned} \tag{1.143}$$

However the representation above is not unique. For example, the same quaternion can be also represented as:

$$\begin{bmatrix} a & d & -b & -c \\ -d & a & c & -b \\ b & -c & a & -d \\ c & b & d & a \end{bmatrix} = a \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} + b \begin{bmatrix} 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} + c \begin{bmatrix} 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} + d \begin{bmatrix} 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \tag{1.144}$$

There exist 48 distinct matrix representations of thei form in which one of the matrices represents the scalar part and the other three are all skew-symmetric.

1.5.7 Exponential, logarithm and power functions

Given a quaternion:

$$\mathbf{q} = a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k} = r + \vec{\mathbf{v}} \tag{1.145}$$

1. the exponential is computed as

$$\exp(\mathbf{q}) = \sum_{n=0}^{\infty} \frac{\mathbf{q}^n}{n!} = e^a \left(\cos \|\vec{\mathbf{v}}\| + \frac{\vec{\mathbf{v}}}{\|\vec{\mathbf{v}}\|} \sin \|\vec{\mathbf{v}}\| \right) \tag{1.146}$$

2. the logarithm is:

$$\ln(\mathbf{q}) = \ln\|\mathbf{q}\| + \frac{\vec{\mathbf{v}}}{\|\vec{\mathbf{v}}\|} \arccos\frac{r}{\|\mathbf{q}\|} \tag{1.147}$$

3. it follows that the polar decomposition of a quaternion may be written

$$\mathbf{q} = \|\mathbf{q}\| e^{\hat{\mathbf{n}}\varphi} = \|\mathbf{q}\| \left(\cos(\varphi) + \hat{\mathbf{n}}\sin(\varphi)\right) \tag{1.148}$$

   where the angle φ is defined as

$$r = \|\mathbf{q}\| \cos(\varphi) \tag{1.149}$$

   and the unit vecotr $\hat{\mathbf{n}}$ is defined by

$$\vec{\mathbf{v}} = \hat{\mathbf{n}}\|\vec{\mathbf{v}}\| = \hat{\mathbf{n}}\|\mathbf{q}\| \sin(\varphi) \tag{1.150}$$

   Any unit quaternion may be expressed in polar form as:

$$\mathbf{q} = \exp(\hat{\mathbf{n}}\varphi) \tag{1.151}$$

4. The power of a quaternion raised to an arbitrary (real) exponend x is given by:

$$\mathbf{q}^x = \|\mathbf{q}\|^x e^{\hat{\mathbf{n}}x\varphi} = \|\mathbf{q}\|^x \left(\cos(x\varphi) + \hat{\mathbf{n}}\sin(x\varphi)\right) \tag{1.152}$$

1.5.8 Quaternions used for coordinate transformation

One can think of **quaternions** as a stored information about coordinate mapping from one space to another. We can split arbitrary **quaternion** $\mathbf{q} = a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$ into **two** parts:

1. a **real** part $r = a$ and
2. a **vector** part $\mathbf{v} = [b, c, d]$.

Next, if we **impose** restrictions on such **quaternions** that:

1. the **real** part must satisfy: $r \in \langle -\pi; \pi \rangle$
2. the **vector** part must have **unit** length: $|\mathbf{v}| = \sqrt{b^2 + c^2 + d^2} = 0$

Then, if one thinks of the complex numbers **i**, **j** and **k** as of basis vectors of cartesian system **x**, **y** and **z**, where

$$\begin{aligned} \mathbf{x} &= [1.0, 0.0, 0.0] \\ \mathbf{y} &= [0.0, 1.0, 0.0] \\ \mathbf{z} &= [0.0, 0.0, 1.0] \end{aligned} \tag{1.153}$$

the quaternion multiplication table 1.1 makes sense in relation to **vector cross product**:

r × c	**1**	**x**	**y**	**z**
1	**1**	**x**	**y**	**z**
x	**x**	−**1**	**z**	−**y**
y	**y**	−**z**	−**1**	**x**
z	**z**	**y**	−**x**	−**1**

Table 1.2: Quaternion cartesian base vectors multiplication table

In 3-dimensional space, according to the **Euler's rotation theorem**, any rotation or a sequence of rotations of a rigid body or coordinate system about a fixed point is equivalent to a single rotation by a given angle θ about a fixed axis (called the *Euler axis*) that runs through the fixed point. The Euler axis is typically represented by a **unit vector $\vec{\mathbf{u}}$**. Therefore, any rotation in three dimensions can be represented as via a **vector $\vec{\mathbf{u}}$** and and angle θ.
Quaternions give a simple way to encode this **axis-angle** representation using four real numbers and can be used to apply (calculate) the corresponding rotation to a **position vector** $[x, y, z]$, representing a point relative to the origin in $\mathbb{R}^3$.

Euclidean vectors such as $[a_x, a_y, a_z]$ can be rewritten as $a_x\mathbf{i}+a_y\mathbf{j}+a_z\mathbf{k}$, where **i**, **j** and **k** are unit vectors representing the three **Cartesian axies** (traditionally **x**, **y** and **z**) and also obey the **multiplication** rules of the fundamental quaternion units by interpreting the Euclidean vector $[a_x, a_y, a_z]$ as the vector part of the pure quaternion $(0, a_x, a_y, a_z)$.

A rotation angle θ around the axis defined by the unit vector

$$\vec{\mathbf{u}} = (u_x, u_y, u_z) = u_x\mathbf{i} + u_y\mathbf{j} + u_z\mathbf{k} \tag{1.154}$$

can be represented by **conjugation** by a unit quaternion **q**.

Since the quaternion product

$$(0 + u_x\mathbf{i} + u_y\mathbf{j} + u_z\mathbf{k})(0 - u_x\mathbf{i} - u_y\mathbf{j} - u_z\mathbf{k}) = -1 \tag{1.155}$$

using **Taylor series** of the exponential function, the extension of the Euler's formula results in:

$$\begin{aligned}\mathbf{q} &= e^{\frac{\theta}{2}(u_x\mathbf{i}+u_y\mathbf{j}+u_z\mathbf{k})}\\ &= \cos\frac{\theta}{2} + (u_x\mathbf{i} + u_y\mathbf{j} + u_z\mathbf{k})\sin\frac{\theta}{2}\\ &= \cos\frac{\theta}{2} + \vec{\mathbf{u}}\sin\frac{\theta}{2}\end{aligned} \tag{1.156}$$

It can be shown that the desired rotation can be applied to an ordinary vector

$$\mathbf{p} = (p_x, p_y, p_z) = p_x\mathbf{i} + p_y\mathbf{j} + p_z\mathbf{k} \tag{1.157}$$

in 3-dimensional space, considered as the vector part of the pure quaternion $\mathbf{p}'$, by evaluating the **conjugation** of $\mathbf{p}'$ by **q**, given by:

$$\begin{aligned}L(\mathbf{p}') &:= \mathbf{q}\mathbf{p}'\mathbf{q}^{-1}\\ r &= \left(\cos^2\frac{\theta}{2} - \|\mathbf{u}\|^2\right) + 2(\mathbf{u}\cdot\mathbf{p})\mathbf{u} + 2\cos\frac{\theta}{2}(\mathbf{u}\times\mathbf{p})\end{aligned} \tag{1.158}$$

using the **Hamilton product** where the vector part of the pute quaternion $L(\mathbf{p}') = (0, r_x, r_y, r_z)$ is the new position vector after the rotation. In a programmatic implementation **the conjugation** is achieved by constructing a pure quaternion whise vector part is $\mathbf{p}$, and then performing the quaternion conjugation. The vector part of the resulting pure quaternion is the desired vector $\mathbf{r}$. Clearly L provides a linear transformation of the quaternion space to itself, also, since $\mathbf{q}$ is unitary, the transformation is an isometry. Also $L(\mathbf{q}) = \mathbf{q}$ ans so L leaves vectors parallel to $\mathbf{q}$ invariant. So, by decomposing $\mathbf{p}$ as a vector parallel to the vector part $(u_x, u_y, u_z)\sin\frac{\theta}{2}$ of $\mathbf{q}$ and a vector normal to the vector part of $\mathbf{q}$ and showing that the application of L to the normal component of $\mathbf{p}$ rotates it, the claim is shown.

So let $\mathbf{n}$ be the component of $\mathbf{p}$ orthogonal to the vector part of $\mathbf{q}$ and let $\mathbf{n}_T = \mathbf{n} \times \mathbf{u}$. It turns out that the vector part of $L(0, \mathbf{n})$ is given by:

$$\left(\cos^2\frac{\theta}{2} - \sin^2\frac{\theta}{2}\right)\mathbf{n} + 2\left(\cos^2\frac{\theta}{2}\sin^2\frac{\theta}{2}\right)\mathbf{n}_T = \cos\theta\mathbf{n} + \sin\theta\mathbf{n}_T \tag{1.159}$$

A geometric fact independent of quaternion is the existence of **two-to-one** mapping from physical rotations to rotational transformation matrices.

If $0 \le \theta \le 2\pi$, a physical rotaion about $\vec{\mathbf{u}}$ by θ and a physical rotation about $-\vec{\mathbf{u}}$ by $2\pi - \theta$ both achieve the **same final orientation** by disjoint paths through intermediate orientations.

By inserting those vectors and angles into the formula for $\mathbf{q}$ above, one findes that if $\mathbf{q}$ represents the first rotation, $-\mathbf{q}$ represents the second rotation. This is a geometric proof that conjugation by $\mathbf{q}$ and by $-\mathbf{q}$ must produce the same rotational transformation matrix. That fact is confirmed algebraically by noting that the conjugation is quadratic in $\mathbf{q}$, so the sign of $\mathbf{q}$ cancels, and does not affect the result.

If both rotations are a half-turn ($\theta = \pi$), both $\mathbf{q}$ and $-\mathbf{q}$ will have real coordinate equal to zero. Otherwise, one will have a positive real part, representing a rotation by an angle less than π, and the other will have a negative real part, representing a rotation by an angle greater than π.

Mathemativally, this operation carries the set of all "pure" quaterninos $\mathbf{p}$ (thise with real part equal to zero) - which consitute a 3-dimensional space among the quaternions - into itself, by the desired rotation about the axis **u**, by the angle θ.

The rotation is **clockwise** if our line of sight points in the same direction as $\vec{\mathbf{u}}$.

In the instance that **q** is a **unit quaternion** and

$$\mathbf{q}^{-1} = e^{-\frac{\theta}{2}(u_x\mathbf{i}+u_y\mathbf{j}+u_z\mathbf{k})} = \cos\frac{\theta}{2} - (u_x\mathbf{i} + u_y\mathbf{j} + u_z\mathbf{k})\sin\frac{\theta}{2} \tag{1.160}$$

It follows that conjugation by the product of two quaternions is the composition of conjugations by these quaternions. If **p** and **q** are unit quaternions, then conjugation (**rotation**) by **pq** is:

$$\mathbf{pq}\vec{\mathbf{v}}(\mathbf{pq})^{-1} = \mathbf{pq}\vec{\mathbf{v}}\mathbf{q}^{-1}\mathbf{p}^{-1} = \mathbf{p}(\mathbf{q}\vec{\mathbf{v}}\mathbf{q}^{-1})\mathbf{p}^{-1} \tag{1.161}$$

which is **the same** as conjugating (**rotating**) by **q** and then by **p**. The scalar component of the result is **necessarily zero**.

The quaternion **inverse** of a rotation is the opposite rotation, since

$$\mathbf{q}^{-1}(\mathbf{q}\vec{\mathbf{v}}\mathbf{q}^{-1})\mathbf{q} = \vec{\mathbf{v}} \tag{1.162}$$

The square of a quaternion rotation is a rotation by twice the angle around the same axis.

Chapter 2

Boundary conditions not in GCS

2.1 Skew boundary conditions

Skew boundary conditions, also known as **Rotated BCs** are implemented after assembling the global stiffness matrix $\mathbf{K}$.

Basic equation:

$$\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{f} \tag{2.1}$$

First we need to identify all prescribed BCs that do not correspond to the defined assemblage DOFs and transform the finite equilibrium equations to correspond to the prescribed BCs. Thus we write:

$$\mathbf{u} = \mathbf{T}\bar{\mathbf{u}} \tag{2.2}$$

where $\bar{\mathbf{u}}$ is the vector of nodal point DOFs in the required directions. The transformation matrix $\mathbf{T}$ is an identity matrix that has been altered by the directional sines and cosines of the components $\bar{\mathbf{u}}$ measured in the original displacement directions. Using (2.2), we obtain:

$$\overline{\mathbf{M}}\ddot{\bar{\mathbf{u}}} + \overline{\mathbf{C}}\dot{\bar{\mathbf{u}}} + \overline{\mathbf{K}}\bar{\mathbf{u}} = \bar{\mathbf{f}} \tag{2.3}$$

where:

$$\begin{aligned}
\overline{\mathbf{M}} &= \mathbf{T}^T\mathbf{M}\mathbf{T} \\
\overline{\mathbf{C}} &= \mathbf{T}^T\mathbf{C}\mathbf{T} \\
\overline{\mathbf{K}} &= \mathbf{T}^T\mathbf{K}\mathbf{T} \\
\bar{\mathbf{f}} &= \mathbf{T}^T\mathbf{f}
\end{aligned} \tag{2.4}$$

We should note that the matrix multiplications in (2.4) involve changes only in those columns and rows of $\mathbf{M}$, $\mathbf{C}$, $\mathbf{K}$ and $\mathbf{f}$ that are actually affected. In practice, the transformation is carried out effectively on the element level just prior to adding the element matrices to the global matrices.

2.2 Cartesian CSYS

When a node or an SPC is defined as a **rotated/skewed** or in a different type of Coordinate system (e.g. cylindrical) a transformation matrix needs to be applied to such nodes.

Cartesian Coordinate system definition by angle (2D):

$$\mathbf{u}_l = \mathbf{T}\mathbf{u}_g \tag{2.5}$$

Unpacked:

$$\begin{bmatrix} u_l \\ w_l \\ \psi_l \end{bmatrix} = \begin{bmatrix} \cos\alpha & \sin\alpha & 0 \\ -\sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_g \\ w_g \\ \psi_g \end{bmatrix} \tag{2.6}$$

Cartesian Coordinate system definition by vectors (3D):

when a **CSYS** is defined by an **origin** and **two vectors** $\mathbf{x} = \begin{pmatrix} x_1 & x_2 & x_3 \end{pmatrix}$ and $\mathbf{y} = \begin{pmatrix} y_1 & y_2 & y_3 \end{pmatrix}$, then for a Cartesian CSYS it suffices to:

$$\begin{aligned} \bar{\mathbf{x}} &= \frac{\mathbf{x}}{||\mathbf{x}||} \\ \bar{\mathbf{y}} &= \frac{\mathbf{y}}{||\mathbf{y}||} \\ \bar{\mathbf{z}} &= \bar{\mathbf{x}} \times \bar{\mathbf{y}} \end{aligned} \tag{2.7}$$

The transformation relation:

$$\mathbf{u}_l = \mathbf{T}\mathbf{u}_g \tag{2.8}$$

when unpacked:

$$\begin{bmatrix} u_l \\ v_l \\ w_l \\ \phi_l \\ \psi_l \\ \rho_l \end{bmatrix} = \begin{bmatrix} \mathbf{x} & \mathbf{0} \\ \mathbf{y} & \mathbf{0} \\ \mathbf{z} & \mathbf{0} \\ \mathbf{0} & \mathbf{x} \\ \mathbf{0} & \mathbf{y} \\ \mathbf{0} & \mathbf{z} \end{bmatrix} \begin{bmatrix} u_g \\ v_g \\ w_g \\ \phi_g \\ \psi_g \\ \rho_g \end{bmatrix} \tag{2.9}$$

Where $\mathbf{0}$ is a 3×1 zero vector.

For a **2D case** just **one vector** is enough, the $\bar{\mathbf{z}} = \begin{pmatrix} 0 & 0 & 1 \end{pmatrix}$ and following relations apply:

$$\begin{aligned}
\bar{\mathbf{x}} &= \bar{\mathbf{y}} \times \bar{\mathbf{z}} \\
\bar{\mathbf{y}} &= \bar{\mathbf{z}} \times \bar{\mathbf{x}} \\
\bar{\mathbf{z}} &= \bar{\mathbf{x}} \times \bar{\mathbf{y}}
\end{aligned} \tag{2.10}$$

This means that a **CSYS** defined by an **origin** point $\mathbf{O} = \begin{pmatrix} x_O & y_O & z_O \end{pmatrix}$, point laying on **x axis** $\mathbf{P}_x = \begin{pmatrix} x_{P_x} & y_{P_x} & z_{P_x} \end{pmatrix}$ and a point laying in **xy plane** $\mathbf{P}_{xy} = \begin{pmatrix} x_{P_{xy}} & y_{P_{xy}} & z_{P_{xy}} \end{pmatrix}$ define a CSYS followingly:

$$\begin{aligned}
\mathbf{x} &= \begin{pmatrix} x_{P_x} - x_O & y_{P_x} - y_O & z_{P_x} - z_O \end{pmatrix} \\
\bar{\mathbf{x}} &= \frac{\mathbf{x}}{||\mathbf{x}||} \\
\hat{\mathbf{y}} &= \begin{pmatrix} x_{P_{xy}} - x_O & y_{P_{xy}} - y_O & z_{P_{xy}} - z_O \end{pmatrix} \\
\bar{\hat{\mathbf{y}}} &= \frac{\hat{\mathbf{y}}}{||\hat{\mathbf{y}}||} \\
\bar{\mathbf{z}} &= \bar{\mathbf{x}} \times \bar{\hat{\mathbf{y}}} \\
\bar{\mathbf{y}} &= \bar{\mathbf{z}} \times \bar{\mathbf{x}}
\end{aligned} \tag{2.11}$$

2.3 Cylindrical CSYS

The basic relation between a point in **cartesian** and **cylindrical** CSYS is (when the cartesian CSYS has the same origin as the cylindrical one and both CSYS z axes overlay):

$$\begin{aligned} x &= r\cos\varphi \\ y &= r\sin\varphi \\ z &= z \end{aligned} \tag{2.12}$$

in one direction and:

$$\begin{aligned} r &= \sqrt{x^2+y^2} \\ \varphi &= \begin{cases} indeterminate & if\ x=0\ and\ y=0 \\ \arcsin\frac{y}{r} & if\ x\geq 0 \\ -\arcsin\frac{y}{r}+\pi & if\ x<0\ and\ y\geq 0 \\ -\arcsin\frac{y}{r}+\pi & if\ x<0\ and\ y<0 \end{cases} \end{aligned} \tag{2.13}$$

in the other. One can also use the arctan function to compute φ:

$$\varphi = \begin{cases} indeterminate & if\ x=0\ and\ y=0 \\ \frac{\pi}{2}\frac{y}{|y|} & if\ x=0\ and\ y\neq 0 \\ \arctan\frac{y}{x} & if\ x>0 \\ \arctan\frac{y}{x}+\pi & if\ x<0\ and\ y\geq 0 \\ \arctan\frac{y}{x}-\pi & if\ x<0\ and\ y<0 \end{cases} \tag{2.14}$$

This is also called the $\mathrm{atan2}(y,x)$ function!

To transform node coordinates from **cartesian** to **cylindrical**, three transforms are in order:

1. translate the coordinates so that **origin** of the new CSYS conforms to the **origin** of the **cylidrical** CSYS.

$$\mathbf{x}_1 = \mathbf{T}_T\mathbf{x}_0 \tag{2.15}$$

$$\begin{bmatrix} x_1 \\ y_1 \\ z_1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & -x_O^c \\ 0 & 1 & 0 & -y_O^c \\ 0 & 0 & 1 & -z_O^c \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \\ z_0 \\ 1 \end{bmatrix} \tag{2.16}$$

where $\mathbf{x}_O^c$ denotes the **cylindrical** CSYS origin and $\mathbf{T}_T$ denotes the translation matrix.

2. **rotate** the node to a new **cartesian** CSYS so that the new CSYS **x-axis** conforms to the cylindrical CSYS **r-axis** and the new CSYS **z-axis** conforms to the cylindrical **z-axis**.

$$\mathbf{x}_2 = \mathbf{T}_R \mathbf{x}_1 \tag{2.17}$$

When the cylindrical CSYS is defined by three points - **origin** $\mathbf{x}_O^c = \begin{bmatrix} x_O^C & y_O^c & z_O^c \end{bmatrix}^T$, point on **r-axis** $\mathbf{x}_R^c$ and a point in **rz-plane** $\mathbf{x}_{RZ}^c$, then:

$$\vec{\mathbf{r}} = \frac{\mathbf{x}_R^c - \mathbf{x}_O^c}{||\mathbf{x}_R^c - \mathbf{x}_O^c||} \tag{2.18}$$

$$\hat{\mathbf{z}} = \frac{\mathbf{x}_{RZ}^c - \mathbf{x}_O^c}{||\mathbf{x}_{RZ}^c - \mathbf{x}_O^c||} \tag{2.19}$$

$$\vec{\mathbf{y}} = \hat{\mathbf{z}} \times \vec{\mathbf{r}} \tag{2.20}$$

$$\vec{\mathbf{z}} = \vec{\mathbf{r}} \times \vec{\mathbf{y}} \tag{2.21}$$

then the transformation can be written:

$$\begin{bmatrix} x_2 \\ y_2 \\ z_2 \\ 1 \end{bmatrix} = \begin{bmatrix} \vec{\mathbf{r}}^T & 0 \\ \vec{\mathbf{y}}^T & 0 \\ \vec{\mathbf{z}}^T & 0 \\ \vec{\mathbf{0}}^T & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ z_1 \\ 1 \end{bmatrix} \tag{2.22}$$

where $\vec{\mathbf{0}} = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^T$

Combining $\mathbf{T}_R$ and $\mathbf{T}_T$ together:

$$\mathbf{T} = \mathbf{T}_R \mathbf{T}_T \tag{2.23}$$

$$\mathbf{T} = \begin{bmatrix} r_1 & r_2 & r_3 & 0 \\ y_1 & y_2 & y_3 & 0 \\ z_1 & z_2 & z_3 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -x_O^c \\ 0 & 1 & 0 & -y_O^c \\ 0 & 0 & 1 & -z_O^c \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.24}$$

$$\mathbf{T} = \begin{bmatrix} r_1 & r_2 & r_3 & -r_1 x_O^c - r_2 y_O^c - r_3 z_O^c \\ y_1 & y_2 & y_3 & -y_1 x_O^c - y_2 y_O^c - y_3 z_O^c \\ z_1 & z_2 & z_3 & -z_1 x_O^c - z_2 y_O^c - z_3 z_O^c \end{bmatrix} \tag{2.25}$$

3. transform the coordinates to the **cylindrical** CSYS.

$$\begin{bmatrix} r \\ \varphi \\ z \end{bmatrix} = \begin{bmatrix} \sqrt{x_2^2 + y_2^2} \\ \operatorname{atan2}(y_2, x_2) \\ z_2 \end{bmatrix} \tag{2.26}$$

A **cylindrical** coordinate system in FEM in the range of **small displacements** is basically a Cartesian one, just skewed for each node separately so that its $\mathbf{x}$ axis coincides with $\mathbf{r}$ axis.
Then the cylindrical to global coordinate transformation is such that:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix}_{global} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \begin{bmatrix} r\cos\varphi \\ r\sin\varphi \\ z \end{bmatrix}_{local} + \begin{bmatrix} x \\ y \\ z \end{bmatrix}_{origin} \tag{2.27}$$

Therefore the Cartesian CSYS transformation matrix for definition of boundary conditions is:

1. first subtract the cylindrical coordinate origin from the global coordinates:
$$\begin{bmatrix} x \\ y \\ z \end{bmatrix}_{local} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}_{global} - \begin{bmatrix} x \\ y \\ z \end{bmatrix}_{origin} \tag{2.28}$$

2. then create a transformation matrix that maps the global csys to rotated one such that the $\mathbf{r}$ axis coincides with the $\mathbf{x}$ axis, $\boldsymbol{\varphi}$ axis coincides with the new $\mathbf{y}$ axis and $\mathbf{z}$ is the same:
$$\mathbf{r}' = \frac{\mathbf{x}_{local}}{||\mathbf{x}_{local}||} \tag{2.29}$$
$$\mathbf{z} = \frac{\mathbf{z}_{cyl}}{||\mathbf{z}_{cyl}||} \tag{2.30}$$
$$\boldsymbol{\varphi} = \mathbf{z} \times \mathbf{r}' \tag{2.31}$$
$$\mathbf{r} = \boldsymbol{\varphi} \times \mathbf{z} \tag{2.32}$$
the transformation matrix is then:
$$\mathbf{T} = \begin{bmatrix} r_1 & r_2 & r_3 \\ \varphi_1 & \varphi_2 & \varphi_3 \\ z_1 & z_2 & z_3 \end{bmatrix} \tag{2.33}$$

3. This transformation matrix $\mathbf{T}$ is finally applied and **BC**s are imposed.
$$\mathbf{T}^T\mathbf{K}\mathbf{T}\bar{\mathbf{u}} = \mathbf{T}^T\mathbf{f} \tag{2.34}$$

If there already is a transformation matrix for the cylindrical system prepared, such that:

$$\mathbf{x}_{local} = \mathbf{T}_{cyl} \left(\mathbf{x}_{global} - \mathbf{x}_{origin} \right) \tag{2.35}$$

or:

$$\mathbf{x}_{global} = \mathbf{T}_{cyl}^{T} \mathbf{x}_{local} + \mathbf{x}_{origin} \tag{2.36}$$

then:

$$r = \sqrt{x_{local}^2 + y_{local}^2} \tag{2.37}$$

$$\varphi = \operatorname{atan2} \frac{y_{local}}{x_{local}} \tag{2.38}$$

$$z = z_{local} \tag{2.39}$$

because $\cos\varphi = x/r$ and $\sin\varphi = y/r$, then the new transformation matrix is composed:

$$\mathbf{T} = \begin{bmatrix} \cos & -\sin & 0 \\ \sin & \cos & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} x/r & -y/r & 0 \\ y/r & x/r & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{2.40}$$

and finally displacements are transformed from global to local:

$$\mathbf{u}_{local} = \mathbf{T}\mathbf{T}_{cyl} \mathbf{u}_{global} \tag{2.41}$$

Chapter 3

Multifreedom Constraints

A finite element model is assembled as if its nodal degrees of freedom were independent. Engineering models, however, frequently require some of those freedoms to satisfy additional relations. Two nodes may have to move together, a displacement may have to equal a weighted combination of several others, or a geometric relation may have to remain satisfied while the structure deforms. Such relations are called *multifreedom constraints* (**MFCs**).

The important distinction in this chapter is between the *definition* of a constraint and the *numerical method used to impose it.* A rigid link, for example, describes an engineering relation between motions. Elimination, penalty augmentation and Lagrange multipliers are alternative algorithms for making the finite element equations obey that relation. Chapter 5 uses these methods to construct rigid and interpolating couplings, while Chapter 4 extends the same ideas to unilateral constraints whose active state can change.

This chapter deliberately moves between three representations of the same idea:

$$\textbf{engineering relation} \longleftrightarrow \textbf{matrix formulation} \longleftrightarrow \textbf{discrete algorithm}.$$

The first tells us what motion is physically admissible. The second exposes rank, work conjugacy, conditioning and other mathematical properties. The third shows what a finite element program must actually store, index, assemble and solve. A good computational model requires all three viewpoints.

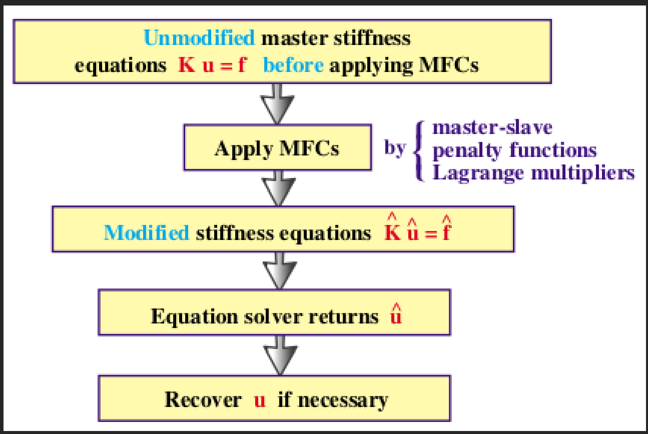


Figure 3.1: Schematic examples of multifreedom constraints. The engineering relation is converted into algebraic relations between discrete nodal DOFs before being imposed by a numerical constraint algorithm.

Python examples in this chapter.
The examples are intentionally small, reusable implementations of the equations. `numpy` is used for arrays, indexing, elementary arithmetic and matrix products only. No `numpy.linalg`, sparse solver, eigensolver or other NumPy submodule is used. Small dense systems are solved by an explicit Gaussian elimination routine so that the transition from a matrix equation to loops and array operations remains visible.
These functions are educational rather than production finite element code. Real solvers use sparse storage, graph algorithms, symbolic preprocessing, rank-revealing factorizations and carefully selected equation solvers, but the underlying operations are the same.

3.1 Constraint Definitions and Matrix Form

A general bilateral constraint may be written as

$$c_i(\mathbf{u}) = 0, \tag{3.1}$$

where $\mathbf{u}$ is the vector of nodal displacement components. A set of m constraints is collected as

$$\mathbf{c}(\mathbf{u}) = \mathbf{0}. \tag{3.2}$$

The constraint is *linear* if it can be written

$$\boxed{\mathbf{A}\mathbf{u} = \mathbf{g}}, \tag{3.3}$$

where $\mathbf{A} \in \mathbb{R}^{m\times n}$ contains the constraint coefficients, $\mathbf{u} \in \mathbb{R}^n$ contains the model DOFs and $\mathbf{g} \in \mathbb{R}^m$ contains the prescribed values. It is *homogeneous* if $\mathbf{g} = \mathbf{0}$ and *nonhomogeneous* otherwise.

An individual row may therefore look like

$$\begin{bmatrix} 1 & -2 & 1 \end{bmatrix} \begin{bmatrix} u_{x2} \\ u_{x4} \\ u_{x6} \end{bmatrix} = 0.25. \tag{3.4}$$

The compact row is useful for an engineer writing the relation. A finite element program, however, ultimately needs to know *which global DOF column* each coefficient belongs to. If the model contains twelve DOFs, the same row is stored conceptually as

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & -2 & 0 & 0 & 0 & 1 & 0 \end{bmatrix} \mathbf{u} = 0.25. \tag{3.5}$$

Most real programs do not store all those zeros. They store a short list such as

```
[(dof 3, 1.0), (dof 7, -2.0), (dof 11, 1.0)].
```

The dense matrix **A** is therefore a mathematical representation of a data structure that is usually sparse in implementation.

3.1.1 From Constraint Terms to a Matrix

The following function converts a list of constraint terms into the dense matrix form used throughout the derivations. The external DOF IDs are kept separate from Python's zero-based array indices. That distinction, trivial in a seven-DOF example, becomes essential in a general finite element program.

```python
import numpy as np


def dof_index(dof_ids, dof_id):
    for i, value in enumerate(dof_ids):
        if int(value) == int(dof_id):
            return i
    raise ValueError("unknown DOF id: %s" % dof_id)


def build_constraint_matrix(dof_ids, constraints):
    """
    constraints = [
        ([(dof_id, coefficient), ...], rhs),
        ...
    ]
    """
    A = np.zeros((len(constraints), len(dof_ids)), dtype=float)
    g = np.zeros(len(constraints), dtype=float)

    for row, (terms, rhs) in enumerate(constraints):
        for dof_id, coefficient in terms:
            col = dof_index(dof_ids, dof_id)
            A[row, col] += coefficient
```

```
        g[row] = rhs

    return A, g


dof_ids = np.arange(1, 8, dtype=int)
constraints = [
    ([(2, 1.0), (6, -1.0)], 0.0),        # u2 - u6 = 0
    ([(1, 1.0), (4, 4.0)], 0.2),         # u1 + 4 u4 = 0.2
]

A, g = build_constraint_matrix(dof_ids, constraints)
print(A)
print(g)
```

The two nested loops are the discrete counterpart of "placing coefficients in a matrix row." The same pattern appears repeatedly in finite element software: local information is mapped to global equation numbers and accumulated.

3.1.2 Constraint Scaling

The exact equations

$$\mathbf{au} = g \qquad \text{and} \qquad s\mathbf{au} = sg, \quad s \neq 0, \tag{3.6}$$

represent precisely the same admissible displacement set. Numerically they are not always equivalent. Row scaling affects pivot selection in elimination, changes the numerical value of a Lagrange multiplier and, unless the weight is adjusted, changes the stiffness of a penalty constraint by s^2.

This is especially important when one equation contains coefficients associated with quantities of different characteristic scale, for example translations and rotations. A robust implementation commonly normalizes constraint rows or uses explicit scaling information. The physical relation should therefore be kept separate from the arbitrary numerical scale used to write it.

Engineering note – A constraint equation has no privileged scale. Multiplying an exact MFC by a constant does not change its kinematics. If the numerical result changes appreciably after such a rescaling, the effect belongs to the enforcement algorithm, conditioning or tolerances rather than to the physical constraint itself.

3.2 A Common Example Structure

The one-dimensional bar chain in Figure 3.2 is used to compare the enforcement methods. It consists of six axial elements and seven nodes, with one displacement DOF u_i at each node.

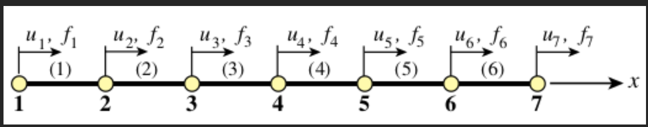


Figure 3.2: One-dimensional bar chain used for the constraint examples.

For equal element stiffness $k = EA/L$, assembly gives

$$\mathbf{K} = \begin{bmatrix} k & -k & 0 & 0 & 0 & 0 & 0 \\ -k & 2k & -k & 0 & 0 & 0 & 0 \\ 0 & -k & 2k & -k & 0 & 0 & 0 \\ 0 & 0 & -k & 2k & -k & 0 & 0 \\ 0 & 0 & 0 & -k & 2k & -k & 0 \\ 0 & 0 & 0 & 0 & -k & 2k & -k \\ 0 & 0 & 0 & 0 & 0 & -k & k \end{bmatrix}. \tag{3.7}$$

For the numerical examples we use $k = 100$, prescribe $u_1 = 0$, apply $f_7 = 100$, and additionally require

$$u_2 - u_6 = 0. \tag{3.8}$$

The exact constrained solution is

$$\mathbf{u}^T = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 & 2 \end{bmatrix}. \tag{3.9}$$

The following code shows how the stiffness matrix itself is assembled. The mathematical summation of element matrices becomes element loops, DOF lookup and entry-by-entry accumulation.

```python
def assemble_bar_chain(node_ids, element_nodes, element_stiffness):
    K = np.zeros((len(node_ids), len(node_ids)), dtype=float)

    for e, (node_a, node_b) in enumerate(element_nodes):
        ke = element_stiffness[e] * np.array([
            [ 1.0, -1.0],
            [-1.0,  1.0],
        ])

        dofs = [
            dof_index(node_ids, node_a),
            dof_index(node_ids, node_b),
        ]

        for i_local, i_global in enumerate(dofs):
            for j_local, j_global in enumerate(dofs):
                K[i_global, j_global] += ke[i_local, j_local]

    return K


node_ids = np.arange(1, 8, dtype=int)
element_nodes = [(1,2), (2,3), (3,4), (4,5), (5,6), (6,7)]
element_stiffness = np.full(6, 100.0)
K = assemble_bar_chain(node_ids, element_nodes, element_stiffness)

f = np.zeros(7)
f[dof_index(node_ids, 7)] = 100.0
```

This is a central computational-mechanics idea: the global matrix is not written down directly. It is *constructed* by repeated local contributions. Constraint matrices and penalty contributions follow the same pattern.

3.2.1 A Minimal Dense Solver and Prescribed DOFs

To keep later examples executable without hiding the linear algebra in a library, we use explicit Gaussian elimination with partial pivoting. The helper also accepts several right-hand sides, which is useful when constructing elimination transformations.

```python
def solve_dense(A, b, tol=1.0e-12):
    A = np.array(A, dtype=float, copy=True)
    b = np.array(b, dtype=float, copy=True)

    vector_rhs = (b.ndim == 1)
    if vector_rhs:
        b = b.reshape((-1, 1))

    n = A.shape[0]
    if A.shape[1] != n or b.shape[0] != n:
        raise ValueError("incompatible dense system")

    # Forward elimination.
    for pivot_col in range(n):
        pivot_row = pivot_col
        pivot_value = abs(A[pivot_row, pivot_col])

        for row in range(pivot_col + 1, n):
            candidate = abs(A[row, pivot_col])
            if candidate > pivot_value:
                pivot_row = row
                pivot_value = candidate

        if pivot_value < tol:
            raise ValueError("singular or rank-deficient system")

```

```python
        if pivot_row != pivot_col:
            A[[pivot_col, pivot_row], :] = A[[pivot_row, pivot_col], :]
            b[[pivot_col, pivot_row], :] = b[[pivot_row, pivot_col], :]

        for row in range(pivot_col + 1, n):
            factor = A[row, pivot_col] / A[pivot_col, pivot_col]
            A[row, pivot_col:] -= factor * A[pivot_col, pivot_col:]
            b[row, :] -= factor * b[pivot_col, :]

    # Back substitution.
    x = np.zeros_like(b)
    for row in range(n - 1, -1, -1):
        rhs = b[row, :].copy()
        for col in range(row + 1, n):
            rhs -= A[row, col] * x[col, :]
        x[row, :] = rhs / A[row, row]

    return x[:, 0] if vector_rhs else x


def eliminate_prescribed_dofs(K, f, dof_ids, prescribed):
    """prescribed = {dof_id: value, ...}"""
    free_ids = []
    fixed_ids = []

    for dof_id in dof_ids:
        if int(dof_id) in prescribed:
            fixed_ids.append(int(dof_id))
        else:
            free_ids.append(int(dof_id))

    Kff = np.zeros((len(free_ids), len(free_ids)))
    ff = np.zeros(len(free_ids))

    for i, dof_i in enumerate(free_ids):
        I = dof_index(dof_ids, dof_i)
        ff[i] = f[I]

        for fixed_id in fixed_ids:
            J = dof_index(dof_ids, fixed_id)
            ff[i] -= K[I, J] * prescribed[fixed_id]
```

```

        for j, dof_j in enumerate(free_ids):
            J = dof_index(dof_ids, dof_j)
            Kff[i, j] = K[I, J]

    return Kff, ff, np.array(free_ids, dtype=int)


Kf, ff, free_ids = eliminate_prescribed_dofs(
    K, f, node_ids, prescribed={1: 0.0}
)
```

A production code would of course use a sparse direct or iterative solver. The purpose here is to make the algorithmic content visible: pivot search, row operations and back substitution are what the compact notation $\mathbf{K}^{-1}\mathbf{f}$ conceals.

3.3 Three Ways to Impose the Same Constraint

Starting from the assembled equilibrium equations

$$\mathbf{K}\mathbf{u} = \mathbf{f}, \tag{3.10}$$

we now impose Eq. (3.8) by three methods:

1. **elimination**: remove dependent coordinates by an exact kinematic transformation;
2. **penalty augmentation**: add a large artificial stiffness that discourages violation;
3. **Lagrange multipliers**: add unknown reaction variables and enforce the constraint exactly in an enlarged system.

The engineering relation is identical in all three cases. What changes is the algebraic system submitted to the equation solver.

3.4 Elimination by Kinematic Transformation

3.4.1 Engineering Interpretation

If one constrained DOF can be expressed exactly in terms of others, it need not remain an independent unknown. For

$$u_2 - u_6 = 0, \tag{3.11}$$

we may choose u_6 as dependent and write simply

$$u_6 = u_2. \tag{3.12}$$

The choice of which DOF is eliminated is numerical bookkeeping; the physical constraint itself does not designate a preferred dependent coordinate.

After $u_1 = 0$ has been treated as the prescribed boundary condition from Chapter 2, the remaining DOFs are

$$\mathbf{u}_f = \begin{bmatrix} u_2 & u_3 & u_4 & u_5 & u_6 & u_7 \end{bmatrix}^T. \tag{3.13}$$

With u_6 eliminated, choose

$$\mathbf{q} = \begin{bmatrix} u_2 & u_3 & u_4 & u_5 & u_7 \end{bmatrix}^T \tag{3.14}$$

and construct

$$\mathbf{u}_f = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}}_{\mathbf{T}} \mathbf{q}. \tag{3.15}$$

Virtual work gives the reduced force vector $\hat{\mathbf{f}} = \mathbf{T}^T \mathbf{f}_f$, and substitution into equilibrium gives

$$\boxed{\hat{\mathbf{K}} = \mathbf{T}^T \mathbf{K}_f \mathbf{T}, \qquad \hat{\mathbf{f}} = \mathbf{T}^T \mathbf{f}_f}. \tag{3.16}$$

The reduced problem is

$$\hat{\mathbf{K}}\mathbf{q} = \hat{\mathbf{f}}. \tag{3.17}$$

For the numerical bar chain,

$$\hat{\mathbf{K}} = \begin{bmatrix} 400 & -100 & 0 & -100 & -100 \\ -100 & 200 & -100 & 0 & 0 \\ 0 & -100 & 200 & -100 & 0 \\ -100 & 0 & -100 & 200 & 0 \\ -100 & 0 & 0 & 0 & 100 \end{bmatrix}, \qquad \hat{\mathbf{f}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 100 \end{bmatrix}. \tag{3.18}$$

Solving returns

$$\mathbf{q}^T = \begin{bmatrix} 1 & 1 & 1 & 1 & 2 \end{bmatrix}, \tag{3.19}$$

and Eq. (3.15) recovers $u_6 = 1$.

3.4.2 General Linear Constraints

For a set of linear constraints

$$\mathbf{A}\mathbf{u} = \mathbf{g}, \tag{3.20}$$

partition the columns into independent coordinates $\mathbf{u}_i$ and selected dependent coordinates $\mathbf{u}_d$:

$$\mathbf{A}_i\mathbf{u}_i + \mathbf{A}_d\mathbf{u}_d = \mathbf{g}. \tag{3.21}$$

If $\mathbf{A}_d$ is square and nonsingular,

$$\mathbf{u}_d = -\mathbf{A}_d^{-1}\mathbf{A}_i\mathbf{u}_i + \mathbf{A}_d^{-1}\mathbf{g}. \tag{3.22}$$

After placing the independent identities and dependent coefficients back into the original DOF order, the full relation becomes

$$\boxed{\mathbf{u} = \mathbf{T}\mathbf{q} + \mathbf{u}_0}. \tag{3.23}$$

The nonhomogeneous vector $\mathbf{u}_0$ is important. Substitution into equilibrium and premultiplication by $\mathbf{T}^T$ gives

$$\boxed{\hat{\mathbf{K}} = \mathbf{T}^T\mathbf{K}\mathbf{T}, \qquad \hat{\mathbf{f}} = \mathbf{T}^T(\mathbf{f} - \mathbf{K}\mathbf{u}_0)}. \tag{3.24}$$

This is the same congruent transformation that later appears in model reduction and component-mode formulations.

3.4.3 Programming the Transformation

The following function constructs $\mathbf{T}$ and $\mathbf{u}_0$ from the constraint matrix and an explicit list of dependent DOF IDs. It uses the same dense solver defined earlier rather than calling a matrix inverse.

```
def build_elimination_transform(dof_ids, A, g, dependent_ids):
    dependent_cols = [dof_index(dof_ids, d) for d in dependent_ids]
    independent_ids = [
        int(d) for d in dof_ids if int(d) not in set(dependent_ids)
    ]
    independent_cols = [dof_index(dof_ids, d) for d in independent_ids]
```

```

    if A.shape[0] != len(dependent_cols):
        raise ValueError("one dependent DOF is required per constraint")

    Ad = np.zeros((A.shape[0], len(dependent_cols)))
    Ai = np.zeros((A.shape[0], len(independent_cols)))

    for row in range(A.shape[0]):
        for j, col in enumerate(dependent_cols):
            Ad[row, j] = A[row, col]
        for j, col in enumerate(independent_cols):
            Ai[row, j] = A[row, col]

    # Ad ud = g - Ai ui
    X = solve_dense(Ad, -Ai)
    y = solve_dense(Ad, g)

    T = np.zeros((len(dof_ids), len(independent_ids)))
    u0 = np.zeros(len(dof_ids))

    for j, col in enumerate(independent_cols):
        T[col, j] = 1.0

    for i, col in enumerate(dependent_cols):
        T[col, :] = X[i, :]
        u0[col] = y[i]

    return T, u0, np.array(independent_ids, dtype=int)


# Constraint on the free system: u2 - u6 = 0.
A, g = build_constraint_matrix(
    free_ids,
    [([(2, 1.0), (6, -1.0)], 0.0)],
)

T, u0, reduced_ids = build_elimination_transform(
    free_ids, A, g, dependent_ids=[6]
)

Kr = T.T @ Kf @ T
```

```
fr = T.T @ (ff - Kf @ u0)
q = solve_dense(Kr, fr)
uf = T @ q + u0

print(reduced_ids)    # [2 3 4 5 7]
print(uf)             # [1. 1. 1. 1. 1. 2.]
```

The code reveals several operations hidden by Eq. (3.22): columns must be classified, the $\mathbf{A}_d$ subsystem must be assembled in a consistent order, its rank must be adequate, and the resulting coefficients must be scattered back into the global transformation matrix.

3.4.4 Dependent-DOF Selection, Rank and Constraint Graphs

The elimination method is exact but requires a valid dependent set. With m independent constraints, m dependent scalar DOFs must be selected such that $\mathbf{A}_d$ is nonsingular. A poor automatic choice can create a singular subsystem even when the complete constraint set is perfectly valid.

In a large model this is not merely a matrix-algebra problem. Constraint rows and DOFs form a graph. Selecting dependent DOFs resembles matching constraints to unknowns while avoiding cycles and preserving numerical pivots. Production implementations therefore use graph processing, symbolic ordering and rank-revealing numerical checks rather than simply choosing the largest coefficient in each row independently.

Redundant constraints are another difficulty. If one row of $\mathbf{A}$ is a linear combination of others, the true number of restrictions is $\operatorname{rank}(\mathbf{A})$, not the number of input records. Eliminating one DOF for every input row would remove too many coordinates.

Programming insight – The matrix is the end product, not the input.
A user may enter constraints one record at a time. Before elimination, a solver must build a dependency graph, choose a consistent dependent set, detect cycles or redundant rows, determine equation ordering, and only then construct a usable transformation. The compact equation $\mathbf{u} = \mathbf{T}\mathbf{q}$ hides this preprocessing.

3.4.5 Model Reduction as the Same Algebra

A transformation need not originate from an exact constraint. One may deliberately choose a low-dimensional interpolation

$$\mathbf{u} \approx \mathbf{T}\mathbf{q} \tag{3.25}$$

and project the model through

$$\hat{\mathbf{K}} = \mathbf{T}^T\mathbf{K}\mathbf{T}, \qquad \hat{\mathbf{f}} = \mathbf{T}^T\mathbf{f}. \tag{3.26}$$

For the seven-node chain, retaining only the end motions and linearly interpolating the interior gives

$$\mathbf{T} = \begin{bmatrix} 1 & 0 \\ 5/6 & 1/6 \\ 4/6 & 2/6 \\ 3/6 & 3/6 \\ 2/6 & 4/6 \\ 1/6 & 5/6 \\ 0 & 1 \end{bmatrix}. \tag{3.27}$$

This reduces seven coordinates to two. Unlike exact MFC elimination, however, this interpolation is generally an approximation to structural deformation. The same matrix algebra therefore appears in two very different engineering uses: exact coordinate elimination and approximate model reduction. Chapter 10 develops static condensation and component-mode reduction in detail.

3.4.6 Assessment of Elimination

Elimination has several attractive properties:

- exact satisfaction of compatible linear constraints;
- fewer unknowns after transformation;
- preservation of symmetry through congruent transformation;
- direct applicability to stiffness, mass and damping matrices.

Its disadvantages are mostly algorithmic:

- dependent DOFs must be selected consistently;
- constraint chains can create dense transformation rows;
- sparse matrix fill-in can increase after $\mathbf{T}^T\mathbf{K}\mathbf{T}$;
- arbitrary nonlinear constraints require repeated, formulation-specific updates rather than one fixed transformation.

Thus the method is mathematically simple but can be demanding to implement as a general-purpose black-box facility.

3.5 Penalty Augmentation

3.5.1 Engineering Interpretation

For $u_2 = u_6$, imagine inserting an artificial spring between nodes 2 and 6 as shown in Figure 3.3. If the spring stiffness w is large compared with the physical structure, relative motion becomes small. Unlike elimination, however, the constraint is not exact for finite w.

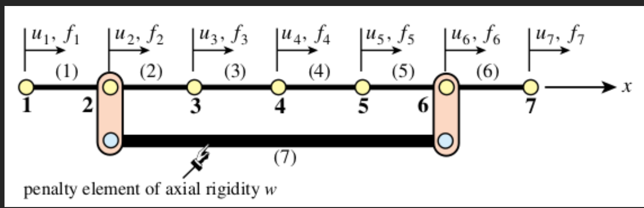


Figure 3.3: Penalty interpretation of $u_2 = u_6$: a fictitious high-stiffness element penalizes relative motion.

For the scalar constraint

$$\mathbf{au} = g, \tag{3.28}$$

the penalty energy is

$$P = \frac{1}{2} w (\mathbf{au} - g)^2. \tag{3.29}$$

Its contribution to equilibrium is

$$w \mathbf{a}^T \mathbf{au} = w \mathbf{a}^T g. \tag{3.30}$$

For several constraints,

$$\boxed{\left(\mathbf{K} + \mathbf{A}^T \mathbf{W} \mathbf{A}\right) \mathbf{u} = \mathbf{f} + \mathbf{A}^T \mathbf{W} \mathbf{g}}, \tag{3.31}$$

where $\mathbf{W}$ is commonly diagonal.

For $u_2 - u_6 = 0$, the local penalty matrix is simply

$$w \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \tag{3.32}$$

which is assembled into global positions 2 and 6 exactly like an ordinary finite element stiffness matrix.

3.5.2 Penalty Assembly as Loops

The compact product $\mathbf{A}^T\mathbf{W}\mathbf{A}$ can obscure how easy the method is to implement. Each constraint row acts like a small numerical element. The following routine assembles its stiffness and force contributions one coefficient at a time.

```python
def apply_penalty(K, f, A, g, weights):
    Kp = np.array(K, dtype=float, copy=True)
    fp = np.array(f, dtype=float, copy=True)

    for p in range(A.shape[0]):
        w = float(weights[p])

        for i in range(A.shape[1]):
            ai = A[p, i]
            if ai == 0.0:
                continue

            fp[i] += w * ai * g[p]

            for j in range(A.shape[1]):
                aj = A[p, j]
                if aj != 0.0:
                    Kp[i, j] += w * ai * aj

    return Kp, fp


A, g = build_constraint_matrix(
    free_ids,
    [([(2, 1.0), (6, -1.0)], 0.0)],
)

for w in (1.0e2, 1.0e4, 1.0e6):
    Kp, fp = apply_penalty(Kf, ff, A, g, weights=[w])
    uf = solve_dense(Kp, fp)
    violation = (A @ uf - g)[0]
    print(w, violation, uf)

```

```
# 1e2 : violation = -8.000000e-01
# 1e4 : violation = -9.975062e-03
# 1e6 : violation = -9.999750e-05
```

The code is almost identical to ordinary element assembly. This is one reason penalty methods are attractive in finite element implementations: constraints can be inserted into an existing assembly framework with little restructuring.

3.5.3 Weight Selection and Conditioning

For large w, the violation

$$\mathbf{e}_c = \mathbf{A}\mathbf{u} - \mathbf{g} \tag{3.33}$$

generally decreases approximately in proportion to $1/w$. At the same time, $\mathbf{A}^T\mathbf{W}\mathbf{A}$ introduces stiffness scales much larger than those of the physical model. The augmented matrix therefore becomes increasingly ill-conditioned.

This is the characteristic penalty trade-off:

$$\text{large } w \Rightarrow \text{small constraint error but poorer conditioning.}$$

A useful order-of-magnitude guideline for double precision is to place the penalty scale several orders above the relevant physical stiffness rather than trying to approximate mathematical infinity. The often quoted "square-root rule" balances expected floating-point error against constraint violation by placing the penalty roughly halfway, on a logarithmic scale, between the physical matrix magnitude and the reciprocal machine precision. It is a heuristic, not a universal constant.

More important than any fixed number is dimensional consistency. If $\mathbf{au} - g$ has units of displacement, w has units of stiffness. If the constraint function is dimensionless or combines scaled quantities, the units and useful magnitude of w change accordingly.

Common mistake – Treating the penalty weight as dimensionless. The numerical value of a suitable penalty depends on how the constraint row is scaled and on the units of the underlying quantities. A penalty value copied from another model can be meaningless after changing from metres to millimetres or after multiplying the constraint equation by a constant.

3.5.4 Redundant Constraints under Penalty Enforcement

Penalty enforcement does not become algebraically singular merely because the same constraint is entered twice. If a row $\mathbf{a}$ with weight w is duplicated, its artificial stiffness contribution simply doubles:

$$w\mathbf{a}^T\mathbf{a} + w\mathbf{a}^T\mathbf{a} = 2w\mathbf{a}^T\mathbf{a}. \tag{3.34}$$

This robustness can also hide modelling errors. A redundant record changes the effective penalty stiffness and therefore the solution error and conditioning. The absence of a singularity does not mean the input is conceptually clean.

3.5.5 Assessment of the Penalty Method

Advantages are straightforward implementation, unchanged DOF count, preservation of positive definiteness when the original stiffness is positive definite, and natural extension to many nonlinear constraints.

The price is approximate enforcement and weight sensitivity. Excessive penalty stiffness can degrade iterative convergence, contaminate high-frequency dynamic behaviour and amplify round-off effects. Penalty methods are therefore best understood as controlled approximations rather than as “nearly exact” rigid links.

3.6 Lagrange Multiplier Adjunction

3.6.1 Engineering Interpretation

An exact constraint can instead be maintained by whatever reaction force is necessary to prevent forbidden motion. For $u_2 - u_6 = 0$, introduce an unknown scalar λ. With the row

$$\mathbf{a} = \begin{bmatrix} 1 & 0 & 0 & 0 & -1 & 0 \end{bmatrix} \tag{3.35}$$

on the free-DOF system, the equilibrium equations are augmented by the constraint force pattern $\mathbf{A}^T\boldsymbol{\lambda}$ and by the constraint equations themselves:

$$\boxed{\begin{bmatrix} \mathbf{K} & \mathbf{A}^T \\ \mathbf{A} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ \boldsymbol{\lambda} \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ \mathbf{g} \end{bmatrix}}. \tag{3.36}$$

The matrix is symmetric but indefinite. The number of unknowns increases by the number of multiplier equations.

With the sign convention in Eq. (3.36), structural equilibrium may be written

$$\mathbf{K}\mathbf{u} = \mathbf{f} - \mathbf{A}^T\boldsymbol{\lambda}. \tag{3.37}$$

Thus the physical constraint-force vector acting on the structure is

$$\boxed{\mathbf{f}_c = -\mathbf{A}^T\boldsymbol{\lambda}}. \tag{3.38}$$

The sign of an individual multiplier is not absolute: reversing a constraint row reverses the corresponding λ. The physical vector $-\mathbf{A}^T\boldsymbol{\lambda}$ remains unchanged.

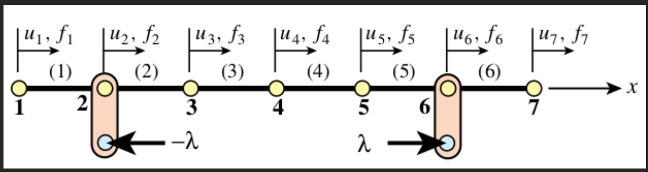


Figure 3.4: Physical interpretation of a Lagrange multiplier for $u_2 = u_6$: the multiplier generates the equal-and-opposite reaction pattern required to enforce the relation exactly.

3.6.2 Programming the Bordered System

The implementation is direct: allocate a matrix with m extra rows and columns, copy $\mathbf{K}$, then place $\mathbf{A}$ and $\mathbf{A}^T$ in the borders.

```
def build_lagrange_system(K, f, A, g):
    n = K.shape[0]
    m = A.shape[0]

    B = np.zeros((n + m, n + m), dtype=float)
    rhs = np.zeros(n + m, dtype=float)

    B[:n, :n] = K
    rhs[:n] = f

    for p in range(m):
        rhs[n + p] = g[p]
        for i in range(n):
            B[i, n + p] = A[p, i]
            B[n + p, i] = A[p, i]

    return B, rhs


A, g = build_constraint_matrix(
    free_ids,
    [([(2, 1.0), (6, -1.0)], 0.0)],
)
```

```

B, rhs = build_lagrange_system(Kf, ff, A, g)
solution = solve_dense(B, rhs)

uf = solution[:len(free_ids)]
lambda_ = solution[len(free_ids):]
constraint_force = -(A.T @ lambda_)

print(uf)                 # [1. 1. 1. 1. 1. 2.]
print(lambda_)            # [-100.]
print(constraint_force)   # [ 100.   0.   0.   0. -100.   0.]
```

The scalar multiplier is -100 for the chosen row orientation. What carries physical meaning is the nodal force pair: $+100$ at DOF 2 and -100 at DOF 6.

3.6.3 Variational Interpretation

For the unconstrained potential energy

$$\Pi(\mathbf{u}) = \frac{1}{2}\mathbf{u}^T\mathbf{K}\mathbf{u} - \mathbf{u}^T\mathbf{f}, \tag{3.39}$$

form the Lagrangian

$$\mathcal{L}(\mathbf{u}, \boldsymbol{\lambda}) = \Pi(\mathbf{u}) + \boldsymbol{\lambda}^T(\mathbf{A}\mathbf{u} - \mathbf{g}). \tag{3.40}$$

Stationarity with respect to $\mathbf{u}$ and $\boldsymbol{\lambda}$ gives Eq. (3.36). This derivation explains why the constraint coefficients appear twice: once as equilibrium force directions and once as kinematic equations.

3.6.4 Scaling and the Meaning of the Multiplier

Suppose one row is multiplied by s. To describe the same physical constraint, $\mathbf{a}$ and g both become $s\mathbf{a}$ and sg. The multiplier solving the new system becomes approximately λ/s, so

$$-(s\mathbf{a})^T(\lambda/s) = -\mathbf{a}^T\lambda. \tag{3.41}$$

The physical force is invariant even though the reported multiplier changes. This is important when interpreting solver output: a multiplier is conjugate to the particular mathematical form of the constraint, not automatically a force in newtons. Only after applying the constraint coefficients and their units does one obtain the physical nodal reaction vector.

3.6.5 Redundancy and Indefiniteness

If two multiplier rows impose the same exact relation, the structural configuration may still be unique but the individual multipliers are not. There are infinitely many ways to split the same total constraint force between two identical equations. The bordered matrix therefore becomes singular.

More generally, the zero block in Eq. (3.36) means the augmented matrix is indefinite even for a positive-definite structural $\mathbf{K}$. Cholesky-type solvers that rely on positive definiteness cannot be used unchanged. Production codes employ symmetric-indefinite factorization, constraint condensation or specialized iterative methods.

3.6.6 Assessment of the Lagrange Multiplier Method

The method enforces compatible constraints exactly, needs no penalty weight and returns constraint reactions naturally. It is very general and extends well to nonlinear formulations.

Its disadvantages are additional unknowns, an indefinite augmented matrix and sensitivity to dependent constraint rows. In dynamic eigenproblems the multiplier coordinates also have no physical inertia, which creates a singular mass block; this issue is discussed later in this chapter and again from the engineering coupling viewpoint in Chapter 5.

3.7 Augmented Lagrangian Method

Penalty and multiplier methods can be combined so that a moderate penalty stiffness supplies numerical regularization while iterative multiplier updates remove the remaining constraint error.

Using

$$\Pi_a = \Pi + \boldsymbol{\lambda}^T(\mathbf{Au} - \mathbf{g}) + \frac{1}{2}(\mathbf{Au} - \mathbf{g})^T\mathbf{W}(\mathbf{Au} - \mathbf{g}), \tag{3.42}$$

the displacement solve at iteration k is

$$\left(\mathbf{K} + \mathbf{A}^T\mathbf{WA}\right)\mathbf{u}^{(k)} = \mathbf{f} + \mathbf{A}^T\mathbf{Wg} - \mathbf{A}^T\boldsymbol{\lambda}^{(k)}, \tag{3.43}$$

followed by

$$\boxed{\boldsymbol{\lambda}^{(k+1)} = \boldsymbol{\lambda}^{(k)} + \mathbf{W}\left(\mathbf{Au}^{(k)} - \mathbf{g}\right)}. \tag{3.44}$$

The penalty matrix remains fixed while the multiplier accumulates the reaction needed to drive the residual to zero.

```python
def augmented_lagrangian(K, f, A, g, weights, iterations):
    weights = np.array(weights, dtype=float)
    lambda_ = np.zeros(A.shape[0], dtype=float)

    # The augmented stiffness is constant for linear constraints.
    Ka, dummy = apply_penalty(
        K, np.zeros_like(f), A, np.zeros_like(g), weights
    )

    history = []

    for k in range(iterations):
```

```
        rhs = np.array(f, copy=True)

        for p in range(A.shape[0]):
            for i in range(A.shape[1]):
                rhs[i] += A[p, i] * (
                    weights[p] * g[p] - lambda_[p]
                )

        u = solve_dense(Ka, rhs)
        residual = A @ u - g

        for p in range(A.shape[0]):
            lambda_[p] += weights[p] * residual[p]

        history.append((k, residual.copy(), lambda_.copy()))

    return u, lambda_, history


u, lambda_, history = augmented_lagrangian(
    Kf, ff, A, g, weights=[100.0], iterations=6
)

for k, residual, multiplier in history:
    print(k, residual[0], multiplier[0])

# 0  -0.800000  -80.0000
# 1  -0.160000  -96.0000
# 2  -0.032000  -99.2000
# 3  -0.006400  -99.8400
# 4  -0.001280  -99.9680
# 5  -0.000256  -99.9936
```

A penalty weight of only 100 gave a violation of 0.8 in the one-pass penalty solution. Repeated multiplier updates reduce the residual by orders of magnitude without increasing the penalty stiffness. This is why augmented Lagrangian ideas are so attractive in nonlinear contact and constraint algorithms.

3.8 Nonlinear Constraints: From Equations to Iterations

A nonlinear relation

$$\mathbf{c}(\mathbf{u}) = \mathbf{0} \tag{3.45}$$

cannot in general be represented by one fixed matrix $\mathbf{A}$. At the current iterate $\mathbf{u}^{(k)}$, define the Jacobian

$$\mathbf{G}^{(k)} = \left.\frac{\partial \mathbf{c}}{\partial \mathbf{u}}\right|_{\mathbf{u}^{(k)}}. \tag{3.46}$$

The first-order correction must satisfy

$$\mathbf{G}^{(k)} \Delta\mathbf{u} = -\mathbf{c}(\mathbf{u}^{(k)}). \tag{3.47}$$

The mathematical statement has now become an algorithm:

1. evaluate the current geometry and constraint residual;
2. build the Jacobian coefficients $\mathbf{G}^{(k)}$;
3. assemble the chosen constraint contribution into the tangent system;
4. solve for $\Delta\mathbf{u}$ and possibly $\Delta\boldsymbol{\lambda}$;
5. update the configuration and repeat.

For exact multiplier enforcement, the Newton system has the generic form

$$\begin{bmatrix} \mathbf{K}_t + \mathbf{K}_c & \mathbf{G}^T \\ \mathbf{G} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \Delta\mathbf{u} \\ \Delta\boldsymbol{\lambda} \end{bmatrix} = \begin{bmatrix} \mathbf{r} \\ -\mathbf{c} \end{bmatrix}. \tag{3.48}$$

Here $\mathbf{K}_c$ denotes tangent terms arising from the dependence of the constraint-force directions on configuration. Omitting such terms may still produce a useful quasi-Newton iteration, but changes convergence behaviour.

A fixed-distance constraint from Chapter 5 is a good example. The exact geometry is nonlinear even though its tangent at any one configuration is a linear relation. A program must therefore repeatedly recompute the direction vector; merely assembling the initial tangent once does not create a finite-rotation constraint.

Programming insight – Nonlinearity means rebuilding data.
In a linear MFC, coefficients can be generated once during preprocessing. In a geometrically nonlinear constraint, coefficients depend on the current nodal coordinates. They belong to the iteration loop. The difference is not merely "linear equation versus nonlinear equation"; it changes when and how the constraint data are generated, assembled and updated.

3.9 Constraint Methods in Dynamics and Eigenvalue Extraction

For linear dynamics,

$$\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{f}, \tag{3.49}$$

the enforcement choice affects more than the static stiffness equations.

3.9.1 Elimination

For a constant homogeneous transformation $\mathbf{u} = \mathbf{T}\mathbf{q}$,

$$\boxed{\hat{\mathbf{M}} = \mathbf{T}^T\mathbf{M}\mathbf{T}, \quad \hat{\mathbf{C}} = \mathbf{T}^T\mathbf{C}\mathbf{T}, \quad \hat{\mathbf{K}} = \mathbf{T}^T\mathbf{K}\mathbf{T}}. \tag{3.50}$$

The reduced eigenproblem is therefore an ordinary one:

$$\left(\hat{\mathbf{K}} - \omega^2 \hat{\mathbf{M}}\right) \boldsymbol{\phi} = \mathbf{0}. \tag{3.51}$$

This is why fixed linear couplings are naturally compatible with conventional modal extraction when they are eliminated or transformed consistently.

3.9.2 Penalty Enforcement

Penalty enforcement changes stiffness but normally not physical mass:

$$\mathbf{K}_p = \mathbf{K} + \mathbf{A}^T \mathbf{W} \mathbf{A}. \tag{3.52}$$

Large $\mathbf{W}$ creates very stiff artificial modes associated with residual constraint deformation. These modes can appear far above the physical frequency range, but they still affect conditioning and may interfere with eigenvalue or iterative algorithms.

3.9.3 Lagrange Multipliers

A multiplier dynamic system has the descriptor form

$$\begin{bmatrix} \mathbf{M} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \ddot{\mathbf{u}} \\ \ddot{\boldsymbol{\lambda}} \end{bmatrix} + \begin{bmatrix} \mathbf{K} & \mathbf{A}^T \\ \mathbf{A} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ \boldsymbol{\lambda} \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ \mathbf{g} \end{bmatrix}. \tag{3.53}$$

The multiplier coordinates carry no inertia, so the augmented mass matrix is singular. This does not make the formulation physically invalid, but a standard eigensolver that assumes a nonsingular mass matrix cannot necessarily process it directly. Solvers may eliminate the constraints internally, use a descriptor formulation, or provide a specialized eigenalgorithm.

These numerical consequences explain why two solver features that impose the same engineering relation may have different admissibility in a particular modal analysis. Chapter 5 discusses this from the viewpoint of RBE, remote and joint definitions.

3.10 Comparison of the Enforcement Methods

Property	**Elimination**	**Penalty**	**Lagrange multiplier**
Constraint satisfaction	Exact for compatible linear relations	Approximate for finite weight	Exact for compatible relations
Unknown count	Reduced	Unchanged	Increased by multiplier DOFs
Matrix character	Congruently transformed; symmetry retained	Added positive semidefinite stiffness	Symmetric saddle-point / indefinite
Primary numerical issue	Dependent-DOF selection, fill-in, rank	Weight choice and conditioning	Constraint dependence and indefinite solve
Constraint reactions	Recovered by postprocessing	Approximate from penalty force	Directly available through multipliers
Nonlinear extension	Formulation dependent; often awkward	Natural	Natural
Conventional modes	Natural after transforming $\mathbf{M}$ and $\mathbf{K}$	Artificial stiff modes may appear	Requires handling singular multiplier mass block
Programming viewpoint	Preprocess dependency graph and build $\mathbf{T}$	Assemble constraint rows like elements	Border global system with $\mathbf{A}$ and $\mathbf{A}^T$

No method is universally superior. The best choice depends on the engineering constraint, desired accuracy, matrix structure, nonlinear behaviour, solver technology and the outputs of interest.

3.11 Implementation Lessons

Several broad lessons recur in real finite element programs.

3.11.1 DOF Identity Is Data

The coefficient A_{ij} has meaning only because column j corresponds to a specific physical DOF. Mesh numbering, active element DOFs, coordinate systems, constraint equations and solver equation numbers therefore need explicit maps. Many implementation bugs that appear to be "matrix errors" are actually index mapping errors.

3.11.2 Sparse Assembly Is Accumulation

Most finite element matrices are sparse. A production solver does not allocate dense zero-filled arrays and then multiply them. It gathers nonzero terms, translates local to global indices, adds repeated contributions and stores only the resulting graph. The dense examples in this chapter expose the same algorithm without the storage machinery.

3.11.3 Rank Is an Engineering and Numerical Property

A redundant constraint may originate from duplicated input, overlapping coupling definitions or a genuine physical mechanism description. Mathematically the issue is rank. Numerically its symptom depends on enforcement: failed dependent selection, a singular multiplier block, or merely a stronger penalty. The constraint graph should therefore be inspected before solver tolerances are changed.

3.11.4 Exact Kinematics Do Not Determine Every Reaction Split

Several exact constraints may determine motion uniquely while leaving individual reaction contributions nonunique. If engineering load sharing between bolts, bearings or mounts is required, sufficient physical compliance must exist in the model to determine that split. This point becomes especially important in the coupling examples of Chapter 5.

3.11.5 The Solver Is Part of the Formulation

The equations shown in a textbook are only one stage of computation. The final behaviour also depends on preprocessing, scaling, storage, pivoting, nonlinear updates, convergence tests and the linear/eigen solver used. Understanding the matrix formulation allows an engineer to reason about these algorithmic choices instead of treating them as inaccessible solver internals.

3.12 Summary

A multifreedom constraint defines an admissible relation between discrete nodal DOFs. For linear bilateral constraints this relation is

$$\mathbf{A}\mathbf{u} = \mathbf{g}. \tag{3.54}$$

The same engineering relation can be imposed numerically in fundamentally different ways:

- **elimination** constructs a transformation $\mathbf{u} = \mathbf{T}\mathbf{q}+\mathbf{u}_0$ and solves a smaller system;
- **penalty augmentation** adds $\mathbf{A}^T\mathbf{W}\mathbf{A}$ and accepts a controlled constraint violation;
- **Lagrange multipliers** border the system with $\mathbf{A}$ and $\mathbf{A}^T$ and solve directly for exact constraint reactions;
- **augmented Lagrangian methods** combine moderate penalty stiffness with iterative multiplier updates.

The matrix notation explains the numerical properties, but implementation requires another level of reasoning: DOF maps, loops, assembly, rank detection, dependency graphs, matrix storage and solution algorithms. The Python examples make those operations explicit while remaining close to the governing equations.

The present chapter therefore answers primarily *how a constraint is imposed.* Chapter 5 asks the complementary engineering question: *what kinematic relation should be imposed?* Chapter 4 then develops the unilateral case, where the set of active constraints itself becomes part of the solution.

Bibliography

Felippa – *Introduction to Finite Element Methods*, chapter on multifreedom constraints.

Zienkiewicz and Taylor – *The Finite Element Method.*

Chapter 4

Contact Formulations

Contact describes the interaction of two bodies whose surfaces may touch, transmit forces and subsequently separate or slide. At first sight this may look like another application of the **multifreedom constraints** treated in the previous chapter. There is, however, one fundamental difference.

An ordinary MFC is an *equality constraint*. Once imposed, it remains active throughout the analysis. Contact in the normal direction is generally a *unilateral constraint*: the surfaces must not penetrate each other, but they are allowed to separate. Consequently, the constraint is active only while the surfaces are closed.

The simplest scalar distinction is

$$\underbrace{a(\mathbf{u}) = 0}_{\text{equality constraint}} \qquad \text{versus} \qquad \underbrace{g_n(\mathbf{u}) \geq 0}_{\text{unilateral contact constraint}} \quad . \tag{4.1}$$

Here g_n is the *normal gap*. In this chapter the sign convention is:

$$g_n > 0 \quad \text{open gap}, \qquad g_n = 0 \quad \text{closed contact}, \qquad g_n < 0 \quad \text{penetration}. \tag{4.2}$$

A contact definition is therefore not merely a special stiffness placed between two nodes. A complete definition has to specify at least four things:

1. how the geometrical gap and relative sliding are measured,
2. which normal and tangential motions are admissible,
3. how the corresponding constraints are imposed numerically,
4. how points on one discretized surface interact with the other surface.

Terms such as *frictionless*, *rough*, *frictional*, *no separation* and *bonded* primarily describe the admissible relative motion and the tractions that may be transmitted. They should not be confused with *penalty*, *Lagrange multiplier* and *augmented Lagrangian*, which describe numerical enforcement methods. Nor should they be confused with *node-to-surface* or *surface-to-surface*, which describe contact discretizations.

4.1 A One-Dimensional Contact Example

The basic behaviour of normal contact can be understood without surface geometry. Consider a node attached to a spring of stiffness k. A positive external force f moves the node to the right. A rigid wall is located at $u = g_0$, as sketched conceptually by

$$\text{spring } k \quad \longrightarrow \quad u \qquad | \qquad \text{rigid wall at } g_0. \tag{4.3}$$

The distance from the node to the wall is

$$g_n = g_0 - u. \tag{4.4}$$

Thus $u < g_0$ means that the node and wall are separated, $u = g_0$ means that they touch, and $u > g_0$ would represent an inadmissible penetration.

If the wall is absent, equilibrium gives

$$ku = f, \tag{4.5}$$

and therefore $u = f/k$. Two cases are possible.

1. If $f/k < g_0$, the unconstrained displacement does not reach the wall. The contact remains open and the wall exerts no force.
2. If $f/k \geq g_0$, the unconstrained displacement would cross the wall. Contact must become active, the displacement is limited to $u = g_0$, and the wall supplies a reaction force.

Let the compressive contact reaction be denoted by $p_n \geq 0$. It acts in the direction opposite to increasing u. Equilibrium of the node is

$$ku + p_n = f. \tag{4.6}$$

The exact solution is therefore

$$\begin{aligned} u &= \frac{f}{k}, \quad p_n = 0, && \text{if } f < kg_0, \\ u &= g_0, \quad p_n = f - kg_0, && \text{if } f \geq kg_0. \end{aligned} \tag{4.7}$$

This small example already contains the central difficulty of contact: before solving the equilibrium equations, it is generally not known whether the contact reaction is zero or whether the gap is zero. The set of active contact constraints is part of the solution.

4.2 The Unilateral Normal Constraint

For a frictionless normal contact pair the admissible states are described by three conditions:

$$\boxed{g_n \geq 0, \qquad p_n \geq 0, \qquad p_n g_n = 0.} \tag{4.8}$$

These are the *Karush–Kuhn–Tucker* conditions for unilateral contact. Their meaning is more important than their name.

1. $g_n \geq 0$ states that penetration is not admissible.
2. $p_n \geq 0$ states that ordinary contact can push but cannot pull. A tensile normal traction would require an adhesive or bonded interface.
3. $p_n g_n = 0$ is the *complementarity condition*. Either the gap is open and the pressure is zero, or the pressure is positive and the gap is closed. They cannot both be positive.

The two admissible states may be written explicitly as

$$\begin{aligned} \text{open:} \quad & g_n > 0, \quad p_n = 0, \\ \text{closed:} \quad & g_n = 0, \quad p_n \geq 0. \end{aligned} \tag{4.9}$$

At the instant of first touch both $g_n = 0$ and $p_n = 0$ are possible. This transition point separates the open and closed branches.

Equation (4.8) also explains why replacing contact by a permanent equality $g_n = 0$ is generally incorrect. Such an equality would prevent separation and could transmit tension. It would describe a *no-separation* or tied normal interface, not ordinary contact.

4.2.1 Contact Status as an Active Set

Suppose that a finite element model contains many possible contact locations. At a given nonlinear iteration the solver classifies each location as active or inactive:

$$\begin{aligned} \mathcal{A} &= \{i \mid \text{contact constraint } i \text{ is active}\}, \\ \mathcal{I} &= \{i \mid \text{contact constraint } i \text{ is inactive}\}. \end{aligned} \tag{4.10}$$

For an inactive constraint, $p_{n,i} = 0$. For an active constraint, the solver attempts to enforce $g_{n,i} = 0$, either exactly or approximately. After the displacement update, the status is checked again. A contact may close, open, stick or slip, so the active set changes during the nonlinear solution.

This status update is one reason why a structurally linear model may still require a nonlinear contact analysis. Even if the material law and the small-displacement stiffness matrix are linear, the switch between $p_n = 0$ and $g_n = 0$ is not.

4.3 Contact Kinematics

The one-dimensional gap (4.4) is replaced in a continuum model by the distance between two candidate surfaces. It is common to label one side *slave* and the other side *master*. These labels describe the search and projection procedure; they do not imply that one physical body is more important than the other.

Let a point on the slave surface have reference position $\mathbf{X}_s$ and current position

$$\mathbf{x}_s = \mathbf{X}_s + \mathbf{u}_s. \tag{4.11}$$

Let the master surface be parametrized by surface coordinates $\boldsymbol{\xi}$:

$$\mathbf{x}_m(\boldsymbol{\xi}) = \mathbf{X}_m(\boldsymbol{\xi}) + \mathbf{u}_m(\boldsymbol{\xi}). \tag{4.12}$$

A corresponding master point $\mathbf{x}_m(\bar{\boldsymbol{\xi}})$ is found, most often by a closest-point projection. If $\mathbf{n}_m$ is the outward unit normal of the master surface at that point, the normal gap is

$$g_n = \left(\mathbf{x}_s - \mathbf{x}_m(\bar{\boldsymbol{\xi}})\right)^T \mathbf{n}_m. \tag{4.13}$$

The orientation of $\mathbf{n}_m$ must be chosen consistently with the adopted sign convention. Reversing the normal reverses the sign of g_n and of the normal traction definition.

4.3.1 Closest-Point Projection

For a smooth master surface, the closest point satisfies orthogonality of the slave-to-master vector to both surface tangent directions:

$$\left(\mathbf{x}_s - \mathbf{x}_m(\bar{\boldsymbol{\xi}})\right)^T \frac{\partial \mathbf{x}_m}{\partial \xi_\alpha} = 0, \qquad \alpha = 1, 2. \tag{4.14}$$

Equation (4.14) is itself nonlinear when the surface is curved or undergoing finite motion. In practice the projection has to determine both the contacted element or segment and the local coordinates inside that element.

If the projection falls outside the candidate face, the solver may test an adjacent face, edge or vertex. Consequently, contact search is a geometrical problem in addition to the mechanical equilibrium problem.

4.3.2 Normal and Tangential Decomposition

The normal projector and the tangential projector are

$$\mathbf{P}_n = \mathbf{n}\mathbf{n}^T, \qquad \mathbf{P}_t = \mathbf{I} - \mathbf{n}\mathbf{n}^T. \tag{4.15}$$

Any relative displacement increment $\Delta\mathbf{u}_{rel}$ may be decomposed as

$$\Delta \mathbf{u}_{rel} = \underbrace{\mathbf{P}_n \Delta \mathbf{u}_{rel}}_{\text{normal part}} + \underbrace{\mathbf{P}_t \Delta \mathbf{u}_{rel}}_{\text{tangential part}} . \tag{4.16}$$

The incremental tangential slip is commonly written

$$\Delta \mathbf{g}_t = \mathbf{P}_t \left(\Delta \mathbf{u}_s - \Delta \mathbf{u}_m \right). \tag{4.17}$$

For finite sliding, the tangential basis rotates and the projected master point moves across element boundaries. Therefore $\mathbf{g}_t$ is normally updated incrementally rather than treated as a single difference between initial and current Cartesian coordinates.

4.3.3 Small and Finite Sliding

A *small-sliding* formulation assumes that the contacted point remains near its initial projection. The contact normal and the interpolation of the opposite surface can then be updated only mildly or even kept fixed within an increment. This is efficient when relative tangential motion is small compared with the local element dimensions.

A *finite-sliding* formulation allows the slave point to travel over many master faces. The closest point, surface normal, tangent basis and contact interpolation must be updated as the bodies move. This is more general but also more expensive and more sensitive to surface discretization.

Small versus finite sliding is a kinematic choice. It is independent of whether the interface is frictionless or frictional and independent of whether a penalty or multiplier method is used.

4.4 Normal Constraint Enforcement

The conditions (4.8) state the exact mechanical problem, but they do not by themselves prescribe how the algebraic equations should be formed. The same three families introduced for MFCs appear again:

1. penalty enforcement,
2. Lagrange multiplier enforcement,
3. augmented Lagrangian enforcement.

The important difference from ordinary MFCs is that these methods are applied only to the currently active contact constraints.

4.4.1 The Penalty Method

Define the positive-part or Macaulay bracket by

$$\langle x \rangle = \max(x, 0). \tag{4.18}$$

With the present gap convention, a simple normal penalty law is

$$\boxed{p_n = \epsilon_n \langle -g_n \rangle,} \tag{4.19}$$

where $\epsilon_n > 0$ is the normal penalty stiffness. If the gap is open, $g_n > 0$, then $p_n = 0$. If penetration occurs, $g_n < 0$, then the contact pressure grows in proportion to the penetration $-g_n$.

The corresponding penalty potential per contact point is

$$\Pi_c = \frac{1}{2}\epsilon_n \langle -g_n \rangle^2. \tag{4.20}$$

The contact force follows from variation of this potential with respect to the nodal displacements. If $\mathbf{B}_n = \partial g_n / \partial \mathbf{u}$ denotes the gap-displacement matrix, the contact contribution to the residual has the form

$$\mathbf{r}_c = -\mathbf{B}_n^T p_n, \tag{4.21}$$

up to the sign convention used for the global residual.

Penalty Solution of the One-Dimensional Example

For the node-wall example, penetration is $u - g_0$. Once contact is active, choose

$$p_n = \epsilon_n (u - g_0). \tag{4.22}$$

Substitution into equilibrium (4.6) gives

$$ku + \epsilon_n (u - g_0) = f, \tag{4.23}$$

and therefore

$$u = \frac{f + \epsilon_n g_0}{k + \epsilon_n}. \tag{4.24}$$

The penetration error is

$$u - g_0 = \frac{f - k g_0}{k + \epsilon_n}. \tag{4.25}$$

As $\epsilon_n \to \infty$, the penetration tends to zero and $u \to g_0$. For any finite ϵ_n, however, some penetration is required to generate the contact reaction. This is not a geometrical prediction of the physical model. It is the constraint violation introduced by penalty enforcement.

The tradeoff is the same as for penalty MFCs. A small ϵ_n gives a soft interface and excessive penetration. A very large ϵ_n improves constraint accuracy but increases the condition number of the tangent stiffness matrix and can impair Newton convergence.

4.4.2 Choosing the Normal Penalty Stiffness

There is no universal penalty stiffness that is suitable for every contact pair. The value has to be large relative to the normal compliance of the adjacent finite elements, but not so large that the contact terms dominate the structural stiffness by many unnecessary orders of magnitude.

A useful dimensional estimate is

$$\epsilon_n \sim \alpha \frac{E_{eff} A_c}{h_c}, \tag{4.26}$$

for a discrete contact point carrying representative area A_c, or

$$\bar{\epsilon}_n \sim \alpha \frac{E_{eff}}{h_c} \tag{4.27}$$

for a traction-based surface formulation. Here E_{eff} is an effective normal modulus of the contacting materials, h_c is a representative element length in the contact-normal direction, and α is a dimensionless scaling factor.

These expressions are scaling arguments, not exact laws. Element type, integration scheme, shell thickness, material incompressibility and the chosen surface discretization all affect the useful range. The computed penetration and nonlinear convergence should therefore be checked rather than relying on a large default number alone.

4.4.3 The Lagrange Multiplier Method

A Lagrange multiplier can enforce the active normal constraint exactly. For a known active set, collect the active gaps in vector $\mathbf{g}_n$ and linearize them as

$$\Delta \mathbf{g}_n = \mathbf{A}_n \Delta \mathbf{u}, \tag{4.28}$$

where $\mathbf{A}_n$ is the active normal constraint matrix. The Newton correction has the bordered form

$$\begin{bmatrix} \mathbf{K}_T & \mathbf{A}_n^T \\ \mathbf{A}_n & \mathbf{0} \end{bmatrix} \begin{bmatrix} \Delta \mathbf{u} \\ \Delta \boldsymbol{\lambda}_n \end{bmatrix} = - \begin{bmatrix} \mathbf{r} \\ \mathbf{g}_n \end{bmatrix}. \tag{4.29}$$

The multiplier $\boldsymbol{\lambda}_n$ represents the contact pressure or an equivalent nodal contact force, depending on the discretization. The gap of an active constraint can be driven to zero without selecting a penalty stiffness.

This exactness has a price. Additional unknowns are introduced, the coefficient matrix becomes indefinite, and the active set must still be determined. A multiplier that becomes tensile is inadmissible for ordinary contact; the corresponding constraint must be released.

For the one-dimensional example, active contact gives the exact equations

$$\begin{bmatrix} k & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} u \\ p_n \end{bmatrix} = \begin{bmatrix} f \\ g_0 \end{bmatrix}. \tag{4.30}$$

The solution is $u = g_0$ and $p_n = f - kg_0$. If this computed p_n is negative, the assumed active status was wrong and the contact must be opened.

4.4.4 The Augmented Lagrangian Method

The augmented Lagrangian method combines a moderate penalty stiffness with an iteratively corrected pressure estimate. With iteration index j, one common normal update is

$$\boxed{p_n^{j+1} = \left\langle p_n^j - \epsilon_n g_n^{j+1} \right\rangle. \qquad (4.31)}$$

If penetration remains, $g_n < 0$, the pressure estimate increases. If the gap opens, the positive-part projection prevents a tensile contact pressure and ultimately returns the pressure to zero.

The displacement problem inside an augmentation step resembles a penalty problem, but the multiplier estimate carries information from one augmentation to the next. Hence the gap can be reduced without increasing ϵ_n to an excessively large value.

The augmented Lagrangian method is therefore often a useful compromise:

1. it is more accurate than a penalty method using the same stiffness,
2. it avoids the full multiplier system in some implementations,
3. it requires additional status and augmentation iterations.

The exact implementation varies. Some formulations keep explicit multiplier unknowns, while others update pressure-like history variables outside the main Newton solve. The mechanical meaning remains the same: a penalty regularization is combined with iterative multiplier correction.

4.4.5 Assessment of Normal Enforcement Methods

The three methods should be compared by their numerical consequences rather than by the apparent elegance of their equations.

1. *Penalty enforcement* retains the original displacement unknowns and is relatively easy to assemble. The contact constraint is only approximate and its accuracy depends on the penalty stiffness.
2. *Lagrange multiplier enforcement* can satisfy the active gap exactly and gives the contact reactions directly. It adds unknowns, produces an indefinite system and requires careful treatment of dependent constraints.
3. *Augmented Lagrangian enforcement* reduces the dependence on a very large penalty stiffness, but adds an outer correction process and still requires contact-status decisions.

No method removes the underlying nonsmoothness caused by opening and closing. A sophisticated enforcement method cannot compensate for an incorrect contact normal, poor surface discretization or a failed search projection.

4.5 Tangential Contact and Friction

Normal contact decides whether the surfaces may approach or separate. Tangential contact decides whether they may slide and which shear traction is transmitted while they are closed.

Let $\mathbf{t}_t$ denote the tangential traction vector and p_n the compressive normal pressure. A friction law acts only while contact is closed. If the surfaces are open, both normal and tangential contact tractions vanish:

$$g_n > 0 \quad \Longrightarrow \quad p_n = 0, \qquad \mathbf{t}_t = \mathbf{0}. \tag{4.32}$$

4.5.1 Frictionless Contact

Frictionless contact imposes the unilateral normal condition but no tangential constraint:

$$g_n \geq 0, \qquad p_n \geq 0, \qquad p_n g_n = 0, \qquad \mathbf{t}_t = \mathbf{0}. \tag{4.33}$$

The surfaces may separate and may slide freely. Compressive normal traction is transmitted, but no shear traction and no tensile normal traction are carried.

Frictionless does not mean that the bodies are disconnected. While closed, the normal relative motion is constrained and normal forces can be large. It only means that the tangential relative motion does not generate contact shear.

4.5.2 Coulomb Friction

The classical Coulomb law limits the tangential traction by the current normal pressure:

$$\boxed{\phi = \|\mathbf{t}_t\| - \mu p_n \leq 0, \qquad (4.34)}$$

where $\mu \geq 0$ is the coefficient of friction. Two states are possible.

1. *Stick*: $\phi < 0$. The tangential traction lies inside the friction bound and the incremental tangential slip is constrained to zero, apart from any tangential penalty compliance.
2. *Slip*: $\phi = 0$. The traction magnitude reaches μp_n and its direction opposes the sliding direction.

For nonzero sliding velocity $\mathbf{v}_t$, the ideal slip law is

$$\mathbf{t}_t = -\mu p_n \frac{\mathbf{v}_t}{\|\mathbf{v}_t\|}. \tag{4.35}$$

The minus sign states that friction resists relative sliding. At zero velocity the direction is undefined, which is why the stick state has to be treated as an inequality rather than by direct substitution into (4.35).

4.5.3 A Tangential Trial-State Algorithm

The stick-slip update resembles an elastic predictor and plastic corrector. Let ϵ_t be a tangential penalty stiffness and let $\Delta\mathbf{g}_t$ be the tangential relative-displacement increment. A trial traction is

$$\mathbf{t}_t^{trial} = \mathbf{t}_t^n - \epsilon_t \Delta\mathbf{g}_t. \tag{4.36}$$

The trial friction function is

$$\phi^{trial} = \|\mathbf{t}_t^{trial}\| - \mu p_n. \tag{4.37}$$

If $\phi^{trial} \leq 0$, the contact sticks and

$$\mathbf{t}_t^{n+1} = \mathbf{t}_t^{trial}. \tag{4.38}$$

If $\phi^{trial} > 0$, the contact slips and the traction is returned to the friction limit:

$$\mathbf{t}_t^{n+1} = \mu p_n \frac{\mathbf{t}_t^{trial}}{\|\mathbf{t}_t^{trial}\|}. \tag{4.39}$$

The signs in (4.36) depend on the adopted traction and slip conventions. The important construction is invariant: first predict the traction assuming stick, then check the Coulomb bound, and if the bound is exceeded project the traction back to the admissible friction surface.

Because the limit μp_n changes with normal pressure, the normal and tangential problems are coupled. Opening the contact immediately removes the friction traction. A change in normal pressure changes the maximum transferable shear.

4.5.4 Regularized Friction

Ideal Coulomb friction is nonsmooth at the transition from stick to slip and at zero sliding velocity. Some formulations replace the discontinuous law by a smooth approximation, for example

$$\mathbf{t}_t = -\mu p_n \tanh\left(\frac{\|\mathbf{v}_t\|}{v_0}\right) \frac{\mathbf{v}_t}{\|\mathbf{v}_t\|}, \tag{4.40}$$

with a small regularization velocity v_0. Such a law can improve nonlinear convergence, but it introduces artificial microslip and rate dependence. The regularization parameter is therefore part of the numerical model and should be reported when it materially affects the result.

4.6 Common Contact Definitions

The common names used in finite element programs can now be stated in terms of normal and tangential admissibility. The names are convenient, but their exact implementation may differ between programs. The solver-independent mechanical meaning is described below.

4.6.1 Frictionless

A frictionless interface has unilateral normal contact and zero tangential traction:

$$g_n \geq 0, \qquad p_n \geq 0, \qquad p_n g_n = 0, \qquad \mathbf{t}_t = \mathbf{0}. \tag{4.41}$$

It can transmit compression. It cannot transmit tension or shear. It allows both separation and unrestricted sliding.

4.6.2 Frictional

A frictional interface has unilateral normal contact together with the Coulomb bound

$$\|\mathbf{t}_t\| \leq \mu p_n. \tag{4.42}$$

It can transmit compression and a pressure-dependent amount of shear. It allows separation. While closed, it may stick or slide depending on whether the required shear traction is below or at the friction limit.

The frictionless definition is recovered as the special case $\mu = 0$.

4.6.3 Rough

A rough interface prevents tangential sliding while the surfaces are in contact:

$$g_n = 0 \quad \Longrightarrow \quad \Delta \mathbf{g}_t = \mathbf{0}. \tag{4.43}$$

It may be viewed as the limiting case of friction with an unbounded friction capacity, although implementations often impose the tangential constraint directly rather than using a very large value of μ.

Rough contact may transmit whatever tangential traction is required to prevent slip while the normal contact remains active. It still permits separation and normally cannot transmit tensile normal traction. This distinguishes rough contact from bonded contact.

4.6.4 No Separation

A no-separation interface prevents opening in the normal direction once the interface is closed, but allows tangential sliding:

$$g_n = 0, \qquad \mathbf{t}_t = \mathbf{0} \tag{4.44}$$

for the ideal frictionless no-separation case. Because the normal relation is bilateral, the interface can develop both compressive and tensile normal reactions. It is therefore not ordinary unilateral contact.

The phrase "once closed" is important. Some implementations tie surfaces that are initially within a specified tolerance; others first allow them to close and then prevent reopening. Initial status and program convention must therefore be checked.

No separation is useful when two surfaces may slide but are physically retained against opening, for example by an idealized enclosure. It is not appropriate when real lift-off is expected.

4.6.5 Bonded

An ideal bonded interface enforces displacement continuity in both normal and tangential directions:

$$\boxed{\mathbf{u}_s - \mathbf{u}_m = \mathbf{0}. \qquad (4.45)}$$

Equivalently,

$$g_n = 0, \qquad \mathbf{g}_t = \mathbf{0}. \qquad (4.46)$$

A bonded interface can transmit compression, tension and shear. It allows neither separation nor sliding. Mechanically it is a tied interface or a set of MFCs, even if it is created through contact-surface machinery.

Bonded contact should not be interpreted as a universal model of adhesive failure. The ideal constraint has no strength limit and no fracture energy. Debonding requires an additional interface law, such as a cohesive-zone model, a damage law or an explicit release criterion.

4.6.6 Comparison of the Definitions

The principal differences are summarized below. "Yes" means that the idealized definition admits the motion or force without adding another constitutive law.

Definition	Separation	Sliding	Tensile normal force	Shear transfer
Frictionless	Yes	Yes	No	No
Frictional	Yes	Conditional	No	Limited by μp_n
Rough	Yes	No while closed	No	Unlimited while closed
No separation	No	Yes	Yes	No, if frictionless
Bonded	No	No	Yes	Yes

This table describes the ideal mechanics, not the numerical enforcement. For example, bonded contact may be implemented by direct elimination, MFCs, penalty terms, multipliers or mortar constraints. Two models with the same word "bonded" may therefore have different constraint accuracy and conditioning.

4.6.7 Cohesive and Adhesive Interfaces

A cohesive interface is different from an ideal bond because the transmitted traction is related to an interface separation. In a simple uncoupled form,

$$t_n = t_n(\delta_n), \qquad \mathbf{t}_t = \mathbf{t}_t(\boldsymbol{\delta}_t), \tag{4.47}$$

where δ_n and $\boldsymbol{\delta}_t$ are normal and tangential separations. The law may initially be stiff, reach a strength limit and then soften as damage grows. The area under a traction-separation curve represents fracture energy.

Thus a cohesive interface is a constitutive model with finite strength and possible failure. Bonded contact is an ideal kinematic constraint with unlimited strength unless an additional failure rule is supplied.

4.7 Contact Discretizations

The continuous contact conditions apply over surfaces, but a finite element model contains nodes, edges, faces and interpolation functions. A contact discretization determines where the gap is evaluated and how the resulting tractions are distributed to the nodes.

4.7.1 Node-to-Node Contact

Node-to-node contact connects predefined pairs of nodes. For a pair with normal $\mathbf{n}$, the gap may be written

$$g_n = (\mathbf{x}_s - \mathbf{x}_m)^T \mathbf{n}. \tag{4.48}$$

The method is simple and inexpensive, but it requires compatible node locations and a known pairing. It is poorly suited to general sliding because a node that moves tangentially no longer remains opposite its original partner.

Node-to-node contact is best understood as a collection of discrete contact springs or constraints, not as an accurate representation of arbitrary smooth surface interaction.

4.7.2 Node-to-Surface Contact

In node-to-surface contact, each slave node is projected onto a master face. The gap is evaluated between the slave node and the interpolated master point. The contact force acting at the projection is distributed to the master nodes through the face shape functions.

This permits a slave node to move over the master mesh and does not require matching nodes. However, the result is biased: penetration is prevented at the slave nodes, but not necessarily at points between them. A coarse slave surface may therefore pass locally through the master surface between slave nodes.

As a general practical rule, the more finely discretized or more deformable surface is often selected as slave, while the smoother, coarser or stiffer surface is selected as master. This is a useful starting point, not a universal law. Curvature, shell normals, expected sliding direction and element order also matter.

4.7.3 Surface-to-Surface Contact

Surface-to-surface formulations evaluate contact over slave segments or integration points rather than only at slave nodes. The contact virtual work may be written schematically as

$$\delta W_c = \int_{\Gamma_c} \left(p_n \delta g_n + \mathbf{t}_t^T \delta \mathbf{g}_t \right) d\Gamma. \tag{4.49}$$

Numerical quadrature converts this surface integral into contact contributions at integration points. Surface-to-surface contact generally represents pressure more smoothly and reduces nodal bias, especially for higher-order elements and curved surfaces.

The term does not uniquely identify one algorithm. Different programs may use one-pass, two-pass, segment-to-segment or mortar-like formulations under the broad surface-to-surface name.

4.7.4 Mortar Contact

Mortar methods impose contact constraints in a weak, integral sense using interpolation spaces for the contact pressure or multipliers. Instead of matching the gap independently at isolated nodes, a weighted residual condition is used:

$$\int_{\Gamma_c} N_{\lambda i} g_n \, d\Gamma = 0 \tag{4.50}$$

for the active contact region, where $N_{\lambda i}$ are multiplier or mortar shape functions.

Mortar methods are well suited to nonmatching meshes and can provide improved conservation and reduced master-slave bias. Their formulation, integration and active-set treatment are more involved than classical node-to-surface contact.

4.7.5 One-Pass and Two-Pass Contact

A one-pass method projects one selected side onto the other. It is inherently asymmetric because exchanging master and slave changes the discrete equations.

A two-pass or symmetric method performs the constraint construction in both directions. This may reduce bias, but simply duplicating constraints can also overconstrain the interface, especially for matching meshes or nearly incompressible response. A properly designed symmetric or mortar formulation is not equivalent to blindly applying two independent node-to-surface pairs.

The words *symmetric* and *asymmetric* should therefore be read as properties of the discrete contact algorithm, not of the physical friction law.

4.8 Linearization and Nonlinear Solution

The finite element equilibrium equations including contact may be written

$$\mathbf{r}(\mathbf{u}) = \mathbf{f}_{int}(\mathbf{u}) + \mathbf{f}_c(\mathbf{u}) - \mathbf{f}_{ext} = \mathbf{0}. \tag{4.51}$$

A Newton iteration solves

$$\mathbf{K}_T^i \Delta\mathbf{u}^i = -\mathbf{r}^i, \qquad \mathbf{u}^{i+1} = \mathbf{u}^i + \Delta\mathbf{u}^i, \tag{4.52}$$

where

$$\mathbf{K}_T = \frac{\partial \mathbf{f}_{int}}{\partial \mathbf{u}} + \frac{\partial \mathbf{f}_c}{\partial \mathbf{u}}. \tag{4.53}$$

The contact tangent contains more than a normal spring stiffness. Depending on the formulation it includes derivatives of the gap, surface normal, projection coordinates, pressure law and friction law. In finite sliding, changes of the contact geometry produce additional geometric terms.

A typical contact iteration consists conceptually of:

1. predict or update the current geometry,
2. search for candidate surface pairs,
3. compute projections, gaps and relative tangential increments,
4. classify each contact point as open, closed-stick or closed-slip,
5. assemble contact residuals and tangent contributions,
6. solve the Newton correction,
7. repeat until equilibrium and contact conditions are satisfied.

The actual order may differ, and some solvers update status only at selected stages to improve robustness.

4.8.1 Contact Chattering

A contact point may alternate between open and closed states in successive iterations. Similarly, a friction point may alternate between stick and slip. This behaviour is called *chattering*. It can occur near the transition where both branches are nearly admissible.

Chattering may be reduced by smaller load increments, line search, status tolerances, augmented Lagrangian updates or mild regularization. Excessive stabilization, however, can alter the physical force-displacement response.

4.8.2 Initial Gap and Initial Penetration

Before the first equilibrium iteration, the discretized surfaces may have an initial gap or may overlap because of geometry tolerances, mesh approximation, shell offsets or intentional interference.

An initial penetration may be treated in several conceptually different ways:

1. preserve it and generate an initial contact force,
2. shift the contact reference so that the initial configuration is considered just touching,
3. move nodes or surfaces to remove the overlap,
4. ramp the interference over several increments.

These choices do not describe the same model. Removing an initial penetration changes the reference geometry, while preserving it may introduce prestress. The adopted treatment must therefore be understood rather than accepted as a purely numerical setting.

4.8.3 Search Distance and Candidate Detection

To avoid testing every surface face against every other face, contact algorithms use spatial search structures and a finite candidate distance. A contact pair outside the search region is ignored during that search pass.

A search distance that is too small can miss rapidly closing or initially separated surfaces. An excessively large distance increases search cost and can create unintended candidate pairs. Search tolerance affects detection; it does not by itself define the physical normal stiffness.

4.9 Numerical Behaviour and Assessment

Contact results should not be assessed only by whether the nonlinear solver reports convergence. A converged solution may still contain excessive penalty penetration, unintended contact pairs or mesh-dependent pressure peaks.

4.9.1 Quantities to Check

Useful checks include:

1. maximum and representative normal penetration,
2. normal contact pressure and integrated contact force,
3. open, closed, sticking and sliding status distributions,
4. frictional traction relative to the bound μp_n,
5. equilibrium residual and reaction-force balance,
6. sensitivity to contact mesh refinement,
7. sensitivity to penalty stiffness and load-step size.

A penetration should be judged relative to a meaningful geometrical scale, such as local element thickness, surface curvature radius or allowed interface deflection. Reporting only its absolute value can be misleading when model units or component sizes differ.

4.9.2 Pressure Singularities

High contact pressure near a sharp edge, corner or the end of a contact region may be a genuine mathematical singularity. Mesh refinement can then increase the peak pressure rather than converge it to a finite value. Integrated force, contact width or pressure away from the singular point may be more meaningful quantities.

A smooth pressure contour is not proof of correctness. Nodal averaging and postprocessing can hide local oscillations or violations of the contact conditions.

4.9.3 Mesh Compatibility and Bias

Nonmatching contact meshes are permissible, but the relative element sizes and orders influence pressure transfer. A very coarse surface contacting a very fine surface may produce oscillatory tractions or artificial local compliance.

Changing master and slave sides is a useful diagnostic for a biased one-pass formulation. Large changes in global response indicate that the contact discretization, not only the physical model, controls the answer.

4.9.4 Contact in Dynamic Problems

In dynamics, closing contact can introduce an impact. A purely displacement-based penalty law stores elastic contact energy

$$\Pi_c = \frac{1}{2}\epsilon_n \langle -g_n \rangle^2 \tag{4.54}$$

and may generate high-frequency oscillations. Contact damping or an impact law may be added, but such additions dissipate energy and must be distinguished from material damping.

The stable time step of an explicit method can be strongly reduced by a large penalty stiffness because the contact stiffness increases the highest system frequency. Thus the penalty value affects not only penetration but also computational cost.

For implicit dynamics, abrupt status changes and frictional slip can make the Newton iterations difficult. Time-step reduction often helps because it limits the amount of contact-status change within one increment.

4.10 Relation to Multifreedom Constraints

The connection to Chapter 3 can now be stated precisely.

A bonded interface is essentially a set of displacement equality constraints:

$$\mathbf{A}\mathbf{u} = \mathbf{0}. \tag{4.55}$$

A no-separation normal interface is a bilateral equality constraint on the normal gap. A sticking tangential state is an equality constraint on incremental tangential slip.

Ordinary normal contact is different because it combines an inequality with a reaction-sign condition and complementarity:

$$g_n \geq 0, \qquad p_n \geq 0, \qquad p_n g_n = 0. \tag{4.56}$$

Friction adds a second inequality and a second status decision:

$$\|\mathbf{t}_t\| - \mu p_n \leq 0. \tag{4.57}$$

Therefore contact may be regarded as a changing collection of MFC-like constraints whose membership is unknown in advance. Penalty, multiplier and augmented Lagrangian methods reappear, but they are embedded in geometry search, active-set logic and constitutive stick-slip updates.

This viewpoint is useful because it separates three questions that are often mixed together:

1. *What relative motion is physically allowed?* This determines frictionless, frictional, rough, no-separation or bonded behaviour.

2. *How is the constraint represented on the mesh?* This determines node-to-node, node-to-surface, surface-to-surface or mortar contact.

3. *How is the discrete constraint imposed?* This determines penalty, Lagrange multiplier, augmented Lagrangian or direct elimination treatment.

A contact setting is properly understood only when all three questions can be answered.

4.11 Summary

Contact is a unilateral interaction. The normal gap may remain positive with zero pressure, or the gap may close and develop a compressive pressure. These alternatives are expressed by

$$g_n \geq 0, \qquad p_n \geq 0, \qquad p_n g_n = 0. \tag{4.58}$$

The contact type describes admissible relative motion and transmitted traction:

1. frictionless contact allows separation and sliding and transmits only compression,

2. frictional contact adds a pressure-dependent shear limit,

3. rough contact prevents sliding while closed but still allows separation,

4. no-separation contact prevents opening but allows tangential sliding,

5. bonded contact prevents both opening and sliding and can transmit tension, compression and shear.

Penalty, Lagrange multiplier and augmented Lagrangian methods describe how the normal and tangential constraints are enforced numerically. Node-to-node, node-to-surface, surface-to-surface and mortar methods describe how the continuous interface is discretized.

No single contact definition or algorithm is uniformly best. The appropriate choice depends on the expected motion, required force transfer, mesh relation, constraint accuracy and nonlinear robustness. Contact settings should therefore be selected from their mechanical meaning first and tuned numerically second.

The next chapter returns to permanent bilateral relations. There, the same separation between physical constraint definition and numerical enforcement is used to organize rigid elements, interpolation couplings, fixed-distance constraints and remote coupling objects.

Chapter 5

Kinematic Constraints and Rigid Couplings

Finite element models frequently need relations between degrees of freedom that are not represented by the ordinary structural elements themselves. A flange may be treated as rigid, a force may be introduced through a point outside the mesh, a bearing may be represented by a bushing acting on an entire hole, or two nodes may be required to remain at a prescribed distance. These modelling operations look different in a graphical user interface, but they are all examples of *kinematic constraints*: relations that prescribe which relative motions are admissible.

The mathematical machinery for imposing such relations was developed in Chapter 3. The present chapter therefore does not repeat the penalty, elimination, Lagrange multiplier and augmented Lagrangian methods. Instead, it asks a different question: *what constraint should be constructed for a particular engineering purpose?*

This distinction is important. The same engineering constraint may be enforced by different numerical techniques, and two solver commands that look similar may produce different algebraic systems. Conversely, commands with very different names may generate essentially the same constraint equations. The familiar names RBE1, RBE2 and RBE3 are consequently used in this book as general engineering names for three widely recognized classes of coupling, not as an attempt to follow the terminology of a particular finite element program.

Contact provides the complementary case. Chapter 4 showed that ordinary contact is not a permanent equality relation but a unilateral constraint whose active set is part of the solution. The distinction can be summarized as

$$\begin{aligned} \text{bilateral kinematic constraint:} \quad & \mathbf{g}(\mathbf{u}) = \mathbf{0}, \\ \text{unilateral contact constraint:} \quad & g_n(\mathbf{u}) \geq 0, \end{aligned} \tag{5.1}$$

with the contact reaction additionally governed by the complementarity conditions introduced in Chapter 4. Figure 5.1 places these relations in a common engineering framework and also emphasizes that the choice of numerical enforcement remains a separate question.

5.1 From General Constraints to Couplings

A general linear multifreedom constraint may be written in the form used in Chapter 3,

$$\mathbf{A}\mathbf{u} = \mathbf{b}. \tag{5.2}$$

The coefficients of $\mathbf{A}$ may be entered explicitly, but doing so becomes cumbersome when the relation is dictated by geometry. A rigid spider with one reference point and fifty attachment nodes already requires hundreds of coefficients. Repeating the construction for each connection is tedious and, more importantly, easy to get wrong.

Rigid and interpolation couplings can therefore be understood as *constraint generators*. The analyst specifies geometry, active degrees of freedom and, where necessary, weights; the program constructs the corresponding equations. The generated relation can then be imposed by one of the numerical procedures discussed in Chapter 3.

This viewpoint also clarifies why a coupling should not automatically be regarded as a physical finite element. An ideal RBE2 or RBE3 usually has no

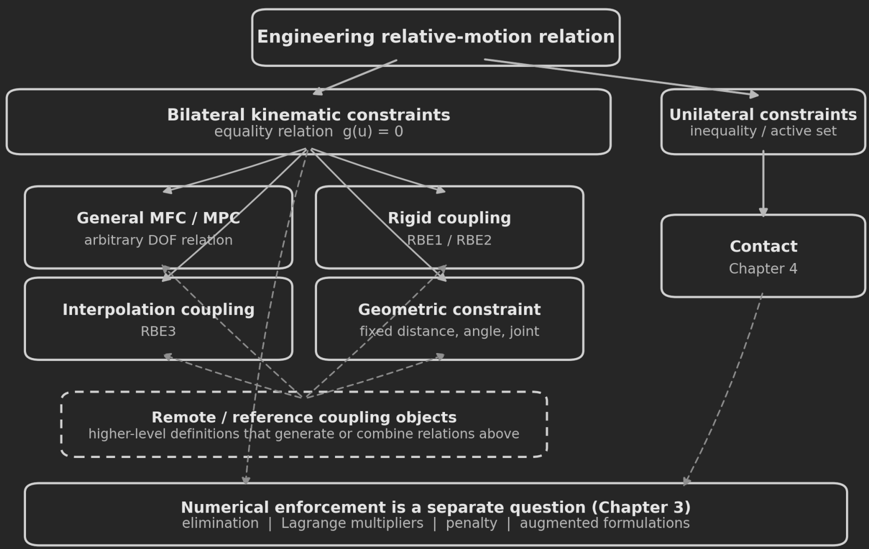


Figure 5.1: Classification of engineering relative-motion constraints used throughout this chapter. General multifreedom constraints, rigid couplings, RBE3 interpolation and geometric constraints are bilateral equality relations; contact is the complementary unilateral case discussed in Chapter 4. Remote or reference coupling objects are higher-level modelling definitions that may generate or combine several of these relations. The enforcement technique is a separate numerical choice treated in Chapter 3.

constitutive law, no Young's modulus and no strain energy of its own. It changes the set of admissible displacement fields. A beam, spring, bushing or adhesive layer is fundamentally different because it stores energy according to a constitutive law.

Engineering note – Constraint definition and constraint enforcement are different questions.
A fixed-distance relation, an RBE2 rigid coupling and an RBE3 interpolation coupling describe different admissible motions. Each may in principle be imposed by elimination, multipliers, a penalty-like method or another solver-specific algorithm. The physical definition should therefore be selected first; the numerical enforcement method is a second decision.

5.2 Evolution of Kinematic Couplings

Early finite element programs exposed general multipoint or constraint equations directly. This remains the most general representation: any linear relation that can be written as Eq. (5.2) can be entered coefficient by coefficient. Recurrent engineering patterns, however, soon led to specialized definitions.

The RBE family is best understood as an evolution of these recurring patterns:

1. **RBE1** provides a general rigid coupling between selected sets of independent and dependent degrees of freedom. It is close in spirit to a structured multipoint-constraint definition.
2. **RBE2** expresses the dependent-node motion directly from the translation and rotation of a reference node. It therefore creates an exactly rigid kinematic region in the selected degrees of freedom.
3. **RBE3** was introduced for the opposite modelling need: transfer forces, moments and generalized motion through a set of surrounding nodes *without making that surrounding region rigid.* Its reference motion is obtained by interpolation, commonly by a weighted least-squares construction.

Modern programs frequently place another abstraction above these definitions: remote points, reference points, surface-based constraints, joints and coupling objects. Such objects may internally generate classical constraint equations, interpolation relations, contact-based MPCs or dedicated constraint elements. The preprocessor object is consequently not the formulation. The engineering behaviour of the underlying relation is what matters. Figure 5.2 summarizes this development from explicit constraint coefficients toward increasingly specialized and higher-level engineering definitions.

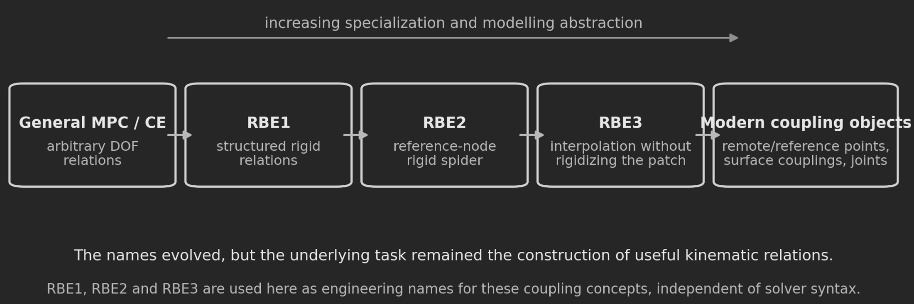


Figure 5.2: Schematic evolution of commonly used kinematic coupling definitions. The sequence emphasizes the engineering progression from explicit general constraints, through the familiar RBE family, toward higher-level coupling objects. It is not intended as a solver-specific command or release chronology.

5.3 Classification of Kinematic Constraints

It is useful to classify a coupling by the relative motion it permits before considering its software implementation.

Constraint type	**Prescribed relation**	**Typical use**
Equal DOF	Selected degrees of freedom have the same value	Coincident-node tying, symmetry, simple joints
General linear MPC	Arbitrary linear combination of DOFs	Periodicity, special kinematic relations
RBE1	Selected rigid relations between sets of DOFs	General rigid coupling
RBE2	Rigid-body motion about a reference node	Rigid flange, rigid spider, ideal rigid attachment
RBE3	Weighted interpolation of reference motion	Load distribution, remote support, deformable attachment
Fixed distance	Constant distance between geometric points	Links, rods, mechanism constraints
Fixed angle or joint	Selected relative rotations or translations suppressed	Revolute, cylindrical, spherical and other joints
Remote or reference coupling	Higher-level interface coordinates combined with a rigid, interpolating or other coupling law	Remote loads, displacements, springs, bushings and joints
Contact	Inequality and complementarity conditions	Separation, sliding, impact and friction

The last row is deliberately included even though contact was treated in the preceding chapter. It emphasizes that contact and permanent couplings answer the same broad engineering question – which relative motion is admissible – but have fundamentally different mathematical status. A remote or reference coupling is listed separately because it is usually a higher-level modelling interface: its underlying relation may be rigid, interpolating or something more specialized.

Throughout this chapter, *reference node* or *reference point* denotes the retained generalized coordinates of a coupling. The term *dependent node* is reserved for a node whose selected DOFs are prescribed by an exact kinematic relation such as an RBE2. *Attachment node* is used for a mesh node participating in an interpolation such as an RBE3, where local relative deformation is intentionally retained. *Remote point* is used primarily when discussing a particular preprocessor abstraction. These words are not interchangeable: they describe different mathematical roles, not merely alternative GUI labels.

Python examples in this chapter.
The Python examples are written as small reusable functions rather than as one-off calculations. `numpy` is used only for storing vectors and matrices and for elementary array arithmetic and matrix products. No `numpy.linalg`, eigensolver, sparse-solver or other NumPy submodule is used. Where a small linear system is unavoidable, the elimination steps are written explicitly in the example code rather than delegated to a numerical library.
The examples use the common structural-analysis DOF numbering $1 = u_x$, $2 = u_y$, $3 = u_z$, $4 = \theta_x$, $5 = \theta_y$, $6 = \theta_z$. The RBE builders accept node IDs and coordinates together with the selected independent/dependent or master/slave DOFs, so changing the coupling geometry does not require rewriting its coefficient matrix by hand. The functions are intended as transparent executable forms of the derivations, not as production finite element utilities.

5.4 Mathematical Background

For a bilateral nonlinear constraint the most general notation used in this chapter is

$$\mathbf{g}(\mathbf{u}) = \mathbf{0}. \tag{5.3}$$

Linearization around the current iterate $\mathbf{u}^{(k)}$ gives

$$\mathbf{g}(\mathbf{u}^{(k)}) + \mathbf{G}^{(k)}\Delta\mathbf{u} = \mathbf{0}, \qquad \mathbf{G}^{(k)} = \left.\frac{\partial \mathbf{g}}{\partial \mathbf{u}}\right|_{\mathbf{u}^{(k)}}. \tag{5.4}$$

If the relation is linear and fixed in the initial configuration, $\mathbf{G} = \mathbf{A}$ is constant and Eq. (5.2) is recovered. This apparently small distinction becomes crucial for large rotations. A constraint constructed once from the initial geometry follows the initial tangent; a geometrically exact constraint must update its direction and moment arms as the configuration changes.

When the constraint is enforced by Lagrange multipliers, for example, the linearized equilibrium system has the familiar saddle-point form

$$\begin{bmatrix} \mathbf{K}_t & \mathbf{G}^T \\ \mathbf{G} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \Delta\mathbf{u} \\ \Delta\lambda \end{bmatrix} = \begin{bmatrix} \mathbf{r} \\ -\mathbf{g} \end{bmatrix}. \tag{5.5}$$

The multipliers λ have the physical interpretation of constraint reactions. Equation (5.5) is included here only to show the distinction between the *definition* $\mathbf{g}$ and its *enforcement*; the numerical properties of multiplier systems were discussed in Chapter 3.

5.5 RBE1 – General Rigid Coupling

5.5.1 Engineering Meaning

RBE1 is the most general rigid member of the familiar RBE family. Its defining feature is not a particular spider shape but the way the rigid-body coordinates are selected. In an RBE2, the retained motion is normally represented by the translation and rotation of one reference node. In an RBE1, the independent scalar degrees of freedom may instead be distributed among several nodes. They must jointly be sufficient to describe the rigid-body motion of the coupled set. Selected dependent components are then obtained from those independent components by rigid-body compatibility.

For a three-dimensional rigid body, six independent scalar components are needed to span the six rigid-body motions. They need not all belong to the same node. This makes RBE1 useful when the natural retained coordinates are already spread over several points, or when selected translational or rotational releases are desired without introducing a separate six-DOF reference node. Components that are not listed in the coupling remain outside that rigid relation.

The distinction from a completely general MPC is therefore one of structure. The analyst identifies independent and dependent components, while the rigid-body geometry determines the coefficients that relate them. In compact form,

$$\mathbf{q}_D = \mathbf{T}_{DI}\mathbf{q}_I, \tag{5.6}$$

where $\mathbf{q}_I$ contains the selected independent scalar DOFs and $\mathbf{q}_D$ the selected dependent DOFs. The transformation $\mathbf{T}_{DI}$ is assembled from rigid-body kinematics. In the classical three-dimensional case $\mathbf{q}_I$ contains six independent scalar components, but their distribution among physical nodes is arbitrary provided that the selected set has full rigid-body rank.

This last requirement is essential. Choosing six scalar DOFs is not sufficient if their geometry cannot distinguish all rigid motions. For example, a set of translational components located on a degenerate or poorly chosen geometry may fail to determine one of the rotations. The resulting problem is one of kinematic rank, not material stiffness.

Engineering note – The main difference between RBE1 and RBE2 is where the retained rigid-body coordinates live.
RBE2 collects the retained coordinates at one reference node and maps them to all dependent nodes. RBE1 allows the independent scalar components to be spread among several nodes. Both describe rigid-body compatibility; RBE1 simply offers more freedom in how the independent and dependent components are selected.

5.5.2 Worked Example – Planar RBE1

The three-dimensional definition is easiest to understand through its planar analogue. A rigid body moving in the x–y plane has three rigid-body degrees of freedom: two translations u_0, v_0 and one small rotation θ_z. Consider the square

$$A = (0,0), \qquad B = (L,0), \qquad C = (0,L), \qquad D = (L,L). \tag{5.7}$$

For small rotation, the displacement of any point (x, y) is

$$u(x,y) = u_0 - \theta_z y, \qquad v(x,y) = v_0 + \theta_z x. \tag{5.8}$$

Instead of introducing a separate reference node with (u_0, v_0, θ_z), choose the following three independent scalar DOFs:

$$\mathbf{q}_I = \begin{bmatrix} u_A & v_A & v_B \end{bmatrix}^T. \tag{5.9}$$

The first two components define the translation of point A. The vertical motion of point B then determines the rotation because

$$v_B = v_A + L\theta_z, \qquad \theta_z = \frac{v_B - v_A}{L}. \tag{5.10}$$

The selected dependent components may now be recovered from rigid-body compatibility. For the remaining translation at B and the translations at C and D,

$$u_B = u_A, \tag{5.11}$$
$$u_C = u_A - L\theta_z = u_A + v_A - v_B, \tag{5.12}$$
$$v_C = v_A, \tag{5.13}$$
$$u_D = u_A - L\theta_z = u_A + v_A - v_B, \tag{5.14}$$
$$v_D = v_B. \tag{5.15}$$

Thus

$$\underbrace{\begin{bmatrix} u_B \\ u_C \\ v_C \\ u_D \\ v_D \end{bmatrix}}_{\mathbf{q}_D} = \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & -1 \\ 0 & 1 & 0 \\ 1 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix}}_{\mathbf{T}_{DI}} \underbrace{\begin{bmatrix} u_A \\ v_A \\ v_B \end{bmatrix}}_{\mathbf{q}_I}. \tag{5.16}$$

The example shows the characteristic RBE1 idea: the independent coordinates do not have to be collected at one reference point. One component of node B is independent while another component at the same node can be dependent, and the other dependent nodal motions follow from the same rigid-body field.

For a numerical check, let $L = 100$ mm and prescribe

$$u_A = 1 \text{ mm}, \qquad v_A = 2 \text{ mm}, \qquad v_B = 3 \text{ mm}. \tag{5.17}$$

The implied small rotation is

$$\theta_z = \frac{3-2}{100} = 0.01 \text{ rad}. \tag{5.18}$$

Equation (5.16) gives

$$u_B = 1 \text{ mm}, \qquad (u_C, v_C) = (0, 2) \text{ mm}, \qquad (u_D, v_D) = (0, 3) \text{ mm}. \tag{5.19}$$

Substitution into Eq. (5.8) gives the same values, so the selected components lie on one planar rigid-body displacement field. The three independent scalar DOFs in this two-dimensional example play the same role as the six independent scalar components required to span a general rigid-body motion in three dimensions.

The calculation can also be generated from node coordinates and a list of independent and dependent scalar DOFs. The first listing introduces three small utilities reused by the later examples: node lookup, one rigid-body coefficient row, and an explicit Gauss–Jordan solver for small dense systems. The RBE1 builder then constructs $\mathbf{T}_{DI}$ from the selected components rather than hard-coding the matrix of Eq. (5.16):

```python
import numpy as np


def node_position(node_ids, coordinates, node_id):
```

```
    for i, nid in enumerate(node_ids):
        if int(nid) == int(node_id):
            return coordinates[i]
    raise ValueError("node ID not found")


def rigid_dof_row(r, dof):
    rx, ry, rz = r
    B = np.array([
        [1, 0, 0,   0,  rz, -ry],
        [0, 1, 0, -rz,   0,  rx],
        [0, 0, 1,  ry, -rx,   0],
        [0, 0, 0,   1,   0,   0],
        [0, 0, 0,   0,   1,   0],
        [0, 0, 0,   0,   0,   1],
    ], dtype=float)
    if dof < 1 or dof > 6:
        raise ValueError("DOF must be in 1..6")
    return B[dof - 1]


def solve_small_system(A, b, tol=1e-12):
    A = np.array(A, dtype=float, copy=True)
    b = np.array(b, dtype=float, copy=True)

    vector_rhs = (b.ndim == 1)
    if vector_rhs:
        b = b.reshape(-1, 1)

    n = A.shape[0]
    if A.shape[1] != n or b.shape[0] != n:
        raise ValueError("incompatible dimensions")

    aug = np.hstack((A, b))
    for col in range(n):
        pivot = col
        for row in range(col + 1, n):
            if abs(aug[row, col]) > abs(aug[pivot, col]):
                pivot = row

        if abs(aug[pivot, col]) < tol:
```

```
            raise ValueError("singular or rank-deficient system")

        if pivot != col:
            tmp = aug[col].copy()
            aug[col] = aug[pivot]
            aug[pivot] = tmp

        aug[col] = aug[col] / aug[col, col]
        for row in range(n):
            if row != col:
                aug[row] -= aug[row, col] * aug[col]

    x = aug[:, n:]
    return x[:, 0] if vector_rhs else x


def build_rbe1(node_ids, coordinates, independent_dofs,
               dependent_dofs, rigid_dofs, origin_id):
    x0 = node_position(node_ids, coordinates, origin_id)
    columns = [dof - 1 for dof in rigid_dofs]

    A_I = np.vstack([
        rigid_dof_row(
            node_position(node_ids, coordinates, nid) - x0, dof
        )[columns]
        for nid, dof in independent_dofs
    ])
    A_D = np.vstack([
        rigid_dof_row(
            node_position(node_ids, coordinates, nid) - x0, dof
        )[columns]
        for nid, dof in dependent_dofs
    ])

    if A_I.shape[0] != A_I.shape[1]:
        raise ValueError("independent DOFs do not span rigid motion")

    # T_DI = A_D A_I^-1 without calling an inverse routine.
    T_DI = solve_small_system(A_I.T, A_D.T).T
    return T_DI
```

```python

L = 100.0
node_ids = np.array([1, 2, 3, 4], dtype=int)
coordinates = np.array([
    [0.0, 0.0, 0.0],  # A
    [L,   0.0, 0.0],  # B
    [0.0, L,   0.0],  # C
    [L,   L,   0.0],  # D
])

independent = [(1, 1), (1, 2), (2, 2)]  # u_A, v_A, v_B
dependent = [(2, 1), (3, 1), (3, 2), (4, 1), (4, 2)]

T_DI = build_rbe1(
    node_ids, coordinates,
    independent, dependent,
    rigid_dofs=(1, 2, 6),
    origin_id=1,
)

q_I = np.array([1.0, 2.0, 3.0])
q_D = T_DI @ q_I

print(T_DI)
print(q_D)
# q_D = [1. 0. 2. 0. 3.]
```

The function interface exposes the actual RBE1 definition: geometry, independent scalar components, dependent scalar components and the rigid-body modes that are to be represented. The rank test is performed by the explicit elimination routine; no matrix-rank or inversion function is hidden in NumPy.

5.5.3 Modelling Consequences

RBE1 is useful when only selected components are to participate in the rigid relation or when a natural set of retained coordinates is distributed over several points. The additional freedom is also what makes the definition easier to misuse. The independent set must span the intended rigid-body modes, and a DOF chosen as dependent must not simultaneously be prescribed by another exact constraint unless the combined equations are intentionally compatible.

The resulting coupling is still rigid in the selected components. It can therefore suppress structural deformation just as an RBE2 can, and the same advanced considerations apply: thermal expansion, finite rotations, modal admissibility and interaction with other constraints all depend on the actual kinematic equations and their implementation.

Common mistake – Counting independent DOFs without checking their rank.
A full three-dimensional RBE1 requires an independent set capable of representing all six rigid-body motions. Six listed scalar components can still be deficient if their directions and locations are geometrically degenerate. The correct question is whether the selected components span the rigid-body motion, not merely whether their count equals six.

5.6 RBE2 – Rigid-Body Coupling

5.6.1 Engineering Meaning

An RBE2-type coupling imposes rigid-body compatibility between a reference node and one or more dependent nodes. In the traditional terminology the reference node is often called the *master* and the coupled nodes *slaves*; in the following, *reference* and *dependent* are used where this makes the physical role clearer.

The essential assumption is that the relative geometry of the coupled nodes cannot deform. A dependent node may translate because the reference node translates and because the rigid arm between the two nodes rotates, but it cannot move independently of that rigid-body motion in the coupled degrees of freedom. An RBE2 is therefore not a very stiff beam or spring. It is an exact kinematic restriction on the admissible displacement field.

This distinction explains both the usefulness and the most common misuse of an RBE2. If a flange, insert or attachment is physically much stiffer than the surrounding structure, replacing it by a rigid coupling can be an efficient and accurate idealization. If the spider is introduced merely to spread a load over a flexible sheet or flange, however, the same constraint suppresses local bending and warping and can make the model substantially too stiff.

Engineering note – RBE2 stiffening is kinematic, not constitutive. An ideal RBE2 does not need a Young's modulus and does not store elastic strain energy of its own. The apparent infinite stiffness results from removing all relative deformation modes that violate the rigid-body constraint. The surrounding structure therefore becomes stiffer because its admissible displacement space has been reduced.

5.6.2 Small-Rotation Kinematics

Let the reference node be located at $\mathbf{X}_r$ and dependent node i at $\mathbf{X}_i$. The initial rigid arm is

$$\mathbf{r}_i = \mathbf{X}_i - \mathbf{X}_r = \begin{bmatrix} r_{x,i} & r_{y,i} & r_{z,i} \end{bmatrix}^T . \tag{5.20}$$

Thus $\mathbf{r}_i$ points from the reference node to the dependent node. For a small reference rotation

$$\theta_r = \begin{bmatrix} \varphi_r & \psi_r & \theta_r \end{bmatrix}^T , \tag{5.21}$$

the rigid-body translation of the dependent node is

$$\mathbf{u}_i = \mathbf{u}_r + \theta_r \times \mathbf{r}_i. \tag{5.22}$$

It is convenient to introduce the matrix that maps the reference rotation to the translational increment,

$$\mathbf{B}(\mathbf{r}_i) = \begin{bmatrix} 0 & r_{z,i} & -r_{y,i} \\ -r_{z,i} & 0 & r_{x,i} \\ r_{y,i} & -r_{x,i} & 0 \end{bmatrix}, \tag{5.23}$$

so that

$$\theta_r \times \mathbf{r}_i = \mathbf{B}(\mathbf{r}_i)\theta_r. \tag{5.24}$$

If rotational DOFs exist at the dependent node and are included in the coupling, they follow the reference rotation,

$$\theta_i = \theta_r. \tag{5.25}$$

The six-DOF relation can therefore be written

$$\boxed{\underbrace{\begin{bmatrix} \mathbf{u}_i \\ \theta_i \end{bmatrix}}_{\mathbf{q}_i} = \underbrace{\begin{bmatrix} \mathbf{I} & \mathbf{B}(\mathbf{r}_i) \\ \mathbf{0} & \mathbf{I} \end{bmatrix}}_{\mathbf{D}_i} \underbrace{\begin{bmatrix} \mathbf{u}_r \\ \theta_r \end{bmatrix}}_{\mathbf{q}_r}. \qquad (5.26)}$$

For solid-element nodes, which ordinarily possess only translational DOFs, only the upper three rows are required. Likewise, an implementation may intentionally couple only selected translational or rotational components. The term RBE2 therefore identifies the rigid-body *kinematics*; the exact active DOF set still belongs to the element or solver definition.

Equation (5.22) is the linearized rigid-body relation. Its arm $\mathbf{r}_i$ is taken from the configuration in which the constraint equations are constructed. The consequences for large rotations are discussed later in Section 5.10.1.

5.6.3 Several Dependent Nodes

For n dependent nodes, the individual relations are stacked vertically,

$$\mathbf{q}_d = \mathbf{D}\mathbf{q}_r, \qquad \mathbf{D} = \begin{bmatrix} \mathbf{D}_1 \\ \mathbf{D}_2 \\ \vdots \\ \mathbf{D}_n \end{bmatrix}. \tag{5.27}$$

The important asymmetry is immediately visible: the reference DOFs are retained, whereas the dependent DOFs are determined by them. The constraint consequently reduces the number of independent kinematic variables if it is imposed by the dependent-DOF elimination procedure of Chapter 3.

5.6.4 Force and Moment Transfer

The same transformation that maps displacements determines the statically equivalent load transfer. For virtual work of a force $\mathbf{F}_i$ and, where applicable, a nodal moment $\mathbf{M}_i$ at dependent node i,

$$\delta W_i = \mathbf{F}_i^T \delta\mathbf{u}_i + \mathbf{M}_i^T \delta\theta_i. \tag{5.28}$$

Using Eq. (5.26) gives the equivalent reference resultants

$$\mathbf{F}_{r,i} = \mathbf{F}_i, \qquad \mathbf{M}_{r,i} = \mathbf{r}_i \times \mathbf{F}_i + \mathbf{M}_i. \tag{5.29}$$

For all dependent nodes,

$$\boxed{\mathbf{F}_r = \sum_i \mathbf{F}_i, \qquad \mathbf{M}_r = \sum_i \left(\mathbf{r}_i \times \mathbf{F}_i + \mathbf{M}_i\right). \tag{5.30}}$$

In matrix form this is simply the transpose rule

$$\mathbf{f}_r = \mathbf{D}^T \mathbf{f}_d. \tag{5.31}$$

This result is sometimes misinterpreted as a prescribed load-distribution rule. It is not. An RBE2 prescribes rigid motion. If a load is applied at the reference node, the individual reactions that develop at the dependent nodes are determined by the stiffness and connectivity of the attached structure while satisfying the rigid kinematics and overall equilibrium. This is one of the key differences from the weighting concept used by an RBE3.

5.6.5 Worked Example – RBE2 Coefficients

The coefficient construction is sufficiently important that it is useful to work through one complete spider rather than showing only the compact matrix notation. Consider the reference node

$$\mathbf{X}_r = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}^T, \tag{5.32}$$

and four dependent nodes forming a square in the plane $z = 0$,

$$\begin{aligned} \mathbf{X}_1 &= \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^T, & \mathbf{X}_2 &= \begin{bmatrix} 2 & 0 & 0 \end{bmatrix}^T, \\ \mathbf{X}_3 &= \begin{bmatrix} 2 & 2 & 0 \end{bmatrix}^T, & \mathbf{X}_4 &= \begin{bmatrix} 0 & 2 & 0 \end{bmatrix}^T. \end{aligned} \tag{5.33}$$

The first step is purely geometric. For each spider leg, form the rigid arm from the reference node to the dependent node,

$$\begin{aligned} \mathbf{r}_1 &= \mathbf{X}_1 - \mathbf{X}_r = \begin{bmatrix} -1 & -1 & -1 \end{bmatrix}^T, & \mathbf{r}_2 &= \mathbf{X}_2 - \mathbf{X}_r = \begin{bmatrix} 1 & -1 & -1 \end{bmatrix}^T, \\ \mathbf{r}_3 &= \mathbf{X}_3 - \mathbf{X}_r = \begin{bmatrix} 1 & 1 & -1 \end{bmatrix}^T, & \mathbf{r}_4 &= \mathbf{X}_4 - \mathbf{X}_r = \begin{bmatrix} -1 & 1 & -1 \end{bmatrix}^T. \end{aligned} \tag{5.34}$$

Substitution of each arm into Eq. (5.26) produces one 6×6 transformation for each dependent node. For node 1,

$$\begin{bmatrix} u_1 \\ v_1 \\ w_1 \\ \varphi_1 \\ \psi_1 \\ \theta_1 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 & -1 & 1 \\ 0 & 1 & 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & -1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}}_{\mathbf{D}_1} \begin{bmatrix} u_r \\ v_r \\ w_r \\ \varphi_r \\ \psi_r \\ \theta_r \end{bmatrix}. \tag{5.35}$$

The other three legs are obtained in exactly the same way,

$$\begin{bmatrix} u_2 \\ v_2 \\ w_2 \\ \varphi_2 \\ \psi_2 \\ \theta_2 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 & -1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & -1 & -1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}}_{\mathbf{D}_2} \begin{bmatrix} u_r \\ v_r \\ w_r \\ \varphi_r \\ \psi_r \\ \theta_r \end{bmatrix}, \tag{5.36}$$

$$\begin{bmatrix} u_3 \\ v_3 \\ w_3 \\ \varphi_3 \\ \psi_3 \\ \theta_3 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 & -1 & -1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & -1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}}_{\mathbf{D}_3} \begin{bmatrix} u_r \\ v_r \\ w_r \\ \varphi_r \\ \psi_r \\ \theta_r \end{bmatrix}, \tag{5.37}$$

and

$$\begin{bmatrix} u_4 \\ v_4 \\ w_4 \\ \varphi_4 \\ \psi_4 \\ \theta_4 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 & -1 & -1 \\ 0 & 1 & 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}}_{\mathbf{D}_4} \begin{bmatrix} u_r \\ v_r \\ w_r \\ \varphi_r \\ \psi_r \\ \theta_r \end{bmatrix}. \tag{5.38}$$

These four equations are the individual spider legs written explicitly. Stacking them gives the complete RBE2 coefficient matrix,

$$
\underbrace{\begin{bmatrix} u_1 \\ v_1 \\ w_1 \\ \varphi_1 \\ \psi_1 \\ \theta_1 \\ u_2 \\ v_2 \\ w_2 \\ \varphi_2 \\ \psi_2 \\ \theta_2 \\ u_3 \\ v_3 \\ w_3 \\ \varphi_3 \\ \psi_3 \\ \theta_3 \\ u_4 \\ v_4 \\ w_4 \\ \varphi_4 \\ \psi_4 \\ \theta_4 \end{bmatrix}}_{\mathbf{q}_d} = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 & -1 & 1 \\ 0 & 1 & 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & -1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & -1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & -1 & -1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & -1 & -1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & -1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & -1 & -1 \\ 0 & 1 & 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}}_{\mathbf{D}} \underbrace{\begin{bmatrix} u_r \\ v_r \\ w_r \\ \varphi_r \\ \psi_r \\ \theta_r \end{bmatrix}}_{\mathbf{q}_r} . \tag{5.39}
$$

The full matrix is long, but it is useful because it makes the nature of an RBE2 explicit: every dependent DOF is simply a linear combination of the six retained reference DOFs. For example,

$$\begin{aligned}
u_1 &= u_r - \psi_r + \theta_r, \\
v_1 &= v_r + \varphi_r - \theta_r, \\
w_1 &= w_r - \varphi_r + \psi_r, \\
&\vdots \\
\theta_4 &= \theta_r.
\end{aligned} \tag{5.40}$$

Thus the complete coefficient relation is simply the stacked form already introduced in Eq. (5.27),

$$\mathbf{q}_d = \mathbf{D}\mathbf{q}_r. \tag{5.41}$$

A numerical motion provides a useful independent check of the signs. Let the reference node translate by 0.1 in x and rotate by $\theta_r = 0.01\,\text{rad}$ about the z axis, with all other components zero. The four dependent translations become

$$\begin{aligned}
\mathbf{u}_1 &= \begin{bmatrix}0.11 & -0.01 & 0\end{bmatrix}^T, \quad & \mathbf{u}_2 &= \begin{bmatrix}0.11 & 0.01 & 0\end{bmatrix}^T, \\
\mathbf{u}_3 &= \begin{bmatrix}0.09 & 0.01 & 0\end{bmatrix}^T, \quad & \mathbf{u}_4 &= \begin{bmatrix}0.09 & -0.01 & 0\end{bmatrix}^T.
\end{aligned} \tag{5.42}$$

The square therefore translates and rotates as one rigid object. No relative in-plane deformation of the four attachment nodes is admissible.

The same geometry also gives a simple load-transfer check. A force $\mathbf{F}_1 = \begin{bmatrix}0 & 100 & 0\end{bmatrix}^T$ applied at node 1 is equivalent at the reference point to

$$\mathbf{F}_{r,1} = \begin{bmatrix}0 & 100 & 0\end{bmatrix}^T, \qquad \mathbf{M}_{r,1} = \mathbf{r}_1 \times \mathbf{F}_1 = \begin{bmatrix}100 & 0 & -100\end{bmatrix}^T. \tag{5.43}$$

The example therefore illustrates both sides of the same transformation: rigid kinematics map reference motion to the dependent nodes, while the transpose maps dependent loads back to an equivalent force and moment at the reference point.

The same spider can be assembled from the data normally used to define it: a node-coordinate table, one master/reference node ID and its retained DOFs, and the slave/dependent node IDs with their selected DOFs. The variable names `master` and `slave` are used here because they mirror common solver input syntax; the chapter terminology remains reference and dependent. The returned `slave_map` records the row ordering of the transformation, so loads can be assigned by node ID and DOF rather than by hard-coded vector indices.

```python
import numpy as np


def build_rbe2(node_ids, coordinates, master_id, master_dofs,
               slave_ids, slave_dofs):
    x_master = node_position(node_ids, coordinates, master_id)
    columns = [dof - 1 for dof in master_dofs]

    rows = []
    slave_map = []
    for nid, dofs in zip(slave_ids, slave_dofs):
        r = node_position(node_ids, coordinates, nid) - x_master
        for dof in dofs:
            rows.append(rigid_dof_row(r, dof)[columns])
            slave_map.append((nid, dof))

    return np.vstack(rows), slave_map


node_ids = np.array([100, 1, 2, 3, 4], dtype=int)
coordinates = np.array([
    [1.0, 1.0, 1.0],  # master/reference node 100
    [0.0, 0.0, 0.0],
    [2.0, 0.0, 0.0],
    [2.0, 2.0, 0.0],
    [0.0, 2.0, 0.0],
])

D, slave_map = build_rbe2(
    node_ids, coordinates,
    master_id=100,
```

```
    master_dofs=(1, 2, 3, 4, 5, 6),
    slave_ids=(1, 2, 3, 4),
    slave_dofs=[(1, 2, 3, 4, 5, 6)] * 4,
)

q_master = np.array([0.1, 0.0, 0.0, 0.0, 0.0, 0.01])
q_slave = D @ q_master
print(q_slave.reshape(4, 6)[:, :3])

# Apply Fy = 100 N at slave node 1.
f_slave = np.zeros(len(slave_map))
f_slave[slave_map.index((1, 2))] = 100.0
f_master = D.T @ f_slave
print(f_master)
# [   0.  100.    0.  100.    0. -100.]
```

The two products `D @ q_master` and `D.T @ f_slave` are the kinematic and virtual-work sides of the same transformation. The matrix itself is constructed entirely from the supplied IDs, coordinates and DOF lists; changing the spider geometry therefore requires changing only those inputs.

5.6.6 Elimination and Congruent Transformation

The algebraic reduction is exactly the dependent-DOF transformation developed in Chapter 3. It is repeated here only in the form needed to show the structural consequence of the RBE2.

Partition the complete displacement vector into unconstrained DOFs $\mathbf{u}_u$, reference DOFs $\mathbf{q}_r$ and dependent DOFs $\mathbf{q}_d$. With Eq. (5.27),

$$\underbrace{\begin{bmatrix} \mathbf{u}_u \\ \mathbf{q}_r \\ \mathbf{q}_d \end{bmatrix}}_{\mathbf{u}} = \underbrace{\begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \mathbf{D} \end{bmatrix}}_{\mathbf{T}} \underbrace{\begin{bmatrix} \mathbf{u}_u \\ \mathbf{q}_r \end{bmatrix}}_{\hat{\mathbf{u}}}. \tag{5.44}$$

Substitution into the assembled equilibrium equations and preservation of virtual work gives the familiar congruent transformation

$$\boxed{\hat{\mathbf{K}} = \mathbf{T}^T\mathbf{K}\mathbf{T}, \qquad \hat{\mathbf{f}} = \mathbf{T}^T\mathbf{f}, \qquad \hat{\mathbf{K}}\hat{\mathbf{u}} = \hat{\mathbf{f}}.} \tag{5.45}$$

If

$$\mathbf{K} = \begin{bmatrix} \mathbf{K}_{uu} & \mathbf{K}_{ur} & \mathbf{K}_{ud} \\ \mathbf{K}_{ru} & \mathbf{K}_{rr} & \mathbf{K}_{rd} \\ \mathbf{K}_{du} & \mathbf{K}_{dr} & \mathbf{K}_{dd} \end{bmatrix}, \qquad \mathbf{f} = \begin{bmatrix} \mathbf{f}_u \\ \mathbf{f}_r \\ \mathbf{f}_d \end{bmatrix}, \tag{5.46}$$

then the reduced blocks are

$$\begin{aligned} \hat{\mathbf{K}}_{ur} &= \mathbf{K}_{ur} + \mathbf{K}_{ud}\mathbf{D}, \\ \hat{\mathbf{K}}_{rr} &= \mathbf{K}_{rr} + \mathbf{K}_{rd}\mathbf{D} + \mathbf{D}^T\mathbf{K}_{dr} + \mathbf{D}^T\mathbf{K}_{dd}\mathbf{D}, \\ \hat{\mathbf{f}}_r &= \mathbf{f}_r + \mathbf{D}^T\mathbf{f}_d. \end{aligned} \tag{5.47}$$

The final line is important: loads applied directly to dependent DOFs are not lost when those DOFs are eliminated. They are transformed to statically equivalent generalized loads at the retained reference DOFs.

For a nonhomogeneous relation

$$\mathbf{q}_d = \mathbf{D}\mathbf{q}_r + \mathbf{g}, \tag{5.48}$$

write

$$\mathbf{u} = \mathbf{T}\hat{\mathbf{u}} + \mathbf{u}_0, \qquad \mathbf{u}_0 = \begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{g} \end{bmatrix}^T. \tag{5.49}$$

The correct reduced right-hand side is then

$$\hat{\mathbf{K}}\hat{\mathbf{u}} = \mathbf{T}^T\left(\mathbf{f} - \mathbf{K}\mathbf{u}_0\right). \tag{5.50}$$

This compact expression is preferable to deriving special load corrections case by case and is identical to the general nonhomogeneous dependent-DOF result in Chapter 3.

After solving for $\hat{\mathbf{u}}$, the full displacement field is recovered from Eq. (5.44). Prescribed single-point constraints, rotated nodal bases and reaction-force recovery are then handled by the general procedures already developed in Chapters 1–3; they are not specific to RBE2.

5.6.7 Modelling Consequences and Typical Uses

RBE2 is appropriate when the neglected deformation of the coupled region is small compared with the deformation modes of interest. Typical examples include an idealized rigid insert, a stiff end plate, a rigid test fixture, or a reference point used to represent an intentionally rigid attachment.

The same coupling should be used cautiously for thin shells, compliant flanges, bearing seats and load-introduction regions. Coupling a large patch rigidly can remove precisely the local bending or ovalization that controls stresses and natural frequencies.

Common mistake – Using RBE2 merely because a force is remote from the mesh.
A remote force does not by itself imply a rigid physical connection. If the attached surface should remain deformable, an interpolation coupling such as RBE3 is generally more appropriate. RBE2 should be selected because the *kinematics are rigid*, not because the geometry happens to look like a spider.

Interactions between RBE2 regions, prescribed boundary conditions, joints and contact are treated together in Section 5.10.4, where the distinction between independent, redundant and incompatible constraints is made explicit.

5.7 RBE3 – Interpolation Coupling

5.7.1 Why RBE3 Is Different from RBE2

An RBE2 answers the question: *how must the surrounding nodes move if the connection is perfectly rigid?* An RBE3 answers the complementary question: *what generalized motion of a reference point best represents the motion of a set of surrounding nodes?*

This reversal is essential. In an RBE2, the reference motion is known and the motion of every dependent node is prescribed from it. In an RBE3, the attachment nodes remain mutually free to deform and the reference motion is constructed as a weighted interpolation of their motion. Loads applied to the reference point are then transferred back to the attachment nodes by work equivalence. The interpolation therefore introduces kinematic relations, but no material stiffness of its own.

RBE3 details vary between finite element programs: implementations may allow multiple attachment-DOF groups, different weights and selected reference DOFs. The derivation below uses the clearest common case, in which three translational DOFs at each attachment node recover six generalized reference translations and rotations. It contains the governing principles of the usual engineering RBE3 spider.

Figure 5.3 summarizes this reversal visually.

5.7.2 Weighted Least-Squares Interpolation

Let attachment node i have initial position $\mathbf{x}_i$, translational displacement $\mathbf{u}_i$ and positive weight w_i. The weighted centroid of the attachment set is

$$\mathbf{x}_c = \frac{\sum_i w_i \mathbf{x}_i}{\sum_i w_i}, \qquad \mathbf{r}_i = \mathbf{x}_i - \mathbf{x}_c. \tag{5.51}$$

The centroid is not introduced merely as a convenient geometric point. Because

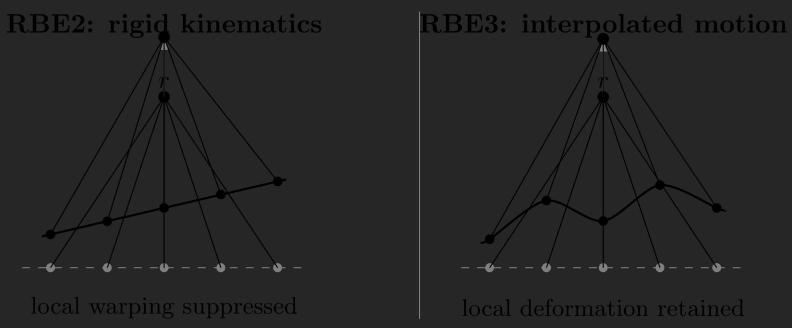


Figure 5.3: Conceptual difference between RBE2 and RBE3. The RBE2 prescribes the attachment-node motion from the reference motion and therefore suppresses relative deformation in the coupled DOFs. The RBE3 obtains a generalized reference motion from the attachment nodes, so local deformation modes can remain. The sketches are schematic rather than a literal depiction of a particular solver implementation.

$$\sum_i w_i \mathbf{r}_i = \mathbf{0}, \tag{5.52}$$

it decouples the weighted average translation from the best-fit rotation.

For small rotations, a rigid motion fitted to attachment node i would predict

$$\mathbf{u}_i^{\text{fit}} = \mathbf{u}_c + \theta \times \mathbf{r}_i. \tag{5.53}$$

Introduce the skew-symmetric matrix $\mathbf{S}(\mathbf{r})$ such that $\mathbf{S}(\mathbf{r})\mathbf{a} = \mathbf{r} \times \mathbf{a}$. Since $\theta \times \mathbf{r} = -\mathbf{S}(\mathbf{r})\theta$, Eq. (5.53) becomes

$$\mathbf{u}_i^{\text{fit}} = \underbrace{\begin{bmatrix} \mathbf{I} & -\mathbf{S}(\mathbf{r}_i) \end{bmatrix}}_{\mathbf{A}_i} \underbrace{\begin{bmatrix} \mathbf{u}_c \\ \theta \end{bmatrix}}_{\mathbf{q}_c}. \tag{5.54}$$

Stacking all n attachment translations gives

$$\mathbf{u}_a = \begin{bmatrix} \mathbf{u}_1 \\ \mathbf{u}_2 \\ \vdots \\ \mathbf{u}_n \end{bmatrix}, \qquad \mathbf{A} = \begin{bmatrix} \mathbf{A}_1 \\ \mathbf{A}_2 \\ \vdots \\ \mathbf{A}_n \end{bmatrix}, \tag{5.55}$$

and define the block-diagonal weighting matrix

$$\mathbf{W} = \operatorname{diag}\left(w_1\mathbf{I}, w_2\mathbf{I}, \ldots, w_n\mathbf{I}\right). \tag{5.56}$$

The RBE3 reference motion is obtained by fitting the rigid-motion field to the actual nodal translations in a weighted least-squares sense. Thus $\mathbf{q}_c$ minimizes

$$\Phi(\mathbf{q}_c) = \frac{1}{2}(\mathbf{u}_a - \mathbf{A}\mathbf{q}_c)^T\mathbf{W}(\mathbf{u}_a - \mathbf{A}\mathbf{q}_c). \tag{5.57}$$

Stationarity gives the normal equations

$$\mathbf{A}^T\mathbf{W}\mathbf{A}\mathbf{q}_c = \mathbf{A}^T\mathbf{W}\mathbf{u}_a. \tag{5.58}$$

Because the origin was chosen at the weighted centroid, $\mathbf{A}^T\mathbf{W}\mathbf{A}$ takes the particularly simple form

$$\mathbf{A}^T\mathbf{W}\mathbf{A} = \begin{bmatrix} w_\Sigma\mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{J}_w \end{bmatrix}, \qquad w_\Sigma = \sum_i w_i, \tag{5.59}$$

where

$$\mathbf{J}_w = \sum_i w_i \left[(\mathbf{r}_i^T\mathbf{r}_i)\mathbf{I} - \mathbf{r}_i\mathbf{r}_i^T\right]. \tag{5.60}$$

The matrix $\mathbf{J}_w$ has the same geometric form as an inertia tensor. The least-squares translation and rotation are therefore

$$\mathbf{u}_c = \frac{1}{w_\Sigma} \sum_i w_i \mathbf{u}_i, \tag{5.61}$$

$$\mathbf{J}_w \theta = \sum_i w_i \, \mathbf{r}_i \times (\mathbf{u}_i - \mathbf{u}_c). \tag{5.62}$$

Equation (5.60) also exposes an important geometric requirement. The attachment set must span the motions that the reference point is expected to represent. A collinear set of translational nodes cannot identify rotation about its own line, because the corresponding component of $\mathbf{J}_w$ is singular.

When $\mathbf{J}_w$ is nonsingular, the centroidal generalized motion can be written as

$$\mathbf{q}_c = \underbrace{(\mathbf{A}^T \mathbf{W} \mathbf{A})^{-1} \mathbf{A}^T \mathbf{W}}_{\mathbf{H}_c} \, \mathbf{u}_a. \tag{5.63}$$

The physical reference point need not coincide with the weighted centroid. Let

$$\mathbf{d} = \mathbf{x}_r - \mathbf{x}_c. \tag{5.64}$$

Its translation is

$$\mathbf{u}_r = \mathbf{u}_c + \theta \times \mathbf{d} = \mathbf{u}_c - \mathbf{S}(\mathbf{d})\theta, \tag{5.65}$$

while its rotation is the same fitted rotation θ. Therefore

$$\underbrace{\begin{bmatrix} \mathbf{u}_r \\ \theta \end{bmatrix}}_{\mathbf{q}_r} = \underbrace{\begin{bmatrix} \mathbf{I} & -\mathbf{S}(\mathbf{d}) \\ \mathbf{0} & \mathbf{I} \end{bmatrix}}_{\mathbf{P}} \mathbf{q}_c. \tag{5.66}$$

Combining Eqs. (5.63) and (5.66) gives the complete interpolation relation

$$\boxed{\mathbf{q}_r = \mathbf{H}\mathbf{u}_a, \qquad \mathbf{H} = \mathbf{P}\mathbf{H}_c. \qquad (5.67)}$$

The contrast with RBE2 is now explicit. The RBE2 matrix $\mathbf{D}$ maps a small set of reference DOFs to many attachment DOFs. The RBE3 matrix $\mathbf{H}$ maps many attachment DOFs to a small set of generalized reference DOFs.

5.7.3 Load Transfer from Virtual Work

The cleanest derivation of RBE3 load distribution does not start from an assumed nodal-force pattern. It follows directly from virtual work. Let the generalized reference load be

$$\mathbf{f}_r = \begin{bmatrix} \mathbf{F}_r \\ \mathbf{M}_r \end{bmatrix}. \tag{5.68}$$

For compatible virtual displacements,

$$\delta\mathbf{q}_r = \mathbf{H}\delta\mathbf{u}_a. \tag{5.69}$$

Equating virtual work at the reference point and at the attachment nodes,

$$\mathbf{f}_r^T\delta\mathbf{q}_r = \mathbf{f}_a^T\delta\mathbf{u}_a, \tag{5.70}$$

immediately gives

$$\boxed{\mathbf{f}_a = \mathbf{H}^T\mathbf{f}_r. \qquad (5.71)}$$

Thus the same interpolation that recovers the generalized motion also defines the work-equivalent load distribution. No spring stiffness has been introduced and no rigid plate has been imposed. The attachment nodes are coupled only through the selected generalized reference motion.

5.7.4 Worked Example – RBE3 Coefficients

To make the difference from RBE2 as direct as possible, consider exactly the same geometry used in the RBE2 worked example. The reference point is

$$\mathbf{X}_r = \begin{bmatrix}1 & 1 & 1\end{bmatrix}^T, \tag{5.72}$$

and the four attachment nodes are

$$\begin{aligned}\mathbf{X}_1 &= \begin{bmatrix}0 & 0 & 0\end{bmatrix}^T, & \mathbf{X}_2 &= \begin{bmatrix}2 & 0 & 0\end{bmatrix}^T, \\ \mathbf{X}_3 &= \begin{bmatrix}2 & 2 & 0\end{bmatrix}^T, & \mathbf{X}_4 &= \begin{bmatrix}0 & 2 & 0\end{bmatrix}^T.\end{aligned} \tag{5.73}$$

Let all four nodes have unit weight. Their weighted centroid is

$$\mathbf{X}_c = \begin{bmatrix}1 & 1 & 0\end{bmatrix}^T, \tag{5.74}$$

and the centroidal position vectors are

$$\begin{aligned}\mathbf{r}_1 &= \begin{bmatrix}-1 & -1 & 0\end{bmatrix}^T, & \mathbf{r}_2 &= \begin{bmatrix}1 & -1 & 0\end{bmatrix}^T, \\ \mathbf{r}_3 &= \begin{bmatrix}1 & 1 & 0\end{bmatrix}^T, & \mathbf{r}_4 &= \begin{bmatrix}-1 & 1 & 0\end{bmatrix}^T.\end{aligned} \tag{5.75}$$

For node 1, Eq. (5.53) gives

$$\begin{bmatrix}u_1 \\ v_1 \\ w_1\end{bmatrix}^{\text{fit}} = \underbrace{\begin{bmatrix}1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & -1 & 1 & 0\end{bmatrix}}_{\mathbf{A}_1} \begin{bmatrix}u_c \\ v_c \\ w_c \\ \varphi \\ \psi \\ \theta\end{bmatrix}. \tag{5.76}$$

The remaining three geometric blocks are

$$\mathbf{A}_2 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & -1 & -1 & 0 \end{bmatrix}, \qquad \mathbf{A}_3 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & -1 & 0 \end{bmatrix}, \tag{5.77}$$

$$\mathbf{A}_4 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix}. \tag{5.78}$$

Stacking the four blocks produces the 12×6 matrix $\mathbf{A}$. With equal weights, $\mathbf{W} = \mathbf{I}$ and the normal matrix is

$$\mathbf{A}^T \mathbf{A} = \operatorname{diag}(4, 4, 4, 4, 4, 8). \tag{5.79}$$

The simple diagonal form is a direct consequence of choosing the weighted centroid as the origin of the rigid-motion fit. Inverting Eq. (5.79) gives the centroidal interpolation matrix

$$\underbrace{\begin{bmatrix} u_c \\ v_c \\ w_c \\ \varphi \\ \psi \\ \theta \end{bmatrix}}_{\mathbf{q}_c} = \underbrace{\begin{bmatrix} \frac{1}{4} & 0 & 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} & 0 & 0 \\ 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} & 0 \\ 0 & 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} \\ 0 & 0 & -\frac{1}{4} & 0 & 0 & -\frac{1}{4} & 0 & 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} \\ 0 & 0 & \frac{1}{4} & 0 & 0 & -\frac{1}{4} & 0 & 0 & -\frac{1}{4} & 0 & 0 & \frac{1}{4} \\ \frac{1}{8} & -\frac{1}{8} & 0 & \frac{1}{8} & \frac{1}{8} & 0 & -\frac{1}{8} & \frac{1}{8} & 0 & -\frac{1}{8} & -\frac{1}{8} & 0 \end{bmatrix}}_{\mathbf{H}_c} \underbrace{\begin{bmatrix} u_1 \\ v_1 \\ w_1 \\ u_2 \\ v_2 \\ w_2 \\ u_3 \\ v_3 \\ w_3 \\ u_4 \\ v_4 \\ w_4 \end{bmatrix}}_{\mathbf{u}_a}. \tag{5.80}$$

Several rows can be read directly. For example,

$$
\begin{aligned}
u_c &= \frac{u_1 + u_2 + u_3 + u_4}{4}, \\
w_c &= \frac{w_1 + w_2 + w_3 + w_4}{4}, \\
\varphi &= \frac{-w_1 - w_2 + w_3 + w_4}{4}, \\
\theta &= \frac{u_1 - v_1 + u_2 + v_2 - u_3 + v_3 - u_4 - v_4}{8}.
\end{aligned}
\tag{5.81}
$$

The physical reference point lies one unit above the centroid,

$$
\mathbf{d} = \mathbf{X}_r - \mathbf{X}_c = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}^T. \tag{5.82}
$$

Therefore

$$
u_r = u_c + \psi, \qquad v_r = v_c - \varphi, \qquad w_r = w_c, \tag{5.83}
$$

and Eq. (5.66) yields the complete 6×12 reference interpolation matrix

$$
\underbrace{\begin{bmatrix} u_r \\ v_r \\ w_r \\ \varphi_r \\ \psi_r \\ \theta_r \end{bmatrix}}_{\mathbf{q}_r}
=
\underbrace{\begin{bmatrix}
\frac{1}{4} & 0 & \frac{1}{4} & \frac{1}{4} & 0 & -\frac{1}{4} & \frac{1}{4} & 0 & -\frac{1}{4} & \frac{1}{4} & 0 & \frac{1}{4} \\
0 & \frac{1}{4} & \frac{1}{4} & 0 & \frac{1}{4} & \frac{1}{4} & 0 & \frac{1}{4} & -\frac{1}{4} & 0 & \frac{1}{4} & -\frac{1}{4} \\
0 & 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} \\
0 & 0 & -\frac{1}{4} & 0 & 0 & -\frac{1}{4} & 0 & 0 & \frac{1}{4} & 0 & 0 & \frac{1}{4} \\
0 & 0 & \frac{1}{4} & 0 & 0 & -\frac{1}{4} & 0 & 0 & -\frac{1}{4} & 0 & 0 & \frac{1}{4} \\
\frac{1}{8} & -\frac{1}{8} & 0 & \frac{1}{8} & \frac{1}{8} & 0 & -\frac{1}{8} & \frac{1}{8} & 0 & -\frac{1}{8} & -\frac{1}{8} & 0
\end{bmatrix}}_{\mathbf{H}}
\underbrace{\begin{bmatrix} u_1 \\ v_1 \\ w_1 \\ u_2 \\ v_2 \\ w_2 \\ u_3 \\ v_3 \\ w_3 \\ u_4 \\ v_4 \\ w_4 \end{bmatrix}}_{\mathbf{u}_a}.
\tag{5.84}
$$

This is the RBE3 counterpart of the RBE2 matrix in Eq. (5.39), but the mapping is reversed: twelve attachment translations determine six generalized reference motions.

The transpose gives the corresponding load distribution. For a pure normal force F_z applied at the reference point,

$$\mathbf{F}_1 = \mathbf{F}_2 = \mathbf{F}_3 = \mathbf{F}_4 = \begin{bmatrix}0 & 0 & F_z/4\end{bmatrix}^T. \tag{5.85}$$

For a pure reference moment M_z, Eq. (5.71) gives

$$\begin{aligned}\mathbf{F}_1 &= \begin{bmatrix}M_z/8 & -M_z/8 & 0\end{bmatrix}^T, & \mathbf{F}_2 &= \begin{bmatrix}M_z/8 & M_z/8 & 0\end{bmatrix}^T,\\ \mathbf{F}_3 &= \begin{bmatrix}-M_z/8 & M_z/8 & 0\end{bmatrix}^T, & \mathbf{F}_4 &= \begin{bmatrix}-M_z/8 & -M_z/8 & 0\end{bmatrix}^T.\end{aligned} \tag{5.86}$$

The four forces sum to zero, while

$$\sum_i \mathbf{r}_i \times \mathbf{F}_i = \begin{bmatrix}0 & 0 & M_z\end{bmatrix}^T. \tag{5.87}$$

The offset of the reference point is also handled automatically. A force $\mathbf{F}_r = \begin{bmatrix}F_x & 0 & 0\end{bmatrix}^T$ applied one unit above the surface produces

$$\begin{aligned}\mathbf{F}_1 &= \begin{bmatrix}F_x/4 & 0 & F_x/4\end{bmatrix}^T, & \mathbf{F}_2 &= \begin{bmatrix}F_x/4 & 0 & -F_x/4\end{bmatrix}^T,\\ \mathbf{F}_3 &= \begin{bmatrix}F_x/4 & 0 & -F_x/4\end{bmatrix}^T, & \mathbf{F}_4 &= \begin{bmatrix}F_x/4 & 0 & F_x/4\end{bmatrix}^T.\end{aligned} \tag{5.88}$$

The nodal forces recover both the applied force and the moment generated by its one-unit lever arm about the surface centroid,

$$\sum_i \mathbf{F}_i = \begin{bmatrix}F_x & 0 & 0\end{bmatrix}^T, \qquad \sum_i \mathbf{r}_i \times \mathbf{F}_i = \begin{bmatrix}0 & F_x & 0\end{bmatrix}^T. \tag{5.89}$$

Finally, the example also demonstrates what the RBE3 does *not* constrain. Consider the local warping pattern

$$w_1 = \delta, \qquad w_2 = -\delta, \qquad w_3 = \delta, \qquad w_4 = -\delta, \tag{5.90}$$

with all other attachment translations zero. Substitution into Eq. (5.84) gives

$$\mathbf{q}_r = \mathbf{0}. \tag{5.91}$$

The four nodes can therefore undergo this relative warping without moving the reference point. An RBE2 applied to the same four nodes would suppress this mode completely. This is the central mechanical distinction between a rigid coupling and an interpolation coupling.

The following function turns the engineering definition directly into an interpolation matrix. The inputs are a node-coordinate table, the slave/reference node ID and retained DOFs, the master/attachment node IDs and selected DOFs, and one weight per master node. From these it constructs the weighted centroid, rigid-fit matrix, normal equations and final interpolation $\mathbf{H}$. To remain consistent with the derivation above, this compact function permits only translational master DOFs 1–3; solver-specific RBE3 definitions may support additional component choices. The small normal system is solved by the explicit elimination routine introduced in the RBE1 example rather than by `numpy.linalg`.

```python
import numpy as np


def build_rbe3(node_ids, coordinates, slave_id, slave_dofs,
               master_ids, master_dofs, master_weights):
    if not (len(master_ids) == len(master_dofs) == len(master_weights)):
        raise ValueError("master arrays must have the same length")

    # One scalar weight per master node; it is applied to every
    # selected DOF of that node in this compact implementation.
    x_centroid = np.zeros(3)
    weight_sum = 0.0
    for nid, weight in zip(master_ids, master_weights):
        x_centroid += weight * node_position(
            node_ids, coordinates, nid
        )
        weight_sum += weight
    if weight_sum == 0.0:
```

```
            raise ValueError("sum of master weights must be non-zero")
        x_centroid /= weight_sum

        rows = []
        scalar_weights = []
        master_map = []
        for nid, dofs, weight in zip(
            master_ids, master_dofs, master_weights
        ):
            if weight <= 0.0:
                raise ValueError("master weights must be positive")
            r = node_position(node_ids, coordinates, nid) - x_centroid
            for dof in dofs:
                if dof not in (1, 2, 3):
                    raise ValueError("this RBE3 example uses master DOFs 1..3")
                rows.append(rigid_dof_row(r, dof))
                scalar_weights.append(weight)
                master_map.append((nid, dof))

        A = np.vstack(rows)
        w = np.array(scalar_weights, dtype=float)

        normal = A.T @ (w[:, None] * A)
        rhs = A.T * w
        H_centroid = solve_small_system(normal, rhs)

        r_slave = (
            node_position(node_ids, coordinates, slave_id) - x_centroid
        )
        S = np.vstack([
            rigid_dof_row(r_slave, dof) for dof in slave_dofs
        ])

        H = S @ H_centroid
        return H, master_map, x_centroid


node_ids = np.array([100, 1, 2, 3, 4], dtype=int)
coordinates = np.array([
    [1.0, 1.0, 1.0],  # slave/reference node 100
    [0.0, 0.0, 0.0],
```

```
    [2.0, 0.0, 0.0],
    [2.0, 2.0, 0.0],
    [0.0, 2.0, 0.0],
])

H, master_map, x_centroid = build_rbe3(
    node_ids, coordinates,
    slave_id=100,
    slave_dofs=(1, 2, 3, 4, 5, 6),
    master_ids=(1, 2, 3, 4),
    master_dofs=[(1, 2, 3)] * 4,
    master_weights=(1.0, 1.0, 1.0, 1.0),
)

# Pure normal force: 400 N becomes four 100 N nodal forces.
f_slave = np.array([0.0, 0.0, 400.0, 0.0, 0.0, 0.0])
f_master = H.T @ f_slave
print(f_master.reshape(4, 3))

# Checkerboard warping does not move the slave/reference node.
u_master = np.zeros(len(master_map))
warp = {1: 1.0, 2: -1.0, 3: 1.0, 4: -1.0}
for i, (nid, dof) in enumerate(master_map):
    if dof == 3:
        u_master[i] = warp[nid]

q_slave = H @ u_master
print(q_slave)
# [0. 0. 0. 0. 0. 0.]
```

The returned `master_map` makes the column ordering of $\mathbf{H}$ explicit. The two essential RBE3 operations then remain visible: `H @ u_master` interpolates attachment motion to the slave/reference coordinates, while `H.T @ f_slave` distributes a generalized load back to the selected master DOFs. The zero checkerboard result is the numerical counterpart of the interpolation null space discussed above.

5.7.5 RBE3 as a Deformable Distributed Support

One of the most useful applications of an interpolation coupling is to connect a spring, bushing or joint to an extended *deformable* region. The reference point then acts as a generalized interface: it sees the weighted translation and rotation of the attachment patch, while the patch itself is not forced to move as a rigid body.

Let the RBE3 interpolation be

$$\mathbf{q}_r = \mathbf{H}\mathbf{u}_a, \tag{5.92}$$

and let a six-degree-of-freedom spring or bushing connect the reference point to a support with generalized motion $\mathbf{q}_g$. For a linear support stiffness $\mathbf{K}_s$, the stored energy is

$$\Pi_s = \frac{1}{2}(\mathbf{H}\mathbf{u}_a - \mathbf{q}_g)^T \mathbf{K}_s (\mathbf{H}\mathbf{u}_a - \mathbf{q}_g). \tag{5.93}$$

Differentiation with respect to the attachment DOFs gives the internal restoring-force contribution of the support (using the usual finite element internal-force sign convention),

$$\mathbf{f}_a^{(s)} = \mathbf{H}^T \mathbf{K}_s (\mathbf{H}\mathbf{u}_a - \mathbf{q}_g), \tag{5.94}$$

and, for a fixed support $\mathbf{q}_g = \mathbf{0}$, the corresponding contribution to the attachment stiffness is

$$\boxed{\mathbf{K}_a^{(s)} = \mathbf{H}^T \mathbf{K}_s \mathbf{H}}. \tag{5.95}$$

Equation (5.95) is the mathematical reason why an RBE3 can be used to create a deformable distributed support. The spring or bushing supplies the constitutive stiffness; the RBE3 only maps that stiffness onto the surrounding mesh. No fictitious plate stiffness is inserted between the attachment nodes.

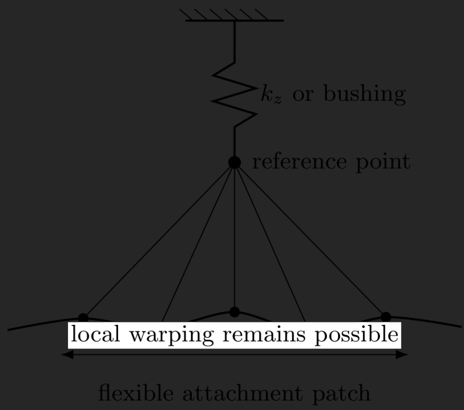


Figure 5.4: RBE3 used as a deformable distributed support. The interpolation maps the generalized motion of the flexible patch to a reference point; the spring or bushing supplies the actual stiffness. Modes lying in the null space of the interpolation are not directly stiffened by the support.

The mechanical picture corresponding to this projection is shown in Figure 5.4.

An especially important consequence follows immediately. If a local deformation mode $\mathbf{u}_0$ satisfies

$$\mathbf{H}\mathbf{u}_0 = \mathbf{0}, \tag{5.96}$$

then

$$\mathbf{u}_0^T \mathbf{K}_a^{(s)} \mathbf{u}_0 = 0. \tag{5.97}$$

The support therefore stores no energy in this deformation mode. Only the generalized translations and rotations represented by $\mathbf{H}$ are supported; local warping that does not change those generalized quantities remains governed by the stiffness of the actual structure.

Worked Example – One Translational Spring on the Four-Node Patch

The four-node RBE3 example of Eq. (5.84) gives for the normal translation of the reference point

$$w_r = \frac{1}{4}(w_1 + w_2 + w_3 + w_4). \tag{5.98}$$

Attach a vertical spring of stiffness k_z from the reference point to ground. Its energy is

$$\Pi_s = \frac{1}{2}k_z w_r^2 = \frac{k_z}{32}(w_1 + w_2 + w_3 + w_4)^2. \tag{5.99}$$

The spring therefore contributes the following internal force and stiffness to the four vertical attachment DOFs:

$$\begin{bmatrix} F_{z1} \\ F_{z2} \\ F_{z3} \\ F_{z4} \end{bmatrix} = \underbrace{\frac{k_z}{16}\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}}_{\mathbf{K}_w^{(s)}} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ w_4 \end{bmatrix}. \tag{5.100}$$

This matrix has rank one. It resists only the average normal displacement of the patch. The remaining three independent combinations of w_i are invisible to this spring. For example, the checkerboard warping mode already considered in the RBE3 worked example,

$$\mathbf{w} = \delta \begin{bmatrix} 1 & -1 & 1 & -1 \end{bmatrix}^T, \tag{5.101}$$

satisfies

$$\mathbf{K}_w^{(s)} \mathbf{w} = \mathbf{0}. \tag{5.102}$$

Thus even an extremely stiff vertical spring can restrain the *mean* motion of the surface while leaving this local warping mode untouched. The structural shells or solids still determine the shape of the attachment patch. An RBE2 connection to the same nodes would instead eliminate the relative warping by kinematic constraint before the spring stiffness was even considered.

More generally, if $\mathbf{K}_s$ contains stiffness in all six generalized directions, the distributed contribution $\mathbf{H}^T\mathbf{K}_s\mathbf{H}$ has rank no greater than six, regardless of how many attachment nodes are used. The support can therefore restrain up to six generalized motions without turning an arbitrarily fine attachment mesh into a six-degree-of-freedom rigid plate.

Coincident Reference Nodes and Bushings

A practical construction uses two coincident reference nodes. One belongs to the RBE3 spider; the other belongs to ground, another flexible body, or a second spider. A spring or bushing between the coincident points then defines which relative generalized motions are compliant and which are nearly rigid.

For two deformable attachment regions,

$$\mathbf{q}_1 = \mathbf{H}_1\mathbf{u}_1, \qquad \mathbf{q}_2 = \mathbf{H}_2\mathbf{u}_2, \tag{5.103}$$

and a bushing stiffness $\mathbf{K}_b$ between the coincident reference points stores

$$\Pi_b = \frac{1}{2}(\mathbf{H}_1\mathbf{u}_1 - \mathbf{H}_2\mathbf{u}_2)^T\mathbf{K}_b(\mathbf{H}_1\mathbf{u}_1 - \mathbf{H}_2\mathbf{u}_2). \tag{5.104}$$

The corresponding interface stiffness is

$$\begin{bmatrix} \mathbf{K}_{11} & \mathbf{K}_{12} \\ \mathbf{K}_{21} & \mathbf{K}_{22} \end{bmatrix} = \begin{bmatrix} \mathbf{H}_1^T\mathbf{K}_b\mathbf{H}_1 & -\mathbf{H}_1^T\mathbf{K}_b\mathbf{H}_2 \\ -\mathbf{H}_2^T\mathbf{K}_b\mathbf{H}_1 & \mathbf{H}_2^T\mathbf{K}_b\mathbf{H}_2 \end{bmatrix}. \tag{5.105}$$

This formulation is useful for bearings, elastomeric mounts, compliant joints, connector stiffnesses and condensed representations of detailed attachments.

The two surrounding surfaces may deform independently; only their selected *generalized relative motion* is seen by the bushing.

If selected spring directions are chosen very stiff rather than exactly rigid, the construction becomes a penalty-like approximation to a constraint in those directions. The stiffness should be large enough that the unwanted relative motion is negligible, but not made arbitrarily enormous: excessive stiffness can worsen matrix conditioning and unnecessarily increase the difficulty of the linear or nonlinear solve. If a direction is physically intended to be exactly constrained, an exact kinematic constraint may be preferable to an artificially huge spring constant.

Engineering note – "Deformable support" does not mean a soft RBE3.
The RBE3 contains no support stiffness that is tuned to make it deformable. The compliance comes from the attached structure and from the spring, bushing or other support element. The RBE3 defines only which generalized motion the support sees and how the corresponding reaction is distributed back to the structure. Equation (5.95) makes this separation explicit.

5.7.6 RBE3 Is Not Automatically a Pressure Load

A common modelling shortcut is to connect an RBE3 to nodes on a surface and apply a force at the reference point. The resulting nodal force field should not be confused automatically with a prescribed uniform pressure.

For a true uniform pressure p, the consistent nodal resultant follows from surface integration. A useful engineering approximation is tributary-area weighting,

$$F_i \approx pA_i, \tag{5.106}$$

where A_i is the surface area represented by node i. Consider, for example, a regular 2×2 patch of four equal bilinear quadrilateral elements. If the pressure contribution of each element is divided equally among its four corner nodes, a geometric corner node belongs to one element, a non-corner edge node to two, and the interior node to four. Their tributary areas, and therefore their nodal forces under uniform pressure, occur in the ratio

$$F_{\text{corner}} : F_{\text{edge}} : F_{\text{interior}} = 1 : 2 : 4. \tag{5.107}$$

Thus the smaller force obtained at the edge of a surface is not necessarily a poor distribution. It may simply reflect the fact that an edge node represents less loaded area than an interior node. Surface-generated distributing couplings often exploit this kind of geometric or integration-based weighting.

Figure 5.5 illustrates this tributary-area effect for the same 2×2 patch.

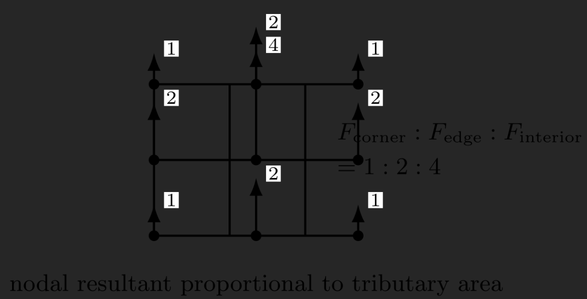


Figure 5.5: Uniform pressure on a regular 2×2 quadrilateral patch does not produce equal nodal resultants. Corner, edge and interior nodes represent different tributary areas; the arrows show their relative resultant magnitudes. An equal-weight RBE3 is therefore not, in general, equivalent to a uniform pressure load.

An explicitly defined RBE3 with equal nodal weights is different. Its translational share is equal at every selected node, regardless of how much surface area that node represents. On a regular mesh, equal weights therefore over-represent corner and edge nodes per unit tributary area. Neither the name “RBE3” nor the phrase “equal weights” is synonymous with “uniform pressure”.

Common mistake – Judging an RBE3 distribution only by equal nodal forces.
The correct question is whether the chosen weights represent the intended load path. If a uniform traction is physically important, apply a true element surface load or construct weights from the surface integration or tributary areas. If only the remote resultant matters sufficiently far from the load introduction, a simpler distribution may be entirely adequate by Saint-Venant's principle.

5.7.7 Choosing the Spider Legs

Connecting every available mesh node is rarely a goal in itself. In many industrial models, engineers deliberately attach spider legs only to representative corner, perimeter or otherwise well-spaced nodes. The reason is not that corner nodes have a special mathematical status, but that a relatively small geometric skeleton can often span the generalized translations and rotations that the coupling must transmit.

There are several practical reasons for such selections:

1. **Geometric rank matters more than node count.** The selected nodes must span the reference motions. Adding ten nodes nearly on top of one another contributes less than adding one node in a previously unrepresented direction.

2. **Higher-order meshes can contain many redundant interpolation locations.** Corner nodes often already define the low-order geometry of a face. Including every midside or finely spaced node may add many constraint coefficients while changing the recovered generalized motion very little.

3. **The interpolation equations become long.** In the translational formulation above, n attachment nodes give a matrix $\mathbf{H}$ with $3n$ columns. Each generalized reference equation can therefore couple a large part of the selected surface. Large spiders increase assembly memory, matrix connectivity and the amount of work required to process the constraints. This

cost can be more noticeable for RBE3 than for a simple rigid transformation because the weighted interpolation involves many contributing DOFs in each generalized relation.

4. **Large spiders interact more easily with other constraints.** Contact, neighbouring spiders, symmetry conditions and prescribed displacements may act on the same surface. A very broad interpolation region can then create redundant or unintended load paths.

There is no universal maximum number of RBE3 legs. Some implementations impose explicit coefficient or node-count limits, while others are constrained mainly by memory and solver architecture. The engineering rule is to use enough well-distributed attachment nodes to represent the intended generalized motion and load path, and no more merely because the mesh contains them.

This recommendation does not mean that a coarse corner-only spider is always best. If the local traction field is important, or if a curved surface must be represented accurately, more attachment points and physically meaningful weights may be required. The appropriate number of legs is therefore a modelling convergence question, not a fixed recipe.

Engineering note – More RBE3 legs do not monotonically improve a model.
Mesh convergence and coupling convergence are separate questions. Once the selected legs adequately span the intended region and load path, adding hundreds of additional attachment nodes can change little physically while increasing the cost of constraint assembly and solution.

5.8 Fixed-Distance and Other Geometric Constraints

A fixed-distance relation is fundamentally different from RBE1, RBE2 and RBE3. For two points with current positions

$$\mathbf{x}_1 = \mathbf{X}_1 + \mathbf{u}_1, \qquad \mathbf{x}_2 = \mathbf{X}_2 + \mathbf{u}_2, \tag{5.108}$$

and prescribed distance L, a convenient exact constraint is

$$g(\mathbf{u}) = \frac{1}{2}\left[(\mathbf{x}_2 - \mathbf{x}_1)^T(\mathbf{x}_2 - \mathbf{x}_1) - L^2\right] = 0. \tag{5.109}$$

Its linearization uses the current connecting vector $\mathbf{r} = \mathbf{x}_2 - \mathbf{x}_1$,

$$\delta g = \mathbf{r}^T(\delta\mathbf{u}_2 - \delta\mathbf{u}_1). \tag{5.110}$$

Unlike the ordinary small-displacement RBE2 relation, the exact constraint is nonlinear because its direction changes as the points rotate. A solver may enforce Eq. (5.109) through Lagrange multipliers, penalty or augmented Lagrangian techniques. A multiplier implementation is especially transparent: the multiplier is the axial constraint force and the constraint adds an extra unknown rather than eliminating a dependent DOF.

This is why a contact- or constraint-element implementation of a fixed distance can behave numerically very differently from a classical RBE even if both are used to make a connection appear “rigid” in a graphical model. The kinematic relation and the enforcement technique must be identified separately.

Joints can be constructed in the same way by combining geometric constraints. A revolute joint removes five relative rigid-body motions and leaves one rotation; a spherical joint enforces point coincidence but leaves relative rotation; a cylindrical joint leaves one translation and one rotation along a common axis. Dedicated joint elements merely package these relations, often with nonlinear updates, limits, friction or stiffness added.

5.8.1 Worked Example – Exact Distance versus the Initial Tangent

Consider a two-dimensional link of prescribed length

$$L = 100 \text{ mm} \tag{5.111}$$

with point 1 fixed at the origin and point 2 initially at

$$\mathbf{X}_2 = \begin{bmatrix} 100 & 0 \end{bmatrix}^T \text{ mm}. \tag{5.112}$$

Suppose point 2 is moved transversely by

$$u_{2y} = 20 \text{ mm}. \tag{5.113}$$

The exact fixed-distance constraint requires the current point to remain on the circle $x_2^2 + y_2^2 = L^2$. Its horizontal coordinate must therefore be

$$x_2 = \sqrt{L^2 - u_{2y}^2} = \sqrt{100^2 - 20^2} = 97.9796 \text{ mm}, \tag{5.114}$$

so that the accompanying axial displacement is

$$\boxed{u_{2x} = x_2 - L = -2.0204 \text{ mm}}. \tag{5.115}$$

This axial correction is not caused by axial elasticity; it is required purely by geometry. The exact constraint rotates with the link.

Now linearize the same relation in the initial configuration. With $\mathbf{r}_0 = [L, 0]^T$, Eq. (5.109) gives, to first order,

$$\mathbf{r}_0^T \delta\mathbf{u}_2 = 0 \quad \Longrightarrow \quad L\,\delta u_{2x} = 0. \tag{5.116}$$

The initial tangent therefore requires only

$$\delta u_{2x} = 0 \tag{5.117}$$

and places no first-order restriction on δu_{2y}. If the full 20 mm transverse motion is applied while this initial tangent is kept fixed, the predicted point becomes $(100, 20)$ mm and its distance from the origin is

$$L_{\mathrm{lin}} = \sqrt{100^2 + 20^2} = 101.9804 \text{ mm}. \tag{5.118}$$

Thus the linearized relation has allowed a 1.9804 mm length error, about 1.98% of the prescribed length. A nonlinear constraint avoids this by relinearizing about the current direction

$$\mathbf{n} = \frac{\mathbf{r}}{L} = \begin{bmatrix} 0.9798 & 0.2000 \end{bmatrix}^T, \tag{5.119}$$

so the current incremental condition is

$$\boxed{\mathbf{n}^T(\delta\mathbf{u}_2 - \delta\mathbf{u}_1) = 0}. \tag{5.120}$$

This is also the form seen by a linear perturbation or modal analysis about the deformed operating point: the nonlinear geometric relation is replaced by its current tangent. The example therefore illustrates why a fixed-distance link can be perfectly appropriate in a nonlinear transient analysis while its ordinary eigenshape extraction requires a frozen, linearized operating configuration.

The same comparison can be wrapped in a reusable geometric check. The function accepts a coordinate table, the two node IDs and trial nodal displacement vectors, and returns both the initial-tangent residual and the exact change of length. A second scalar helper evaluates the exact axial correction for the planar example:

```
import numpy as np


def fixed_distance_trial(node_ids, coordinates, node_1, node_2,
                         du_1, du_2):
    x1 = node_position(node_ids, coordinates, node_1)
```

```
    x2 = node_position(node_ids, coordinates, node_2)

    r0 = x2 - x1
    dr = np.array(du_2, dtype=float) - np.array(du_1, dtype=float)

    L0 = (r0 @ r0) ** 0.5
    r_trial = r0 + dr
    L_trial = (r_trial @ r_trial) ** 0.5

    linear_residual = r0 @ dr
    exact_length_error = L_trial - L0
    return L0, L_trial, linear_residual, exact_length_error


def exact_planar_axial_correction(length, transverse_motion):
    x_new = (length * length - transverse_motion**2) ** 0.5
    return x_new - length


node_ids = np.array([1, 2], dtype=int)
coordinates = np.array([
    [0.0,   0.0, 0.0],
    [100.0, 0.0, 0.0],
])

L0, L_trial, linear_residual, length_error = fixed_distance_trial(
    node_ids, coordinates,
    node_1=1,
    node_2=2,
    du_1=(0.0, 0.0, 0.0),
    du_2=(0.0, 20.0, 0.0),
)
ux_exact = exact_planar_axial_correction(L0, 20.0)

print("exact ux =", ux_exact, "mm")
print("linearized length =", L_trial, "mm")
print("length error =", length_error, "mm")
print("initial tangent residual =", linear_residual)
# exact ux = -2.020410... mm
# length error = 1.980390... mm
```

```
# initial tangent residual = 0.0
```

The final zero is precisely the point: the transverse displacement satisfies the *initial linear tangent* even though it violates the exact fixed-distance constraint by almost 2 mm.

> **Engineering note – A fixed-distance link is not a very stiff spring.** An exact distance constraint carries whatever axial reaction is required to remain on the constraint manifold, but it introduces no axial compliance from which that reaction could be predicted independently. A spring or truss instead relates axial force to extension through a constitutive law. The two models may look similar in a preprocessor but represent different mechanics.

5.9 Remote Couplings and Reference Points

Modern preprocessors commonly expose a *remote point*, *reference point* or *remote coupling* rather than asking the analyst to construct a classical spider manually. In the solver-independent discussion below, *reference point* denotes the generalized interface coordinates, while *remote point* is treated as one common implementation name. The most useful interpretation is therefore to regard the reference point as a set of generalized coordinates $\mathbf{q}_r$ through which loads, prescribed motions, springs or joints can be connected to an extended region of the finite element mesh.

This interpretation separates two questions:

1. **How is the mesh region coupled to the generalized point?** A rigid coupling may use a relation such as $\mathbf{u}_a = \mathbf{D}\mathbf{q}_r$, whereas a distributing coupling may use $\mathbf{q}_r = \mathbf{H}\mathbf{u}_a$.
2. **What is attached to that generalized point?** It may carry a force or moment, a prescribed displacement, a point mass, a spring or bushing, a joint, or another coupling.

The reference point is therefore best viewed as a *generalized mechanical interface*, not as a particular finite element formulation. The same apparent GUI object may be implemented internally by direct constraint equations, surface-based MPCs, contact-style constraint elements, Lagrange multipliers or other methods depending on solver and analysis type.

5.9.1 Remote Loads

For an interpolation coupling

$$\mathbf{q}_r = \mathbf{H}\mathbf{u}_a, \tag{5.121}$$

a generalized force and moment vector $\mathbf{f}_r$ applied at the remote point is transferred by virtual work as

$$\mathbf{f}_a = \mathbf{H}^T\mathbf{f}_r. \tag{5.122}$$

Thus a remote force is not merely a concentrated nodal force moved graphically away from the mesh. The coupling defines how its resultant force and moment are represented by nodal forces on the attachment region. If the remote point is offset from the surface, the lever-arm moment is automatically included in the mapping, as demonstrated by the RBE3 worked example.

For a rigid coupling, the same virtual-work argument applies in the opposite kinematic direction: $\mathbf{u}_a = \mathbf{D}\mathbf{q}_r$ gives generalized reactions $\mathbf{f}_r = \mathbf{D}^T\mathbf{f}_a$. The important difference is not the ability to carry a resultant load; it is whether the attachment region is required to follow rigid body motion while doing so.

5.9.2 Remote Displacements

A prescribed remote displacement applies a boundary condition to selected components of $\mathbf{q}_r$. With a rigid coupling this can rigidize the entire selected attachment region in the active directions. With an interpolation coupling it instead constrains only the generalized weighted motion represented by $\mathbf{H}$; local modes belonging to the nullspace of the interpolation may remain free.

This distinction is easy to miss in a graphical interface because both options may show the same reference-point symbol. A condition such as $w_r = 0$ can mean "every attached point participates in a rigid plane whose reference translation is zero" or merely "the weighted average normal translation is zero", depending on the coupling definition.

Rotational remote constraints also require sufficient geometry. A collection of coincident or collinear translational attachment nodes cannot in general define all three average rotations. The issue is geometric rank, not a missing solver setting. The same rank condition appeared in the RBE3 least-squares matrix $\mathbf{A}^T\mathbf{W}\mathbf{A}$.

5.9.3 Remote Springs, Bushings and Joints

Once the generalized coordinates $\mathbf{q}_r$ have been defined, constitutive or kinematic objects may be attached to them without modifying the surrounding mesh. The deformable-support formulation of Section 5.7.5 is the simplest example. A six-degree-of-freedom bushing acts in generalized space and its stiffness is projected back to the attachment mesh through $\mathbf{H}^T\mathbf{K}_s\mathbf{H}$.

This is an important modelling pattern. A detailed bearing seat, rubber mount or bolted interface may be represented by a flexible FE region whose generalized motion is connected to a compact spring, bushing or joint law. The connection stiffness and the load-distribution rule remain separate modelling decisions. The analyst can therefore change the support law without replacing the local mesh by an artificial rigid plate.

5.9.4 Worked Example – Deformable Remote Flange on a Bushing

Consider a square flange represented by four attachment nodes at

$$(x_i, y_i) = (-a, -a),\ (a, -a),\ (a, a),\ (-a, a), \qquad a = 50 \text{ mm}. \tag{5.123}$$

For clarity, retain only the normal nodal displacements w_i and two generalized remote coordinates: normal translation w_r and rotation θ_y. Equal weights give the interpolation

$$\begin{bmatrix} w_r \\ \theta_y \end{bmatrix} = \underbrace{\begin{bmatrix} \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4a} & -\frac{1}{4a} & -\frac{1}{4a} & \frac{1}{4a} \end{bmatrix}}_{\mathbf{H}_{w\theta}} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ w_4 \end{bmatrix}. \tag{5.124}$$

The first row measures the mean normal motion. The second row measures the best-fit rotation about y; the signs follow from $w \approx w_r - \theta_y x$. Attach the remote coordinates to ground through the diagonal bushing stiffness

$$\mathbf{K}_b = \begin{bmatrix} k_z & 0 \\ 0 & k_{\theta y} \end{bmatrix}, \qquad k_z = 40{,}000 \text{ N/mm}, \qquad k_{\theta y} = 100 \times 10^6 \text{ Nmm/rad}. \tag{5.125}$$

Apply at the remote point

$$F_z = 8{,}000 \text{ N}, \qquad M_y = 200{,}000 \text{ Nmm}. \tag{5.126}$$

In generalized coordinates the bushing response is immediately

$$\begin{bmatrix} w_r \\ \theta_y \end{bmatrix} = \mathbf{K}_b^{-1} \begin{bmatrix} F_z \\ M_y \end{bmatrix} = \begin{bmatrix} 0.200 \text{ mm} \\ 0.002 \text{ rad} \end{bmatrix}. \tag{5.127}$$

The generalized load is distributed to the four attachment nodes by virtual work,

$$\mathbf{f}_a = \mathbf{H}_{w\theta}^T \begin{bmatrix} F_z \\ M_y \end{bmatrix}. \tag{5.128}$$

Since $1/(4a) = 1/200 \text{ mm}^{-1}$, each node first receives $F_z/4 = 2{,}000$ N. The moment adds $+1{,}000$ N to the two nodes at $x = -a$ and $-1{,}000$ N to the two nodes at $x = +a$. Hence

$$\boxed{\mathbf{f}_a = \begin{bmatrix} 3000 & 1000 & 1000 & 3000 \end{bmatrix}^T \text{N}}. \tag{5.129}$$

The resultant force is 8 kN. The nodal-force couple recovers the applied moment, $M_y = 200$ kN mm. The bushing therefore sees a compact two-coordinate interface while the finite element mesh receives a statically equivalent distributed load.

The crucial distinction from a rigid remote coupling appears in the kinematics. If the four nodes were rigidly tied to the remote point, their normal displacements would be forced to the plane

$$w_i = w_r - \theta_y x_i, \tag{5.130}$$

which gives 0.300 mm at $x = -50$ mm and 0.100 mm at $x = +50$ mm. An interpolation coupling does not impose those four values. It requires only that the actual nodal field has the generalized mean and slope of Eq. (5.124). For example, the local warping field

$$\delta\mathbf{w} = \delta \begin{bmatrix} 1 & -1 & 1 & -1 \end{bmatrix}^T \tag{5.131}$$

satisfies

$$\mathbf{H}_{w\theta}\, \delta\mathbf{w} = \mathbf{0}. \tag{5.132}$$

It can therefore be superposed on the flange deformation without changing w_r, θ_y or the bushing force. The local flange stiffness determines that warping; the bushing acts only on the generalized interface coordinates. This is the mathematical content of a *deformable remote support.*

A compact function keeps the generalized interface and the bushing law visibly separate. Rather than hard-coding $\mathbf{H}_{w\theta}$, the example reuses the general `build_rbe3` function with the flange coordinates, master IDs, master DOFs and weights. The bushing response is then evaluated from the resulting interpolation and the supplied bushing matrix.

```python
import numpy as np


def remote_bushing_response(H, K_bushing, remote_load):
    q_remote = solve_small_system(K_bushing, remote_load)
    f_master = H.T @ remote_load
    K_master = H.T @ K_bushing @ H
    return q_remote, f_master, K_master


a = 50.0  # mm
node_ids = np.array([100, 1, 2, 3, 4], dtype=int)
coordinates = np.array([
    [ 0.0,  0.0, 0.0],  # remote/slave node 100
    [-a,   -a,   0.0],
    [ a,   -a,   0.0],
    [ a,    a,   0.0],
    [-a,    a,   0.0],
])

# Reuse the general RBE3 builder; retain only w and theta_y
# at the remote point while the master nodes use translations.
H, master_map, _ = build_rbe3(
    node_ids, coordinates,
    slave_id=100,
    slave_dofs=(3, 5),
    master_ids=(1, 2, 3, 4),
    master_dofs=[(1, 2, 3)] * 4,
    master_weights=(1.0, 1.0, 1.0, 1.0),
)

K_bushing = np.array([
    [40_000.0, 0.0],
    [0.0, 100e6],
])
remote_load = np.array([8_000.0, 200_000.0])

q_remote, f_master, K_master = remote_bushing_response(
    H, K_bushing, remote_load
)
```

```
print("[w_r, theta_y] =", q_remote)

z_forces = []
for i, (nid, dof) in enumerate(master_map):
    if dof == 3:
        z_forces.append((nid, float(f_master[i])))
print("nodal z forces =", z_forces)
# [(1, 3000.), (2, 1000.), (3, 1000.), (4, 3000.)]

# A local warping mode remains invisible to the generalized interface.
warp_master = np.zeros(len(master_map))
warp = {1: 1.0, 2: -1.0, 3: 1.0, 4: -1.0}
for i, (nid, dof) in enumerate(master_map):
    if dof == 3:
        warp_master[i] = warp[nid]
print("H @ warp =", H @ warp_master)
# [0. 0.]
```

The returned `K_master` is exactly the projected support stiffness $\mathbf{H}^T\mathbf{K}_b\mathbf{H}$ derived in Section 5.7.5. The same general RBE3 builder is therefore used for both load interpolation and the deformable support; only the selected slave DOFs and the attached bushing law change.

Engineering note – Separate the interface kinematics from the support law.
Changing an interpolating attachment to a rigid one changes the flange kinematics even if the same bushing matrix is retained; changing the bushing stiffness changes the support compliance without changing how flange motion is measured. A higher-level remote-point object may realize either behaviour using solver-specific equations, so "rigid" or "deformable" should be interpreted from the generated kinematics rather than assumed to be literally RBE2 or RBE3.

5.9.5 Example Implementations in Commercial Solvers

The following names are included only to help map the solver-independent concepts onto familiar software. They are not a substitute for the respective solver manuals, and the exact formulation can depend on analysis type and selected options.

Concept	**ANSYS examples**	**PERMAS examples**
General linear constraint	`CE`	`MPC_GENERAL`
Equal/coupled DOF	`CP`, `CE`	`MPC_SAME`, `MPC_ASSIGN`
Rigid coupling	`CERIG`, rigid remote behaviour	`MPC_RIGID`
Interpolation coupling	`RBE3`, deformable/distributing remote behaviour	`MPC_WLSCON`
Other specialized couplings	surface-based MPCs, joint/constraint elements	`MPC_JOIN` and other specialized MPCs

The names illustrate a broader point: a mature solver usually contains several constraint families because equal DOFs, rigid-body kinematics, weighted least-squares interpolation and nonlinear geometric relations are not the same mathematical problem. Higher-level remote coupling objects may deliberately use an implementation different from the classical command bearing the most similar name. This can be advantageous. For example, ANSYS Mechanical Remote Points are generally realized through surface-based MPC constraints rather than by simply issuing the classical `CERIG` or `RBE3` command; in a large-deflection analysis the underlying element-shape interpolation can be reformed as the configuration evolves. This is exactly why an engineering behaviour that is "RBE3-like" should not automatically be assumed to have the limitations of a particular classical RBE3 command.

5.10 Advanced Considerations

The equations developed in the preceding sections describe the intended kinematic relation, but practical behaviour also depends on the analysis in which that relation is used. Finite rotation changes the geometry of the constraint, thermal expansion can change its stress-free dimensions, eigenshape extraction requires a fixed linear tangent, and neighbouring constraints can remove motions that neither definition was intended to suppress on its own. These effects are considered together below because they determine whether a formally correct coupling remains a correct engineering model.

5.10.1 Small-Displacement and Finite-Rotation Formulations

Consider a reference point at initial position $\mathbf{x}_{r0}$ and a dependent point at

$$\mathbf{x}_{s0} = \mathbf{x}_{r0} + \mathbf{r}_0, \tag{5.133}$$

where $\mathbf{r}_0$ is the initial rigid arm. If the reference point undergoes a translation $\mathbf{u}_r$ and a finite rotation represented by the orthogonal rotation tensor $\mathbf{R}$, the exact rigid-body relation is

$$\boxed{\mathbf{x}_s = \mathbf{x}_r + \mathbf{R}\mathbf{r}_0}. \tag{5.134}$$

Since $\mathbf{x}_r = \mathbf{x}_{r0} + \mathbf{u}_r$, the corresponding dependent displacement is

$$\mathbf{u}_s = \mathbf{u}_r + (\mathbf{R} - \mathbf{I})\mathbf{r}_0. \tag{5.135}$$

The familiar linear RBE2 equation follows by linearizing the rotation tensor about the initial configuration,

$$\mathbf{R} \approx \mathbf{I} + [\theta]_\times, \tag{5.136}$$

which gives

$$\boxed{\mathbf{u}_s \approx \mathbf{u}_r + \boldsymbol{\theta} \times \mathbf{r}_0}. \tag{5.137}$$

Equation (5.137) is not the exact rigid-body motion; it is its tangent at the initial configuration. The rigid arm $\mathbf{r}_0$ remains fixed in the constraint coefficients. Consequently, a point subjected to a large rotation follows the *initial tangent* rather than the circular path implied by Eq. (5.134).

Pure translation does not cause this error: a translation contains no changing orientation and Eq. (5.137) reproduces it exactly. The limitation is specifically associated with finite rotation and other configuration-dependent geometry.

Worked Example – A Rigid Arm Rotated Through 90°

Consider a planar rigid arm of length

$$L = 100 \text{ mm} \tag{5.138}$$

with its reference point at the origin and its dependent point initially at

$$\mathbf{r}_0 = \begin{bmatrix} 100 & 0 & 0 \end{bmatrix}^T \text{ mm}. \tag{5.139}$$

Let the reference point rotate about the global z axis while remaining fixed in translation. The exact position after a rotation θ is

$$\mathbf{x}_{s,\text{exact}} = \begin{bmatrix} L\cos\theta \\ L\sin\theta \\ 0 \end{bmatrix}, \tag{5.140}$$

whereas the small-rotation constraint predicts

$$\mathbf{x}_{s,\mathrm{lin}} = \mathbf{r}_0 + \boldsymbol{\theta} \times \mathbf{r}_0 = \begin{bmatrix} L \\ L\theta \\ 0 \end{bmatrix}, \tag{5.141}$$

with θ expressed in radians. Equation (5.141) is literally the tangent line to the circular trajectory at $\theta = 0$.

For $\theta = 90° = \pi/2$, the exact rigid-body motion gives

$$\mathbf{x}_{s,\mathrm{exact}} = \begin{bmatrix} 0 \\ 100 \\ 0 \end{bmatrix} \text{mm}, \qquad \mathbf{u}_{s,\mathrm{exact}} = \begin{bmatrix} -100 \\ 100 \\ 0 \end{bmatrix} \text{mm}. \tag{5.142}$$

The linearized RBE instead gives

$$\mathbf{x}_{s,\mathrm{lin}} = \begin{bmatrix} 100 \\ 157.08 \\ 0 \end{bmatrix} \text{mm}, \qquad \mathbf{u}_{s,\mathrm{lin}} = \begin{bmatrix} 0 \\ 157.08 \\ 0 \end{bmatrix} \text{mm}. \tag{5.143}$$

The geometric difference between the exact and tangent trajectories is shown in Figure 5.6.

The predicted point is therefore not merely slightly misplaced: it remains on the vertical tangent through the initial end point rather than rotating around the reference node. The distance between the exact and tangent predictions is approximately 115.1 mm, greater than the original arm length.

The growth of this purely geometric error is summarized below. The error is the distance between Eqs. (5.140) and (5.141) for $L = 100$ mm.

Rotation	**Position error**	**Error / arm length**
1°	0.015 mm	0.015%
5°	0.381 mm	0.381%
10°	1.522 mm	1.522%
30°	13.604 mm	13.604%
90°	115.144 mm	115.144%

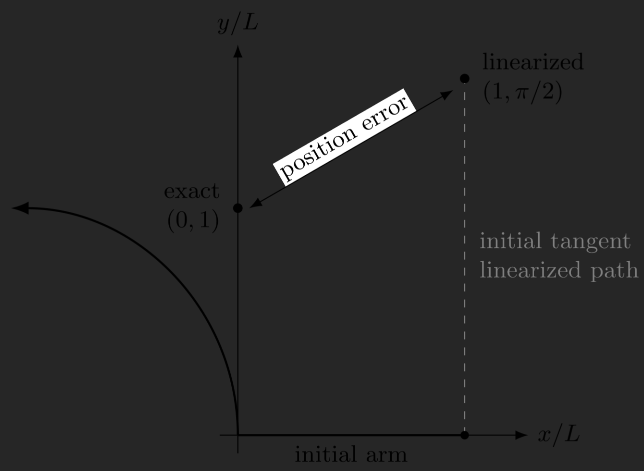


Figure 5.6: Finite rotation versus the small-rotation tangent approximation for a rigid arm. The exact dependent point follows a circular arc, whereas a constraint linearized in the initial configuration moves along the initial tangent. At 90° the two positions are radically different.

There is no universal rotation angle at which a small-displacement model becomes "invalid." The acceptable error depends on the geometry and on what quantity is important. The table is intended only to show how rapidly the tangent approximation deteriorates once rotations cease to be small.

Engineering note — small strain is not small rotation. A thin or otherwise flexible structure can experience very small strains while undergoing a large rigid-body rotation. Material linearity and small strain do not justify Eq. (5.137) if the orientation of the coupling changes substantially. Geometric linearity and material linearity are separate assumptions.

Updated Constraints in a Geometrically Nonlinear Analysis

A finite-rotation formulation evaluates the constraint in the current configuration. Conceptually, the current arm is

$$\mathbf{r} = \mathbf{R}\mathbf{r}_0, \tag{5.144}$$

and the constraint is relinearized about the current equilibrium iterate. The incremental tangent therefore follows the rotating arm rather than remaining attached to the initial direction. The precise tangent matrix depends on the rotation parametrization and on the solver implementation, but the essential point is independent of those details: the constraint coefficients become configuration dependent.

This distinction also affects remote loads. If a force $\mathbf{F}$ acts at an offset point, the moment transferred to the reference point contains the current lever arm,

$$\mathbf{M} = \mathbf{r} \times \mathbf{F}. \tag{5.145}$$

In a geometrically linear formulation $\mathbf{r}$ is the initial arm $\mathbf{r}_0$. In a finite-rotation formulation it is the rotated arm. Whether the force *direction* also rotates is a separate question: a follower force rotates with the structure, whereas a force prescribed in a fixed global direction does not. Lever-arm update and follower-load definition should therefore not be confused.

Turning on geometric nonlinearity does not necessarily upgrade every legacy constraint object automatically. A solver may contain both classical linear RBE commands and newer surface-based, joint or constraint formulations capable of finite rotation. The analyst must therefore distinguish the engineering concept — for example, “rigid coupling” — from the numerical formulation used by the chosen solver object.

5.10.2 Thermal Expansion of Rigid Couplings

Thermal loading creates a second geometry-related issue. Consider again a rigid arm $\mathbf{r}_0$ connecting a reference point to a point on a structure. If the surrounding material has coefficient of thermal expansion α_s, a uniform temperature change ΔT produces the free thermal change

$$\Delta\mathbf{r}_{\text{free}} \approx \alpha_s \Delta T\, \mathbf{r}_0 \tag{5.146}$$

for isotropic expansion and constant α_s. An ordinary rigid-body constraint that preserves the initial arm length corresponds, in this sense, to a coupling with zero thermal expansion. It can therefore suppress the free thermal change in Eq. (5.146) and generate reactions that originate only from the numerical coupling.

Some implementations avoid this problem by allowing the rigid or constraint entity to carry its own coefficient of thermal expansion α_r. The target arm then changes approximately according to

$$\mathbf{r}_T = (1 + \alpha_r \Delta T)\, \mathbf{r}_0. \tag{5.147}$$

In a small-rotation formulation this appears as a nonhomogeneous constraint,

$$\mathbf{u}_s - \mathbf{u}_r - \theta_r \times \mathbf{r}_0 = \alpha_r \Delta T\, \mathbf{r}_0. \tag{5.148}$$

Equation (5.148) connects directly to the nonhomogeneous multifreedom constraints discussed in Chapter 3: the thermal expansion changes the right-hand side of the kinematic relation rather than adding an ordinary elastic stiffness to the rigid element.

For temperature-dependent expansion, the scalar factor in Eq. (5.147) is more generally obtained from

$$\lambda_T = 1 + \int_{T_0}^{T} \alpha_r(\vartheta)\, \mathrm{d}\vartheta, \qquad \mathbf{r}_T = \lambda_T \mathbf{r}_0. \tag{5.149}$$

A finite-rotation thermomechanical coupling combines both effects,

$$\boxed{\mathbf{x}_s = \mathbf{x}_r + \mathbf{R}\mathbf{r}_T}. \tag{5.150}$$

Thus rotation changes the *direction* of the arm while thermal expansion changes its stress-free *length*.

Worked Example – Artificial Thermal Restraint

Consider an idealized steel member for which a numerical rigid coupling spans

$$L_0 = 100 \text{ mm}. \tag{5.151}$$

Let

$$E = 210 \text{ GPa}, \qquad \alpha_s = 12 \times 10^{-6} \text{ K}^{-1}, \qquad \Delta T = 100 \text{ K}. \tag{5.152}$$

The surrounding material would freely increase the 100 mm distance by

$$\Delta L_{\text{free}} = \alpha_s \Delta T L_0 = 0.12 \text{ mm}. \tag{5.153}$$

If a rigid coupling forces the distance to remain exactly 100 mm, then in the corresponding one-dimensional idealization the actual axial strain is zero while the free thermal strain is

$$\varepsilon_{\text{th}} = \alpha_s \Delta T = 1.2 \times 10^{-3}. \tag{5.154}$$

The resulting mechanical strain is therefore

$$\varepsilon_{\text{mech}} = -1.2 \times 10^{-3}, \tag{5.155}$$

and the artificial compressive stress would be

$$\sigma = E\varepsilon_{\text{mech}} \approx -252 \text{ MPa}. \tag{5.156}$$

The numerical value is intentionally simple and should not be interpreted as the stress state of a real two- or three-dimensional spider. It demonstrates the scale of the error that can arise when a coupling intended only to transfer load accidentally suppresses thermal expansion.

If instead the coupling is assigned the same thermal expansion coefficient as the surrounding steel, $\alpha_r = \alpha_s$, its stress-free length becomes 100.12 mm and this artificial one-dimensional restraint disappears. Conversely, if the rigid region represents a real insert with a different coefficient of thermal expansion, choosing $\alpha_r \neq \alpha_s$ may be exactly the intended physical model.

Engineering note – Thermal compatibility is a system property. An interpolation coupling does not rigidly preserve every pairwise distance of the attachment patch, but a restrained reference point can still react the generalized thermal motion it measures. Conversely, assigning one expansion coefficient to a rigid coupling cannot reproduce an arbitrary temperature gradient over a large region. Assess the coupling together with its supports and temperature field; use a more local or explicitly deformable connection when differential expansion is important.

5.10.3 Modal and Dynamic Analyses

The phrase "can this coupling be used in dynamics?" is ambiguous. A constraint may be perfectly admissible in a full nonlinear transient analysis, where the equations are updated and solved incrementally in time, yet be unsuitable for a conventional eigenshape extraction. Conversely, a linear MPC that has no constitutive stiffness of its own fits naturally into a modal analysis because it can be imposed once on both the stiffness and mass matrices.

It is therefore useful to distinguish at least three questions:

1. Can the coupling be represented by a *fixed linear homogeneous* relation for the vibration perturbations?
2. If not, can it be *linearized about an equilibrium state* and then frozen for the modal calculation?
3. Does the selected solver and mode-extraction algorithm support the numerical enforcement method used by that particular implementation?

The first question is mathematical, the second is physical, and the third is solver-specific.

Linear Homogeneous Constraints Fit Naturally into an Eigenproblem

For an undamped linear structure the free-vibration equations are

$$\mathbf{M}\ddot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{0}. \tag{5.157}$$

Assume that the admissible displacement field is restricted by a set of linear homogeneous constraints

$$\mathbf{C}\mathbf{u} = \mathbf{0}. \tag{5.158}$$

If the admissible displacement space is parameterized by

$$\mathbf{u} = \mathbf{T}\mathbf{q}, \qquad \mathbf{C}\mathbf{T} = \mathbf{0}, \tag{5.159}$$

then substitution into Eq. (5.157) and premultiplication by $\mathbf{T}^T$ gives

$$\underbrace{\mathbf{T}^T\mathbf{M}\mathbf{T}}_{\widehat{\mathbf{M}}}\,\ddot{\mathbf{q}} + \underbrace{\mathbf{T}^T\mathbf{K}\mathbf{T}}_{\widehat{\mathbf{K}}}\,\mathbf{q} = \mathbf{0}. \tag{5.160}$$

The usual harmonic ansatz $\mathbf{q} = \phi\exp(i\omega t)$ therefore yields

$$\boxed{\left(\widehat{\mathbf{K}} - \omega^2\widehat{\mathbf{M}}\right)\phi = \mathbf{0}}. \tag{5.161}$$

This is the same ordinary generalized eigenvalue problem as for an unconstrained finite element model, only expressed in a smaller admissible coordinate space. Consequently, a *constant linear kinematic coupling imposed by exact DOF elimination* is not an obstacle to mode extraction. The same transformation must simply be applied consistently to both stiffness and mass.

This category includes, in principle,

- zero prescribed displacements and ordinary fixed supports,
- equal-DOF couplings,
- general linear homogeneous MPC equations,
- linear RBE1- and RBE2-type rigid couplings,
- linear RBE3-type interpolation couplings,
- linear tied or interpolation constraints with a fixed topology and fixed coefficients.

The important property is not the name of the coupling but the existence of a constant transformation $\mathbf{T}$ for the perturbation DOFs.

Nonzero prescribed values. A stand-alone natural-frequency calculation is a free-vibration problem. A constraint

$$\mathbf{C}\mathbf{u} = \mathbf{d}, \qquad \mathbf{d} \neq \mathbf{0}, \tag{5.162}$$

therefore cannot simply be interpreted as a nonzero oscillatory boundary value in the eigensolution. If the imposed displacement creates stress or changes the geometry, the correct procedure is normally to solve the loaded equilibrium state first and then extract modes of the *incremental* perturbation. The incremental constraint becomes homogeneous,

$$\mathbf{C}_0\,\delta\mathbf{u} = \mathbf{0}, \tag{5.163}$$

while the prestress and, where appropriate, geometric stiffness are carried into the tangent eigenproblem.

Engineering note – Modal analysis acts on perturbations.
A prestressed body can be displaced, rotated, thermally expanded or otherwise far from its original configuration. The mode shapes are nevertheless small *incremental* motions about that operating point. This is why a nonlinear base analysis and a linear modal perturbation are not contradictory.

RBE2 and RBE3 in Modal Analysis

The RBE2 transformation is a direct example of Eq. (5.159). As derived in Section 5.6, its kinematic reduction acts on the mass matrix as well as on the stiffness matrix,

$$\widehat{\mathbf{K}} = \mathbf{T}^T\mathbf{K}\mathbf{T}, \qquad \widehat{\mathbf{M}} = \mathbf{T}^T\mathbf{M}\mathbf{T}. \tag{5.164}$$

The effect on natural frequencies therefore has two distinct sources. First, the rigid coupling removes local deformation from the admissible displacement space. Second, any mass associated with dependent DOFs is redistributed through the same rigid-body transformation. Raising a natural frequency is not intrinsically an RBE2 error: it is correct if the real connection is rigid. It becomes artificial when the spider is used only as a convenient load-introduction device over a region that should remain flexible.

A linear RBE3 also fits naturally into an eigenproblem. If

$$\mathbf{q}_r = \mathbf{H}\mathbf{u}_a, \tag{5.165}$$

and a mass or rotary inertia $\mathbf{M}_r$ is attached at the remote point, its kinetic energy is

$$T_r = \frac{1}{2}\dot{\mathbf{q}}_r^T\mathbf{M}_r\dot{\mathbf{q}}_r = \frac{1}{2}\dot{\mathbf{u}}_a^T\underbrace{\mathbf{H}^T\mathbf{M}_r\mathbf{H}}_{\mathbf{M}_a^{(r)}}\dot{\mathbf{u}}_a. \tag{5.166}$$

Thus the remote inertia is distributed consistently to the attachment patch as

$$\boxed{\mathbf{M}_a^{(r)} = \mathbf{H}^T\mathbf{M}_r\mathbf{H}}. \tag{5.167}$$

Equation (5.167) is the dynamic analogue of the support-stiffness projection in Eq. (5.95). Local deformation modes in the nullspace of $\mathbf{H}$ remain available, while the generalized inertia associated with the reference point participates in the modes seen by the interpolation.

This is one reason RBE3-type attachments are often preferred for accelerometers, shakers, concentrated equipment masses or elastic supports on flexible panels: the remote quantity can participate dynamically without converting the entire attachment patch into a rigid plate.

Penalty Couplings Can Be Used, but They Create Constraint Modes

A linear penalty enforcement replaces the exact constraint by a large artificial stiffness. For

$$\mathbf{C}\mathbf{u} \approx \mathbf{0}, \tag{5.168}$$

one obtains approximately

$$\mathbf{K}_p = \mathbf{K} + \alpha \mathbf{C}^T \mathbf{C}, \tag{5.169}$$

where α is a large penalty parameter. The eigenproblem remains a standard displacement eigenproblem,

$$\left(\mathbf{K}_p - \omega^2 \mathbf{M}\right) \phi = \mathbf{0}, \tag{5.170}$$

so mode extraction is mathematically possible. The price is that the constraint is no longer exact and the artificial stiffness creates very high-frequency *constraint modes.* A characteristic penalty frequency scales roughly as

$$\omega_p \sim \sqrt{\frac{\alpha}{m_{\text{eff}}}}. \tag{5.171}$$

If α is sufficiently high, these modes lie well above the frequency range of interest and the low modes approximate the exactly constrained system. If it is too small, the intended connection becomes visibly compliant. If it is made arbitrarily large, conditioning deteriorates and the artificial modes can enter the extracted frequency range or make iterative eigensolvers inefficient.

Penalty-based RBE, tie, contact or joint formulations can therefore be used in modal analysis only with the understanding that the penalty stiffness is part of the eigenvalue problem. "Rigid" then means *very stiff*, not exactly kinematically eliminated.

Lagrange-Multiplier Constraints Are Eigensolver Dependent

An exact constraint may instead be enforced by Lagrange multipliers. For the linear relation $\mathbf{Cu} = \mathbf{0}$, the free-vibration equations become

$$\begin{bmatrix} \mathbf{K} & \mathbf{C}^T \\ \mathbf{C} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \phi \\ \lambda \end{bmatrix} = \omega^2 \begin{bmatrix} \mathbf{M} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \phi \\ \lambda \end{bmatrix}. \tag{5.172}$$

The multiplier coordinates carry constraint force but no physical inertia, so the augmented mass matrix is singular. Equation (5.172) is therefore not the ordinary positive-definite displacement eigenproblem assumed by many classical mode extractors.

This does *not* mean that Lagrange-multiplier couplings are physically forbidden in modal analysis. A solver may

- eliminate or transform the multiplier equations internally,
- use a mode extractor designed for the augmented saddle-point system,
- support only selected multiplier elements or selected eigensolvers,
- reject the formulation for eigenshape extraction even though the same element is valid in static or transient analysis.

Consequently, the statement "this joint uses Lagrange multipliers" is not by itself enough to decide modal admissibility. The particular solver and extraction method must be checked.

Nonlinear Geometric Constraints Require Linearization

A fixed-distance constraint illustrates the difference particularly clearly. For two points with current separation

$$\mathbf{r} = \mathbf{x}_2 - \mathbf{x}_1, \tag{5.173}$$

an exact fixed length L may be written as

$$g(\mathbf{x}) = \mathbf{r}^T \mathbf{r} - L^2 = 0. \tag{5.174}$$

This is a nonlinear geometric relation and cannot be inserted unchanged into an ordinary linear eigenproblem. Linearization about an equilibrium configuration $\mathbf{r}_0$ gives

$$\delta g = 2\mathbf{r}_0^T (\delta \mathbf{u}_2 - \delta \mathbf{u}_1) = 0. \tag{5.175}$$

Equation (5.175) is now a linear homogeneous MPC for the modal perturbation. It restrains only the relative displacement component parallel to the current link direction. Transverse relative motion changes the link length only to second order and is therefore not constrained by the first-order tangent equation.

This is not a defect. It is the correct local kinematics of an ideal pin-ended fixed-length link. If the link carries a finite axial reaction in the equilibrium state, the second variation of the constraint also contributes geometric stiffness. The prestressed tangent eigenproblem then contains both the linearized kinematic relation and the effect of the equilibrium constraint force.

The same logic applies to finite-rotation joints, spherical constraints, point-on-line constraints and other nonlinear holonomic relations. A standard modal analysis can use their *tangent* form about a specified operating point, provided the solver supports that linearization. It cannot follow a changing joint geometry during a single linear mode shape.

Engineering note – A nonlinear joint may be valid in transient dynamics but not directly in modal dynamics.
In a full nonlinear transient analysis the joint equations can be updated every time step. In a conventional modal analysis the joint must be represented by one fixed tangent model. Modal superposition based on that eigenspace therefore also inherits the same fixed linearization and cannot reproduce opening, locking, sliding or other state changes without additional nonlinear treatment.

Contact, Gaps and One-Sided Supports

Contact is even more restrictive because its topology is not only nonlinear but also conditional. The normal constraint from Chapter 4 contains an active-set decision: an open pair contributes no normal contact stiffness, while a closed pair does. Friction adds a further distinction between stick and slip.

A conventional eigenshape extraction cannot repeatedly decide whether contact is open or closed as the eigenvector changes sign. Instead, it uses a *frozen linearization*. Typical possibilities are

- use the initial contact status in a stand-alone linear modal analysis,
- solve a nonlinear static equilibrium first and carry the converged contact status and tangent stiffness into a prestressed modal analysis,
- deliberately replace the contact by a tied or MPC coupling when the interface is physically known to remain closed over the vibration range.

The resulting mode shape is therefore a mode of the structure with that contact state held fixed. It does not mean that the real nonlinear interface will remain closed, sticking or bonded at arbitrary vibration amplitude.

The same caveat applies to compression-only supports, gap springs, cable/slack conditions and unilateral stops. If their state changes with the sign or amplitude of motion, a single linear modal basis cannot reproduce that change.

Springs, Bushings, Dampers and Frequency-Dependent Couplings

A linear spring or bushing adds an ordinary stiffness matrix and is naturally compatible with eigenshape extraction. The RBE3-supported bushing of Section 5.7.5, for example, contributes

$$\mathbf{K}_a^{(s)} = \mathbf{H}^T \mathbf{K}_s \mathbf{H} \tag{5.176}$$

and can be included directly in Eq. (5.161).

A purely viscous damper is different. In the undamped real eigenproblem it does not affect the extracted frequencies or real mode shapes because no damping matrix is present. To include viscous damping explicitly one needs a damped or complex eigenvalue formulation, schematically

$$\left(\mathbf{K} + i\omega \mathbf{C}_d - \omega^2 \mathbf{M}\right) \phi = \mathbf{0}. \tag{5.177}$$

If the support stiffness or damping itself depends strongly on frequency, the problem is no longer a simple linear generalized eigenproblem because the system matrix depends on the unknown eigenfrequency. Solvers may use special complex mode algorithms, evaluate the support at a reference frequency, or prohibit the feature in ordinary normal-mode extraction. The same issue occurs with active control laws and other frequency- or state-dependent couplings.

Which Coupling Classes Can Be Used for Eigenshape Extraction?

Tables 5.1 and 5.2 summarize the general rule. They are intentionally organized by mathematical behaviour rather than by solver command. A particular commercial implementation may impose additional restrictions.

Coupling or support class	**Ordinary linear modes**	**Reason / required treatment**
Zero displacement, equal DOF, linear homogeneous CE/MPC	Yes	Constant linear constraint; eliminate or transform the DOFs consistently in $\mathbf{K}$ and $\mathbf{M}$.
Linear RBE1/RBE2/RBE3	Yes	Constant kinematic transformation. RBE2 removes local deformation; RBE3 preserves interpolation nullspace modes.
Linear spring or bushing	Yes	Adds ordinary stiffness; attached remote mass/inertia must also be mapped consistently.
Penalty rigid/tied coupling	Yes, with caution	Standard eigenproblem, but artificial high-frequency constraint modes and conditioning sensitivity remain.
Lagrange-multiplier MPC or joint	Solver dependent	Massless multiplier DOFs give a singular mass block; supported condensation or a compatible eigensolver is required.

Table 5.1: Couplings that can enter a linear eigenshape extraction directly, subject to the stated numerical caveats.

The remaining classes require an operating-point linearization or a different eigenvalue formulation rather than direct insertion into the ordinary undamped normal-mode problem.

Coupling or support class	**Ordinary linear modes**	**Reason / required treatment**
Nonzero prescribed displacement	Not directly	Solve the equilibrium/prestress state first; modal perturbations normally use homogeneous incremental constraints.
Fixed distance, finite-rotation joint, other nonlinear bilateral constraint	After linearization	Use the tangent constraint about equilibrium; include geometric stiffness from equilibrium reactions when relevant.
Contact, gap, compression-only support, stop	Not with changing status	Freeze/linearize the active state, preferably from a prestressed equilibrium when the operating contact condition matters.
Frictional contact with possible stick/slip change	Tangent only	A mode uses one fixed stick/slip tangent; changing friction state belongs to nonlinear dynamics.
Nonlinear spring/bushing	Tangent only	Linearize the force-displacement law at the operating point.
Pure viscous damper	Not in undamped real modes	Use a damped/complex eigensolution if its effect on modes is to be represented.
Frequency-dependent or active coupling	Special treatment	Leads to a frequency-dependent or state-space eigenproblem; ordinary normal-mode extraction is generally insufficient.

Table 5.2: Couplings requiring linearization, a fixed active state, or a nonstandard eigenvalue formulation.

Examples of Solver-Specific Modal Restrictions

The previous tables state the governing principles. Commercial solvers then add implementation-specific restrictions that can change between element families and eigensolvers. Two examples illustrate why the manual still matters even when the underlying mechanics is clear.

ANSYS.

Linear constraint equations are supported in modal systems and are normally handled by eliminating a suitable dependent DOF. The classical `CERIG` and `RBE3` commands generate linear constraint equations and are small-deflection formulations, which makes them conceptually compatible with a linear modal model about their reference configuration. ANSYS `MPC184` rigid link/beam elements using direct elimination are explicitly available for modal analysis. Lagrange-multiplier variants are more restrictive: selected `MPC184` rigid links, rigid beams and joint types are supported with the PCG Lanczos mode extractor through a dedicated treatment, but this should not be generalized to every multiplier element or every mode-extraction method.

ANSYS contact in a linear modal analysis is not allowed to open and close during the eigensolution. Its stiffness is based on a fixed contact state. For a deformed or contact-loaded operating condition, a prestressed modal / linear perturbation analysis is therefore the appropriate interpretation: the nonlinear static solution determines the operating geometry, stress state and contact status, and the eigensolution acts on the tangent system about that state.

PERMAS.

The ordinary linear MPC family belongs naturally to the displacement-reduction category described by Eq. (5.161). Thus linear definitions such as `MPC_RIGID`, `MPC_WLSCON` and `MPC_GENERAL` are natural candidates for a dynamic eigenvalue model. The same is true of simpler assignment or joining MPCs. PERMAS documentation also describes explicit MPC handling in dynamic eigenvalue reduction and linear MPC surface couplings that are available for dynamic analyses. By contrast, contact or other state-dependent connections must first be represented by a fixed linearized state; PERMAS provides contact-locking concepts precisely to permit a transition from a contact solution to linear dynamics.

These examples are deliberately not exhaustive command references. Their purpose is to reinforce the general engineering rule: determine whether the intended connection has a constant linear tangent for the modal perturbation, then verify that the chosen solver object and eigensolver support the numerical enforcement used to realize it.

5.10.4 Interaction with Contact and Other Constraints

A kinematic coupling is rarely the only constraint in an industrial FE model. The same region may also contain supports, joints, contact definitions, symmetry conditions, other spiders, or prescribed motions. Each individual definition may be correct in isolation and still become incorrect when combined with the others. The resulting problem is commonly called *overconstraint*, although three different situations should be distinguished:

1. **Independent constraints:** every additional equation removes an additional admissible motion. This may be exactly what the physical connection requires.
2. **Redundant constraints:** an added equation repeats information already contained in the existing set. The admissible displacement space does not become smaller, but some numerical enforcement methods become singular or ill-conditioned.
3. **Incompatible constraints:** two or more equations prescribe mutually inconsistent motion. No displacement field can satisfy them simultaneously.

This distinction is more useful than the informal rule that "too many constraints are bad". A structure may legitimately require many independent constraint equations, while only two equations are sufficient to create a singularity if one is an exact duplicate of the other.

Constraint Rank and Compatibility

Consider the linear constraint system introduced in Chapter 3,

$$\mathbf{C}\mathbf{u} = \mathbf{d}, \tag{5.178}$$

with n structural DOFs and m scalar constraint equations. Let

$$r = \operatorname{rank}(\mathbf{C}). \tag{5.179}$$

Only r of the m equations are independent. If the constraints are compatible, the dimension of the admissible displacement space is

$$n_{\text{free}} = n - r. \tag{5.180}$$

Redundancy is therefore present whenever

$$r < m. \tag{5.181}$$

For nonhomogeneous constraints, rank alone is not sufficient. The equations are compatible only when

$$\operatorname{rank}(\mathbf{C}) = \operatorname{rank}\begin{bmatrix}\mathbf{C} & \mathbf{d}\end{bmatrix}. \tag{5.182}$$

Equation (5.182) is the algebraic statement that there exists at least one displacement vector satisfying all prescribed relations. In FE practice the solver seldom performs this textbook rank test explicitly on the complete constraint set, but zero pivots, singular multiplier blocks, repeated dependent-DOF messages and inconsistent boundary-condition warnings are numerical manifestations of the same issue.

Worked Example – One Physical Constraint Written Twice

Let two scalar DOFs be required to move together,

$$u_2 - u_1 = 0. \tag{5.183}$$

Suppose the same relation is accidentally entered a second time, merely scaled by a factor of two:

$$2u_2 - 2u_1 = 0. \tag{5.184}$$

The assembled constraint matrix is

$$\mathbf{C} = \begin{bmatrix} -1 & 1 \\ -2 & 2 \end{bmatrix}, \qquad \operatorname{rank}(\mathbf{C}) = 1. \tag{5.185}$$

The two equations therefore impose only one independent kinematic condition. The displacement solution may still be perfectly well defined, but the numerical consequences depend strongly on how the equations are enforced.

Elimination. One equation is sufficient to write

$$u_2 = u_1. \tag{5.186}$$

The second equation adds no information. A global constraint-reduction algorithm may detect and discard it. A simpler sequential elimination scheme, however, may report that the selected dependent DOF has already been used, or may fail when it encounters a cyclic dependency. This is why many solver manuals impose apparently restrictive rules on which DOFs may be selected as dependent variables: the rules ensure that a unique acyclic transformation can be constructed.

Lagrange multipliers. With one multiplier for each entered equation, the equilibrium equations have the saddle-point form

$$\begin{bmatrix} \mathbf{K} & \mathbf{C}^T \\ \mathbf{C} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ \lambda \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ \mathbf{0} \end{bmatrix}. \tag{5.187}$$

Because the rows of $\mathbf{C}$ are linearly dependent, the augmented matrix is singular. More importantly, the individual constraint reactions are not unique. The structural equilibrium contains

$$\mathbf{C}^T \lambda = \begin{bmatrix} -\lambda_1 - 2\lambda_2 \\ \lambda_1 + 2\lambda_2 \end{bmatrix}, \tag{5.188}$$

so only the combination $\lambda_1 + 2\lambda_2$ has physical effect. Infinitely many pairs (λ_1, λ_2) produce the same nodal reaction. The structural displacement can therefore be unique while the reactions assigned to the individual redundant constraints are indeterminate.

Penalty enforcement. For the elementary penalty functional

$$\Pi_p = \frac{\alpha}{2}(\mathbf{Cu})^T(\mathbf{Cu}), \tag{5.189}$$

the additional stiffness is

$$\mathbf{K}_p = \alpha \mathbf{C}^T \mathbf{C}. \tag{5.190}$$

Using Eq. (5.185),

$$\mathbf{K}_p = 5\alpha \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}. \tag{5.191}$$

Had the first equation been entered only once, the coefficient would be α rather than 5α. Thus a raw penalty formulation is not invariant to duplicating or even rescaling the same constraint equation. Practical implementations may normalize the constraint rows or use more sophisticated penalty scaling, but the example explains why redundant penalty definitions can silently change the stiffness and conditioning instead of producing an obvious singularity.

Engineering note – Redundant constraints are not numerically neutral.
An exact duplicate may be harmless after an ideal global elimination, singular with Lagrange multipliers, and artificially stiffer under a penalty formulation. The engineering relation is the same; the numerical symptom depends on the enforcement method discussed in Chapter 3.

If the second equation were instead

$$u_2 - u_1 = \Delta, \qquad \Delta \neq 0, \tag{5.192}$$

while the first still required $u_2 - u_1 = 0$, the constraints would be incompatible rather than redundant. Exact elimination or multiplier methods would have no admissible solution. A penalty method could still return a numerical displacement by compromising between the two requirements, but the large artificial reactions and penalty energy would represent a modelling conflict rather than a physical deformation mode.

Overlapping RBE2 Spiders

Overlapping RBE2 spiders are one of the most common ways of creating a hidden constraint. Suppose a node P is included as a dependent node of two rigid spiders with reference coordinates $\mathbf{q}_A$ and $\mathbf{q}_B$. The two RBE2 relations require simultaneously

$$\mathbf{q}_P = \mathbf{D}_A \mathbf{q}_A, \qquad \mathbf{q}_P = \mathbf{D}_B \mathbf{q}_B. \tag{5.193}$$

Eliminating $\mathbf{q}_P$ gives the hidden relation

$$\mathbf{D}_A \mathbf{q}_A - \mathbf{D}_B \mathbf{q}_B = \mathbf{0}. \tag{5.194}$$

The shared node has therefore coupled the two reference points directly. One shared translational DOF removes one relative motion between the spiders. A sufficiently rich set of shared non-collinear nodes can force the two reference frames to behave as one larger rigid body even though no explicit RBE was drawn between them.

This hidden coupling is illustrated in Figure 5.7.

This may be intentional, for example when two rigid idealizations are genuinely part of the same stiff component. More often it occurs accidentally where two

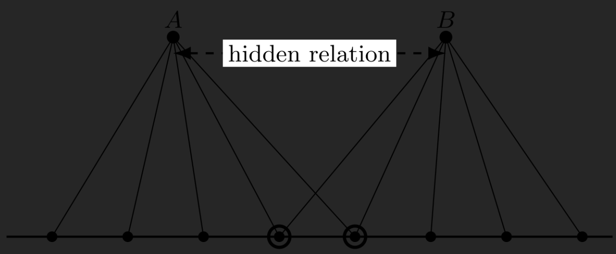


Figure 5.7: Overlapping RBE2 spiders. A node that is dependent on both reference systems must satisfy both rigid-body relations simultaneously. The overlap therefore introduces an implicit relation between reference points A and B; enough shared nodes can effectively merge the two spiders into one rigid region.

bolt-head spiders, flange couplings or bearing spiders overlap on a fine mesh. The consequences are then not limited to a solver warning. The overlap can suppress structural deformation, alter the load path and raise natural frequencies.

Common mistake – "A node can belong to both spiders because both are rigid."
If the same dependent motion is prescribed by two different reference systems, the node does not merely receive two equivalent descriptions. It creates an additional relation between the reference systems themselves. Check the implied constraint of Eq. (5.194) before allowing rigid spiders to overlap.

A related problem is a chain or loop of elimination constraints. Relations such as

$$u_2 = u_1, \qquad u_3 = u_2, \qquad u_1 = u_3 \tag{5.195}$$

contain only two independent equations; the third closes a redundant cycle. If one relation contains a different offset or transformation, the cycle may instead become inconsistent. General constraint solvers can sometimes resolve such graphs globally, whereas simple dependent-DOF elimination requires an acyclic ordering. A good modelling rule is therefore to keep the dependency hierarchy obvious even when the solver is capable of more general algebra.

Overlapping RBE3 Couplings Are Different

The same geometric overlap does not automatically have the same meaning for an RBE3. In the interpolation form

$$\mathbf{q}_{r1} = \mathbf{H}_1\mathbf{u}, \qquad \mathbf{q}_{r2} = \mathbf{H}_2\mathbf{u}, \tag{5.196}$$

an attachment-node displacement is not prescribed by either RBE3; it contributes to the generalized motion of the reference point. A physical mesh node may therefore participate in two load-distribution interpolations without being rigidly assigned two different motions.

However, overlap is not automatically harmless. If both reference-point motions are subsequently prescribed, joined, or made dependent in other MPCs, the two relations become constraints on overlapping weighted averages of the same mesh DOFs. Those constraints may be independent, redundant or incompatible depending on $\mathbf{H}_1$, $\mathbf{H}_2$ and the prescribed generalized motions. Large overlapping RBE3 regions can also create unintended parallel load paths even when no formal rank deficiency exists.

The useful distinction is therefore not "RBE2 overlap is bad, RBE3 overlap is good". The distinction is that RBE2 directly prescribes attachment-node kinematics, whereas RBE3 defines generalized coordinates from them. The complete constraint graph must still be considered.

Coupling Plus Prescribed Boundary Conditions

A prescribed DOF on a node that already participates in a coupling deserves the same rank-based interpretation. For an RBE2 dependent node,

$$\mathbf{q}_i = \mathbf{D}_i \mathbf{q}_r. \tag{5.197}$$

Prescribing one component of $\mathbf{q}_i$ therefore imposes a corresponding linear condition on the reference coordinates $\mathbf{q}_r$. This is mathematically valid if it represents the intended support. It becomes redundant if the same reference motion has already been prescribed elsewhere, and incompatible if the two prescriptions disagree.

Many elimination-based implementations prohibit or restrict direct boundary conditions on dependent DOFs because a DOF that has already been removed cannot also serve as an independently prescribed variable. The underlying engineering condition can usually be expressed instead at the reference point or by choosing a different independent/dependent assignment.

For an RBE3 reference point the situation is almost the mirror image. Prescribing

$$\mathbf{q}_r = \bar{\mathbf{q}}_r, \qquad \mathbf{q}_r = \mathbf{H}\mathbf{u}_a \tag{5.198}$$

constrains a weighted generalized motion of the attachment patch. It does not, by itself, fix every attachment node. This can be exactly the desired "deformable remote support" discussed in Section 5.7.5. If the attachment nodes are also individually fixed, however, the generalized constraint may become redundant.

Coupling and Contact

Contact and kinematic couplings are frequently used in the same region, but they play fundamentally different roles. Contact from Chapter 4 determines whether a unilateral constraint is active. An RBE or MPC remains bilateral as long as it exists. Combining them requires asking which motions are being controlled by each formulation.

Several common situations should be distinguished.

- **Rigid platen or rigid tool.** Coupling all nodes of a contact surface by an RBE2 may be exactly correct if that surface is intended to behave as a rigid body. The resulting contact-pressure distribution is then the pressure under a rigid surface, not under a deformable one.

- **Flexible contact surface with remote loading.** An RBE3-type interpolation can introduce the resultant load or connect a support while leaving local contact deformation available. This is often more appropriate than rigidizing the whole patch.
- **Bonded/tied contact plus rigid coupling.** If both definitions enforce the same relative motion, they create redundant parallel constraints. If one rigidizes a region that the tied interface was intended to let deform, the model is overconstrained even if the algebra remains nonsingular.
- **Frictional contact plus rigid spider.** Rigidly tying tangential motion over a large patch can suppress local slip and change the stick/slip distribution. The contact law may be correct while the surrounding kinematics make the physical interface too stiff.
- **Active contact against an incompatible prescribed motion.** The contact algorithm may generate very large reactions, repeated active-set changes or convergence failure if a permanent coupling drives the same normal DOF through an impenetrable surface.

The interaction also matters numerically. Some contact formulations themselves use MPCs or multiplier equations when the interface is closed. If a user-defined constraint already consumes the same dependent DOF, the solver may need special constraint ordering or may reject the combination. Such restrictions are implementation details, but they reflect the same underlying requirement that the active constraint Jacobian have the required rank.

Couplings, Joints, Springs and Parallel Load Paths

A particularly useful modelling pattern is

$$\begin{aligned} \text{flexible surface} &\longrightarrow \text{RBE2/RBE3 reference point} \\ &\longrightarrow \text{joint, spring or bushing} \longrightarrow \text{second interface.} \end{aligned} \tag{5.199}$$

There is no redundancy merely because several constraint objects appear in the chain. The surface coupling defines how a distributed FE region maps to a small set of generalized interface coordinates; the joint or bushing then defines the relative motion or constitutive relation between those interface coordinates. The two objects perform different tasks.

Redundancy appears when the same relative motion is also constrained by a second parallel path. For example, connecting two surfaces by a bushing and at the same time rigidly tying them with an RBE2 leaves the bushing no deformation with which to generate its intended force. Similarly, a revolute joint followed by an SPC that fixes its permitted rotation is no longer a revolute connection; it has become fixed in that coordinate.

Parallel paths are not inherently wrong. Real structures often have several bolts, mounts or bearings sharing a load. The important point is that *load sharing requires compliance.* Two mathematically perfect rigid constraints in parallel may produce a unique displacement field while the split of reaction forces between the ideal constraints is indeterminate. If the force in each physical attachment matters, represent the relevant stiffness, contact or local geometry rather than expecting exact rigid MPCs to determine a unique load share.

Engineering note – Exact constraints determine motion better than load sharing.
A rigid MPC is an excellent way to state that a motion is forbidden. It is a poor model of two parallel physical connectors when the engineering quantity of interest is how much load each connector carries. Without relative compliance, that load split may be numerically arbitrary or formally indeterminate.

Consequences in Modal and Dynamic Analysis

The modal discussion in Section 5.10.3 makes constraint interaction especially easy to diagnose physically. Every *independent* exact kinematic constraint removes an admissible perturbation motion. An accidental RBE2 bridge or redundant support can therefore make modes disappear or move frequencies upward because the model has been made kinematically stiffer than intended.

The numerical symptom again depends on enforcement:

- elimination may simply remove the accidentally constrained mode,
- a penalty relation may replace it by an artificial very-high-frequency constraint mode,
- multiplier redundancy may produce a singular augmented eigenproblem or algebraic modes requiring special treatment.

Thus an unexpectedly high natural frequency is not always caused by material or mesh stiffness. It may be the first visible symptom of an unintended constraint path.

Practical Diagnostics

Constraint problems are easier to find when solver messages are interpreted as information about the model rather than obstacles to be suppressed. Table 5.3 summarizes several common symptoms.

Observed symptom	**Typical constraint-related cause to investigate**
Dependent DOF already constrained / already eliminated	The same DOF is used as dependent in more than one elimination relation, or an SPC is applied after the DOF has been eliminated.
Zero pivot or singular constraint/multiplier block	Redundant constraint rows, a closed dependency cycle, or insufficiently independent geometry in a coupling.
No exact equilibrium / very large constraint reactions	Incompatible nonhomogeneous constraints or a prescribed motion competing with contact/support conditions.
Displacements look reasonable but individual MPC reactions are unstable or nonunique	Compatible but redundant exact constraints; only the resultant reaction may be determined.
Natural frequencies unexpectedly high or local modes disappear	Accidental rigidization, overlapping RBE2 regions, excessive penalty stiffness, or duplicated supports.
Very high artificial modes and poor conditioning	Penalty constraints that are too stiff, duplicated penalty relations, or near-rigid springs used as substitutes for exact constraints.
Unexpected contact pressure, sticking or convergence difficulty	Rigid coupling has changed local contact kinematics, or contact and permanent constraints compete for the same motion.
Reaction/load split changes strongly with modelling details while deformation barely changes	Multiple nearly rigid parallel load paths; individual load sharing is not physically resolved by compliance.

Table 5.3: Typical numerical symptoms of interacting or redundant constraints.

A useful debugging procedure is to reason from the constraint graph before changing solver tolerances:

1. Identify every DOF or generalized coordinate constrained by more than one modelling object.
2. Ask whether the equations control the same motion or genuinely different motions.

3. Temporarily remove one path and observe whether the structural response or problematic reaction changes.

4. For linear MPCs, inspect the expected rank and the number of remaining admissible motions using Eq. (5.180).

5. For contact or nonlinear joints, inspect the active tangent constraint set at the operating point rather than only the nominal definitions.

6. If individual connector forces matter, verify that the model contains physical compliance capable of determining their load share.

Final rule for interacting constraints.
Do not count modelling objects; count independent restrictions on motion. Two objects may be perfectly complementary, while one badly chosen overlap can silently remove a real deformation mode.

5.11 Engineering Selection Guidelines

The most useful first question is not which solver command to choose, but which relative motion the physical connection should permit. Table 5.4 collects the main choices developed in this chapter.

Engineering requirement	**Natural starting formulation**	**Main modelling check**
Attached region is intentionally rigid	RBE2-type rigid coupling	Local bending, ovalization and warping are deliberately removed.
Transfer a resultant force, moment or generalized motion while retaining local deformation	RBE3-type interpolation coupling	Choose attachment nodes and weights to represent the intended load path; do not assume equal weights imply uniform traction.
Extended flexible region connected through finite translational and rotational compliance	Interpolation coupling plus spring or bushing	Keep coupling kinematics and support stiffness separate; include real compliance when connector load sharing matters.
Exact constant distance, angle or joint motion	Geometric constraint or dedicated joint	Verify nonlinear updating for finite motion and use the operating-point tangent for linear modes.
Interface may open, close, slide or change state	Contact or other unilateral/nonlinear connection	A permanent RBE/MPC cannot represent changing active status.
Remote load, mass, support or joint needs generalized interface coordinates	Remote/reference coupling	Inspect the underlying rigid/interpolating formulation rather than inferring behaviour from the GUI object name.

Table 5.4: Solver-independent decision guide for the main coupling classes discussed in this chapter.

After selecting the physical relation, four cross-cutting checks remain:

1. **Geometry:** if large rotations are expected, verify that the implementation updates the constraint geometry rather than retaining the initial tangent.

2. **Thermal behaviour:** if thermal expansion matters, verify that the coupling does not suppress the stress-free motion of the represented connection.

3. **Dynamics and load distribution:** for modal analysis, verify that a fixed linear tangent exists and is supported by the eigensolver; for distributed loading, distinguish resultant equivalence from the local traction field.

4. **Constraint interaction and size:** trace other supports, contact, joints and spiders acting on the same DOFs, and use only as many well-spaced spider legs as are needed to span the intended motion and load path.

Final engineering rule.
Choose the *admissible relative motion* first, the *constraint formulation* second and the solver command last. A correct command applied to the wrong physical constraint is still the wrong model.

5.12 Summary

Kinematic couplings define admissible relative motion; Chapter 3 covers how such constraints are enforced. RBE1 distributes retained rigid-body coordinates among nodal components. RBE2 maps reference translation and rotation to dependent nodes and removes their relative deformation. RBE3 instead interpolates generalized reference motion from attachment nodes and returns loads by virtual work, allowing local deformation to remain.

Fixed-distance relations and joints are geometric equality constraints, while contact (Chapter 4) adds unilateral relations with changing active sets. Remote/reference objects provide generalized interfaces above these formulations. Finite rotation, thermal expansion, modal linearization, load weighting and neighbouring constraints can all alter their practical behaviour. The durable rule of Section 5.11 is therefore to define the permitted relative motion first, then choose the formulation and finally its software implementation.

Chapter 6

Governing Equations

6.1 Material Matrices

Problem	Displacement components	Strain vector $\boldsymbol{\epsilon}^T$	Stress vector $\boldsymbol{\tau}^T$
Bar	u	$[\epsilon_{xx}]$	$[\sigma_{xx}]$
Beam	w	$[\kappa_{xx}]$	$[m_{xx}]$
Plane stress	u, v	$[\epsilon_{xx}, \epsilon_{yy}, \gamma_{xy}]$	$[\sigma_{xx}, \sigma_{yy}, \tau_{xy}]$
Plane strain	u, v	$[\epsilon_{xx}, \epsilon_{yy}, \gamma_{xy}]$	$[\sigma_{xx}, \sigma_{yy}, \tau_{xy}]$
Axisymmetric	u, v	$[\epsilon_{xx}, \epsilon_{yy}, \gamma_{xy}, \epsilon_{zz}]$	$[\sigma_{xx}, \sigma_{yy}, \tau_{xy}, \sigma_{zz}]$
3D	u, v, w	$[\epsilon_{xx}, \epsilon_{yy}, \epsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{zx}]$	$[\sigma_{xx}, \sigma_{yy}, \sigma_{zz}, \tau_{xy}, \tau_{yz}, \tau_{zx}]$
Plate bending	w	$[\kappa_{xx}, \kappa_{yy}, \kappa_{xy}]$	$[m_{xx}, m_{yy}, m_{xy}]$

Notation: $\epsilon_{xx} = \frac{\partial u}{\partial x}$, $\epsilon_{yy} = \frac{\partial v}{\partial y}$, $\gamma_{xy} = \frac{\partial u}{\partial y} + \frac{\partial v}{\partial x}$, $\ldots$, $\kappa_{xx} = \frac{\partial^2 w}{\partial x^2}$, $\kappa_{yy} = \frac{\partial^2 w}{\partial y^2}$, $\kappa_{xy} = \frac{\partial^2 w}{\partial x \partial y}$.

Table 6.1: Corresponding kinematic and static variables in various problems.

Problem	Material matrix C
Bar	E
Beam	EI
Plane stress	$\frac{E}{1-\nu^2}\begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1-\nu}{2} \end{bmatrix}$
Plane strain	$\frac{E(1-\nu)}{(1+\nu)(1-2\nu)}\begin{bmatrix} 1 & \frac{\nu}{1-\nu} & 0 \\ \frac{\nu}{1-\nu} & 1 & 0 \\ 0 & 0 & \frac{1-2\nu}{2(1-\nu)} \end{bmatrix}$
Axisymmetric	$\frac{E(1-\nu)}{(1+\nu)(1-2\nu)}\begin{bmatrix} 1 & \frac{\nu}{1-\nu} & 0 & \frac{\nu}{1-\nu} \\ \frac{\nu}{1-\nu} & 1 & 0 & \frac{\nu}{1-\nu} \\ 0 & 0 & \frac{1-2\nu}{2(1-\nu)} & 0 \\ \frac{\nu}{1-\nu} & \frac{\nu}{1-\nu} & 0 & 1 \end{bmatrix}$
3D	$\frac{E(1-\nu)}{(1+\nu)(1-2\nu)}\begin{bmatrix} 1 & \frac{\nu}{1-\nu} & \frac{\nu}{1-\nu} & 0 & 0 & 0 \\ \frac{\nu}{1-\nu} & 1 & \frac{\nu}{1-\nu} & 0 & 0 & 0 \\ \frac{\nu}{1-\nu} & \frac{\nu}{1-\nu} & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1-2\nu}{2(1-\nu)} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1-2\nu}{2(1-\nu)} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1-2\nu}{2(1-\nu)} \end{bmatrix}$
Plate bending	$\frac{Eh^3}{12(1-\nu^2)}\begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1-\nu}{2} \end{bmatrix}$

Notation: E = Young's modulus, ν = Poisson's ratio, h = thickness of plate, I = moment of inertia

Table 6.2: Generalised stress-strain matrices for isotropic materials

Chapter 7

Dynamics

7.1 Function properties

Sine and Cosine functions

Let a fucntion $f(t)$ be in the form:

$$f(t) = A_1 \cos \omega t + B_i \sin \omega t \tag{7.1}$$

then the following relation applies:

$$\begin{aligned} f(t) &= A \cos(\omega t - \phi) \\ A &= \sqrt{A_1^2 + B_1^2} \\ \phi &= \tan^{-1}\left(\frac{B_i}{A_i}\right) \end{aligned} \tag{7.2}$$

also:

$$\begin{aligned} \cos(\omega t - \phi) &= \cos \omega t \cos \phi + \sin \omega t \sin \phi \\ \sin(\omega t - \phi) &= \sin \omega t \cos \phi + \cos \omega t \sin \phi \end{aligned} \tag{7.3}$$

7.2 SDOF system

7.2.1 Definition

Let:

$$m\ddot{u}(t) + c\dot{u}(t) + ku(t) = f(t) \tag{7.4}$$

or:

$$ma(t) + cv(t) + ku(t) = f(t) \tag{7.5}$$

be the **SDOF equation of motion**, then the following applies:

$$\begin{aligned} \omega^2 &= \frac{k}{m} \\ \omega &= \sqrt{\frac{k}{m}} \\ f &= \frac{\omega}{2\pi} \\ T &= \frac{1}{f} \end{aligned} \tag{7.6}$$

where:
ω is radial eigenfrequency [rad/s],
f is eigenfrequency [Hz] and
T is period [s]

The simplest vibratory syslem can be described by a single mass connected to a spring (and possibly a dashpot). The mass is allowed to travel only along the spring elongation direction. Such systems are called *Single Degree-of-Freedom* (**SDOF**) systems and are shown in the figure.

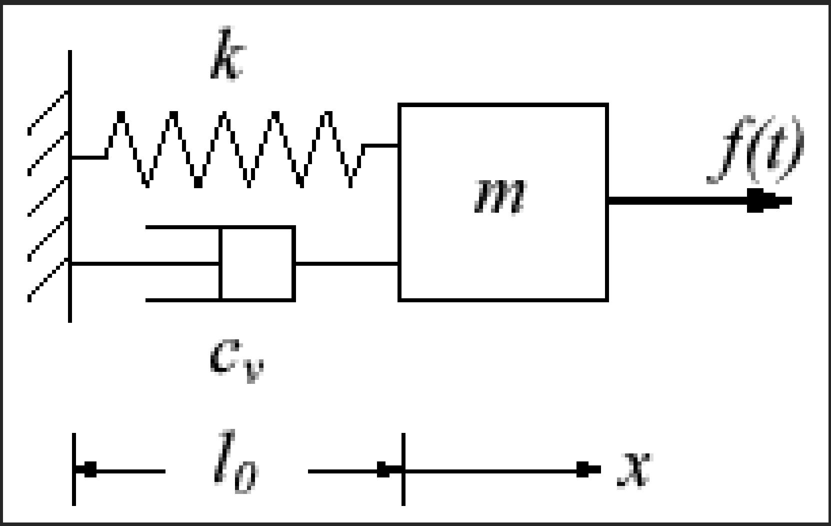


Figure 7.1: SDOF system with a linear spring and dashpot

7.2.2 Equation of Motion for SDOF Systems

SDOF vibration can be analyzed by **Newton's second law of motion**, $F = ma$. The analysis can be easily visualised with the aid of a **free body diagram**,

The resulting equation of motion is a **second order, non-homegeneous, ordinary differential equation**:

$$m\ddot{u} + c\dot{u} + ku = f(t) \begin{cases} u(t=0) = u_0 \\ \dot{u}(t=0) = \dot{u}_0 \end{cases} \tag{7.7}$$

with the initial conditions u_0 and $\dot{u}_0$.

7.2.3 Time Solution for Unforced Undamped SDOF Systems

https://www.efunda.com/formulae/vibrations/sdof_free_undamped.cfm

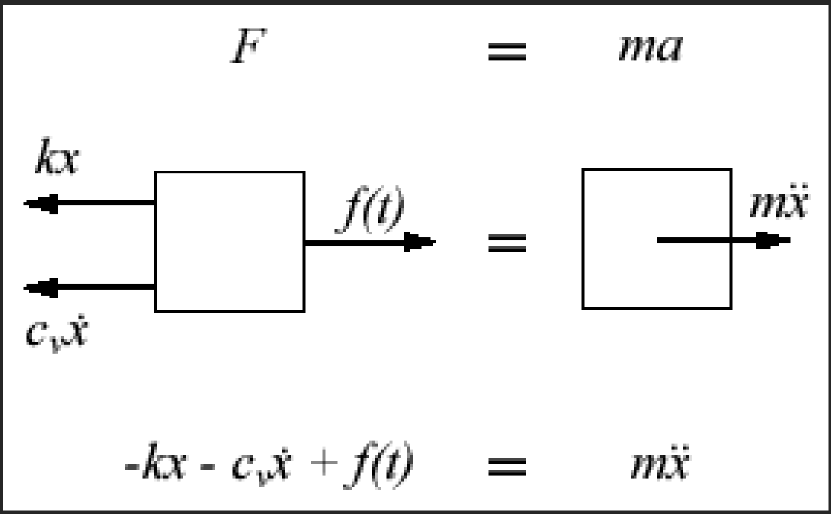


Figure 7.2: SDOF Free Body Diagram

The equation of motion derived in the definition can be simplified to:

$$m\ddot{u} + ku = 0 \begin{cases} u(t=0) = u_0 \\ \dot{u}(t=0) = \dot{u}_0 \end{cases} \tag{7.8}$$

This equation of motion is a **second order, homegeneous, ordinary differential equation** (ODE). If the mass and spring stiffness are constants, the ODE becomes a **linear, homogeneous ODE with constant coefficients** and can be solved by the Characteristic Equation method. The characteristic equation for this problem is:

$$ms^2 + k = 0 \tag{7.9}$$

which determines the 2 independent roots for the undamped vibration problem. The final solution (that contains the 2 independent roots from the characteristic equation and satisfies the initial conditions) is:

$$
\begin{aligned}
u(t) &= c_1 e^{i\omega_n t} + c_2 e^{-\omega_n t} \\
&= d_1 \cos \omega_n t + d_2 \sin \omega_n t \\
\implies u(t) &= u_0 \cos \omega_n t + \frac{\dot{u}_0}{\omega_n} \sin \omega_n t
\end{aligned}
\tag{7.10}
$$

The natural frequency ω_n is defined by:

$$
\omega_n = \sqrt{\frac{k}{m}} \tag{7.11}
$$

and depends only on the system mass and the spring stiffness (i.e. any damping will not change the natural frequency of a system).

Alternatively, the solution may be expressed by the equivalent form,

$$
u(t) = A_0 \cos\left(\omega_n t - \phi_0\right) \tag{7.12}
$$

where the amplitude A_0 and the intial phase ϕ_0 are given by:

$$
\begin{aligned}
A_0 &= \sqrt{u_0^2 + \left(\frac{\dot{u}_0}{\omega_n}\right)^2} \\
\phi_0 &= \tan^{-1} \frac{\dot{u}_0}{u_0 \omega_n}
\end{aligned}
\tag{7.13}
$$

Note that an assumption of zero damping is typically not accurate. In reality, there almost always exists some resistance in vibratory systems. This reistance will damp the vibration and dissipate energy, the oscillatory motion caused by the initial disturbance will eventually be reduced to zero.

7.2.4 Time Solution for Unforced Damped SDOF Systems

https://www.efunda.com/formulae/vibrations/sdof_free_damped.cfm

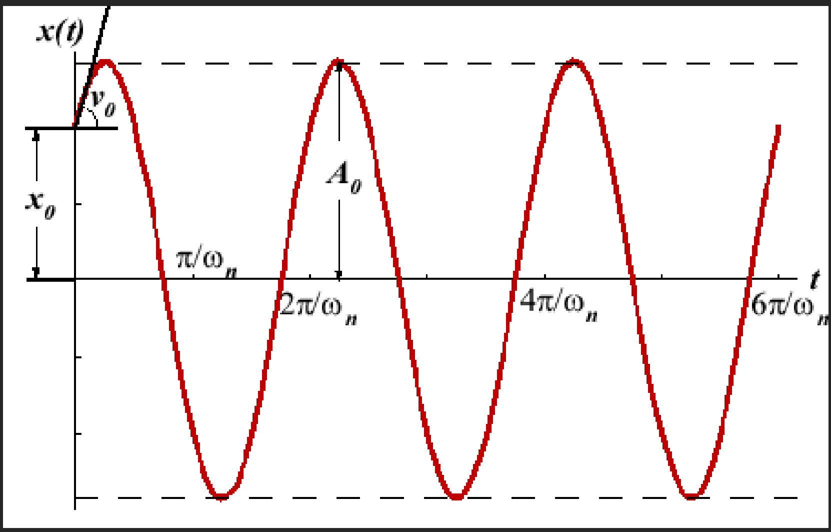


Figure 7.3: SDOF Undamped sample time behavior

Damping that produces a damping force proportional to the mass's velocity is commonly referred to as *viscous damping*, and is denoted graphically by a dashpot.

For an unforced damped **SDOF** system, the general equation of motion becomes:

$$m\ddot{u} + c\dot{u} + ku = 0 \begin{cases} u(t=0) = u_0 \\ \dot{u}(t=0) = \dot{u}_0 \end{cases} \tag{7.14}$$

Tis equation of motion is a **second order, homogeneous, ODE**. If all parameters (mass, stiffness and viscous damping) are constants, the **ODE** becomes a **linear ODE with constant coefficients** and can be solved by the *Characteristic Equation method*. The characteristic equation for this problem is:

$$ms^2 + s_v s + k = 0 \tag{7.15}$$

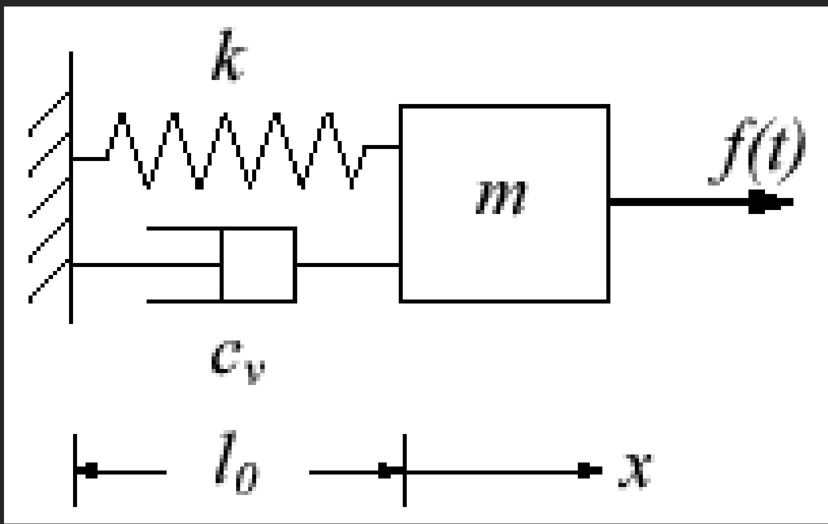


Figure 7.4: SDOF system with a linear spring and dashpot

which determines the 2 independent roots for the damped vibration problem. The roots to the characteristic equation fall into one of the following 3 cases:

- If $c_v^2 - 4mk < 0$, the system is termed **underdamped**. The roots of the characteristic equation are complex conjugates, corresponding to *oscillatory motion* with an *exponential decay* in amplitude.

- If $c_v^2 - 4mk = 0$, the system is termed **critically-derdamped**. The roots of the characteristic equation are repeated, corresponding to *simple decay motion* with at most *one overshoot* of the systems resting position.

- If $c_v^2 - 4mk > 0$, the system is termed **overderdamped**. The roots of the characteristic equation are prurely real and distinct, corresponding to *simple exponentially decaying motion.*

To simplify the solutions coming up, we define the critical damping c_c, the damping ratio ζ, and the damped vibration frequency ω_d as:

$$\begin{aligned} c_c &= 2m\sqrt{\frac{k}{m}} = 2m\omega_n \\ \zeta &= \frac{c_v}{c_c} \\ \omega_d &= \sqrt{1-\zeta^2}\omega_n \end{aligned} \tag{7.16}$$

where the natural frequency of the system ω_n is given by:

$$\omega_n = \sqrt{\frac{k}{m}} \tag{7.17}$$

Note that ω_d will equal ω_n when the damping of the system is zero (i.e. undamped). The time solutions for the free **SDOF** system is presented below for each of the three case scenarios.

Time Solution of Unforced Underdamped SDOF Systems

When $c_v^2 - 4mk < 0$ (equivalent to $\zeta < 1$ or $c_v < c_c$), the characteristic equation has a pair of complex conjugate roots. The displacement solution for this kind of system is:

$$\begin{aligned} u(t) &= c_1 e^{-\zeta + i\sqrt{1-\zeta^2}\omega_n t} + c_2 e^{-\zeta - i\sqrt{1-\zeta^2}\omega_n t} \\ &= e^{-\zeta\omega_n t}\left[d_1 \cos(\omega_d t) + d_2 \sin(\omega_d t)\right] \\ \implies u(t) &= \underbrace{e^{-\zeta\omega_n t}}_{exponential\ decay} \underbrace{\left[u_0 \cos(\omega_d t) + \frac{\dot{u}_0 + \zeta\omega_n u_0}{\omega_d}\sin(\omega_d t)\right]}_{periodic\ motion} \end{aligned} \tag{7.18}$$

An alternate but equivalent solution is given by:

$$u(t) = A_0 \underbrace{e^{-\zeta\omega_n t}}_{exponential\ decay} \underbrace{\cos(\omega_d t - \phi_0)}_{periodic\ motion} \tag{7.19}$$

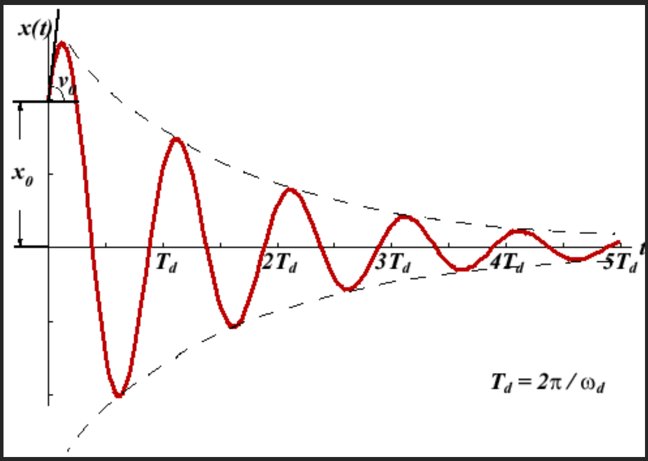


Figure 7.5: The displacement plot of an underdamped system

Note that the displacement amplitude decays exponentially (i.e. the natural logarithm of the amplitude ratio for any two displacements separated in time by a constant ratio is a constant):

$$\begin{aligned}\frac{A_k}{A_{k+1}} &= \frac{A_0 e^{-\zeta\omega_n(kT_d)}\cos(\phi_0)}{A_0 e^{-\zeta\omega_n[(k+1)T_d]}\cos(\phi_0)} \\ &= \frac{e^{-\zeta\omega_n(kT_d)}}{e^{-\zeta\omega_n[(k+1)T_d]}} \\ &= e^{\zeta\omega_n T_d} \\ \Longrightarrow \ln\left(\frac{A_k}{A_{k+1}}\right) &= \zeta\omega_n T_d = \zeta\omega_n\frac{2\pi}{\omega_d} = \frac{2\pi\zeta}{\sqrt{1-\zeta^2}}\end{aligned} \tag{7.20}$$

where $T_d = \frac{1}{f_d} = \frac{2\pi}{\omega_d}$ is the period of the damped vibration.

Time Solution of Unforced Critically-Damped Systems

When $c_v^2 - 4mk = 0$ (equivalent to $\zeta = 1$ or $c_v = c_c$), the characteristic equation has repeated real roots. The displacement solution for this kind of system is:

$$\begin{aligned} u(t) &= (c_1 + c_2 t)\, e^{-\omega_n t} \\ \implies u(t) &= e^{-\omega_n t}\left[u_0 + (v_0 + \omega_n u_0)\, t\right] \end{aligned} \tag{7.21}$$

The critical damping factor c_c can be interpreted as the **minimum damping** that results in non-periodic motion (i.e. simple decay).

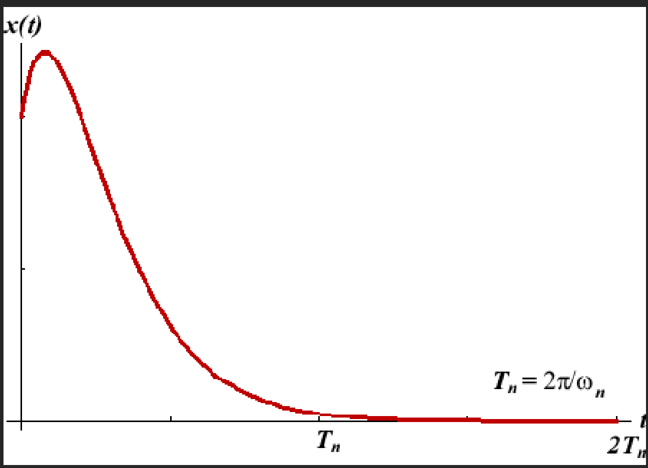


Figure 7.6: The displacement plot of a critically-damped system with positive initial displacement and velocity.

The displacement decays to a negligible level after one natural period T_n. Note that if the initial velocity v_0 is negative while the initial displacement u_0 is positive, there will exist one overshoot of the resting position in the displacement plot.

Time Solution of Unforced Overdamped SDOF Systems

When $c_v^2 - 4mk > 0$ (equivalent to $\zeta > 1$ or $c_v > c_c$), the characteristic equation has two distinct real roots. The displacement solution for this kind of system is:

$$
\begin{aligned}
u(t) &= c_1 e^{\left(-\zeta+\sqrt{\zeta^2-1}\right)\omega_n t} + c_2 e^{\left(-\zeta-\sqrt{\zeta^2-1}\right)\omega_n t} \\
\Longrightarrow \; u(t) &= \frac{u_0\omega_n\left(\zeta+\sqrt{\zeta^2-1}\right)+v_0}{2\omega_n\sqrt{\zeta^2-1}} e^{\left(-\zeta+\sqrt{\zeta^2-1}\right)\omega_n t} + \\
&+ \frac{-u_0\omega_n\left(\zeta-\sqrt{\zeta^2-1}\right)-v_0}{2\omega_n\sqrt{\zeta^2-1}} e^{\left(-\zeta-\sqrt{\zeta^2-1}\right)\omega_n t}
\end{aligned}
\tag{7.22}
$$

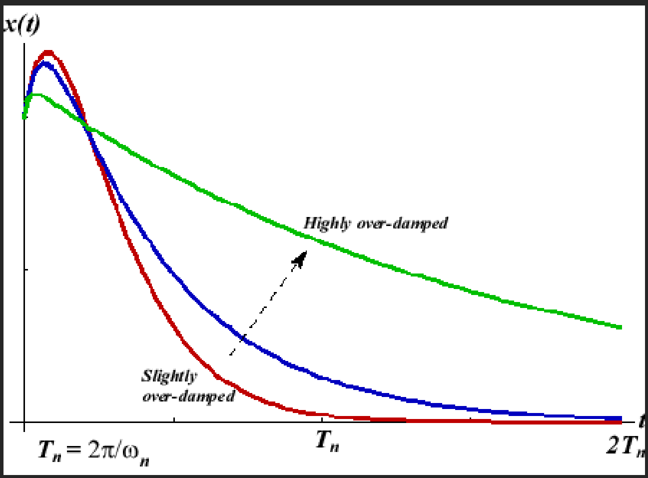


Figure 7.7: The displaceent plot of an overdamped system response

The motion of an overdamped system is non-periodic, regardless of the inital conditions. The larger the damping, the longer time to decay from an initial disturbance.

If the system is heavily damped, $\zeta \gg 1$, the displacement solution takes the approximate form:

$$
u(t) \approx u_0 + \frac{v_0}{2\zeta\omega_n}\left(1 - e^{-2\zeta\omega_n t}\right) \tag{7.23}
$$

7.2.5 SDOF Systems under Harmonic Excitaion

https://www.efunda.com/formulae/vibrations/sdof_harmo.cfm

When a **SDOF** System is forced by $f(t)$, the solution for the displacement $x(t)$ consists of two parts: the *complimentary solution*, and the *particular solution*. The complimentary solution for the problem is given by the **free vibration of an unforced SDOF System**. The **particular solution** depends on the nature fo the forcing function.

When the forcing function is **harmonic** (i.e. it consits of at most a **sine** and **cosine** at the same **frequency**, a quantity that can be expressed by the complex exponentional $e^{i\omega t}$), the method if **Undetermined Coefficients** can be used to find the particular solution. Non-harmonic forcing functions are handled by other techniques.

Consider the SDOF system forced by the harmonic function $f(t)$.

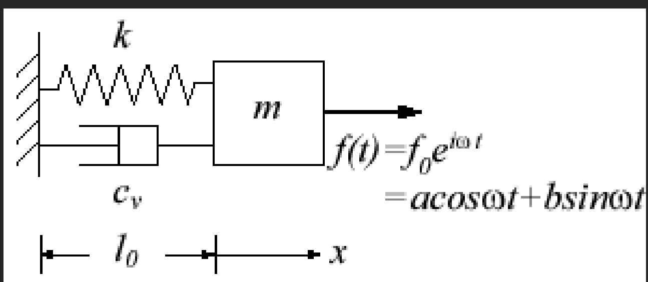


Figure 7.8: SDOF System forced by the harmonic function $f(t)$

The particular solution for this problem is found to be:

$$u_p(t) = \frac{f_0}{(k - m\omega^2) + ic\omega} e^{i\omega t} \tag{7.24}$$

The general solution is given by the sum of the complimentary and particular solutions multiplied by two weighting constants c_1 and c_2,

$$u(t) = c_1 u_c(t) + c_2 u_p(t) \tag{7.25}$$

The values of c_1 and c_2 are found by matching $u(t=0)$ to the initial conditions.

Undamped SDOF Systems under Harmonic Excitation

For an undamped system ($c_v = 0$) the total displacement solution is:

$$\begin{aligned}
u(t) &= d_1 \cos\omega_n t + d_2 \sin\omega_n t + \frac{f_0}{k - m\omega^2} e^{i\omega t} \\
\implies u(t) &= \left(u_0 - \frac{f_0}{k - m\omega^2} \right) \cos\omega_n t \\
&\quad + \left(\frac{v_0 - \frac{i\omega f_0}{k - m\omega^2}}{\omega_n} \right) \sin\omega_n t \\
&\quad + \frac{f_0}{k - m\omega^2} e^{i\omega t}
\end{aligned} \tag{7.26}$$

If the forcing frequency is close to the natural frequency, $\omega \approx \omega_n$, the system will exhibit **resonance** (very large displacements) due to near-zeros in the denominators of $u(t)$.

When the forcing frequency is equal to the natural frequency, we cannot use the $u(t)$ given above as it would give divide-by-zero. Instead we must use **L'Hôspital's Rule** to derive a solution free of zeros in the denominators.

$$\begin{aligned}
u(t) &= u_0 \cos\omega_n t + \frac{v_0}{\omega_n} \sin\omega_n t \\
&\quad + \lim_{\omega \to \omega_n} \left\{ \frac{f_0}{k - m\omega^2} \left(e^{i\omega t} - \cos\omega_n t - i\omega \sin\omega_n t \right) \right\} \\
&= u_0 \cos\omega_n t + \frac{v_0}{\omega_n} \sin\omega_n t - \frac{f_0 \omega_n}{2k} \left(ite^{i\omega)nt} - i \sin\omega_n t \right)
\end{aligned} \tag{7.27}$$

To simplify $u(t)$, let's assume that the driving force consists only of the cosine function $f(t) = f_0 \cos\omega t$.

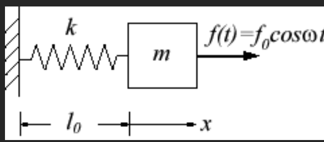


Figure 7.9: SDOF System forced by the harmonic function $f(t) = f_0 \cos \omega t$

The displacement solution reduces to:

$$
\begin{aligned}
u(t) &= u_0 \cos \omega_n t + \frac{v_0}{\omega_n} \sin \omega_n t \\
&\quad + \lim_{\omega \to \omega_n} \left\{ \frac{f_0}{k - m\omega^2} (\cos \omega t - \cos \omega_n t) \right\} \\
&= \underbrace{u_0 \cos \omega_n t + \frac{v_0}{\omega_n} \sin \omega_n t}_{free\ vibration\ (complementary)} + \underbrace{\frac{f_0 \omega_n t}{2k}}_{\substack{amplitude \\ linearly \\ increased}} \sin \omega_n t
\end{aligned}
\tag{7.28}
$$

This solution contains one term multiplied by t. This term will cause the displacement amplitude to increase linearly wit time as the forcing function pumps energy into the system.

The maximum displacement of an undamped system forced at its resonant frequency will increase unbounded according to the solution for $u(t)$ above. However, real systems will inject additional physics once displacements become large enough. These additional physics (nonlinear plastic deformation, heat transfer, buckling etc.) will serve to limit the maximum displacement exhibited by the system, and allow one to escape the "sudden death" impression that such systems will immediately fail.

Damped SDOF Systems under Harmonic Excitation

$$
m\ddot{u} + c\dot{u} + ku = F_0 \cos(\omega t) \tag{7.29}
$$

then **steady state** solution is in the form:

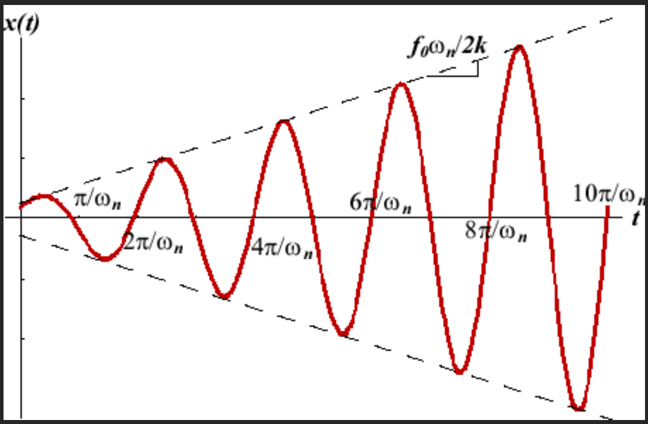


Figure 7.10: Resonance response of undamped SDOF with harmonic excitation

$$u_{s.s.} = u_0 \cos(\omega t - \phi) \tag{7.30}$$

If we plug the **steady state** solution to the **equation of motion**, we get:

$$\begin{aligned} u_o \left[\left(k - m\omega^2 \right) \cos \omega t - \phi - c\omega \sin(\omega t - \phi) \right] &= F_0 \cos(\omega t) \quad | \times \frac{1}{k} \\ u_o \left[\left(1 - \frac{\omega^2}{\omega_n^2} \right) \cos \omega t - \phi - 2\zeta \frac{\omega}{\omega_n} \sin(\omega t - \phi) \right] &= \frac{F_0}{k} \cos(\omega t) \end{aligned} \tag{7.31}$$

because:

$$\frac{k}{m} = \omega_n \tag{7.32}$$

$$\zeta = \frac{c}{c_c} \tag{7.33}$$

$$c = 2m\sqrt{\frac{k}{m}} = 2m\omega_n \tag{7.34}$$

$$\frac{c}{k} = \frac{\zeta c_c}{k} = \frac{\zeta 2m\omega_n}{k} = \frac{2\zeta\omega_n^2}{omega_n^2} = \frac{2\zeta}{\omega_n} \tag{7.35}$$

after utilising the sine and cosine relations:

$$\begin{aligned} \cos(\omega t - \phi) &= \cos\omega t\cos\phi + \sin\omega t\sin\phi \\ \sin(\omega t - \phi) &= \sin\omega t\cos\phi + \cos\omega t\sin\phi \end{aligned} \tag{7.36}$$

we get a set of two equatons:

$$\begin{aligned} u_o\left[\left(1-\frac{\omega^2}{\omega_n^2}\right)\cos\phi + 2\zeta\frac{\omega}{\omega_n}\sin\phi\right]\cos\omega t &= \frac{F_0}{k}\cos(\omega t) \\ u_o\left[\left(1-\frac{\omega^2}{\omega_n^2}\right)\sin\phi - 2\zeta\frac{\omega}{\omega_n}\cos\phi\right]\sin\omega t &= 0 \end{aligned} \tag{7.37}$$

and when solving just for amplitudes this reduces to a set of two algebraic equations which is then solved for u_0 and ϕ:

$$\begin{aligned} u_o\left[\left(1-\frac{\omega^2}{\omega_n^2}\right)\cos\phi + 2\zeta\frac{\omega}{\omega_n}\sin\phi\right] &= \frac{F_0}{k} \\ u_o\left[\left(1-\frac{\omega^2}{\omega_n^2}\right)\sin\phi - 2\zeta\frac{\omega}{\omega_n}\cos\phi\right] &= 0 \end{aligned} \tag{7.38}$$

The solution (**transfer function**) is finally:

$$\begin{aligned} u_0 &= \frac{\frac{F_0}{k}}{\left[\left(1-\frac{\omega^2}{\omega_n^2}\right)^2 + \left(2\zeta\frac{\omega}{\omega_n}\right)^2\right]^{\frac{1}{2}}} \\ \phi &= \tan^{-1}\left(\frac{2\zeta\frac{\omega}{\omega_n}}{1-\frac{\omega^2}{\omega_n^2}}\right) \end{aligned} \tag{7.39}$$

where:
u_0 is displacement amplitude
$\frac{F_0}{k}$ is displacement from static simulation.

$$H_0 = \frac{\frac{F_0}{k}}{\left[\left(1 - \frac{\omega^2}{\omega_n^2}\right)^2 + \left(2\zeta\frac{\omega}{\omega_n}\right)^2\right]^{\frac{1}{2}}} \tag{7.40}$$

7.3 Damping

7.3.1 Rayleigh Damping

https://www.simscale.com/knowledge-base/rayleigh-damping-coefficients/

Let **Rayleigh Damping** be defined as:

$$c = \alpha k + \beta m \tag{7.41}$$

where:
c is damping value [-],
α is stiffness dependent damping coefficient,
β is inertia (mass) dependent damping coefficient,
m is mass and
k is stiffness.

Thus, substituting this relation to the equation of motion:

$$\begin{aligned} m\ddot{(u)} + (\alpha k + \beta m)\,\dot{u} + ku &= f(t) \qquad | \times \frac{1}{m} \\ \ddot{u} + \left(\alpha \frac{k}{m} + \beta \frac{m}{m}\right)\dot{u} + \frac{k}{m}u &= \frac{f(t)}{m} \quad | \frac{k}{m} = \omega_n^2 \\ \ddot{u} + \left(\alpha \omega_n^2 + \beta\right)\dot{u} + \omega_n^2 u &= \frac{f(t)}{m} \end{aligned} \tag{7.42}$$

Damping Ratio:

$$\begin{aligned} \zeta &= \frac{c}{2m\omega_n} \\ \zeta &= \frac{1}{2m\omega_n}(\alpha k + \beta m) \\ \zeta &= \frac{1}{2}\left(\alpha \omega_n + \frac{\beta}{\omega_n}\right) \end{aligned} \tag{7.43}$$

where:
ζ is damping ratio,
c is damping value [-],
m is mass and
ω_n is natural frequency [rad/s]

Substituting back:

$$\ddot{u} + 2\zeta\omega_n\dot{u} + \omega_n^2 ku = \frac{f(t)}{m} \tag{7.44}$$

When **Damping is proportional to inertia**:

In this case, the stiffness coefficient $\alpha = 0$ and thus:

$$\zeta = \frac{\beta}{2\omega_n} \tag{7.45}$$

For a given constant value of β, it is seen that the damping is inversely proportional to the natural frequency, as shown in the illustration:

Moreover, if one computes β from the damping ration ζ_1 at a given natural frequency ω_1, all the natural frequencies below it will be amplified and the frequencies above it will be attenuated. The effect is more dramatic the farther the frequencies are from the reference value.

When **Damping is proportional to stiffness**:

In this case, the mass coefficient $\beta = 0$ and thus:

$$\zeta = \frac{1}{2}\alpha\omega_n \tag{7.46}$$

It is seen that, contrary to the first case, here the damping id directly proportional to the natural frequency:

If one computes α from the damping ratio ζ_1 at a natural frequency ω_1, then the natural frequencies below will be attenuated and the frequencies above will be amplified.

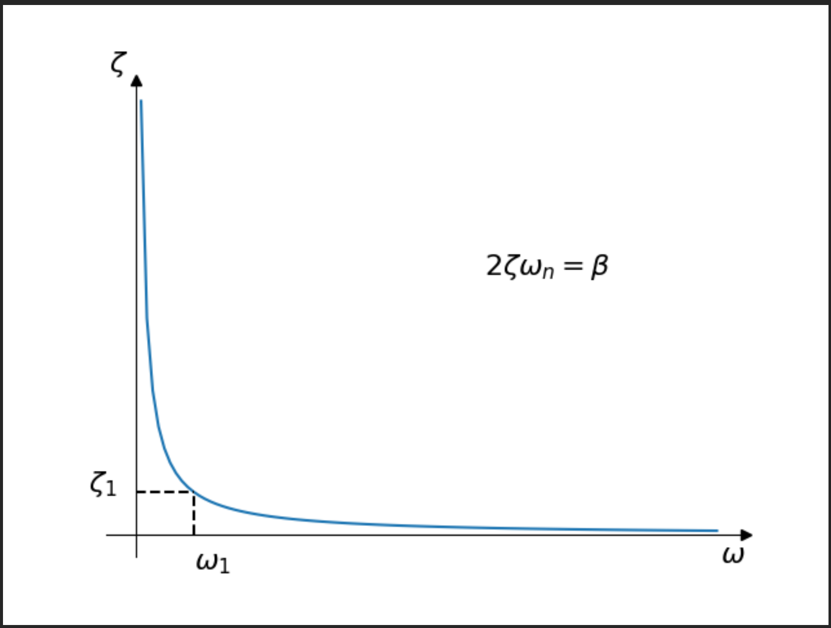


Figure 7.11: Schematic of damping proportional to inertia

General Case:

In the case of using the model with two parameters, the proportionality of damping against frequency is convex:

In this case one needs two damping ratios and two natural frequencies to create a pair of equations and solve for α and β. The model gives some flexibility on where to place the natural frequencies, but in general, frequencies too far away from the ones used in the computation will be amplified.

In the particular case of using equal damping ratios for the two frequencies, it is important to note that the damping ratio **will not be constant inside the range** defined by the sample points, but the inner frequencies will be attenuated. That is, the inner frequencies will have a lower damping ratio.

Computing the Rayleigh Damping Coefficients

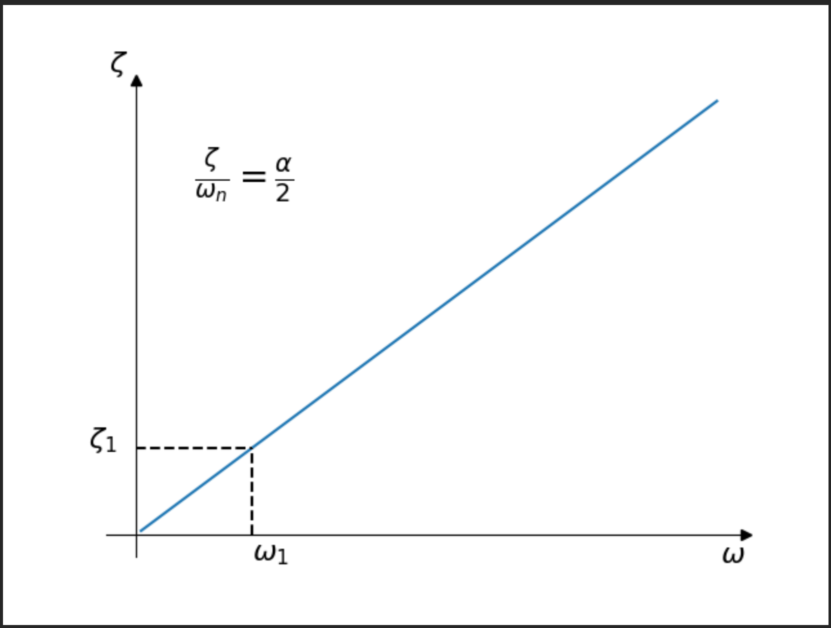


Figure 7.12: Schematic of damping proportional to stiffness

In the most common case, a transient response curve frim the system is obtained and the damping ratio ζ_1 is determined for the lowest natural frequency ω_1 by measuring the (logarithmic) attenuation of successive peaks.

$$\begin{aligned}
\zeta &= \frac{\delta}{\sqrt{\delta^2 + (2\pi)^2}} \\
\delta &= \ln \frac{x_0}{x_1} \\
f &= \frac{1}{T} = \frac{1}{t_1 - t_0} \\
\omega &= 2\pi f
\end{aligned} \tag{7.47}$$

It is then most common to assume the case of damping proportional to the stiffness, that is, $\beta = 0$, and the α stiffness coefficient is computed from:

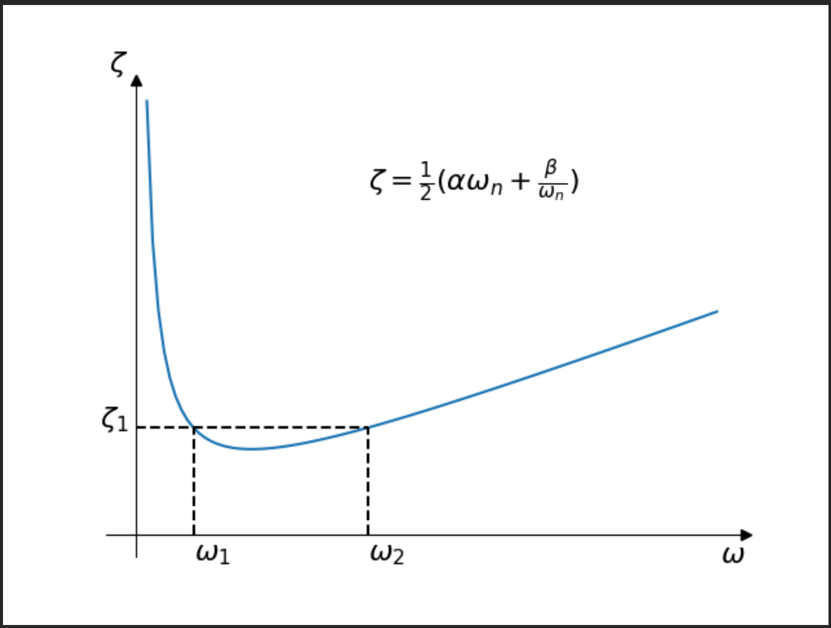


Figure 7.13: Schematic of full damping model

$$\alpha = \frac{2\zeta_1}{\omega_1} = \frac{\zeta + 1}{\pi f_1} \tag{7.48}$$

If the knowledge on the system indicates the case of damping decreasing with the frequency, then one can assume the case of damping proportional to the inertia, where $\alpha = 0$ and determine the mass coefficient β:

$$\beta = 2\zeta_1\omega_1 = 4\pi\zeta_1 f_1 \tag{7.49}$$

If there is not such test data or knowledge of the system, or if one wishes to apply an approximate damping ration over a range of frequencies, then we can use the general case and build a system of two equations:

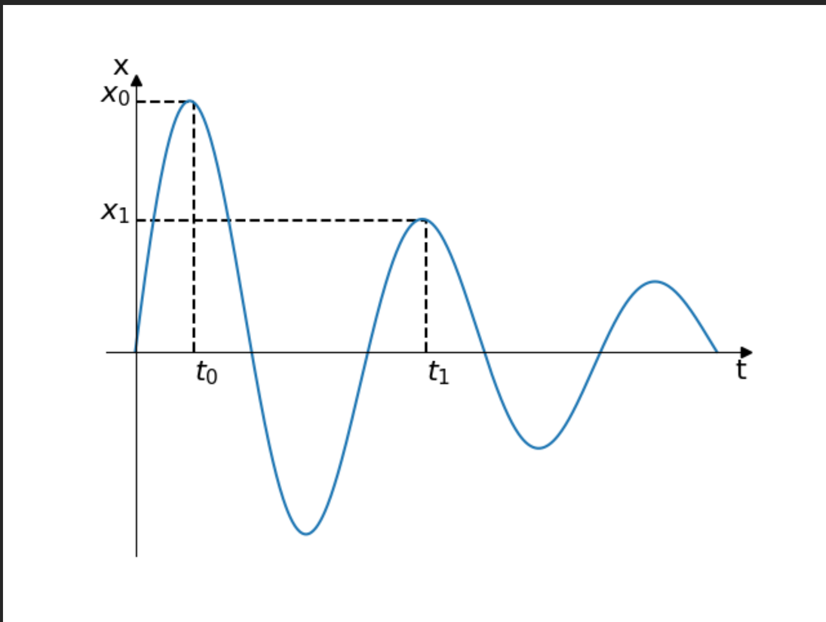


Figure 7.14: Determination of the damping ration from the logarithmic decay

$$
\begin{aligned}
\zeta_1 &= \frac{1}{2}\left(\alpha\omega_1 + \frac{\beta}{\omega_1}\right) \\
\zeta_2 &= \frac{1}{2}\left(\alpha\omega_2 + \frac{\beta}{\omega_2}\right)
\end{aligned}
\tag{7.50}
$$

Then solve for the unknown coefficients, keeping in mint the considerations given above for the general case and the influence of the model on natural frequencies inside and outside the range of interest. That is, perhaps one wants to achieve a mean damping ratio over the range, then compensate the attenuation by modifying the input damping rations, or by performing some least-squares approximation from more than two frequency points.

7.4 Modal decomposition

7.4.1 Definition

Let:

$$\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{F}\left(t\right) \tag{7.51}$$

be the **equation of motion**, then:
$\boldsymbol{\Lambda}$ is a **spectral** matrix
$\boldsymbol{\Psi}$ is a **mode shape** matrix,

such that:

$$\boldsymbol{\Lambda} = \begin{bmatrix} \lambda_1^2 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \lambda_n^2 \end{bmatrix} \tag{7.52}$$

where λ_n^2 is the n-th eigenvalue of the problem $(\mathbf{K} - \lambda^2\mathbf{M})\mathbf{X} = 0$

and:

$$\boldsymbol{\Psi} = \begin{bmatrix} \boldsymbol{\psi}_1 & \dots & \boldsymbol{\psi}_n \end{bmatrix} \tag{7.53}$$

where $\boldsymbol{\psi}_n = \{\psi_{n,1}, \dots, \psi_{n,m}\}^T$ is the eigenvector of the n-th eigenvalue λ_n^2.

The **displacement u** can be expressed as a linear combination of eigenvectors such that:

$$\begin{aligned} \mathbf{u}(t) &= \boldsymbol{\Psi}\mathbf{q}(t) \\ &= \begin{bmatrix} \boldsymbol{\psi}_1 & \dots & \boldsymbol{\psi}_n \end{bmatrix} \mathbf{q}(t) \\ &= \begin{bmatrix} \boldsymbol{\psi}_{1,1} & \dots & \boldsymbol{\psi}_{n,1} \\ \vdots & \vdots & \vdots \\ \boldsymbol{\psi}_{1,m} & \dots & \boldsymbol{\psi}_{n,m} \end{bmatrix} \begin{bmatrix} q_1(t) \\ \vdots \\ q_n(t) \end{bmatrix} \end{aligned} \tag{7.54}$$

where $\mathbf{q}(t)$ is a vector of modal coefficients.

Then:

$$\begin{aligned}\dot{\mathbf{u}}(t) &= \mathbf{\Psi}\dot{\mathbf{q}}(t)\\ \ddot{\mathbf{u}}(t) &= \mathbf{\Psi}\ddot{\mathbf{q}}(t)\end{aligned} \tag{7.55}$$

First we substitute $\mathbf{u} = \mathbf{\Psi}\mathbf{q}$ to the **equation of motion**:

$$\mathbf{M}\mathbf{\Psi}\ddot{q} + \mathbf{C}\mathbf{\Psi}\dot{q} + \mathbf{K}\mathbf{\Psi}\mathbf{q} = \mathbf{F}(t) \tag{7.56}$$

By **premultiplying** with $\mathbf{\Psi}^T$ we get:

$$\mathbf{\Psi}^T\mathbf{M}\mathbf{\Psi}\ddot{\mathbf{q}} + \mathbf{\Psi}^T\mathbf{C}\mathbf{\Psi}\dot{\mathbf{q}} + \mathbf{\Psi}^T\mathbf{K}\mathbf{\Psi}\mathbf{q} = \mathbf{\Psi}\mathbf{F}(t) = \mathbf{Q}(t) \tag{7.57}$$

where $\mathbf{Q}(t) = \mathbf{\Psi}\mathbf{F}(t)$ is called a **modal load**.

7.4.2 Orthonormalised Modal Base

Then one can exploit the properties of **orthonormalised eigenvectors**, where:

$$\boldsymbol{\psi}_r^T\boldsymbol{\psi}_s = 0 \Leftrightarrow r \neq s \tag{7.58}$$

The **mass**, **damping** and **stiffness** matrices are therefore reduced to:

$$
\begin{aligned}
\mathbf{\Psi}^T\mathbf{M}\mathbf{\Psi} &= \begin{bmatrix} m_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & m_n \end{bmatrix} \\
\mathbf{\Psi}^T\mathbf{C}\mathbf{\Psi} &= \begin{bmatrix} c_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & c_n \end{bmatrix} \\
\mathbf{\Psi}^T\mathbf{K}\mathbf{\Psi} &= \begin{bmatrix} k_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & k_n \end{bmatrix}
\end{aligned}
\tag{7.59}
$$

where:
m_i is the **modal mass** of the i-th shape
c_i is the **modal damping** of the i-th shape
k_i is the **modal stiffness** of the i-th shape

Note:
When are the eigenvectors **orthonormalised** to **mass**, the above equations reduce to:

$$\begin{aligned}
\mathbf{\Psi}^T\mathbf{M}\mathbf{\Psi} &= \begin{bmatrix} 1.0 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & 1.0 \end{bmatrix} = \mathbf{I} \\
\mathbf{\Psi}^T\mathbf{C}\mathbf{\Psi} &= \begin{bmatrix} c_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & c_n \end{bmatrix} \\
\mathbf{\Psi}^T\mathbf{K}\mathbf{\Psi} &= \begin{bmatrix} \lambda_1^2 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \lambda_n^2 \end{bmatrix} = \mathbf{\Lambda}^2
\end{aligned} \tag{7.60}$$

Mass normalisation:

$$\boldsymbol{\psi}_{i,mass} = \frac{\boldsymbol{\psi}_i}{\boldsymbol{\psi}_i^T \mathbf{m}_i \boldsymbol{\psi}_i} \tag{7.61}$$

where:
$\mathbf{m}_i$ is the i-th row of the **mass** matrix $\mathbf{M}$.

7.4.3 MDOF to SDOF

The MDOF **equation of motion** can be then rewritten as a set of **SDOF** equations:

$$\begin{gathered}
\mathbf{\Psi}^T\mathbf{M}\mathbf{\Psi}\ddot{\mathbf{q}}(t) + \mathbf{\Psi}^T\mathbf{C}\mathbf{\Psi}\dot{\mathbf{q}}(t) + \mathbf{\Psi}^T\mathbf{K}\mathbf{\Psi}\mathbf{q}(t) = \mathbf{\Psi}^T\mathbf{F}(t) \\
\mathbf{M}_\Psi\ddot{\mathbf{q}}(t) + \mathbf{C}_\Psi\dot{\mathbf{q}}(t) + \mathbf{K}_\Psi\mathbf{q}(t) = \mathbf{Q}(t)
\end{gathered} \tag{7.62}$$

Written explicitely:

$$\begin{bmatrix} m_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & m_n \end{bmatrix} \begin{bmatrix} \ddot{q}_1(t) \\ \vdots \\ \ddot{q}_n(t) \end{bmatrix} + \begin{bmatrix} c_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & c_n \end{bmatrix} \begin{bmatrix} \dot{q}_1(t) \\ \vdots \\ \dot{q}_n(t) \end{bmatrix} + \begin{bmatrix} k_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & k_n \end{bmatrix} \begin{bmatrix} q_1(t) \\ \vdots \\ q_n(t) \end{bmatrix} = \begin{bmatrix} Q_1(t) \\ \vdots \\ Q_n(t) \end{bmatrix} \tag{7.63}$$

It is important to note that all modal matrices (mass, damping and stiffness) are purely diagonal (with simple enough damping), so the **MDOF** problem reduces to a series of **SDOF** problems:

$$\begin{aligned} m_1\ddot{q}_1(t) + c_1\dot{q}_1(t) + k_1 q_1(t) &= Q_1(t) \\ m_2\ddot{q}_2(t) + c_2\dot{q}_2(t) + k_2 q_2(t) &= Q_2(t) \\ &\vdots \\ m_n\ddot{q}_n(t) + c_n\dot{q}_n(t) + k_n q_n(t) &= Q_n(t) \end{aligned} \tag{7.64}$$

which are solved separately and the results are **linearly** combined:

$$\begin{aligned} \mathbf{u} &= \sum_{i=1}^{n} \boldsymbol{\psi}_i q_i \\ \dot{\mathbf{u}} &= \sum_{i=1}^{n} \boldsymbol{\psi}_i \dot{q}_i \\ \ddot{\mathbf{u}} &= \sum_{i=1}^{n} \boldsymbol{\psi}_i \ddot{q}_i \end{aligned} \tag{7.65}$$

Note: Above equation can be also written as:

$$
\begin{aligned}
\mathbf{u} &= \sum_{i=1}^{n} \boldsymbol{\psi}_i q_{u,i} \\
\mathbf{v} &= \sum_{i=1}^{n} \boldsymbol{\psi}_i q_{v,i} \\
\mathbf{a} &= \sum_{i=1}^{n} \boldsymbol{\psi}_i q_{a,i}
\end{aligned}
\tag{7.66}
$$

where:
$\mathbf{u}$ is displacement
$\mathbf{v}$ is velocity
$\mathbf{a}$ is acceleration
and
$\mathbf{q}_u$ is modal displacement
$\mathbf{q}_v$ is modal velocity
$\mathbf{q}_a$ is modal acceleration

7.4.4 Initial Conditions

To get the initial modal conditions, one simply:

$$
\begin{aligned}
\mathbf{q}_0 &= \boldsymbol{\Psi}^{-1}\mathbf{u}_0 \\
\dot{\mathbf{q}}_0 &= \boldsymbol{\Psi}^{-1}\dot{\mathbf{u}}_0 \\
\ddot{\mathbf{q}}_0 &= \boldsymbol{\Psi}^{-1}\ddot{\mathbf{u}}_0
\end{aligned}
\tag{7.67}
$$

where:
$\boldsymbol{\Psi}^{-1}$ is an inverse of eigenshape matrix $\Leftrightarrow$ all eigenvalues and eigenshapes are used.

Note: This is not usually so. Normally one would use **reduced modal base**, where only the first **n** eigenmodes are used. Then $\mathbf{\Psi}^{-1}$ is written as $\mathbf{\Psi}^{+}$ and called the **pseudoinverse of mode shape matrix** defined (at least in **numpy** - *numpy.linal.pinv()*) as:

The pseudo-inverse of a matrix $\mathbf{A}$, denoted $\mathbf{A}^{+}$, is defined as: "the matrix that 'solves' [the least-squares problem] $\mathbf{Ax} = \mathbf{b}$," i.e., if $\overline{\mathbf{x}}$ is said solution, then $\mathbf{A}^{+}$ is that matrix such that $\overline{\mathbf{x}} = \mathbf{A}^{+}\mathbf{b}$.

It can be shown that if $\mathbf{Q}_1\mathbf{\Sigma}\mathbf{Q}_2^T = \mathbf{A}$ is the singular value decomposition of $\mathbf{A}$, then $\mathbf{A}^{+} = \mathbf{Q}_2\mathbf{\Sigma}^{+}\mathbf{Q}_1^T$, where $\mathbf{Q}_{1,2}$ are orthogonal matrices, $\mathbf{\Sigma}$ is a diagonal matrix consisting of $\mathbf{A}$'s so-called singular values, (followed, typically, by zeros), and then $\mathbf{\Sigma}^{+}$ is simply the diagonal matrix consisting of the reciprocals of $\mathbf{A}$'s singular values (again, followed by zeros).

Chapter 8

Kinematics

8.1 Rigid Body Motion

A rigid freebody relation between **slave** (dependent) and **master** (independent) DOFs is calculated following the equation:

$$
\begin{aligned}
\mathbf{T}_s &= \mathbf{T}_m + \mathbf{R}_s \times \mathbf{x} \\
\mathbf{R}_s &= \mathbf{R}_m
\end{aligned}
\tag{8.1}
$$

where $\mathbf{T}$ is a translations vector $= \{\mathbf{T}_1, \mathbf{T}_2, \mathbf{T}_3\}$, $\mathbf{R}$ is a rotations vector $= \{\mathbf{R}_1, \mathbf{R}_2, \mathbf{R}_3\}$, $\mathbf{x}$ is a vector from slave (base) to the master node (tip) $= \{\mathbf{x}_s - \mathbf{x}_m, \mathbf{y}_s - \mathbf{y}_m, \mathbf{z}_s - \mathbf{z}_m\}$ and $\times$ is a vector (cross) product.

Chapter 9

Iterative Methods

9.1 Root finding problems

9.1.1 Incremental Search Method

The approximate locations of the roots are best determined by plotting the function. Often a very rough plot, based on a few points, is sufficient to provide reasonable starting values.

The basic thought behind the incremental search method is simple: If $f(x_1)$ and $f(x_2)$ have opposite signs, then there is at least one root in the interval (x_1, x_2). If the interval is small enough, it is likely to contain a single root. Thus the zeros of $f(x)$ can be detected by evaluating the function $f(x)$ at intervals Δx and looking for a change in sign.

There are several potential problems with the incremental search method:

- It is possible to miss two closely spaced roots if the search increment Δx is larger than the spacing of the roots
- A double root (two roots that coincide) will not be detected.

- Certain singularities (poles) of $f(x)$ can be mistaken for roots. For example, $f(x) = tan(x)$ changes sign at $x \pm \frac{1}{2}n\pi$, $n = 1, 3, 5, \ldots$. However, these locations are not true zeroes, because the function does not cross the x-axis.

Example of incremental function in python:

```python
import math

def rootsearch(f: object, a: float, b: float, dx: float) -> float:
    """function using incremental search on the interval <a, b>, in increments
    of dx, to find the bounds <x1, x2> of the smallest root of f(x).

    Args:
        f (object): function to find the smallest root of
        a (float):  lower bound of the search region
        b (float):  upper bound of the search region
        dx (float): search step

    Returns:
        x1 = x2 = None if no roots were detected.
        x1 < x2 = the bounds specified by dx (x2 = x1 + dx)
    """
    x1, f1 = a, f(a)
    x2, f2 = b, f(b)
    while math.sign(f1) == math.sign(f2):
        if x1 >= b:
            return None, None
        x1, f1 = x2, f2
        x2, f2 = x2 + dx, f(x2 + dx)
    else:
        return x1, x3

```

9.1.2 Bisection Method

After a root $f(x) = 0$ has been bracketed in the interval $< x_1, x_2 >$, several methods can be used to close in on it. The method of **bisection** accomplishes it by *halving* the interval until the error is sufficiently small:

$$\begin{aligned} \varepsilon_{err} &= |x_2 - x_1| \\ \varepsilon_{err} &\leq \varepsilon_{tol} \end{aligned} \tag{9.1}$$

The method of **bisection** uses the same principle as the **incremental** search method. If there is a root $f(x) = 0$ in the interval $x \in < x_1, x_2 >$ then:

$$sign(f(x_1)) \neq sign(f(x_2)) \tag{9.2}$$

To halve the interval, we compute $f(x_3)$, where $x_3 = \frac{1}{2}(x_1 + x_2)$ is the midpoint of the interval. If $sign(f(x_3)) \neq sign(f(x_2))$, then the root must be in the interval $< x_3, x_2 >$ and we replace the x_1 by x_3. Otherwise the root must be in the interval $< x_1, x_3 >$ and we replace thr x_2 by x_3. In either case the interval is **halved**. The bisection is repeated until the interval has been reduced such that $\varepsilon_{err} \leq \varepsilon_{tol}$.

It is easy to compute the number of bisections needed to reach the prescribed ε_{tol}. The original interval Δx is reduced to $\Delta x/2$ after one bisection, $\Delta x/2^2$ after two bisections, and after n bisections it is $\Delta x/2^n$. Setting $\Delta x/2^n = \varepsilon$ and solving for n, we get:

$$n = \frac{\ln(\Delta x/\varepsilon)}{\ln 2} \tag{9.3}$$

Since n must be an integer, the ceiling of n is used (smalles integer greater n).

Python implementation:

```python
import math

def bisection(f: callable, x1: float, x2: float,
              deny_increase: bool = True, tol: float = 1.E-09) -> float:
    """Root finding algorithm using a bisection method. The root must be bracketed in
    (x1, x2).

    Args:
        f (callable):          reference to the function to evaluate
        x1 (float):            lower bounds of the root bracket
        x2 (float):            upper bounds of the root bracket
        deny_increase (bool): returns None if |f(x)| increases upon bisection
                               (default = False)
        tol (float):           allowed tolerance for root finding
                               (default = 1.0E-09)

    Returns:
        (float): if root has been found
        None:    otherwise

    Raises:
        ValueError: The root has not been bracketed.
    """
    f1 = f(x1)
    if f1 == 0.: # jackpot -> no more searching
        return x1

    f2 = f(x2)
    if f2 == 0.: # jackpot -> no more searching
        return x2

    if math.sign(f1) == math.sign(f2):
        raise ValueError(f"The root has not been bracketed in <{x1:f}, {x2:f}>.")

    n = int(math.ceil(math.log(abs(x2 - x1) / tol) / math.log(2.)))

```

```
    # the bisection itself
    for i in range(n):
        x3 = 0.5 * (x1 + x2)
        f3 = f(x3)
        # abs value of f(x) is larger than before
        if deny_increase and ((abs(f3) > abs(f1)) or (abs(f3) > abs(f2))):
            return None
        if f3 == 0.: # jackpot -> no more searching
            return x3
        elif math.sign(f3) != math.sign(f2):
            x1, f1 = x3, f3
        else:
            x2, f2 = x3, f3

    # return the midpoint of the resulting span
    return 0.5 * (x1 + x2)

```

9.1.3 Methods Based on Linear Interpolation

Secant and False Position Methods

The **secant** and **false position** methods are closely related. Both methods require two starting estimates of the root, say x_1 and x_2. The function $f(x)$ is assumed to be approximately linear near the root, so that the improved value x_3 of the root can be estimated by linear interpolation between x_1 and x_2.

This yields the relationship:

$$\frac{f_2}{x_3 - x_2} = \frac{f_1 - f_2}{x_2 - x_1} \tag{9.4}$$

where the notation $f_i = f(x_i)$ is used. Thus the improved estimate of the root is:

$$x_3 = x_2 - f_2 \frac{x_2 - x_1}{f_2 - f_1} \tag{9.5}$$

The difference between **false position** and **secant** method is in the manner the new estimate is dealt with.

- The **false position** method is similar to the **bisection** method where it is necessary that the root is **bracketed** by $x \in < x_1, x_2 >$. When a new estimate x_3 is obtained, then if $sign(f_3) = sign(f_1)$, we let $x_1 \leftarrow x_3$, otherwise we let $x_2 \leftarrow x_3$. In this manner, the root is always bracketed in $< x_1, x_2 >$. Then we run a new iteration until convergence criteria are met.
- The **secant** method is different in two ways. It does not require prior bracketing of the root, and it discards the oldest prior estimate of the root (i.e. after x_3 is computed, we let $x_1 \leftarrow x_2 \leftarrow x_3$).

The convergence of the **secant** method is **superlinear**, with the error behaving as $E_{k+1} = cE_k^{1.618}$, where the exponent 1.618... is called the "golden ratio".

Python implementation of the secant method:

```python
import math

def secant(f: callable, x1: float, x2: float,
           tol: float = 1.E-09, maxiter: int = 1000) -> float:
    """Root finding algorithm using a secant method.

    Args:
        f (callable):  reference to the function to evaluate
        x1 (float):    lower bounds of the root bracket
        x2 (float):    upper bounds of the root bracket
        tol (float):   allowed tolerance for root finding
                       (default = 1.0E-09)
        maxiter (int): maximum number of iterations allowed (default = 1000)

    Returns:
        (float): if root has been found

    Raises:
        ValueError: No convergence is achievable as (f1 == f2, function is constant)
        ValueError: Maximum number of iterations reached.
    """
    f1 = f(x1)
    if f1 == 0.: # jackpot -> no more searching
        return x1

    f2 = f(x2)
    if f2 == 0.: # jackpot -> no more searching
        return x2

    # the secant method itself
    for i in range(maxiter):
        if f1 == f2:
            raise ValueError(f"The function is constant in interval ({x1:f}, {x2:f}).")
        x3 = x2 - f2 * (x2 - x1) / (f2 - f1)
        f3 = f(x3)
```

```python
        if f3 == 0.: # jackpot -> no more searching
            return x3
        elif abs(f3) <= tol:
            return x3
        else:
            # discard x1 and f1
            x1, f1, x2, f2 = x2, f2, x3, f3
    else:
        raise ValueError(f"Maximum number of iterations ({maxiter:d}) reached!")


```

Ridder's Method

Ridder's method is a clever modification of the **false position** method. Assuming ht root is bracketed in $< x_1, x_2 >$, whe first compute $f_3 = f(x_3)$, where x_3 is the midpoint of the bracket.

It can be noted:

$$
\begin{aligned}
x_3 &= \frac{1}{2}(x_1 + x_2) \\
h &= \frac{1}{2}(x_2 - x_1)
\end{aligned}
\tag{9.6}
$$

Next we introduce a new function $g(x)$ such, that:

$$
g(x) = f(x)e^{(x-x_1)Q} \tag{9.7}
$$

where the constant Q is determined by requiring the points (x_1, g_1), (x_2, g_2) and (x_3, g_3) to lie on a straight line. As before, the notation is $g_i = g(x_i)$. The improved value x_4 is then obtained by linear interpolation of $g(x)$ rather than $f(x)$.

From 9.7 we obtain:

$$
\begin{aligned}
g_1 &= f_1 \\
g_2 &= f_2 e^{2hQ} \\
g_3 &= f_3 e^{hQ}
\end{aligned}
\tag{9.8}
$$

where h is obtained from 9.6.

Using the requirement thet the three points g_1, g_2 and g_3 line on straight line or:

$$g_3 = \frac{g_1 + g_2}{2} \tag{9.9}$$

or:

$$f_3 e^{hQ} = \frac{1}{2}\left(f_1 + f_2 e^{2hQ}\right) \tag{9.10}$$

which is a quadratic equation in e^{hQ}. The solution is

$$e^{hQ} = \frac{f_3 \pm \sqrt{f_3^2 - f_1 f_2}}{f_2} \tag{9.11}$$

Linear interpolation based on points (x_1, g_1) and (x_3, g_3) now yields for the improved root:

$$\begin{aligned} x_4 &= x_3 - g_3 \frac{x_3 - x_1}{g_3 - g_1} \\ x_4 &= x_3 - f_3 e^{hQ} \frac{x_3 - x_1}{f_3 e^{hQ} - f_1} \end{aligned} \tag{9.12}$$

where in the last step the 9.8 is used. As the final step, a substitution for e^{hQ} is used from 9.11 and after some algebra the following solution is obtained:

$$x_4 = x_3 \pm (x_3 - x_1) \frac{f_3}{\sqrt{f_3^2 - f_1 f_2}} \tag{9.13}$$

it can be shown that the correct result is obtained by choosing the + sign if $f_1 - f_2 > 0$ and the minus signe if $f_1 - f_2 < 0$.

The 9.13 can be then adjusted as:

$$x_4 = x_3 + sign\,(f_1 - f_2) \times (x_3 - x_1) \frac{f_3}{\sqrt{f_3^2 - f_1 f_2}} \tag{9.14}$$

After the computation of x_4, new brackets are determined for the root and 9.14 is applied again. The procedure is repeated until the difference between two successive values of x_4 becomes negligible.

The **Ridder's** iterative formula in 9.14 has a very useful property: If x_1 and x_2 bracket the root, then $x_4 \in < x_1, x_2 >$ in all cases. In other words, once the root is bracketed, it stays bracketed.

Ridder's method can be shown to converge **quadratically**, making it faster than either the **secant** or the **false position** method.

It is the method to use if the derivative of $f(x)$ is impossible or difficult to compute.

Python implementation:

```python
import math

def ridder(f: callable, x1: float, x2: float,
           rtol: float = 1.E-09, maxiter: int = 1000) -> float:
    """Root finding algorithm using a Ridder's algorithm. The root must be
    bracketed in (x1, x2).

    Args:
        f (callable):  reference to the function to evaluate
        x1 (float):    lower bounds of the root bracket
        x2 (float):    upper bounds of the root bracket
        tol (float):   allowed tolerance for root finding
                       absolute for found root, relative for
                       change between root iteraions
                       (default = 1.0E-09)
        maxiter (int): maximum number of iterations
                       (default = 1000)

    Returns:
        (float): if root has been found
        None:    if root is not bracketed

    Raises:
        ValueError: Maximum number of iterations reached.
    """
    f1 = f(x1)
    if abs(f1) <= tol: # jackpot -> no more searching (absolute error)
        return x1

    f2 = f(x2)
    if abs(f2) <= tol: # jackpot -> no more searching (absolute error)
        return x2

    if math.sign(f1) == math.sign(f2):
        return None

    x3 = 0.5 * (x1 + x2)
```

```python

    # the Ridder's method itself
    xold = x1
    for i in range(maxiter):
        x3 = (x1 + x2) / 2
        f3 = f(x3)
        if abs(f3) <= tol: # jackpot -> no more searching (absolute error)
            return x3
        s = math.sqrt(f_3 ** 2 - f_1 * f_2)
        if s == 0.:
            return None
        dx = (x3 - x1) * f3 / s
        x4 = x3 + math.sign(f1 - f2) * dx
        f4 = f(x4)
        if abs(f4) <= tol: # jackpot -> no more searching (absolute error)
            return x4

        # test for convergence -> relative error
        if abs(x4 - xold) <= tol * abs(x4):
            return x4
        xold = x4

        # Re-bracket the root as tightly as possible
        if math.sign(f3) == math.sign(f4):
            if math.sign(f1) != math.sign(f4):
                # f1  0  f4  f3  f2
                x2, f2 = x4, f4
            else:
                # f1  f3  f4  0  f2
                x1, f1 = x4, f4
        else:
            # f1  f3  0  f4  f2
            x1, f1, x2, f2 = x3, f3, x4, f4

    else:
        raise ValueError(f"Maximum number of iterations ({maxiter:d}) reached!")

```

Newton-Raphson Method

The **Newton-Raphson** algorithm is the best known method of finding roots for a good reason: it is **simple** and **fast**. The only drawback of the method is that it uses the **derivative** $f'(x)$ of the function as well as the function $f(x)$ itself. Therefore, the **Newton-Raphson** method is usable only in problesm where $f'(x)$ can be readily computed.

The **Newton-Raphson** formula can be derived from the **Taylor series** expansion of $f(x)$ about x:

$$f(x_{i+1}) = f(x_i) + f'(x_i)(x_{i+1} - x_i) + \mathcal{O}(x_{i+1} - x_i)^2 \tag{9.15}$$

where $\mathcal{O}(z)$ is to be read as "of the order of z". If x_{i+1} is a root of $f(x) = 0$. The formula 9.15 becomes:

$$0 = f(x_i) + f'(x_i)(x_{i+1} - x_i) + \mathcal{O}(x_{i+1} - x_i)^2 \tag{9.16}$$

Assuming that $x_i \to x_{i+1}$, we can dorp the last term in 9.16 and solve for x_{i+1}.

Then the **Newton-Raphson** formula is:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \tag{9.17}$$

The formula 9.17 appriximates $f(x)$ by the straight line that is tangent to the curve at x_i. This x_{i+1} is at the intersection of the x-axis and the tangent line.

The formula including $f'(x_i)$ can be also derived using the following discrete approach (using Δ as a finite neighborhood of x_i):

$$\begin{aligned} f'(x_1) &= \frac{f(x_i + \Delta/2) - f(x_i - \Delta/2)}{(x_i + \Delta/2) - (x_i - \Delta/2)} \\ f'(x_1) &= \frac{f(x_i + \Delta/2) - f(x_i - \Delta/2)}{\Delta} \end{aligned} \tag{9.18}$$

then setting $dx = \Delta$ and $dy = f(x_i + \Delta/2) - f(x_i - \Delta/2)$:

$$f'(x_1) = \frac{dy}{dx} \tag{9.19}$$

we can write:

$$x_{i+1} = x_i - dx\frac{f(x_i)}{dy} \tag{9.20}$$

The **algorithm** for the **Newton-Raphson** method is simple:

It repeatedly applies 9.17, starting with an initial value x_0, until convergence criterion $|x_{i+1} - x_i| \leq \varepsilon_{tol}$ is reached, the ε_{tol} being the error tolerance. Only the **latest** value of x has to be stored.

The algorithm is here:

1. Let x be an estimate of the root of $f(x) = 0$.
2. Do until $|\Delta x| \leq \varepsilon_{tol}$:
   (a) Compute $\Delta x = -f(x)/f'(x)$
   (b) Let $x \leftarrow x + \Delta x$

The truncation error E in the **Newton-Raphson** formula can be shown to behave as

$$E_{i+1} = -\frac{f''(x)}{2f'(x)}E_i^2 \tag{9.21}$$

where x is the **root**. This indicates that the method converges *quadratically* (the error is the square of the error in the previous step). Consequently the number of significant figures is roughly doubled in every iteration.

Althou the **Newton-Raphson** method converges fast near the root, its global convergence characteristics are poor. The reasion is that the tangent line is not always an acceptable approximation of the function. However, the method can be made nearly fail-safe by **combining** it with **bisection**.

The following *safe version* of the **Newton-Raphson** method assumes that the root to be computed is initially bracketed in $< x_1, x_2 >$. The midpoint of the bracket is used as the initial guess of the root. The brackets are updated after each iteration. If a **Newton-Raphson** iteration does not stay within the brackets, it is disregarded and replaced with bisection. Because the **Newton-Raphson** method uses the function $f(x)$ as well as its derivative, function routines for both (denoted as f and df) must be provided.

Python implementation:

```
import math

def newtonRaphson(f: callable, df: callable, x: float,
                  tol: float = 1.E-09, maxiter: int = 1000) -> float:
    for i in range(maxiter):
        try:
            dx = - f(x) / df(x)
        except ZeroDivisionError as e:
            raise ValueError(f"Function derivation  f'(x = {x:f}) = 0")
        x += dx
        if abs(dx) <= tol * abs(x):
            return x

```

```
    else:
        raise ValueError(f"Maximum iteration number reached ({maxiter:d}).")

def newtonRaphsonImproved(f: callable, df: callable, x1: float, x2: float,
                          tol: float = 1.E-09, maxiter: int = 1000) -> float:
    f1 = f(x1)
    if abs(f1) <= tol:
        return x1

    f2 = f(x2)
    if abs(f2) <= tol:
        return x2

    if math.sign(f1) == math.sign(f2):
        raise ValueError(f"Root is not bracketed by ({x1:f}, {x2:f}).")

    x3 = (x1 + x2) / 2
    for i in range(maxiter):
        f3 = f(x3)
        if abs(f3) <= tol:
            return x3

        # tighten the brackets on the root
        if math.sign(f1) != math.sign(f3):
            x2 = x3
        else:
            x1 = x3

        # try a Newton-Raphson step
        df3 = df(x3)

        # if division by zero, push x3 out of bounds
        try:
            dx = -f3 / df3
        except ZeroDivisionError as e:
            dx = x2 - x1

        x3 += dx

        # if the result is out of brackets, use bisection
        if (x2 - x3) * (x3 - x1) < 0.:
```

```
            dx = (x2 - x1) / 2.
            x3 = x1 + dx

        # check for convergence
        if abs(dx) <= tol * abs(x2):
            return x3

    else:
        raise ValueError(f"Maximum iteration number reached ({maxiter:d}).")
```

9.2 Minimizing function of Multiple variables

Downhill Simplex Method

Downhill Simplex method, also known as **Nelder-Mead** method is best used when optimizing (finding of **minimum**) a function of *multiple* variables, especially when derivatives (the **Jacobian**) are hard or impossible to obtain.

The method is **robust** but **slow**. Special consideration must be taken when preparing the function and when supplying constraints.

The **target function** to optimise usually takes the following form:

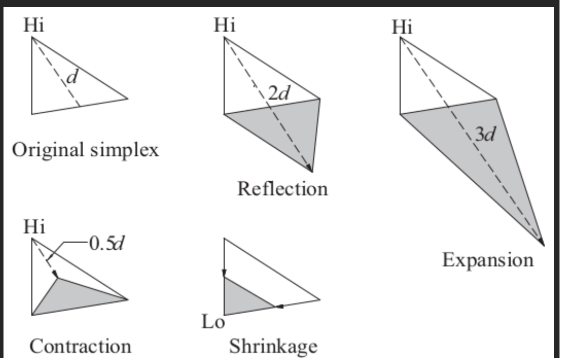
$$S(\mathbf{x}) = F(\mathbf{x}) + \sum_{i=1}^{k} \lambda_i \left(T_i - C_i(\mathbf{x})\right)^2 \tag{9.22}$$

where:
$S(\mathbf{x})$ is the optimised function
$\mathbf{x}$ is a vector of variables $\mathbf{x} = \{x_1, x_2, \ldots, x_n\}$
$F(\mathbf{x})$ is the function to optimise
T_i is the i-th constraint target value
$C_i(\mathbf{x})$ is the i-th constraint function, and
λ_i is the i-th constraint scaling weight factor, usually $\lambda \gg 1.0$.

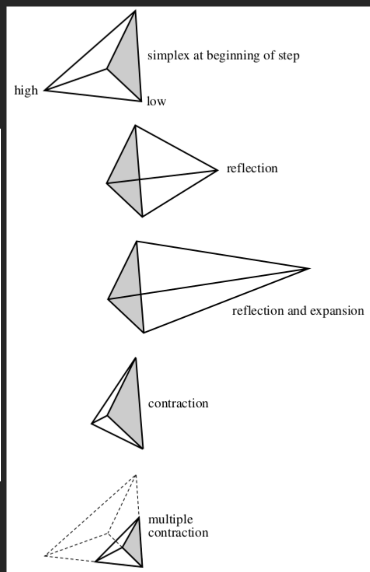
The i-th constraint is $\lambda_i \left(T_i - C_i(\mathbf{x})\right)^2$ which, using the power of 2, is made to have higher weight the more the actual $C_i(\mathbf{x})$ value differs from the constraint target T_i to both sides of the spectra. The λ_i parameter is used to **scale** the not yet met convergence criterion even further. It is especially effective for values close to 0.

> A **maximization** of a function $F(\mathbf{x})$ is actually equal to **minimization** of $-F(\mathbf{x})$.

The idea behind the **downhill simplex** method is to employ a moving **simplex** in the design space to surround the optimal point and then shrink the simplex until its dimensions reach the specified error of tolerance. In n-dimensional space a **simplex** is a figure of $n + 1$ vertices connected by straight lines and bounded by polygonal faces. If $n = 1$, a **simplex** is a line. If $n = 2$, a **simplex** is a triangle, **tetrahedron** if $n = 3$.



(a) Allowed moves for a 2D simplex



(b) Allowed moves for a 3D simplex

Figure 9.1: Allowed simplex moves for $n = 2$ and $n = 3$

The allowed moves of a **two-dimensional simplex** ($n = 2$) and **three-dimensional simplex** ($n = 3$) are illustrated on pictures 9.1.

By applying these moves in a suitable sequence, the **simplex** can **always** hunt down the minimum point, enclose it, and then shrink around it. The direction of a bove is determined by the values of $S(\mathbf{x})$ (the function to be minimised, including constraints) at the **vertices**. The vertex with the **highest** value of S is labeled Hi, and Lo denotes the vertex with the **lowest** value.

The magnitude of a move is controlled by the distance d measured from the Hi vertex to the **centroid** of the opposing vertices.

The **centroid** is simply an arithmetic average of the other vertices.

$$
\begin{aligned}
x_{iHi} &= Hi \\
x_c &= \sum_{i \neq i_{Hi}}^{n} x_i / (n-1) \\
d &= x_c - x_{iHi}
\end{aligned}
\tag{9.23}
$$

The outline of the algorithm is as follows:

- Choose a starting simplex (using side parameter)
- Cycle until $d \leq \epsilon_{tol}$
  - find **vertices** v_{Hi}, v_{Lo} with the highest and lowest value of $S(\mathbf{x})$, respectively
  - compute $d = |v_{Hi} - \sum (v \neq v_{Hi}) / (n-1)|$
  - Try **reflection** $v_n = v_{Hi} - 2d$
  - if $S(v_n) \leq S(v_{Lo})$: accept **reflection** $v_{Hi} = v_n$
    - Try **expansion**: $v_n = v_n - d$
    - if $S(v_n) \leq S(v_{Lo})$: accept **expansion** $v_{Hi} = v_n$
  - else:
    - if $S(v_n) \leq S(v_{Hi})$:
      - Try **contraction**: $v_n = v_{Hi} - d/2$
      - if $S(v_n) \leq S(v_{Hi})$: accept **contraction** $v_{Hi} = v_n$
    - else: use **shrinkage**: $v_i = (v_i + v_{Hi})/2 \quad \forall \quad v_i \neq v_{Hi}$

The **downhill simplex method** is much slower than other methods in most cases, but makes up for it in robustness. It often works for problems where other methods hang up.

The starting simplex for a three parameter optimization (with δ side parameter):

$$\mathbf{x_0} = \{x_1, x_2, x_3\}$$
$$\mathbf{x} = \begin{bmatrix} x_1 & x_2 & x_3 \\ x_1+\delta & x_2 & x_3 \\ x_1 & x_2+\delta & x_3 \\ x_1 & x_2 & x_3+\delta \end{bmatrix} \tag{9.24}$$

Python implementation:

```python
import numpy as np
import math

def downhill(F: callable, x0: np.ndarray,
     side: float = 0.1, tol: float = 1.0e-6, maxiter: int = 500,
     ro: float = 1.0, epsilon: float = 2.,
     gamma: float = 0.5, sigma: float = 0.5) -> np.ndarray:
    '''Downhill simplex method for minimization (Nelder-Mead method)

    Args:
        F (callable):    reference to scalar function that should be minimized
        x0 (np.ndarray): starting vector x = initial estimate
        side (float):    side length of the starting simplex (default = 0.1)
        tol (float):     convergence toelerance (default = 1.0e-06)
        maxiter (int):   maximum number of iterations (default = 500)
        ro (float):      reflection coefficient   (default ro      = 1.0)
        epsilon (float): expansion coefficient    (default epsilon = 2.0)
        gamma (float):   contraction coefficient  (default gamma   = 0.5)
        sigma (float):   shrink coefficient       (default sigma   = 0.5)

    Returns:
        (np.ndarray): the optimized values

    '''
    n = len(x0) # Number of variables

```

```
    # Generate starting simplex
    x = np.repeat(x0.reshape(1,-1), repeats=n+1, axis=0)
    # move each parameter by 'side'
    x[1:,:] += np.eye(n, n) * side

    # Compute values of F at the vertices of the simplex
    f = np.apply_along_axis(F, axis=1, arr=x)

    # Main loop
    for k in range(maxiter):
        # Sort vertices from lowest to highest (indexes: lowest = 0, highest = n)
        idx = np.argsort(f)
        f, x = f[idx], x[idx]

        # The centroid of all vertices except the highest xcog = sum x(i<n) / n
        # Compute the move vector d (from highest vertex to centroid of rest)
        # d = xcog - xn
        d = np.sum(x[:-1], axis=0) / n - x[-1]

        # Check for convergence
        if math.sqrt(np.dot(d, d) / n) < tol:
            print(f"[+] Convergence reached at iteration: {k:d}")
            return x[0]

        # Try reflection: xr = xcog + ro * (xcog - xn)
        xNew = x[-1] + (1.0 + ro) * d
        fNew = F(xNew)
        if fNew <= f[0]: # Accept reflection
            x[-1], f[-1] = xNew, fNew

            # Try expanding the reflection: xe = xcog + epsilon * (xr - xcog)
            xNew = x[-1] + (epsilon - ro) * d
            fNew = F(xNew)

            if fNew <= f[0]: # Accept expansion
                x[-1], f[-1] = xNew, fNew

        # Try reflection again
        elif fNew <= f[-1]: # Accept reflection
            x[-1], f[-1] = xNew, fNew
```

```
        else:
            # Try contraction: xc = xcog + gamma * (xr - xcog)
            xNew = x[-1] + gamma * d
            fNew = F(xNew)

            if fNew <= f[-1]: # Accept contraction
                x[-1], f[-1] = xNew, fNew

            else:
                # Use shrinkage: xi = xi + sigma * (xi - x0)
                x[1:] += sigma * (x[1:] - x[0])
                f[1:] = np.apply_along_axis(F, axis=1, arr=x[1:])

    print(f"[-] Iteration limit exceeded: {maxiter:d}")
    return x[0]
```

9.3 second order ODE's

Most of FEM problems are **second order Ordinary Differential Equations**, e.g:

$$\begin{aligned} m\frac{u''(t)}{d^2t} + c\frac{u'(t)}{dt} + ku(t) &= 0 \\ m\ddot{u} + c\dot{u} + ku &= 0 \\ ma + cv + ku &= 0 \end{aligned} \tag{9.25}$$

while the iterative methods are for **first order ODE's** only. A **substitution** technique is therefore used to transform the **second order ODE's** to a set of **first order ODE's** and these are then solved sequentialy.

First we begin with a **second order ODE** in the form of:

$$a\frac{y''(x)}{d^2x} + b\frac{y'(x)}{dx} + cy = d, \quad with \quad y(0) = y_0 \quad and \quad \frac{y'(0)}{dx} = y_1 \tag{9.26}$$

By stating:

$$\frac{dy}{dx} = z \tag{9.27}$$

and substituting back:

$$a\frac{dz}{dx} + bz + cy = d \tag{9.28}$$

now we can write the following set of equations:

$$\begin{aligned} z &= \frac{dy}{dx} \\ \frac{dz}{dx} &= \frac{d - cy - bz}{a} \end{aligned} \qquad \begin{cases} y(0) = y_0 \\ z(0) = y_1 \end{cases} \tag{9.29}$$

9.4 Explicit

9.4.1 Euler's method

The **Euler's Method** is an iterative method to solve ODEs. This method's huge plus is it is fast. The minus is it gradually diverges from the ideal solution.

The **Euler's Method** uses the following formula:

$$y(t+h) = y(t) + hf(x,y) \tag{9.30}$$

to construct the tangent at point x and obtain the value of $y(x+h)$, whose slope is:

$$f(x,y) \quad or \quad \frac{dy}{dx} \tag{9.31}$$

In Euler's method, you can approximate the curve of the solution byt the tangent in each interval (that is, by a sequence of shor line segments), at steps of h.

In general, if you use small step size, the accuracy of the approximation increases.

General Formula

$$y_{i+1} = y_i + hf(x_i, y_i) \tag{9.32}$$

where:
y_{i+1} is the next estimated solution value,
y_i is the current value,
h is the interval between steps and
$f(x_i, y_i)$ is the value of the derivative at the current (x_i, y_i) point.

Pseudocode:

- define: $f(x,y)$

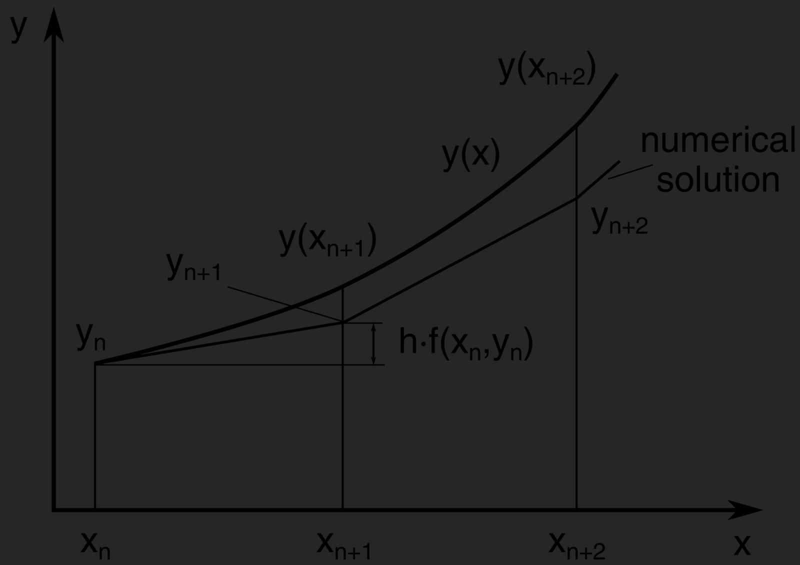


Figure 9.2: Euler's method schematic

- input: x_0, y_0
- input: h, n
- for j from 0 to $(n-1)$ do
  - $y_{j+1} = y_j + hf(x_j, y_j)$
  - $x_{j+1} = x_j + h$
  - Print x_{j+1} and $= y_{j+1}$
- End.

Note:
If thinking about a problem in time domain:

- define: $f(t, x)$
- input: t_0, x_0
- input: dt, n
- for j from 0 to $(n-1)$ do
    - $x_{j+1} = x_j + dtf(t_j, x_j)$
    - $t_{j+1} = t_j + dt$
    - Print t_{j+1} and $= x_{j+1}$
- end.

Python Code for vibration problem:

```python
#!/usr/bin/python3
import numpy as np
import matplotlib.pyplot as plt


def iterate(h, y0, func, rhs):
    num_of_odes = y0.shape[0]
    y1 = np.zeros(num_of_odes, dtype = float)
    for i in range(num_of_odes-1):
        y1[i] = y0[i] + y0[i+1] * h
    y1[-1] = func(y1, rhs)
    return y1


def euler(t, y, func, rhs):
    N = t.shape[0]
    for j in range(N-1):
        dt = t[j+1] - t[j]
        y[j+1,:] = iterate(dt, y[j,:], func, rhs[j])
        # print(y[j+1,:])
    return y


def euler_vibration():
    m = 0.55   # tonnes
    c = 3.     # Ns/mm
    k = 1000.  # N/mm

    freq = np.sqrt(k / m) / (2 * np.pi)

    # equation to solve
    # m * a + c * v + k * u = f
    # a = (f - c * v - k * u) / m
    acceleration = lambda u, f: (f - c * u[1] - k * u[0]) / m

    T = 2.0      # seconds
    dt = 0.0001 # timestep s
```

```
u_0 = 0.    # mm of initial displacement
v_0 = 0.    # mm/s of initial velocity

# times at which to solve
t = np.linspace(0, T, int(T/dt) + 1)

# force vector
F = 5000. # N of max impulse
t0 = 0.1  # impulse start time
t1 = 0.2  # impuls end time
f = np.zeros(t.shape[0], dtype=float)
# create a half sine impulse of force
for i in range(t.shape[0]):
    if t[i] >= t0 and t[i] <= t1:
        f[i] = F * np.sin((t[i] - t0) / (t1 - t0) * np.pi)
    else:
        f[i] = 0.

uva = np.zeros((t.shape[0],3), dtype=float)
uva[0,0] = u_0
uva[0,1] = v_0

uva = euler(t, uva, acceleration, f)

fig, axf = plt.subplots()
fig.subplots_adjust(right=0.60)

p1, = axf.plot(t, f, label='force', color='violet')
axu = axf.twinx()
p2, = axu.plot(t, uva[:,0], label='displacement', color='red')
axv = axf.twinx()
axv.spines.right.set_position(('axes', 1.20))
p3, = axv.plot(t, uva[:,1], label='velocity', color='blue')
axa = axf.twinx()
axa.spines.right.set_position(('axes', 1.40))
p4, = axa.plot(t, uva[:,2], label='acceleration', color='green')

axf.set_xlabel('Time [s]')
axf.set_ylabel('Force [N]')
axu.set_ylabel('Displacement [mm]')
axv.set_ylabel('Velocity [mm/s]')
```

```
    axa.set_ylabel('Acceleration [mm/s2]')

    axf.legend(handles=[p1, p2, p3, p4])

    plt.show()

if __name__ == '__main__':
    euler_vibration()

```

Which results to:

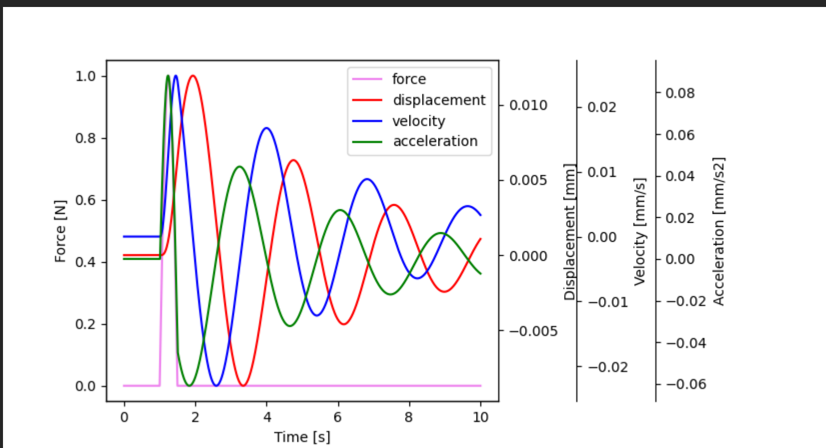


Figure 9.3: Euler's implementation for vibration problem example

The **iterate()** function:

```
def iterate(h, y0, func, rhs):
    num_of_odes = y0.shape[0]
    y1 = np.zeros(num_of_odes, dtype = float)
    for i in range(num_of_odes-1):
        y1[i] = y0[i] + y0[i+1] * h
```

```
    y1[-1] = func(y1, rhs)
    return y1

```

is written for general case of a set of ODEs. When considering the problem of vibration (set of 2 ODEs):

```
def iterate(h, u0, v0, a0, f, m , c, k):
    u1 = u0 + h * v0
    v1 = v0 + h * a0
    # m * a + c * v + k * u = f
    a1 = (f - c * v1 - k * u1) / m
    return np.array([u1, v1, a1], dtype=float)
```

which is the iteration of a set of **first order ODEs**:

$$\begin{aligned} v &= \dot{u} \\ \dot{v} &= \frac{f - c * v - k * u}{m} \end{aligned} \tag{9.33}$$

9.4.2 Runge-Kutta 4th order method

Runge-Kutta 4th order method is another explicit method for solving **first order ODEs**. The basic equation is:

$$\frac{dy}{dx} = f(x, y) \quad , y(0) = y_0 \tag{9.34}$$

The formula for the next value y_{i+1} after a step size equal to h is given by:

$$\begin{aligned}
k_1 &= hf(x_i, y_i) \\
k_2 &= hf(x_i + \frac{h}{2}, y_i + \frac{k_1}{2}) \\
k_3 &= hf(x_i + \frac{h}{2}, y_i + \frac{k_2}{2}) \\
k_4 &= hf(x_i + h, y_i + k_3) \\
y_{i+1} &= y_i + \frac{k_1}{6} + \frac{k_2}{3} + \frac{k_3}{3} + \frac{k_4}{6} + O(h^5)
\end{aligned} \tag{9.35}$$

The formula basically computes next value y_{i+1} using current y_i plus **weighted average of four increments**:

- k_1 is the increment based on the slope at the beginning of the interval, using y
- k_2 is the increment based on the slope at the midpoint of the interval, using $y + hk_1/2$
- k_3 is the increment based on the slope at the midpoint, using $y + hk_2/2$
- k_4 is the increment based on the slope at the end of the interval, using $y + hk_3/2$

The method is a fourth order method, meaning that the local truncation error is on the order of $O(h^5)$, while the total accumulated error is of order $O(h^4)$.

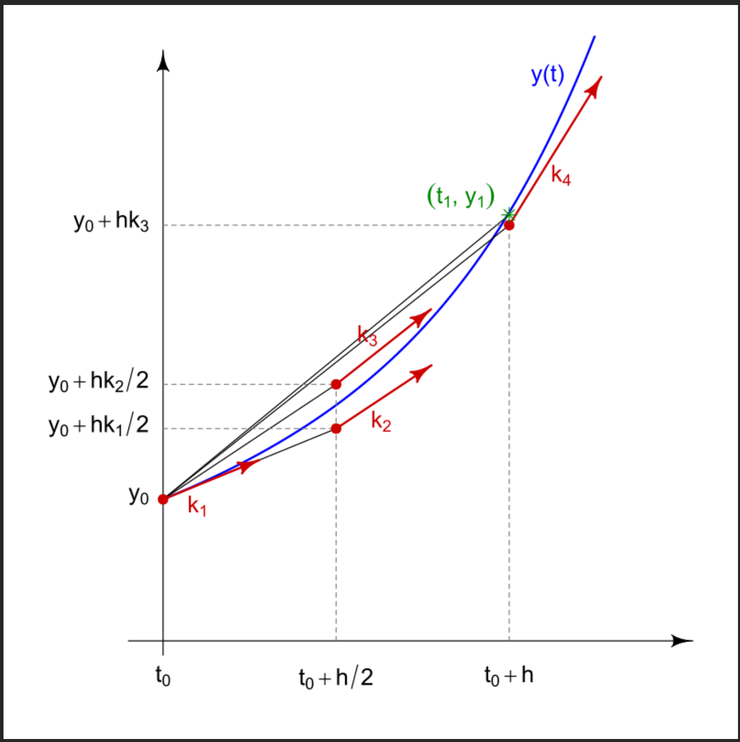


Figure 9.4: 4th Order Runge-Kutta's method schematic

Note:
The formula for the next value y_{i+1} can be also written as:

$$
\begin{aligned}
k_1 &= y_i \\
k_2 &= y_i + \frac{h}{2}\frac{dk_1}{dx} \\
k_3 &= y_i + \frac{h}{2}\frac{dk_2}{dx} \\
k_4 &= y_i + h\frac{k_3}{dx} \\
y_{i+1} &= y_i + \frac{h}{6}(\frac{dk_1}{dx} + 2\frac{dk_2}{dx} + 2\frac{dk_3}{dx} + \frac{dk_4}{dx}) + O(h^5)
\end{aligned}
\tag{9.36}
$$

This notation is more convenient to use when solving sets of ODEs for vibration problem, because it translates to:

$$
\begin{aligned}
u_1 &= u_0 \\
v_1 &= v_0 \\
a_1 &= (f - cv_0 - ku_0)/m \\
u_2 &= u_0 + \frac{h}{2}v_1 \\
v_2 &= v_0 + \frac{h}{2}a_1 \\
a_2 &= (f - cv_1 - ku_1)/m \\
u_3 &= u_0 + \frac{h}{2}v_2 \\
v_3 &= v_0 + \frac{h}{2}a_2 \\
a_3 &= (f - cv_2 - ku_2)/m \\
u_4 &= u_0 + hv_3 \\
v_4 &= v_0 + ha_3 \\
a_4 &= (f - cv_3 - ku_3)/m \\
u_{i+1} &= u_0 + \frac{h}{6}\left(v_1 + 2v_2 + 2v_3 + v_4\right) \\
v_{i+1} &= v_0 + \frac{h}{6}\left(a_1 + 2a_2 + 2a_3 + a_4\right) \\
a_{i+1} &= (f - cv_{i+1} - ku_{i+1})/m
\end{aligned}
\tag{9.37}
$$

where each of the **ODEs** are solved sequentially from the values already known.

Python implementation for vibration problem:

```python
#!/usr/bin/python3
import numpy as np
import matplotlib.pyplot as plt


def iterate(h, y0, y, func, rhs):
    num_of_odes = y0.shape[0]
    y1 = np.zeros(num_of_odes, dtype = float)
    for i in range(num_of_odes-1):
        y1[i] = y0[i] + y[i+1] * h
    y1[-1] = func(y1, rhs)
    return y1


def runge_kutta_4(t, y, func, rhs):
    N = t.shape[0]
    for j in range(N-1):
        dt = t[j+1] - t[j]
        y1 = iterate(0., y[j,:], y[j,:], func, rhs[j])
        y2 = iterate(dt/2, y[j,:], y1, func, 0.5 * (rhs[j+1] + rhs[j]))
        y3 = iterate(dt/2, y[j,:], y2, func, 0.5 * (rhs[j+1] + rhs[j]))
        y4 = iterate(dt, y[j,:], y3, func, rhs[j+1])

        # next step solution
        for i in range(y.shape[1]-1):
            y[j+1,i] = y[j,i] + dt/6 * (y1[i+1] + 2 * y2[i+1] + 2 * y3[i+1] +
            ↪  y4[i+1])
        y[j+1,-1] = func(y[j+1,:], rhs[j+1])
    return y


def rk4_vibration():
    m = 10.    # tonnes
    c = 5.     # Ns/mm
    k = 50.    # N/mm

    freq = np.sqrt(k / m) / (2 * np.pi)
```

```
print(f'f = {freq}')

# equation to solve
# m * a + c * v + k * u = f
# a = (f - c * v - k * u) / m
acceleration = lambda u, f: (f - c * u[1] - k * u[0]) / m

T = 10.      # seconds
dt = 0.01    # timestep s
u_0 = 0.     # mm of initial displacement
v_0 = 0.     # mm/s of initial velocity

# times at which to solve
t = np.linspace(0, T, int(T/dt) + 1)

# force vector
F = 1.     # N of max impulse
t0 = 1.    # impulse start time
t1 = 1.5   # impuls end time
f = np.zeros(t.shape[0], dtype=float)
# create a half sine impulse of force
for i in range(t.shape[0]):
    if t[i] >= t0 and t[i] <= t1:
        f[i] = F * np.sin((t[i] - t0) / (t1 - t0) * np.pi)
    else:
        f[i] = 0.

uva = np.zeros((t.shape[0],3), dtype=float)
uva[0,0] = u_0
uva[0,1] = v_0

uva = runge_kutta_4(t, uva, acceleration, f)

fig, axf = plt.subplots()
fig.subplots_adjust(right=0.60)

p1, = axf.plot(t, f, label='force', color='violet')
axu = axf.twinx()
p2, = axu.plot(t, uva[:,0], label='displacement', color='red')
axv = axf.twinx()
axv.spines.right.set_position(('axes', 1.20))
```

```python
    p3, = axv.plot(t, uva[:,1], label='velocity', color='blue')
    axa = axf.twinx()
    axa.spines.right.set_position(('axes', 1.40))
    p4, = axa.plot(t, uva[:,2], label='acceleration', color='green')

    axf.set_xlabel('Time [s]')
    axf.set_ylabel('Force [N]')
    axu.set_ylabel('Displacement [mm]')
    axv.set_ylabel('Velocity [mm/s]')
    axa.set_ylabel('Acceleration [mm/s2]')

    axf.legend(handles=[p1, p2, p3, p4])

    plt.show()

if __name__ == '__main__':
    rk4_vibration()
```

Which results to:

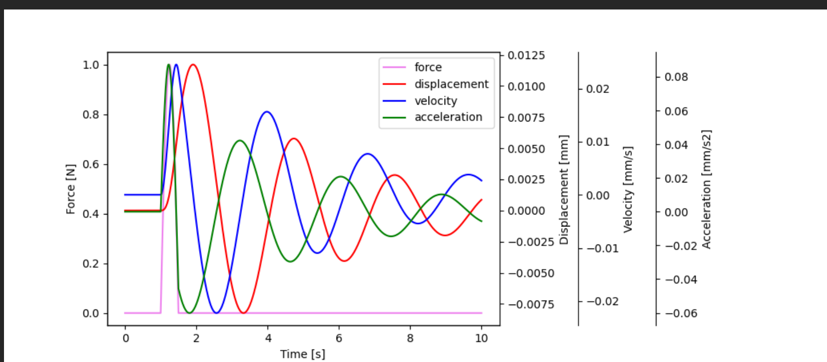


Figure 9.5: 4th order Runge-Kutta's implementation for vibration problem example

9.5 Implicit

The basic approach is an incremental step-by-step solution with an assumtion that the solution for a discrete time t is known and that the solution for the discrete time $t + \Delta t$ is required, where Δt is a suitably chosen time increment. Hence, considering time $t + \Delta t$:

$$\mathbf{F}^{t+\Delta t} - \mathbf{N}^{t+\Delta t} = \mathbf{0} \tag{9.38}$$

Where $\mathbf{F}$ is the vector of **external forces** (loads) and $\mathbf{N}$ is the vector of **inner forces** (stresses).

Assume that $\mathbf{F}^{t+\Delta t}$ is independent of the deformations. Since the solution is known at time t, we can write:

$$\mathbf{N}^{t+\Delta t} = \mathbf{N}^{t} + \Delta\mathbf{N} \tag{9.39}$$

where $\Delta\mathbf{N}$ is the increment in nodal point forces corresponding to the increment in element displacements and stresses from time t to time $t + \Delta t$. This vector can be approximated using a **tangent stiffness matrix $\mathbf{K}^t$** which corresponds to the geometric and material conditions at time t,

$$\Delta\mathbf{N} \doteq \mathbf{K}^{t}\Delta\mathbf{u} \tag{9.40}$$

where $\Delta\mathbf{u}$ is a vector of incremental nodal point displacements and

$$\mathbf{K}^{t} = \frac{\partial\mathbf{N}^{t}}{\partial\mathbf{u}^{t}} \tag{9.41}$$

Hence, the **tangent stiffness matrix** corresponds to the derivative of the internal element nodal point forces $\mathbf{N}^t$ with respect to the nodal point displacements $\mathbf{u}^t$.

Substituting (9.40) and (9.41) into (9.38), we obtain

$$\mathbf{K}^t \Delta\mathbf{u} = \mathbf{F}^{t+\Delta t} - \mathbf{N}^t \tag{9.42}$$

and solving for $\mathbf{u}^{\Delta t}$, we can calculate and approximation to the displacements at time $t + \Delta t$,

$$\mathbf{u}^{t+\Delta t} \doteq \mathbf{u}^t + \Delta\mathbf{u} \tag{9.43}$$

The exact displacements at time $t+\Delta t$ are those that correspond to the applied loads $\mathbf{F}^{t+\Delta t}$. We calculate in (9.43) only an approximation to these displacements because (9.40) was used.

Having evaluated an approximation to the displacements corresponding to time $t+\Delta t$, we could now solve for an approximation to the stresses and corresponding nodal point forces at time $t + \Delta t$, and then proceed to the next time increment calculations. However, because of the assumption (9.40), such a solution may be subject to very significant errors and, depending on the time or load step sizes used, may indeed be unstable. In practice, it is therefore necessary to iterate until the solution of (9.38) is obtained to sufficient accuracy.

9.5.1 Newton-Raphson

Having calculated an *increment* in nodal point displacements, which defines a *new total* displacement vector, we can repeat the incremental solution using the currently known total displacements at time t.

The equations used in **Newton-Raphson** iteration are, $for\ i = 1, 2, 3, \ldots,$

$$\boxed{\begin{aligned} \mathbf{K}_{i-1}^{t+\Delta t} \Delta\mathbf{u}_i &= \mathbf{F}^{t+\Delta t} - \mathbf{N}_{i-1}^{t+\Delta t} \\ \mathbf{u}_i^{t+\Delta t} &= \mathbf{u}_i^{t+\Delta t} + \mathbf{u}_{i-1}^{t+\Delta t} + \Delta\mathbf{u}_i \end{aligned} \qquad (9.44)}$$

with the initial conditions:

$$\begin{aligned}\mathbf{u}_0^{t+\Delta t} &= \mathbf{u}^t \\ \mathbf{K}_0^{t+\Delta t} &= \mathbf{K}^t \\ \mathbf{N}_0^{t+\Delta t} &= \mathbf{N}^t\end{aligned} \tag{9.45}$$

Note that in the first iteration, the relations in (9.44) reduce to the equations (9.42) and (9.43). Then, in subsequent iterations, the latest estimates for the nodal point displacements are used to evaluate the corresponding element stresses and nodal point forces $\mathbf{N}_{i-1}^{t+\Delta t}$ and **tangent stiffness matrix** $\mathbf{K}_{i-1}^{t+\Delta t}$.

The **tangent stiffness matrix** is achieved in the same way as the **global stiffness matrix**, but of the **deformed** geometry, incorporating the model nonlinearities (if applicable), such as:

- **geometric** non-linearities as large rotations, function-dependent springs, etc.
- **configuration** non-linearities as tension only or compression only elements, contact definitions (usually contacts should be realised on the **RHS** of the $\mathbf{Ku} = \mathbf{f}$ equation)
- **material** non-linearities as plasticity etc.

The out-of-balance load vector $\mathbf{F}^{t+\Delta t} - \mathbf{N}_{i-1}^{t+\Delta t}$ corresponds to a load vector that is not yet balanced by element stress, and hence an increment in the nodal point displacement is required. This updating of the nodal point displacement in the iteration is continued until out-of-balance loads and incremental displacements are small enough.

An important point is that the calculation of $\mathbf{N}_{i-1}^{t+\Delta t}$ from $\mathbf{u}_{i-1}^{t+\Delta t}$ is **crucial**. Any errors in this calculation will, in general, result in an incorrect response prediction.

The correct evaluation of the **tangent stiffness matrix** $\mathbf{K}_{i-1}^{t+\Delta t}$ is also important. The use of proper **tangent stiffness matrix** may be necessary for convergence and, in general, will result in fewer iterations until convergence is reached.

However, because the expense involved in evaluating and factoring a new **tangent stiffness matrix**, in practice, it can be more efficient, depending on the nonlinearities present in the analysis, to evaluate a new **tangent stiffness matrix** only at certain times. Specifically, in the **modified Newton-Raphson method** a new **tangent stiffness matrix** is established only at the beginning of each load step, and in **quasi-Newton** methods **secant stiffness matrices** are used instead of the tangent stiffness matrix.

The use of the iterative solution requires appropriate convergence criteria. If inappropriate criteria are used, the iteration may be terminated before the necessary solution accuracy is reached or be continued after the required accuracy has been reached.

Full Newton-Raphson Procedure derivation

The finite element equilibrium requirements amount to finding the solution of the equations:

$$f(\mathbf{u}^*) = 0 \tag{9.46}$$

Where:

$$f(\mathbf{u}^*) = \mathbf{F}^{t+\Delta t}(\mathbf{u}^*) - \mathbf{N}^{t+\Delta t}(\mathbf{u}^*) \tag{9.47}$$

We denote here and in the following the complete array of the solution as $\mathbf{u}^*$ but realize that this vector may also contain variables other than displacements, for example, pressure variables and rotations.

Assume that in the iterative solution we have evaluated $\mathbf{u}_{i-1}^{t+\Delta t}$; then a Taylor series expansion gives:

$$
\begin{aligned}
f(\mathbf{u}^*) = f(\mathbf{u}_{i-1}^{t+\Delta t}) + \left[\frac{\partial N}{\partial \mathbf{u}}\right]\bigg|_{\mathbf{u}_{i-1}^{t+\Delta t}} \left(\mathbf{u}^* - \mathbf{u}_{i-1}^{t+\Delta t}\right) \\
+ \, higher \ order \ terms
\end{aligned}
\tag{9.48}
$$

Substituting from (9.47) into (9.48) and using (9.46), we obtain:

$$
\begin{aligned}
&\left[\frac{\partial \mathbf{N}}{\partial \mathbf{u}}\right]\bigg|_{\mathbf{u}_{i-1}^{t+\Delta t}} \left(\mathbf{u}^* - \mathbf{u}_{i-1}^{t+\Delta t}\right) + higher \ order \ terms \\
&= \mathbf{F}^{t+\Delta t} - \mathbf{N}_{i-1}^{t+\Delta t}
\end{aligned}
\tag{9.49}
$$

where we assumed that the externally applied loads are deformation independent.

Neglecting the higher-order terms in (9.49), we can calculate an increment in the displacements,

$$
\mathbf{K}_{i-1}^{t+\Delta t} \Delta \mathbf{u}_i = \mathbf{F}^{t+\Delta t} - \mathbf{N}_{i-1}^{t+\Delta t} \tag{9.50}
$$

where $\mathbf{K}_{i-1}^{t+\Delta t}$ is the current **tangent stiffness matrix**

$$
\mathbf{K}_{i-1}^{t+\Delta t} = \left[\frac{\partial \mathbf{N}}{\partial \mathbf{u}}\right]\bigg|_{\mathbf{u}_{i-1}^{t+\Delta t}} \tag{9.51}
$$

and the improved displacement solution is:

$$
\mathbf{u}_i^{t+\Delta t} = \mathbf{u}_{i-1}^{t+\Delta t} + \Delta \mathbf{u}_i \tag{9.52}
$$

The relations in (9.50) *and* (9.52) *costitute the Newton-Raphson solution of* (9.38). Since an incremental analysis is performed with time (or load) steps of size Δt, the initial conditions in this iteration are $\mathbf{K}_0^{t+\Delta t} = \mathbf{K}^t$, $\mathbf{N}_0^{t+\Delta t} = \mathbf{N}^t$, and $\mathbf{u}_0^{t+\Delta t} = \mathbf{u}^t$. The iteration is continued **until appropriate convergence criteria are satisfied.**

A characteristic of this iteration is that a new tangent stiffness matrix is calculated in **each** iteration, which is why this method is also referred to as the **full Newton-Raphson method**.

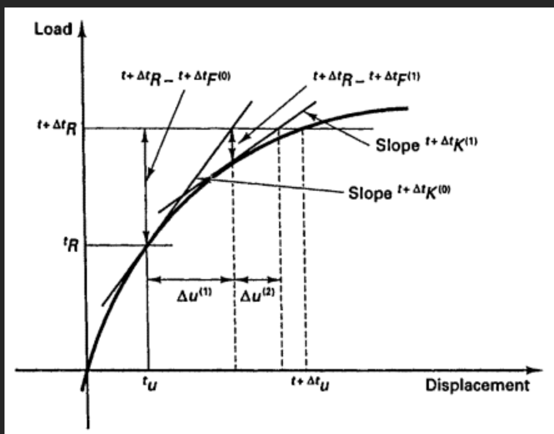


Figure 9.6: Newton-Raphson iteration in solution of SDOF system. **R** is load increment, **F** are internal forces, **u** are displacements.

Chapter 10

Model reduction

10.1 Guyan reduction

Guyan reduction, also known as **static condensation**, is a *dimensionality reduction* method which reduces the number of **DOFs** by ignoring the interial terms of the equilibrium equations and expressing the unloaded **DOFs** in terms of the loaded **DOFs**.

The static equilibrium equation:

$$\begin{gathered}\begin{bmatrix}\mathbf{K}_{mm} & \mathbf{K}_{ms}\\ \mathbf{K}_{sm} & \mathbf{K}_{ss}\end{bmatrix}\begin{bmatrix}\mathbf{u}_m\\ \mathbf{u}_s\end{bmatrix}=\begin{bmatrix}\mathbf{f}_m\\ \mathbf{f}_s\end{bmatrix}\\ \mathbf{K}_{mm}\mathbf{u}_m+\mathbf{K}_{ms}\mathbf{u}_s=\mathbf{f}_m\\ \mathbf{K}_{sm}\mathbf{u}_m+\mathbf{K}_{ss}\mathbf{u}_s=\mathbf{f}_s\end{gathered} \tag{10.1}$$

Where m denotes the **master** DOFs and s the **slave** DOFs. Stating that s section of the problem DOFs is unloaded, we can write that $\mathbf{f}_s=\mathbf{0}$. Then the slave DOFs are expressed by the following equation:

$$\mathbf{K}_{sm}\mathbf{u}_m+\mathbf{K}_{ss}\mathbf{u}_s=\mathbf{0} \tag{10.2}$$

Solving the above equation in terms of the independent (master) DOFs leads to the following dependency relations:

$$\mathbf{u}_s = -\mathbf{K}_{ss}^{-1}\mathbf{K}_{sm}\mathbf{u}_m \tag{10.3}$$

Substituting the dependency relations to the upper (master) partition of the static equilibrium problem condenses away the slave DOFs, leading to the following reduced system of linear equations:

$$\left(\mathbf{K}_{mm} - \mathbf{K}_{ms}\mathbf{K}_{ss}^{-1}\mathbf{K}_{sm}\right)\mathbf{u}_m = \mathbf{f}_m \tag{10.4}$$

The above system of linear equations is equivalent to the original problem, but expressed in terms of **master** DOFs alone. Thus, the **Guyan** reduction method results in a reduced system by condensing away the **slave** DOFs.

The **Guyan reduction** can be also expressed as a *change of basis* which produces a low-dimensional representation of the original space, represented by the **master** DOFs. The linear transformation that maps the reduced space onto the full space is expressed as:

$$\begin{bmatrix} \mathbf{u}_m \\ \mathbf{u}_s \end{bmatrix} = \begin{bmatrix} \mathbf{I} \\ -\mathbf{K}_{ss}^{-1}\mathbf{K}_{sm} \end{bmatrix} \mathbf{u}_m = \mathbf{T}_G\mathbf{u}_m \tag{10.5}$$

where $\mathbf{T}_G$ represents the **Guyan** reduction *transformation matrix*. Thus, the reduced problem is represented as:

$$\mathbf{K}_G\mathbf{u}_m = \mathbf{f}_m \tag{10.6}$$

In the above equation, $\mathbf{K}_G$ represents the reduced system of linear equations that's obtained by applying the **Guyan reduction** transformation on the full system, which is expressed as:

$$\mathbf{K}_G = \mathbf{T}_G^T\mathbf{K}\mathbf{T}_G \tag{10.7}$$

where $\mathbf{T}_G$ has the dimension of $n_{DOF} \times n_m$.

10.2 Craig-Bampton reduction

The **Craig-Bampton reduction**, also known as the **fixed-interface component mode synthesis**, extends the **Guyan reduction** by supplementing the static deformation shapes with a truncated set of dynamic deformation shapes. It therefore retains the exact static relation between the interface and internal DOFs, while approximating the internal dynamic deformation by a modal base.

The dynamic equilibrium equation is partitioned into the **boundary** DOFs b and the **internal** DOFs i:

$$\begin{bmatrix} \mathbf{M}_{bb} & \mathbf{M}_{bi} \\ \mathbf{M}_{ib} & \mathbf{M}_{ii} \end{bmatrix} \begin{bmatrix} \ddot{\mathbf{u}}_b \\ \ddot{\mathbf{u}}_i \end{bmatrix} + \begin{bmatrix} \mathbf{K}_{bb} & \mathbf{K}_{bi} \\ \mathbf{K}_{ib} & \mathbf{K}_{ii} \end{bmatrix} \begin{bmatrix} \mathbf{u}_b \\ \mathbf{u}_i \end{bmatrix} = \begin{bmatrix} \mathbf{f}_b \\ \mathbf{f}_i \end{bmatrix} \tag{10.8}$$

where the boundary DOFs are retained in physical coordinates and the internal DOFs are reduced. In the terminology used for the **Guyan reduction**, the boundary DOFs correspond to the master DOFs and the internal DOFs to the slave DOFs.

10.2.1 Constraint modes

The **constraint modes**, also called the **static constraint modes**, describe the static deformation of the internal DOFs caused by a unit displacement of each individual boundary DOF. The inertia terms and the externally applied forces on the internal DOFs are set to zero:

$$\mathbf{K}_{ib}\mathbf{u}_b + \mathbf{K}_{ii}\mathbf{u}_i = \mathbf{0} \tag{10.9}$$

The internal displacement is then expressed as:

$$\mathbf{u}_i = -\mathbf{K}_{ii}^{-1}\mathbf{K}_{ib}\mathbf{u}_b = \boldsymbol{\Psi}_c\mathbf{u}_b \tag{10.10}$$

where:

$$\boldsymbol{\Psi}_c = -\mathbf{K}_{ii}^{-1}\mathbf{K}_{ib} \tag{10.11}$$

is the **constraint mode matrix**. Each column of $\boldsymbol{\Psi}_c$ is the internal static displacement field corresponding to a unit displacement of one boundary DOF, while all remaining boundary DOFs are held fixed.

The inverse $\mathbf{K}_{ii}^{-1}$ is only a mathematical notation. Numerically, the constraint modes are obtained by solving the multiple right-hand-side problem:

$$\mathbf{K}_{ii}\boldsymbol{\Psi}_c = -\mathbf{K}_{ib} \tag{10.12}$$

The matrix $\mathbf{K}_{ii}$ is factorised only once and the factorisation is reused for all boundary DOFs.

10.2.2 Fixed-interface normal modes

The dynamic deformation that cannot be represented by the constraint modes is approximated using the **fixed-interface normal modes**. These modes are calculated by fixing all boundary DOFs:

$$\mathbf{u}_b = \mathbf{0} \tag{10.13}$$

and solving the internal generalised eigenvalue problem:

$$\left(\mathbf{K}_{ii} - \omega_j^2\mathbf{M}_{ii}\right)\boldsymbol{\varphi}_j = \mathbf{0} \tag{10.14}$$

The retained fixed-interface modes are assembled column-wise into the matrix:

$$\boldsymbol{\Phi}_i = \begin{bmatrix}\boldsymbol{\varphi}_1 & \boldsymbol{\varphi}_2 & \dots & \boldsymbol{\varphi}_{n_\varphi}\end{bmatrix} \tag{10.15}$$

where n_φ is the number of retained fixed-interface modes. When the modes are mass-normalised:

$$
\begin{aligned}
\mathbf{\Phi}_i^T \mathbf{M}_{ii} \mathbf{\Phi}_i &= \mathbf{I} \\
\mathbf{\Phi}_i^T \mathbf{K}_{ii} \mathbf{\Phi}_i &= \mathbf{\Omega}_i^2
\end{aligned}
\tag{10.16}
$$

where:

$$
\mathbf{\Omega}_i^2 = \begin{bmatrix} \omega_1^2 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \omega_{n_\varphi}^2 \end{bmatrix}
\tag{10.17}
$$

is the diagonal matrix of the retained fixed-interface eigenvalues.

10.2.3 Craig-Bampton transformation

The internal displacement is represented as a superposition of the static constraint modes and the fixed-interface normal modes:

$$
\mathbf{u}_i = \mathbf{\Psi}_c \mathbf{u}_b + \mathbf{\Phi}_i \boldsymbol{\eta}
\tag{10.18}
$$

where $\boldsymbol{\eta}$ is the vector of the fixed-interface modal coordinates. The full physical displacement vector is therefore expressed as:

$$
\begin{aligned}
\begin{bmatrix} \mathbf{u}_b \\ \mathbf{u}_i \end{bmatrix} &= \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{\Psi}_c & \mathbf{\Phi}_i \end{bmatrix} \begin{bmatrix} \mathbf{u}_b \\ \boldsymbol{\eta} \end{bmatrix} \\
\mathbf{u} &= \mathbf{T}_{CB} \mathbf{q}_{CB}
\end{aligned}
\tag{10.19}
$$

where:

$$
\begin{aligned}
\mathbf{T}_{CB} &= \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{\Psi}_c & \mathbf{\Phi}_i \end{bmatrix} \\
\mathbf{q}_{CB} &= \begin{bmatrix} \mathbf{u}_b \\ \boldsymbol{\eta} \end{bmatrix}
\end{aligned}
\tag{10.20}
$$

The Craig-Bampton coordinates therefore consist of:

- the physical displacements of all retained boundary DOFs,
- the modal amplitudes of the retained fixed-interface normal modes.

Substituting $\mathbf{u} = \mathbf{T}_{CB}\mathbf{q}_{CB}$ to the full equation of motion and premultiplying it with $\mathbf{T}_{CB}^T$ gives the reduced equation:

$$\mathbf{M}_{CB}\ddot{\mathbf{q}}_{CB} + \mathbf{C}_{CB}\dot{\mathbf{q}}_{CB} + \mathbf{K}_{CB}\mathbf{q}_{CB} = \mathbf{f}_{CB} \tag{10.21}$$

where:

$$\begin{aligned} \mathbf{M}_{CB} &= \mathbf{T}_{CB}^T\mathbf{M}\mathbf{T}_{CB} \\ \mathbf{C}_{CB} &= \mathbf{T}_{CB}^T\mathbf{C}\mathbf{T}_{CB} \\ \mathbf{K}_{CB} &= \mathbf{T}_{CB}^T\mathbf{K}\mathbf{T}_{CB} \\ \mathbf{f}_{CB} &= \mathbf{T}_{CB}^T\mathbf{f} \end{aligned} \tag{10.22}$$

For exact constraint modes and a symmetric stiffness matrix, the reduced stiffness matrix has the following block-diagonal form:

$$\mathbf{K}_{CB} = \begin{bmatrix} \mathbf{K}_G & \mathbf{0} \\ \mathbf{0} & \mathbf{\Omega}_i^2 \end{bmatrix} \tag{10.23}$$

where:

$$\mathbf{K}_G = \mathbf{K}_{bb} - \mathbf{K}_{bi}\mathbf{K}_{ii}^{-1}\mathbf{K}_{ib} \tag{10.24}$$

is the **Guyan-condensed stiffness matrix**. The corresponding mass matrix is generally not block-diagonal:

$$\mathbf{M}_{CB} = \begin{bmatrix} \mathbf{M}_{bb}^{CB} & \mathbf{M}_{b\varphi}^{CB} \\ \mathbf{M}_{\varphi b}^{CB} & \mathbf{I} \end{bmatrix} \tag{10.25}$$

with:

$$\begin{aligned} \mathbf{M}_{bb}^{CB} &= \mathbf{M}_{bb} + \mathbf{M}_{bi}\mathbf{\Psi}_c + \mathbf{\Psi}_c^T\mathbf{M}_{ib} + \mathbf{\Psi}_c^T\mathbf{M}_{ii}\mathbf{\Psi}_c \\ \mathbf{M}_{b\varphi}^{CB} &= \left(\mathbf{M}_{bi} + \mathbf{\Psi}_c^T\mathbf{M}_{ii}\right)\mathbf{\Phi}_i \\ \mathbf{M}_{\varphi b}^{CB} &= \left(\mathbf{M}_{b\varphi}^{CB}\right)^T \end{aligned} \tag{10.26}$$

The coupling in the reduced mass matrix is the reason why the Craig-Bampton coordinates are not normal modes of the unconstrained component. The boundary coordinates remain physical and are dynamically coupled with the fixed-interface modal coordinates.

Note:
The Craig-Bampton transformation does not reduce the boundary DOFs. If a component has n_b boundary DOFs and n_φ retained fixed-interface modes, then the reduced model has:

$$n_{CB} = n_b + n_\varphi \tag{10.27}$$

generalised coordinates. A large number of interface nodes can therefore still result in a relatively large Craig-Bampton model.

10.3 Flexible body modal reduction

A **flexible body** used in a multibody or reduced-order simulation is often represented by a small set of free-interface normal modes instead of keeping all Craig-Bampton boundary coordinates as independent generalised coordinates. This can be obtained by performing a second modal reduction in the Craig-Bampton space.

The Craig-Bampton equation without damping and external forces is:

$$\mathbf{M}_{CB}\ddot{\mathbf{q}}_{CB} + \mathbf{K}_{CB}\mathbf{q}_{CB} = \mathbf{0} \tag{10.28}$$

No boundary coordinates are constrained in the second reduction pass. The free-interface generalised eigenvalue problem is therefore:

$$\left(\mathbf{K}_{CB} - \omega_j^2\mathbf{M}_{CB}\right)\boldsymbol{\xi}_j = \mathbf{0} \tag{10.29}$$

The eigenvectors are assembled into the reduced free-interface mode matrix:

$$\boldsymbol{\Xi} = \begin{bmatrix}\boldsymbol{\xi}_1 & \boldsymbol{\xi}_2 & \dots & \boldsymbol{\xi}_{n_f}\end{bmatrix} \tag{10.30}$$

where n_f is the number of retained free-interface modes. The Craig-Bampton coordinates are then represented by:

$$\mathbf{q}_{CB} = \boldsymbol{\Xi}\mathbf{q}_f \tag{10.31}$$

and the full physical displacement is recovered as:

$$\begin{aligned}\mathbf{u} &= \mathbf{T}_{CB}\mathbf{q}_{CB} \\ &= \mathbf{T}_{CB}\boldsymbol{\Xi}\mathbf{q}_f \\ &= \mathbf{T}_f\mathbf{q}_f\end{aligned} \tag{10.32}$$

where:

$$\mathbf{T}_f = \mathbf{T}_{CB}\boldsymbol{\Xi} \tag{10.33}$$

is the final **flexible body transformation matrix**. The columns of $\mathbf{T}_f$ are free-interface mode shapes expressed at the original FE DOFs. The interface displacement is no longer an independent coordinate. It is approximated by the retained flexible body modes in the same way as the internal displacement.

The final reduced matrices are calculated as:

$$\begin{aligned}
\mathbf{M}_f &= \boldsymbol{\Xi}^T \mathbf{M}_{CB} \boldsymbol{\Xi} = \mathbf{T}_f^T \mathbf{M} \mathbf{T}_f \\
\mathbf{K}_f &= \boldsymbol{\Xi}^T \mathbf{K}_{CB} \boldsymbol{\Xi} = \mathbf{T}_f^T \mathbf{K} \mathbf{T}_f \\
\mathbf{C}_f &= \boldsymbol{\Xi}^T \mathbf{C}_{CB} \boldsymbol{\Xi} = \mathbf{T}_f^T \mathbf{C} \mathbf{T}_f \\
\mathbf{f}_f &= \boldsymbol{\Xi}^T \mathbf{f}_{CB} = \mathbf{T}_f^T \mathbf{f}
\end{aligned} \tag{10.34}$$

When the free-interface eigenvectors are mass-normalised:

$$\begin{aligned}
\boldsymbol{\Xi}^T \mathbf{M}_{CB} \boldsymbol{\Xi} &= \mathbf{I} \\
\boldsymbol{\Xi}^T \mathbf{K}_{CB} \boldsymbol{\Xi} &= \boldsymbol{\Omega}_f^2
\end{aligned} \tag{10.35}$$

The flexible body equation of motion then becomes:

$$\ddot{\mathbf{q}}_f + \mathbf{C}_f \dot{\mathbf{q}}_f + \boldsymbol{\Omega}_f^2 \mathbf{q}_f = \mathbf{f}_f \tag{10.36}$$

For classical modal damping, the damping matrix can be represented by a diagonal matrix:

$$\mathbf{C}_f = \begin{bmatrix} 2\zeta_1\omega_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & 2\zeta_{n_f}\omega_{n_f} \end{bmatrix} \tag{10.37}$$

10.3.1 Rigid body modes

A free three-dimensional component has six **rigid body modes**: three translations and three rotations. Their eigenvalues are theoretically equal to zero because a rigid body displacement produces no elastic strain energy:

$$\mathbf{K}_{CB}\boldsymbol{\xi}_r = \mathbf{0} \tag{10.38}$$

In finite-precision arithmetic, the rigid body eigenvalues are usually small but not exactly zero. They must therefore be identified using a tolerance relative to the characteristic stiffness or to the first elastic eigenvalue, rather than by checking whether the calculated frequency is exactly zero.

The exact treatment of the rigid body modes depends on the formulation of the surrounding simulation:

- when the flexible component is described only by modal coordinates, the rigid body modes must be retained to allow free translation and rotation,
- when the multibody formulation describes the rigid body position and orientation separately, the six rigid body modes are not used as elastic coordinates; the retained elastic modes must nevertheless remain mass-orthogonal to the rigid body motion.

The second case is commonly used by floating-frame flexible body formulations. The large rigid motion is described by the reference frame of the body and the modal coordinates describe only the small elastic deformation relative to this frame.

10.3.2 Mode truncation

If all n_{CB} eigenvectors of the reduced free-interface eigenproblem are kept, the transformation $\boldsymbol{\Xi}$ represents only another basis of the same Craig-Bampton space. The second reduction becomes a dimensionality reduction only when the mode matrix is truncated:

$$n_f < n_{CB} \tag{10.39}$$

This provides the distinction between the two reduced representations:

- the Craig-Bampton model retains all boundary DOFs and therefore preserves an independent physical coordinate at every interface DOF,
- the flexible body modal model retains only selected free-interface modes and therefore approximates both the boundary and internal deformation by a smaller number of modal coordinates.

The accuracy of the final flexible body base is limited by both truncation stages. The fixed-interface base must first be rich enough to represent the required internal dynamics. The free-interface base is then selected according to the frequency range and deformation patterns required by the final simulation. Increasing the number of free-interface modes cannot recover deformation that was already excluded from the Craig-Bampton base.

10.3.3 Numerical procedure

A numerically efficient implementation of the complete reduction can be summarised as follows:

1. Reorder the FE matrices into boundary and internal DOFs.
2. Factorise $\mathbf{K}_{ii}$ and solve $\mathbf{K}_{ii}\mathbf{\Psi}_c = -\mathbf{K}_{ib}$ for all constraint modes.
3. Solve the lowest required eigenpairs of $\mathbf{K}_{ii}\boldsymbol{\varphi} = \omega^2\mathbf{M}_{ii}\boldsymbol{\varphi}$ with the boundary fixed.
4. Mass-normalise the fixed-interface modes and assemble $\mathbf{T}_{CB}$.
5. Calculate $\mathbf{M}_{CB}$, $\mathbf{K}_{CB}$ and, when required, $\mathbf{C}_{CB}$ by congruent transformation.
6. Solve the free-interface eigenproblem $\mathbf{K}_{CB}\boldsymbol{\xi} = \omega^2\mathbf{M}_{CB}\boldsymbol{\xi}$.
7. Identify the rigid body modes and select the required elastic modes.
8. Mass-normalise the retained eigenvectors and assemble $\mathbf{T}_f = \mathbf{T}_{CB}\mathbf{\Xi}$.
9. Calculate the final modal matrices and the load and output projection operators.

The following numerical considerations are important:

- The matrix inverse must not be formed explicitly. A sparse factorisation and multiple right-hand-side solve should be used for the constraint modes.
- The fixed-interface stiffness matrix $\mathbf{K}_{ii}$ must be nonsingular. A singular matrix indicates an internal rigid body motion or a mechanism that is not restrained by the selected boundary DOFs.
- Symmetric generalised eigensolvers preserve the mass orthogonality of distinct modes. An ordinary Euclidean Gram-Schmidt process is therefore not normally applied to the eigenvectors.
- Repeated or tightly clustered eigenvalues can require re-orthogonalisation of the corresponding mode block. This must use the mass inner product $\boldsymbol{\xi}_r^T \mathbf{M}_{CB} \boldsymbol{\xi}_s$.
- The Craig-Bampton matrices should be symmetrised only to remove small round-off asymmetry, not to hide an incorrectly assembled transformation.
- Mode selection should be based on the required frequency range and on the deformation and loading patterns, rather than on the mode count alone.

For a retained mode $\boldsymbol{\xi}_j$, the mass normalisation is performed as:

$$\boldsymbol{\xi}_{j,mass} = \frac{\boldsymbol{\xi}_j}{\sqrt{\boldsymbol{\xi}_j^T \mathbf{M}_{CB} \boldsymbol{\xi}_j}} \tag{10.40}$$

The orthogonality and the eigenpair residual can be checked using:

$$\begin{aligned} \mathbf{E}_M &= \boldsymbol{\Xi}^T \mathbf{M}_{CB} \boldsymbol{\Xi} - \mathbf{I} \\ \mathbf{E}_K &= \boldsymbol{\Xi}^T \mathbf{K}_{CB} \boldsymbol{\Xi} - \boldsymbol{\Omega}_f^2 \\ \mathbf{r}_j &= \mathbf{K}_{CB} \boldsymbol{\xi}_j - \omega_j^2 \mathbf{M}_{CB} \boldsymbol{\xi}_j \end{aligned} \tag{10.41}$$

A valid reduced base should have small off-diagonal entries in $\mathbf{E}_M$ and $\mathbf{E}_K$, and a small residual norm $\|\mathbf{r}_j\|$ relative to the norms of the stiffness and inertia terms.

Note:
The free-interface modes calculated in the Craig-Bampton space are not the original full FE free-free modes. They are approximations restricted to the subspace spanned by $\mathbf{T}_{CB}$. After mapping with $\mathbf{T}_f = \mathbf{T}_{CB}\boldsymbol{\Xi}$, they have the same physical meaning and converge towards the full FE free-free modes as the Craig-Bampton base is enriched.

Scope of the classical formulation:
The above derivation assumes a linear time-invariant system with symmetric mass and stiffness matrices. For a component linearised about a prestressed operating point, the tangent stiffness matrix can be used in the same reduction procedure.
When a velocity-dependent gyroscopic matrix $\mathbf{G}$ is retained, it must be transformed together with the other system matrices:

$$\mathbf{G}_{CB} = \mathbf{T}_{CB}^T \mathbf{G} \mathbf{T}_{CB} \tag{10.42}$$

The free-interface problem then contains the term $i\omega\mathbf{G}_{CB}$ and is no longer the symmetric real-valued eigenproblem described above. Its modes can be complex and the simple mass-orthogonality relations do not generally apply.

Chapter 11

Element Atlas

11.1 Matrices

The inversion formula for 3x3 matrix:

$$
\begin{aligned}
\mathbf{A} &= \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \\
\mathbf{A}^{-1} &= \frac{1}{|\mathbf{A}|} \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix}
\end{aligned}
\tag{11.1}
$$

where:

$$\begin{aligned}
A_{11} &= a_{22}a_{33} - a_{23}a_{32} \\
A_{22} &= a_{33}a_{11} - a_{31}a_{13} \\
A_{33} &= a_{11}a_{22} - a_{12}a_{21} \\
A_{12} &= a_{23}a_{31} - a_{21}a_{33} \\
A_{23} &= a_{31}a_{12} - a_{32}a_{11} \\
A_{31} &= a_{12}a_{23} - a_{13}a_{22} \\
A_{21} &= a_{32}a_{13} - a_{12}a_{33} \\
A_{32} &= a_{13}a_{21} - a_{23}a_{11} \\
A_{13} &= a_{21}a_{22} - a_{31}a_{22} \\
|A_{13}| &= a_{11}A_{11} + a_{12}A_{21} + a_{13}A_{31}
\end{aligned} \tag{11.2}$$

11.2 Numerical Integration

Note:
Quadrature is another term for **Numerical Integration**.

11.2.1 Newton-Cotes quadrature

In the most obvious procedure, points at which the function is to be formed are determined *a priori* - usually at equal intervals - and a polynomial passed through the values of the function at these points and exactly integrated.

As n values define a polynomial of degree $n-1$, the errors will be of the order $O(h^n)$ where h is the element size. This leads to the well-known **Newton-Cotes** 'quadrature formulae'. The integrals can be written as:

$$I = \int_{-1}^{1} f(\xi)d\xi = \sum_{1}^{n} H_i f(\xi_i) \tag{11.3}$$

for the range of integration between -1 and $+1$. For example, if $n = 2$, we have the well-known trapezoidal rule:

$$I = f(-1) + f(1) \tag{11.4}$$

for $n = 3$, the well-known **Simpson** one-third rule:

$$I = \frac{1}{3}\left[f(-1) + 4f(0) + f(1)\right] \tag{11.5}$$

and for $n = 4$:

$$I = \frac{1}{4}\left[f(-1) + 3f(-\frac{1}{3}) + 3f(\frac{1}{3}) + f(1)\right] \tag{11.6}$$

11.2.2 Gauss quadrature

If in place of specifying the position of sampling points *a priori* we allow these to be located at points to be determined so as to aim for best accuracy, then for a given number of sampling points increased accuracy can be obtained. Indeed, if we again consider:

$$I = \int_{-1}^{1} f(\xi)d\xi = \sum_{1}^{n} H_i f(\xi_i) \tag{11.7}$$

and again assume a polynomial expression, it is easy to see that for n sampling points we have $2n$ unknowns (H_i and ξ_i) and hence a polynomial of degree $2n-1$ could be constructed and exactly integrated. The error is thus of order $O(h^{2n})$.

The simultaneous equations involved are difficult to solve, but some mathematical manipulation will show that the solution can be obtained explicitely in terms of **Legendre polynomials**. Thus this particular process is frequently known as **Gauss-Legendre** quadrature.

For purposes of finite elements analysis complex calculations are involved in determining the values of f, the function to be integratet. Thus the Gauss-type processes, requiring the least number of such evaluations, are ideally suited and are mostly used exvlusively.

Other expresssions for integration of functions of the type

$$I = \int_{-1}^{1} w(\xi) f(\xi) d\xi = \sum_{1}^{n} H_i f(\xi_i) \tag{11.8}$$

can be derived for prescribed forms of $w(\xi)$, again integrating up to a certain order of accuracy a polynomial expansion of $f(\xi)$.

11.2.3 Linear and Quadrilateral Elements

For 1D:

$$I = \int_{-1}^{1} f(\xi) d\xi = \sum_{1}^{n} H_i f(\xi_i) \tag{11.9}$$

For 2D:

The most obvious way of obtaining the integral:

$$I = \int_{-1}^{1} \int_{-1}^{1} f(\xi, \eta) d\xi d\eta \tag{11.10}$$

Is to first evaluate the inner integral keeping η constant, i.e.:

$$\int_{-1}^{1} f(\xi, \eta) d\xi = \sum_{j=1}^{n} H_j f(\xi_i, \eta) = \psi(\eta) \tag{11.11}$$

Evaluating the outer integral in a similar manner, we have:

$$
\begin{aligned}
I = \int_{-1}^{1} \psi(\eta) d\eta &= \sum_{i=1}^{n} H_i \psi(\eta_i) \\
&= \sum_{i=1}^{n} H_i \sum_{j=1}^{n} H_j f(\xi_i, \eta_i) \\
&= \sum_{i=1}^{n} \sum_{j=1}^{n} H_i H_j f(\xi_j, \eta_i)
\end{aligned} \tag{11.12}
$$

For 3D:

For a 3D element we have similarly:

$$
\begin{aligned}
I &= \int_{-1}^{1} \int_{-1}^{1} \int_{-1}^{1} f(\xi, \eta, \mu) d\xi d\eta d\mu \\
&= \sum_{i=1}^{n} \sum_{j=1}^{n} \sum_{k=1}^{n} H_i H_j f(\xi_i, \eta_j, \mu_k)
\end{aligned} \tag{11.13}
$$

In the above, the number of integrating points in each direction was assumed to be the same. Clearly this is not necessary and on occasion it may be an advantage to use different numbers in each direction of integration.

To get the **Gauss-Legendre** points and weights in python use:

```python
import numpy as np

# number of integration points
N = 3

points, weights = np.polynomial.legendre.leggauss(N)
```

integration point	polynomial order	weight
0	n = 1	2.0
$-1/\sqrt{3}$	n = 2	1.0
$1/\sqrt{3}$		1.0
$-\sqrt{0.6}$	n = 3	5/9
0.0		8/9
$\sqrt{0.6}$		5/9
-0.861 136 311 594 953	n = 4	0.347 854 845 137 454
-0.339 981 043 584 856		0.652 145 154 862 546
0.339 981 043 584 856		0.652 145 154 862 546
0.861 136 311 594 953		0.347 854 845 137 454
-0.906 179 845 938 664	n = 5	0.236 926 885 056 189
-0.538 469 310 105 683		0.478 628 670 499 366
0.000 000 000 000 000		0.568 888 888 888 889
0.538 469 310 105 683		0.478 628 670 499 366
0.906 179 845 938 664		0.236 926 885 056 189
-0.932 469 514 203 152	n = 6	0.171 324 492 379 170
-0.661 209 386 466 265		0.360 761 573 048 139
-0.238 619 186 083 197		0.467 913 934 572 691
0.238 619 186 083 197		0.467 913 934 572 691
0.661 209 386 466 265		0.360 761 573 048 139
0.932 469 514 203 152		0.171 324 492 379 170
-0.949 107 912 342 759	n = 7	0.129 484 966 168 870
-0.741 531 185 599 394		0.279 705 391 489 277
-0.405 845 151 377 397		0.381 830 050 505 119
0.000 000 000 000 000		0.471 959 183 673 469
0.405 845 151 377 397		0.381 830 050 505 119
0.741 531 185 599 394		0.279 705 391 489 277
0.949 107 912 342 759		0.129 484 966 168 870

Table 11.1: Gauss-Legendre Integration Points and Weights

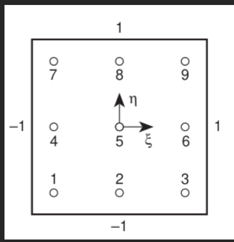


Figure 11.1: Integration points for $n = 3$ in a square region. Exact for a polynomial of fifth order in each direction

11.2.4 Triangular and Tetrahedral elements:

2D

For a triangle, in terms of the **area coordinates** the integrals are of the form:

$$I = \int_0^1 \int_0^{1-\xi} f(\xi, \eta, \mu) d\eta d\xi, \quad \mu = 1 - \xi - \eta \tag{11.14}$$

Once again we could use n Gauss points and arrive at a summation expression of the type used in the previous section. However, the limits of integration now involve the variable itself and it is convenient to use alternative sampling points for the second integration by use of a special Gauss expression for integrals of the type given by equation (11.8) in which w is a linear function. These have been devised by *Radau* and used successfully in finite element context. It is, however, much more desirable (and aesthetically pleasing) to use special formulae in which no bias is given to any of the natural coordinates (ξ, η, μ). Such formulae were first derived by *Hammer et al.* and *Felippa* and a series of necessary sampling points and weights.

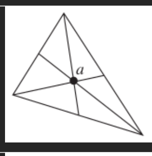
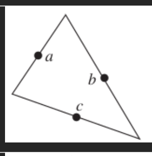
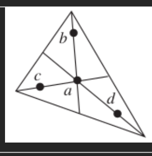
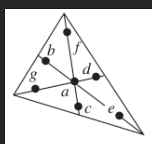
Order	Figure	Error	Points	Triangular Coordinates	Weights
Linear		$R = O(h^2)$	a	$\frac{1}{3}, \frac{1}{3}, \frac{1}{3}$	1
Quadratic		$R = O(h^3)$	a b c	$\frac{1}{2}, \frac{1}{2}, 0$ $0, \frac{1}{2}, \frac{1}{2}$ $\frac{1}{2}, 0, \frac{1}{2}$	$\frac{1}{3}$ $\frac{1}{3}$ $\frac{1}{3}$
Cubic		$R = O(h^4)$	a b c d	$\frac{1}{3}, \frac{1}{3}, \frac{1}{3}$ $0.6, 0.2, 0.2$ $0.2, 0.6, 0.2$ $0.2, 0.2, 0.6$	$-\frac{27}{48}$ $\frac{25}{48}$ $\frac{25}{48}$ $\frac{25}{48}$
Quintic		$R = O(h^6)$	a b c d e f g	$\frac{1}{3}, \frac{1}{3}, \frac{1}{3}$ $\alpha_1, \beta_1, \beta_1$ $\beta_1, \alpha_1, \beta_1$ $\beta_1, \beta_1, \alpha_1$ $\alpha_2, \beta_2, \beta_2$ $\beta_2, \alpha_2, \beta_2$ $\beta_2, \beta_2, \alpha_2$	0.2250000000 0.1323941527 0.1323941527 0.1323941527 0.1259391805 0.1259391805 0.1259391805
			with: $\alpha_1 = 0.059\ 715\ 871\ 7$ $\beta_1 = 0.470\ 142\ 064\ 1$ $\alpha_2 = 0.797\ 426\ 985\ 3$ $\beta_2 = 0.101\ 286\ 507\ 3$		

Table 11.2: Triangular Element integration points and weights

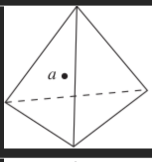
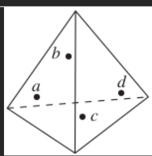
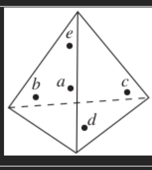
Order	Figure	Error	Points	Tetrahedral Coordinates	Weights
Linear		$R = O(h^2)$	a	$\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}$	1
Quadratic		$R = O(h^6)$	a	$\alpha, \beta, \beta, \beta$	$\frac{1}{4}$
			b	$\beta, \alpha, \beta, \beta$	$\frac{1}{4}$
			c	$\beta, \beta, \alpha, \beta$	$\frac{1}{4}$
			d	$\beta, \beta, \beta, \beta$	$\frac{1}{4}$
			with: $\alpha = 0.585\ 410\ 20$ $\beta = 0.138\ 196\ 60$		
Cubic		$R = O(h^4)$	a	$\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}$	$-\frac{4}{5}$
			b	$\frac{1}{2}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}$	$\frac{9}{20}$
			c	$\frac{1}{6}, \frac{1}{2}, \frac{1}{6}, \frac{1}{6}$	$\frac{9}{20}$
			d	$\frac{1}{6}, \frac{1}{6}, \frac{1}{2}, \frac{1}{6}$	$\frac{9}{20}$
			e	$\frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{2}$	$\frac{9}{20}$

Table 11.3: Tetrahedral Element integration points and weights

11.3 General Element Matrices Procedure

The procedure to generate any element stiffness, mass and load matrices can be generalised to the following steps:

1. Create Element domain in natural coordinates:
   - -1 to 1 for quadrilateral elements
   - 0 to 1 for triangular and linear elements
2. select the number of Gauss points based on the order of the element
   - 1 or 2 for linear elements
   - 3 for quadratic elements

   Example:

   ```
   import numpy as np
   gp, gw = np.polynomial.legendre.leggauss(3)
   ```

3. Generate the shape functions $\mathbf{N}_n^e$ in natural coordinates based on the number of Gauss points, dimension of the element and its domain.

   The $\mathbf{N}_n^e$ matrix has the dimensions of:
   - number of integration points = number of rows
   - number of shape functions = number of columns

   Therefore:

   $$\mathbf{N}_n^e = \begin{bmatrix} N_{1,1}^e & \dots & N_{n,1}^e \\ \vdots & \vdots & \vdots \\ N_{1,m}^e & \dots & N_{n,m}^e \end{bmatrix} \tag{11.15}$$

   where $1 \dots m$ are the integration points and $1 \dots n$ are the shape functions.

4. Transform the shape functions from natural coordinates to global coordinates

$$\mathbf{N}_g^e = \mathbf{N}_n^e \mathbf{x}^e$$
$$\mathbf{N}_g^e = \begin{bmatrix} N_{1,1}^e(\xi) & \dots & N_{n,1}^e(\xi) \\ \vdots & \vdots & \vdots \\ N_{1,m}^e(\xi) & \dots & N_{n,m}^e(\xi) \end{bmatrix} \begin{bmatrix} x_1 & y_1 & z_1 \\ \vdots & \vdots & \vdots \\ x_n & y_n & z_n \end{bmatrix} \tag{11.16}$$

   The $\mathbf{x}^e$ matrix is a matrix of global coordinates of the element nodes.

   The resulting matrix has the dimension of:

   - number of integration points = number of rows
   - number of coordinates = number of columns (3D = 3, 2D = 2)

5. Generate the shape functions derivatives $\mathbf{B}_n^e$ in natural coordinates

   The shape functions derivatives matrix has 3 dimensions being *number of integration points* × *number of natural coordinates* × *number of shape functions*

6. Create the Matrix of Jacobians from the shape functions derivatives:

$$\mathbf{J} = \mathbf{B}_n^e \mathbf{x}^e \tag{11.17}$$

   The Matrix of Jacobians has the dimension of *number of integration points* × *number of global coordinates* × *number of global coordinates*, where number of global coordinates means 2 for 2D and 3 for 3D problem.

7. Get the Matrix of Jacobian determinants and an inverse Jacobian:

$$\mathbf{J}^{-1}\mathbf{J}_d = det\mathbf{J} \tag{11.18}$$

   Example:

```
d_jacobi = np.linalg.det(jacobi)
i_jacobi = np.linalg.inv(jacobi)
```

The Jacobian determinant is computed for each integration point and has a dimension of *number of integration points* $\times$ *1*

The inverse Jacobian has the same dimension as the Jacobian (**must have**)

8. Transform the shape function derviatives from natural coordinates to global coordinates
$$\mathbf{B}_g^e = \mathbf{J}^{-1}\mathbf{B}_n^e \tag{11.19}$$

9. Create the Material Stiffness matrix $\mathbf{C}$

10. Finally for each integration point:

3D:

$$\begin{aligned} \mathbf{B}_{i,x}^e &= \mathbf{B}_g(i,0) \\ \mathbf{B}_{i,y}^e &= \mathbf{B}_g(i,1) \\ \mathbf{B}_{i,z}^e &= \mathbf{B}_g(i,2) \end{aligned} \tag{11.20}$$

where $\mathbf{B}_{i,x}^e$, $\mathbf{B}_{i,y}^e$ and $\mathbf{B}_{i,z}^e$ are vectors of legth = *number of shape functions*

Then matrix $\mathbf{B}$ is:

$$\mathbf{B}_i = \begin{bmatrix} \mathbf{B}_{i,x}^e & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{B}_{i,y}^e & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{B}_{i,z}^e \\ \mathbf{B}_{i,y}^e & \mathbf{B}_{i,x}^e & \mathbf{0} \\ \mathbf{0} & \mathbf{B}_{i,z}^e & \mathbf{B}_{i,y}^e \\ \mathbf{B}_{i,z}^e & \mathbf{0} & \mathbf{B}_{i,x}^e \end{bmatrix} \tag{11.21}$$

and matrix $\mathbf{N}$ is:

$$\mathbf{N}_i = \begin{bmatrix} \mathbf{N}_{n,i}^e & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{N}_{n,i}^e & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{N}_{n,i}^e \end{bmatrix} \tag{11.22}$$

where $\mathbf{0}$ is a zero vector of length = *number of shape functions* and $\mathbf{N}_{n,i}^e$ is a vector of shape functions in natural coordinates pertaining to the i-th integration point.

Then:

(a)

$$\mathbf{K}_i^e = \mathbf{B}_i^T \mathbf{C} \mathbf{B}_i \ det \mathbf{J}_i w_i \tag{11.23}$$

(b)

$$\mathbf{M}_i^e = \rho \mathbf{N}_i^T \mathbf{N}_i \ det \mathbf{J}_i w_i \tag{11.24}$$

(c)

$$\mathbf{F}_i^e = \mathbf{N}_i^T \mathbf{F} \ det \mathbf{J}_i w_i \tag{11.25}$$

where $\mathbf{F} = \begin{bmatrix} f_x \\ f_y \\ f_z \end{bmatrix}$

where w_i is the multiple of gaussian weights for the respective integration point (for 3D case brick where the natural coordinates are ξ, η, μ the $w_i = w_{\xi_i} w_{\eta_i} w_{\mu_i}$

11. Finally:

(a)

$$\mathbf{K}^e = \sum_{i=1}^{n} \mathbf{K}_i^e \tag{11.26}$$

(b)

$$\mathbf{M}^e = \sum_{i=1}^{n} \mathbf{M}_i^e \tag{11.27}$$

(c)

$$\mathbf{F}^e = \sum_{i=1}^{n} \mathbf{F}_i^e \tag{11.28}$$

11.4 ROD

11.4.1 Element Formulation

The **rod** element is a 1-D linear element. It has 2 nodes, and 3 dofs for each node u_x, u_y, u_y (or u, v, w). Rotation **DOF**s are not constrained.

Natural coordinates

The natural coordinate ξ of the **ROD** element goes from 0 to 1. The 2-noded rod element is defined by:

$$\begin{bmatrix} 1 \\ x \\ y \\ z \\ u_x \\ u_y \\ u_z \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ x_1 & x_2 \\ y_1 & y_2 \\ z_1 & z_2 \\ u_{x1} & u_{x2} \\ u_{y1} & u_{y2} \\ u_{z1} & u_{z2} \end{bmatrix} \begin{bmatrix} N_1^e \\ N_2^e \end{bmatrix} \tag{11.29}$$

Shape Functions

The **ROD** natural coordinates are:

node	ξ
1	0
2	1

Table 11.4: ROD corners natural coordinates

The shape functions are:

$$\begin{aligned} N_1^e &= 1 - \xi \\ N_2^e &= \xi \end{aligned} \tag{11.30}$$

Partial Derivatives

The shape function derivatives with respect to the natural coordinate:

$$\begin{aligned} \frac{\partial N_1^e}{\partial \xi} &= -1 \\ \frac{\partial N_2^e}{\partial \xi} &= 1 \end{aligned} \tag{11.31}$$

The Jacobian

The derivatives of the shape functions are given by the usual chain rule formula:

$$\begin{aligned} \frac{\partial N_i^e}{\partial x} &= \frac{\partial N_i^e}{\partial \xi}\frac{\partial \xi}{\partial x} \\ \frac{\partial N_i^e}{\partial y} &= \frac{\partial N_i^e}{\partial \xi}\frac{\partial \xi}{\partial y} \\ \frac{\partial N_i^e}{\partial z} &= \frac{\partial N_i^e}{\partial \xi}\frac{\partial \xi}{\partial z} \end{aligned} \tag{11.32}$$

In matrix form:

$$\begin{bmatrix} \frac{\partial N_i^e}{\partial x} \\ \frac{\partial N_i^e}{\partial y} \\ \frac{\partial N_i^e}{\partial z} \end{bmatrix} = \begin{bmatrix} \frac{\partial \xi}{\partial x} \\ \frac{\partial \xi}{\partial y} \\ \frac{\partial \xi}{\partial z} \end{bmatrix} \begin{bmatrix} \frac{\partial N_i^e}{\partial \xi} \end{bmatrix} \tag{11.33}$$

The 3×1 matrix above is $\mathbf{J}^{-1}$, the inverse of:

$$\mathbf{J} = \frac{\partial(x, y, z)}{\partial(\xi)} = \begin{bmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial y}{\partial \xi} & \frac{\partial z}{\partial \xi} \end{bmatrix} \tag{11.34}$$

Given the element coordinates the Jacobian can be computed:

$$\mathbf{J}_i = \begin{bmatrix} \frac{\partial N_i^e}{\partial \xi} & \frac{\partial N_i^e}{\partial \xi} & \frac{\partial N_i^e}{\partial \xi} \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ z_i \end{bmatrix} \tag{11.35}$$

end then numerically inverted for $\mathbf{J}^{-1}$.

If the $\mathbf{J}$ is not a square matrix, then **Moore-Penrose pseudoinverse** applies:

$$\mathbf{J}^{-1} = \mathbf{J}^{+} = \left(\mathbf{J}^{T}\mathbf{J}\right)^{-1}\mathbf{J}^{T} \tag{11.36}$$

in python:

```
import numpy as np

J_i = np.linalg.pinv(J)
```

Assembling the **Jacobian** thegether gives essentially that for a 1D case

$$\mathbf{J} = l \tag{11.37}$$

and therefore:

$$\mathbf{J}^{-1} = \frac{1}{l} \tag{11.38}$$

$$\det J = l \tag{11.39}$$

afterwards for each integration point:

$$\mathbf{N}_g^i = \mathbf{N}^i\mathbf{u} \tag{11.40}$$

$$\mathbf{B}_g^i = \mathbf{J}^{i-1}\frac{\partial N^i}{\partial \xi} \tag{11.41}$$

then:

$$\begin{aligned}
\mathbf{K}^e &\pm \mathbf{B}_g^{i\,T}\mathbf{D}\mathbf{B}_g^i \det \mathbf{J}^i w^i \\
\mathbf{M}^e &\pm \rho\mathbf{N}^{i\,T}\mathbf{N}^i \det \mathbf{J}^i w^i \\
\mathbf{F}^e &\pm \mathbf{N}^{i\,T}\mathbf{F} \det \mathbf{J}^i w^i
\end{aligned} \tag{11.42}$$

11.4.2 Example 1D

If assembling the element in local CSYS, then $u_y = 0$ and $u_z = 0$. Simplifiying the equations gives that:

$$\mathbf{N} = \begin{bmatrix} N_1^e & N_2^e \end{bmatrix} = \begin{bmatrix} 1-\xi & \xi \end{bmatrix} \tag{11.43}$$

and

$$u^e(\xi) = \mathbf{N} \begin{bmatrix} u_1^e \\ u_2^e \end{bmatrix} \tag{11.44}$$

shape function derivatives:

$$\mathbf{B} = \frac{\partial \mathbf{N}}{\partial \xi} = \begin{bmatrix} -1 & 1 \end{bmatrix} \tag{11.45}$$

the strain-displacement realtion is given by:

$$\boldsymbol{\epsilon} = \mathbf{Bu} \tag{11.46}$$

Integrating the element at midpoint $\xi = 0.5$ with a weight of $w = 1.0$ (for interval $\langle 0, 1 \rangle$, for an interval $\langle -1, 1 \rangle$ the weight is $w = 2.0$)

$$\mathbf{N} = \begin{bmatrix} 0.5 & 0.5 \end{bmatrix} \tag{11.47}$$

Shape Functions in global coordinates:

$$\mathbf{N} = 0.5u_1 + 0.5u_2 = 0.5(u_1 + u_2) \tag{11.48}$$

Shape Function Derivatives in Natural Coordinates:

$$\mathbf{B} = \begin{bmatrix} -1 & 1 \end{bmatrix} \tag{11.49}$$

Jacobian:

$$\mathbf{J} = \mathbf{B}\mathbf{u} = \begin{bmatrix} -1 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = u_2 - u_1 = l \tag{11.50}$$

Jacobian Determinant:

$$\det \mathbf{J} = l \tag{11.51}$$

Jacobian Inverse:

$$\mathbf{J}^{-1} = \frac{1}{l} \tag{11.52}$$

Shape function derivatives in global coordinates:

$$\mathbf{B}_g = \mathbf{J}^{-1}\mathbf{B} = \frac{1}{l} \begin{bmatrix} -1 & 1 \end{bmatrix} \tag{11.53}$$

Material Stiffness Matrix

$$\mathbf{D} = EA \tag{11.54}$$

Then for each integration point (here 1):

$$\begin{aligned}
\mathbf{K}^e &\stackrel{\pm}{=} \mathbf{B}_g^T \mathbf{D} \mathbf{B}_\mathbf{g} \det \mathbf{J} w \\
\mathbf{K}^e &\stackrel{\pm}{=} \frac{1}{l} \begin{bmatrix} -1 \\ 1 \end{bmatrix} EA \frac{1}{l} \begin{bmatrix} -1 & 1 \end{bmatrix} l 1 \\
\mathbf{K}^e &\stackrel{\pm}{=} \frac{EA}{l} \begin{bmatrix} -1 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 1 \end{bmatrix} \\
\mathbf{K}^e &\stackrel{\pm}{=} \frac{EA}{l} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}
\end{aligned} \tag{11.55}$$

11.4.3 Example 2D

If assembling the element in 2D, then $u_z = 0$. Simplifiying the equations gives that:

$$\mathbf{N} = \begin{bmatrix} N_1^e & 0 & N_2^e & 0 \\ 0 & N_1^e & 0 & N_2^e \end{bmatrix} = \begin{bmatrix} 1-\xi & 0 & \xi & 0 \\ 0 & 1-\xi & 0 & \xi \end{bmatrix} \tag{11.56}$$

and

$$\begin{bmatrix} u_x^e(\xi) \\ u_y^e(\xi) \end{bmatrix} = \mathbf{N} \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{x2}^e \\ u_{y2}^e \end{bmatrix} \tag{11.57}$$

or if:

$$\mathbf{N} = \begin{bmatrix} N_1^e & N_2^e \end{bmatrix} = \begin{bmatrix} 1-\xi & \xi \end{bmatrix} \tag{11.58}$$

then:

$$\begin{bmatrix} u_x^e(\xi) \\ u_y^e(\xi) \end{bmatrix} = \mathbf{N} \begin{bmatrix} u_{x1}^e & u_{y1}^e \\ u_{x2}^e & u_{y2}^e \end{bmatrix} \tag{11.59}$$

shape function derivatives:

$$\mathbf{B} = \frac{\partial \mathbf{N}}{\partial \xi} = \begin{bmatrix} -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{bmatrix} \tag{11.60}$$

the strain-displacement realtion is given by:

$$\boldsymbol{\epsilon} = \mathbf{B}\mathbf{u} \tag{11.61}$$

Integrating the element at midpoint $\xi = 0.5$ with a weight of $w = 1.0$ (for interval $\langle 0, 1 \rangle$, for an interval $\langle -1, 1 \rangle$ the weight $w = 2.0$) then numerically:

$$\mathbf{N} = \begin{bmatrix} 0.5 & 0.5 \end{bmatrix} \tag{11.62}$$

Shape Functions in global coordinates:

$$\mathbf{N} = \begin{bmatrix} 1-\xi & 0 & \xi & 0 \\ 0 & 1-\xi & 0 & \xi \end{bmatrix}_{\xi=0.5} \begin{bmatrix} u^e_{x1} \\ u^e_{y1} \\ u^e_{x2} \\ u^e_{y2} \end{bmatrix} = \begin{bmatrix} 0.5u_{x1} + 0.5u_{x2} \\ 0.5u_{y1} + 0.5u_{y2} \end{bmatrix} \tag{11.63}$$

Shape Function Derivatives in Natural Coordinates (there is a row for each integration point if the coordinates are specified as matrix, not a column vector. otherwise for each integration point there is a separate **B** matrix):

$$\frac{\partial \mathbf{N}}{\partial \xi} = \mathbf{B} = \begin{bmatrix} -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{bmatrix} \tag{11.64}$$

Jacobian:

$$\mathbf{J} = \mathbf{B}\mathbf{u} = \begin{bmatrix} -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{bmatrix} \begin{bmatrix} u^e_{x1} \\ u^e_{y1} \\ u^e_{x2} \\ u^e_{y2} \end{bmatrix} = \begin{bmatrix} u^e_{x2} - u^e_{x1} \\ u^e_{y2} - u^e_{y1} \end{bmatrix} = \begin{bmatrix} l_x \\ l_y \end{bmatrix} \tag{11.65}$$

? ?

Jacobian Determinant (when the **Jacobian matrix** is a vector, its determinant is simply its **length**):

¿ ¿

$$\det \mathbf{J} = ||\mathbf{J}|| = \sqrt{J_1^2 + J_2^2} = \sqrt{l_x^2 + l_y^2} = l \tag{11.66}$$

? ?

Jacobian Inverse (when the **Jacobian matrix** is a row vector, its inverse is just its **transpose** divided by its determinant squared):

Moore-Penrose pseudoinverse:

$$\begin{aligned}\boxed{\mathbf{J}^{-1}} = \mathbf{J}^{+} &= \left(\mathbf{J}^T\mathbf{J}\right)^{-1}\mathbf{J}^T \\ &= \left(\begin{bmatrix} l_x & l_y \end{bmatrix}\begin{bmatrix} l_x \\ l_y \end{bmatrix}\right)^{-1}\begin{bmatrix} l_x & l_y \end{bmatrix} \\ &= \left(l_x^2 + l_y^2\right)^{-1}\begin{bmatrix} l_x & l_y \end{bmatrix} \\ &= \frac{1}{l_x^2 + l_y^2}\begin{bmatrix} l_x & l_y \end{bmatrix} \\ &= \frac{1}{\sqrt{l_x^2 + l_y^2}^{\,2}}\begin{bmatrix} l_x & l_y \end{bmatrix} \\ &= \boxed{\frac{1}{l^2}\mathbf{J}^T}\end{aligned} \tag{11.67}$$

¿ ¿

$$\mathbf{J}^{-1} = \frac{1}{l^2}\begin{bmatrix} l_x & l_y \end{bmatrix} = \frac{1}{l}\begin{bmatrix} c & s \end{bmatrix} \tag{11.68}$$

where $c = \cos\varphi = l_x/l$ and $s = \sin\varphi = l_y/l$.

Shape function derivatives in global coordinates:

$$\mathbf{B}_g = \mathbf{J}^{-1}\mathbf{B} = \frac{1}{l}\begin{bmatrix} c & s \end{bmatrix}\begin{bmatrix} -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{bmatrix} = \frac{1}{l}\begin{bmatrix} -c & -s & c & s \end{bmatrix} \tag{11.69}$$

Material Stiffness Matrix

$$\mathbf{D} = EA \tag{11.70}$$

Then for each integration point (here 1):

$$\mathbf{K}^e \stackrel{+}{=} \mathbf{B}_g^T \mathbf{D} \mathbf{B}_\mathbf{g} \det \mathbf{J} w \tag{11.71}$$

$$\begin{aligned}
\mathbf{K}^e &\stackrel{+}{=} \frac{1}{l}\begin{bmatrix} -c \\ -s \\ c \\ s \end{bmatrix} EA \frac{1}{l}\begin{bmatrix} -c & -s & c & s \end{bmatrix} lw \\
&\stackrel{+}{=} \frac{EA}{l}\begin{bmatrix} c^2 & cs & -c^2 & -cs \\ cs & s^2 & -cs & -s^2 \\ -c^2 & -cs & c^2 & cs \\ -cs & -s^2 & cs & s^2 \end{bmatrix} w
\end{aligned} \tag{11.72}$$

which is the same result as when the element stiffness matrix is derived in local coordinate system for a situation where ξ axis coincides with the element x axis and the u_y component is 0. Afterwards, the element **stiffness** and **mass** matrices as well as **forces** vector are tranformed to the global CSYS using a transformation matrix $\mathbf{T}$:

$$\mathbf{K}_l^e = \frac{EA}{l}\begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \tag{11.73}$$

$$\mathbf{T} = \begin{bmatrix} c & s & 0 & 0 \\ -s & c & 0 & 0 \\ 0 & 0 & c & s \\ 0 & 0 & -s & c \end{bmatrix} \tag{11.74}$$

then the global element stiffness matrix is defined:

$$
\begin{aligned}
\mathbf{K}_g^e &= \mathbf{T}^T\mathbf{K}_l^e\mathbf{T} \\
&= \frac{EA}{l}\begin{bmatrix} c & -s & 0 & 0 \\ s & c & 0 & 0 \\ 0 & 0 & c & -s \\ 0 & 0 & s & c \end{bmatrix}\begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}\begin{bmatrix} c & s & 0 & 0 \\ -s & c & 0 & 0 \\ 0 & 0 & c & s \\ 0 & 0 & -s & c \end{bmatrix} \\
&= \frac{EA}{l}\begin{bmatrix} c & -s & 0 & 0 \\ s & c & 0 & 0 \\ 0 & 0 & c & -s \\ 0 & 0 & s & c \end{bmatrix}\begin{bmatrix} c & s & -c & -s \\ 0 & 0 & 0 & 0 \\ -c & -s & c & s \\ 0 & 0 & 0 & 0 \end{bmatrix} \\
&= \frac{EA}{l}\begin{bmatrix} c^2 & cs & -c^2 & -cs \\ cs & s^2 & -cs & -s^2 \\ -c^2 & -cs & c^2 & cs \\ -cs & -s^2 & cs & s^2 \end{bmatrix}
\end{aligned} \tag{11.75}
$$

11.4.4 Example 3D

If assembling the element in 3D, then $u_x \neq 0$, $u_y \neq 0$ and $u_z \neq 0$. The full equations give that:

$$
\mathbf{N} = \begin{bmatrix} N_1^e & 0 & 0 & N_2^e & 0 & 0 \\ 0 & N_1^e & 0 & 0 & N_2^e & 0 \\ 0 & 0 & N_1^e & 0 & 0 & N_2^e \end{bmatrix} = \begin{bmatrix} 1-\xi & 0 & 0 & \xi & 0 & 0 \\ 0 & 1-\xi & 0 & 0 & \xi & 0 \\ 0 & 0 & 1-\xi & 0 & 0 & \xi \end{bmatrix} \tag{11.76}
$$

and

$$
\begin{bmatrix} u_x^e(\xi) \\ u_y^e(\xi) \\ u_z^e(\xi) \end{bmatrix} = \mathbf{N}(\xi)\begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{z1}^e \\ u_{x2}^e \\ u_{y2}^e \\ u_{z2}^e \end{bmatrix} \tag{11.77}
$$

or if:

$$
\mathbf{N} = \begin{bmatrix} N_1^e & N_2^e \end{bmatrix} = \begin{bmatrix} 1-\xi & \xi \end{bmatrix} \tag{11.78}
$$

then:

$$\begin{bmatrix} u^e_x(\xi) \\ u^e_y(\xi) \\ u^e_z(\xi) \end{bmatrix} = \mathbf{N}(\xi) \begin{bmatrix} u^e_{x1} & u^e_{y1} & u^e_{z1} \\ u^e_{x2} & u^e_{y2} & u^e_{z2} \end{bmatrix} \tag{11.79}$$

shape function derivatives:

$$\mathbf{B} = \frac{\partial \mathbf{N}}{\partial \xi} = \begin{bmatrix} -1 & 0 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 0 & 1 \end{bmatrix} \tag{11.80}$$

the strain-displacement relation is given by:

$$\boldsymbol{\epsilon} = \mathbf{B}\mathbf{u} \tag{11.81}$$

Integrating the element at midpoint $\xi = 0.5$ with a weight of $w = 1.0$ (for interval $\langle 0, 1\rangle$, for an interval $\langle -1, 1\rangle$ the weight $w = 2.0$) then numerically:

$$\mathbf{N} = \begin{bmatrix} 0.5 & 0.5 \end{bmatrix} \tag{11.82}$$

Shape Functions in global coordinates:

$$\mathbf{N} = \begin{bmatrix} 1-\xi & 0 & 0 & \xi & 0 & 0 \\ 0 & 1-\xi & 0 & 0 & \xi & 0 \\ 0 & 0 & 1-\xi & 0 & 0 & \xi \end{bmatrix}_{\xi=0.5} \begin{bmatrix} u^e_{x1} \\ u^e_{y1} \\ u^e_{z1} \\ u^e_{x2} \\ u^e_{y2} \\ u^e_{z2} \end{bmatrix} = \begin{bmatrix} 0.5u_{x1} + 0.5u_{x2} \\ 0.5u_{y1} + 0.5u_{y2} \\ 0.5u_{z1} + 0.5u_{z2} \end{bmatrix} \tag{11.83}$$

Shape Function Derivatives in Natural Coordinates (there is a row for each integration point if the coordinates are specified as matrix, not a column vector. otherwise for each integration point there is a separate **B** matrix):

$$\frac{\partial \mathbf{N}}{\partial \xi} = \mathbf{B} = \begin{bmatrix} -1 & 0 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 0 & 1 \end{bmatrix} \tag{11.84}$$

Jacobian:

$$\mathbf{J} = \mathbf{Bu} = \begin{bmatrix} -1 & 0 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 0 & 1 \end{bmatrix}_{\xi=0.5} \begin{bmatrix} u^e_{x1} \\ u^e_{y1} \\ u^e_{z1} \\ u^e_{x2} \\ u^e_{y2} \\ u^e_{z2} \end{bmatrix} = \begin{bmatrix} u^e_{x2} - u^e_{x1} \\ u^e_{y2} - u^e_{y1} \\ u^e_{z2} - u^e_{z1} \end{bmatrix} = \begin{bmatrix} l_x \\ l_y \\ l_z \end{bmatrix} \tag{11.85}$$

? ?

Jacobian Determinant (when the **Jacobian matrix** is a vector, its determinant is simply its **length**):

¿ ¿

$$\det \mathbf{J} = ||\mathbf{J}|| = \sqrt{J_1^2 + J_2^2 + J_3^2} = \sqrt{l_x^2 + l_y^2 + l_y^2} = l \tag{11.86}$$

? ?

Jacobian Inverse (when the **Jacobian matrix** is a row vector, its inverse is just its **transpose** divided by its determinant squared):

Moore-Penrose pseudoinverse:

$$
\begin{aligned}
\boxed{\mathbf{J}^{-1}} = \mathbf{J}^{+} &= \left(\mathbf{J}^T\mathbf{J}\right)^{-1}\mathbf{J}^T \\
&= \left(\begin{bmatrix} l_x & l_y & l_z \end{bmatrix}\begin{bmatrix} l_x \\ l_y \\ l_z \end{bmatrix}\right)^{-1}\begin{bmatrix} l_x & l_y & l_z \end{bmatrix} \\
&= \left(l_x^2 + l_y^2 + l_z^2\right)^{-1}\begin{bmatrix} l_x & l_y & l_z \end{bmatrix} \\
&= \frac{1}{l_x^2 + l_y^2 + l_z^2}\begin{bmatrix} l_x & l_y & l_z \end{bmatrix} \\
&= \frac{1}{\sqrt{l_x^2 + l_y^2 + l_z^2}^2}\begin{bmatrix} l_x & l_y & l_z \end{bmatrix} \\
&= \boxed{\frac{1}{l^2}\mathbf{J}^T}
\end{aligned}
\tag{11.87}
$$

¿ ¿

$$
\mathbf{J}^{-1} = \frac{1}{l^2}\begin{bmatrix} l_x & l_y & l_z \end{bmatrix} \tag{11.88}
$$

Shape function derivatives in global coordinates:

$$
\begin{aligned}
\mathbf{B}_g = \mathbf{J}^{-1}\mathbf{B} = \frac{1}{l^2}\begin{bmatrix} l_x & l_y & l_z \end{bmatrix}_{\xi=0.5}&\begin{bmatrix} -1 & 0 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 0 & 1 \end{bmatrix}_{\xi=0.5} \\
= \frac{1}{l^2}\begin{bmatrix} -l_x & -l_y & -l_z & l_x & l_y & l_z \end{bmatrix}&
\end{aligned}
\tag{11.89}
$$

Material Stiffness Matrix

$$
\mathbf{D} = EA \tag{11.90}
$$

Then for each integration point (here 1):

$$\mathbf{K}^e \doteq \mathbf{B}_g^T \mathbf{D} \mathbf{B}_{\mathbf{g}} \det \mathbf{J} w \tag{11.91}$$

$$\begin{aligned}
\mathbf{K}^e &\doteq \frac{1}{l^2}\begin{bmatrix} -l_x \\ -l_y \\ -l_z \\ l_x \\ l_y \\ l_z \end{bmatrix} EA \frac{1}{l^2}\begin{bmatrix} -l_x & -l_y & -l_z & l_x & l_y & l_z \end{bmatrix} lw \\
&\doteq \frac{EA}{l^3}\begin{bmatrix}
l_x^2 & l_x l_y & l_x l_z & -l_x^2 & -l_x l_y & -l_x l_z \\
l_x l_y & l_y^2 & l_y l_z & -l_x l_y & -l_y^2 & -l_y l_z \\
l_x l_z & l_y l_z & l_z^2 & -l_x l_z & -l_y l_z & -l_z^2 \\
-l_x^2 & -l_x l_y & -l_x l_z & l_x^2 & l_x l_y & l_x l_z \\
-l_x l_y & -l_y^2 & -l_y l_z & l_x l_y & l_y^2 & l_y l_z \\
-l_x l_z & -l_y l_z & -l_z^2 & l_x l_z & l_y l_z & l_z^2
\end{bmatrix} w
\end{aligned} \tag{11.92}$$

11.4.5 Example 2D quadratic

If assembling the element in 2D, then and $u_z = 0$.

The node numbering scheme is $1 - 3 - 2$ and the natural coordinate is defined $\xi \in \langle -1, 1 \rangle$.

The **quadratic** shape functions are:

$$\begin{aligned}
N_1^e &= -\frac{1}{2}\xi\left(1 - \xi\right) \\
N_2^e &= \frac{1}{2}\xi\left(1 + \xi\right) \\
N_3^e &= \left(1 + \xi\right)\left(1 - \xi\right)
\end{aligned} \tag{11.93}$$

$$\mathbf{N} = \begin{bmatrix} N_1^e & 0 & N_2^e & 0 & N_3^e & 0 \\ 0 & N_1^e & 0 & N_2^e & 0 & N_3^e \end{bmatrix} \tag{11.94}$$

$$\mathbf{N} = \begin{bmatrix} -\frac{1}{2}\xi\left(1-\xi\right) & 0 & \frac{1}{2}\xi\left(1+\xi\right) & 0 & \left(1+\xi\right)\left(1-\xi\right) & 0 \\ 0 & -\frac{1}{2}\xi\left(1-\xi\right) & 0 & \frac{1}{2}\xi\left(1+\xi\right) & 0 & \left(1+\xi\right)\left(1-\xi\right) \end{bmatrix} \tag{11.95}$$

$$\mathbf{N} = \begin{bmatrix} \frac{\xi^2}{2} - \frac{\xi}{2} & 0 & \frac{\xi^2}{2} + \frac{\xi}{2} & 0 & 1-\xi^2 & 0 \\ 0 & \frac{\xi^2}{2} - \frac{\xi}{2} & 0 & \frac{\xi^2}{2} + \frac{\xi}{2} & 0 & 1-\xi^2 \end{bmatrix} \tag{11.96}$$

and

$$\begin{bmatrix} u_x^e(\xi) \\ u_y^e(\xi) \end{bmatrix} = \mathbf{N}(\xi) \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{x2}^e \\ u_{y2}^e \\ u_{x3}^e \\ u_{y3}^e \end{bmatrix} \tag{11.97}$$

or if:

$$\mathbf{N} = \begin{bmatrix} N_1^e & N_2^e & N_3^e \end{bmatrix} = \begin{bmatrix} \frac{\xi^2}{2} - \frac{\xi}{2} & \frac{\xi^2}{2} + \frac{\xi}{2} & 1-\xi^2 \end{bmatrix} \tag{11.98}$$

then:

$$\begin{bmatrix} u_x^e(\xi) \\ u_y^e(\xi) \end{bmatrix} = \mathbf{N}(\xi) \begin{bmatrix} u_{x1}^e & u_{y1}^e \\ u_{x2}^e & u_{y2}^e \\ u_{x3}^e & u_{y3}^e \end{bmatrix} \tag{11.99}$$

shape function derivatives:

$$\mathbf{B} = \frac{\partial \mathbf{N}}{\partial \xi} = \begin{bmatrix} \xi - \frac{1}{2} & 0 & \xi + \frac{1}{2} & 0 & -2\xi & 0 \\ 0 & \xi - \frac{1}{2} & 0 & \xi + \frac{1}{2} & 0 & -2\xi \end{bmatrix} \tag{11.100}$$

the strain-displacement realtion is given by:

$$\varepsilon = \mathbf{B}\mathbf{u} \tag{11.101}$$

Integrating the element at two **Gauss-Legendre** points of the natural coordinate $\xi = [-0.5773502692, 0.5773502692]$ with weights of $w = 1.0$ then numerically:

To get the correct interpolation points and weights, python can be used:

```python
import numpy as np

xi, w = np.polynomial.legendre.leggauss(2)
```

for $\xi^1 = -0.5773502692$

$$\mathbf{N}^1 = \begin{bmatrix} 0.45534 & -0.12201 & 0.66667 \end{bmatrix} \tag{11.102}$$

for $\xi^2 = 0.5773502692$

$$\mathbf{N}^2 = \begin{bmatrix} -0.12201 & 0.45534 & 0.66667 \end{bmatrix} \tag{11.103}$$

Shape Functions in global coordinates:

$$\begin{bmatrix} u^i_{x1} \\ u^i_{y1} \end{bmatrix} = \begin{bmatrix} \frac{\xi^2}{2} - \frac{\xi}{2} & 0 & \frac{\xi^2}{2} + \frac{\xi}{2} & 0 & 1-\xi^2 & 0 \\ 0 & \frac{\xi^2}{2} - \frac{\xi}{2} & 0 & \frac{\xi^2}{2} + \frac{\xi}{2} & 0 & 1-\xi^2 \end{bmatrix}_{\xi^i} \begin{bmatrix} u^e_{x1} \\ u^e_{y1} \\ u^e_{x2} \\ u^e_{y2} \\ u^e_{x3} \\ u^e_{y3} \end{bmatrix} \tag{11.104}$$

$$\begin{bmatrix} u^1_x \\ u^1_y \end{bmatrix} = \begin{bmatrix} 0.45534u_{x1} - 0.12201u_{x2} + 0.66667u_{x3} \\ -0.12201u_{y1} + 0.45534u_{y2} + 0.66667u_{y3} \end{bmatrix} \tag{11.105}$$

$$\begin{bmatrix} u^2_x \\ u^2_y \end{bmatrix} = \begin{bmatrix} -0.12201u_{x1} + 0.45534u_{x2} + 0.66667u_{x3} \\ 0.45534u_{y1} - 0.12201u_{y2} + 0.66667u_{y3} \end{bmatrix} \tag{11.106}$$

Shape Function Derivatives in Natural Coordinates (there is a row for each integration point if the coordinates are specified as matrix, not a column vector. otherwise for each integration point there is a separate **B** matrix):

$$\frac{\partial \mathbf{N}}{\partial \xi} = \mathbf{B} = \begin{bmatrix} \xi - \frac{1}{2} & 0 & \xi + \frac{1}{2} & 0 & -2\xi & 0 \\ 0 & \xi - \frac{1}{2} & 0 & \xi + \frac{1}{2} & 0 & -2\xi \end{bmatrix} \tag{11.107}$$

Jacobian:

$$\mathbf{J} = \mathbf{B}\mathbf{u} = \begin{bmatrix} \xi - \frac{1}{2} & 0 & \xi + \frac{1}{2} & 0 & -2\xi & 0 \\ 0 & \xi - \frac{1}{2} & 0 & \xi + \frac{1}{2} & 0 & -2\xi \end{bmatrix} \begin{bmatrix} u^e_{x1} \\ u^e_{y1} \\ u^e_{x2} \\ u^e_{y2} \\ u^e_{x3} \\ u^e_{y3} \end{bmatrix} \tag{11.108}$$

for $\xi^1 = -0.5773502692$

$$\mathbf{J}^1 = \begin{bmatrix} -1.07735u_{x1} - 0.07735u_{x2} + 1.15470u_3 \\ -1.07735u_{y1} - 0.07735u_{y2} + 1.15470u_{y3} \end{bmatrix} \tag{11.109}$$

for $\xi^2 = 0.5773502692$

$$\mathbf{J}^2 = \begin{bmatrix} 0.07735u_{x1} + 1.07735u_{x2} - 1.15470u_3 \\ 0.07735u_{y1} + 1.07735u_{y2} - 1.15470u_{y3} \end{bmatrix} \tag{11.110}$$

Jacobian Determinant (when the **Jacobian matrix** is a vector, its determinant is simply its **length**):

$$\det \mathbf{J} = ||\mathbf{J}|| \tag{11.111}$$

for $\xi^1 = -0.5773502692$

$$\det \mathbf{J}^1 = \sqrt{\left(-1.07735u_{x1} - 0.07735u_{x2} + 1.15470u_3\right)^2 + \left(-1.07735u_{y1} - 0.07735u_{y2} + 1.15470u_{y3}\right)^2} \tag{11.112}$$

for $\xi^2 = 0.5773502692$

$$\det \mathbf{J}^2 = \sqrt{\left(0.07735u_{x1} + 1.07735u_{x2} - 1.15470u_3\right)^2 + \left(0.07735u_{y1} + 1.07735u_{y2} - 1.15470u_{y3}\right)^2} \tag{11.113}$$

Jacobian Inverse (when the **Jacobian matrix** is a row vector, its inverse is just its **transpose** divided by its determinant squared):

Moore-Penrose pseudoinverse:

$$\mathbf{J}^{-1} = \mathbf{J}^{+} = \left(\mathbf{J}^T\mathbf{J}\right)^{-1}\mathbf{J}^T \tag{11.114}$$

for $\xi^1 = -0.5773502692$

$$\mathbf{J}^{1^{-1}} = \frac{1}{\det \mathbf{J}^{1^2}}\mathbf{J}^{1^T} \tag{11.115}$$

for $\xi^2 = 0.5773502692$

$$\mathbf{J}^{2^{-1}} = \frac{1}{\det \mathbf{J}^{2^2}}\mathbf{J}^{2^T} \tag{11.116}$$

Shape function derivatives in global coordinates:

$$\mathbf{B}_g = \mathbf{J}^{-1}\mathbf{B} \tag{11.117}$$

for $\xi^1 = -0.5773502692$

$$\mathbf{B}_g^1 = \mathbf{J}^{1^{-1}}\mathbf{B}^1 \tag{11.118}$$

for $\xi^2 = 0.5773502692$

$$\mathbf{B}_g^2 = \mathbf{J}^{2^{-1}}\mathbf{B}^2 \tag{11.119}$$

Material Stiffness Matrix

$$\mathbf{D} = EA \tag{11.120}$$

Then the stiffness matrix is composed as:

$$\mathbf{K}^e = \sum_{i=1}^{2} \mathbf{B}_g^{i\,T} \mathbf{D} \mathbf{B_g}^i \det \mathbf{J}^i w^i \tag{11.121}$$

11.4.6 Python Implementation

```python
#!/usr/bin/python3

import numpy as np
np.set_printoptions(precision=6, suppress=True)

def rod2(xyz, E, A, rho, fx, fy, fz, gauss_points=1):
    print('\nStarting ROD element generation:\n')
    coors = xyz.reshape(-1, 1)
    print('-------------------------------------------------------------- Input')
    print('coors =\n{0}'.format(coors))
    print(f'{A = } mm2\n{rho = } t/mm2\n{fx = } N/mm\n{fy = } N/mm' +
          f'\n{fz = } N/mm\n{gauss_points = }\n')

```

```python
    domain = np.array([[-1],
                       [ 1]], dtype=float)
    print('Domain: {0}\n{1}\n'.format(domain.shape, domain))

    print('----------------------------------------------------- Gauss-Legendre')
    # gauss-legendre integration
    ip, iw = np.polynomial.legendre.leggauss(gauss_points)
    print('Gauss-Legendre Integration\nip = {0}\n{1}\n'.format(ip, iw))

    full_domain = np.vstack((domain, ip.reshape(-1, 1)))

    print('----------------------------------------------------- Shape Functions')
    # shape functions in natural coordinates
    xi = ip.T
    print('xi = {0}\n'.format(xi))
    psi = []
    for i in range(xi.shape[0]):
        psi.append(np.array([[1/2 - xi[i]/2, 0, 0, 1/2 + xi[i]/2, 0, 0],
                             [0, 1/2 - xi[i]/2, 0, 0, 1/2 + xi[i]/2, 0],
                             [0, 0, 1/2 - xi[i]/2, 0, 0, 1/2 + xi[i]/2]],
                   dtype=float))
    psi = np.array(psi)
    # psi = psi.transpose(0, 2, 1)
    print('Shape Functions: {0}\n{1}\n'.format(psi.shape, psi))

    # integration points in global coordinates
    psi_g = psi @ coors
    print('Integration Points in '
          'Global Coordinates: {0}\n{1}\n'.format(psi_g.shape, psi_g))

    # shape function derivatives in natural coordinates
    dpsi = []
    for i in range(xi.shape[0]):
        dpsi.append(np.array([[-1/2, 0, 0, 1/2, 0, 0],
                              [0, -1/2, 0, 0, 1/2, 0],
                              [0, 0, -1/2, 0, 0, 1/2]], dtype=float))
    dpsi = np.array(dpsi)
    print('Shape Functions Derivatives: {0}\n{1}\n'.format(dpsi.shape, dpsi))

    print('---------------------------------------------------------- Jacobians')
    # Jacobian Matrix
```

```python
    jacobi = dpsi @ coors
    print('Jacobian Matrix: {0}\n{1}\n'.format(jacobi.shape, jacobi))

    # Jacobian Determinants
    d_jacobi = []
    for i in range(xi.shape[0]):
        d_jacobi.append(np.linalg.norm(jacobi[i]))
    d_jacobi = np.array(d_jacobi)
    print('Determinant of Jacobian: {0}\n{1}\n'.format(d_jacobi.shape, d_jacobi))

    # Inverse Jacobian
    i_jacobi = np.linalg.pinv(jacobi)
    print('Inverse Jacobian Matrix: {0}\n{1}\n'.format(i_jacobi.shape, i_jacobi))

    # Shape Function Derivatives in Global Coordinates
    dpsi_g = i_jacobi @ dpsi
    print('Shape Function Derivatives in Global Coordinates:
    ↪ {0}\n{1}\n'.format(dpsi_g.shape, dpsi_g))

    # material stiffness
    D = np.array([[E * A]], dtype=float)
    print('Material Stiffness Matrix: {0}\n{1}\n'.format(D.shape, D))

    # create element stiffness and mass matrix
    Ke = np.zeros((domain.shape[0] * 3, domain.shape[0] * 3), dtype=float)
    Me = np.zeros((domain.shape[0] * 3, domain.shape[0] * 3), dtype=float)
    Fe = np.zeros((domain.shape[0] * 3, 1), dtype=float)

    print('---------------------------------------------------- System Matrices')
    # iterate over integration points
    F = np.array([[fx], [fy], [fz]], dtype=float)
    for i in range(xi.shape[0]):
        B = dpsi_g[i]
        N = psi[i]

        Ke += (B.T @ D @ B) * d_jacobi[i] * iw[i]
        Me += rho * (N.T @ N) * d_jacobi[i] * iw[i]
        Fe += (N.T @ F) * d_jacobi[i] * iw[i]

    print('Element Stiffness Matrix: {0}\n{1}\n'.format(Ke.shape, Ke))
    print('Element Mass Matrix: {0}\n{1}\n'.format(Me.shape, Me))
```

```
    print('Element Volume Force Vector: {0}\n{1}\n'.format(Fe.shape, Fe))

    print('------------------------------------------------------ Displacements')
    # xyz displacements at end nodes
    u = np.array([0.,
                  0.,
                  0.,
                  .001,
                  .001,
                  .001], dtype=float).reshape(-1, 1)
    print('Displacements {0}\n{1}\n'.format(u.shape, u))

    print('------------------------------------------------------------ Strains')
    eps = []
    for i in range(xi.shape[0]):
        eps.append(dpsi_g[i] @ u)
    eps = np.array(eps)
    print('Strains {0}\n{1}\n'.format(eps.shape, eps))

    print('----------------------------------------------------------- Stresses')
    sig = D @ eps
    print('Stresses {0}\n{1}\n'.format(sig.shape, sig))

    return Ke, Me, Fe



if __name__ == '__main__':
    coor = np.array([[    0, 0., 0.],
                     [1000., 0., 0.]], dtype=float)

    E = 210000.0   # MPa steel
    A = 200.       # steel
    rho = 9.81E-9  # steel t/mm3

    fx = 1.  # volumetric continuous load
    fy = 0.  # volumetric continuous load
    fz = 1.  # volumetric continuous load

    rod2(coor, E, A, rho, fx, fy, fz, 1)
```

11.5 HEX8

11.5.1 Element Formulation

The **Hexahedral** element is a 3-D quadrilateral **trilinear** element, otherwise known as a **brick**. Topologically is hexahedron equivalent to a cube. It has eight corners, twelve edges and six faces. Especially the **HEX8** element has 8 interpolation points.

Natural coordinates

The **natural coordinate system** of a hexahedral element are called **isoparametric hexahedral coordinates**, denoted ξ, η and μ. The coordinates go from -1 to 1 and span from each face to the opposite one. Each coordinate is 0 on the plane at midpoint of opposing faces called the **median** face. This choice of limits is to facilitate the use of standard *Gauss integration formulas*.

Corner Numbering Rules

The eight corners of a hexahedron are locally numbered $1, 2, \ldots, 8$. The corner numbering rule is:

- select one face and numbers the corners of this face $1 \ldots 4$ in a **counterclockwise** direction.
- number the corners directly opposite to corners $1, 2, 3, 4$ as $5, 6, 7, 8$ respectively.

The purpose of this numbering manner is so that a positive volume (or more precisely, a positive **Jacobian determinant** at every point).

The definition of ξ, η and μ can be now made more precise:

- ξ goes from -1 from (center of) face 1485 to $+1$ on face 2376

- η goes from -1 from (center of) face 1265 to $+1$ on face 3487
- μ goes from -1 from (center of) face 1234 to $+1$ on face 5678

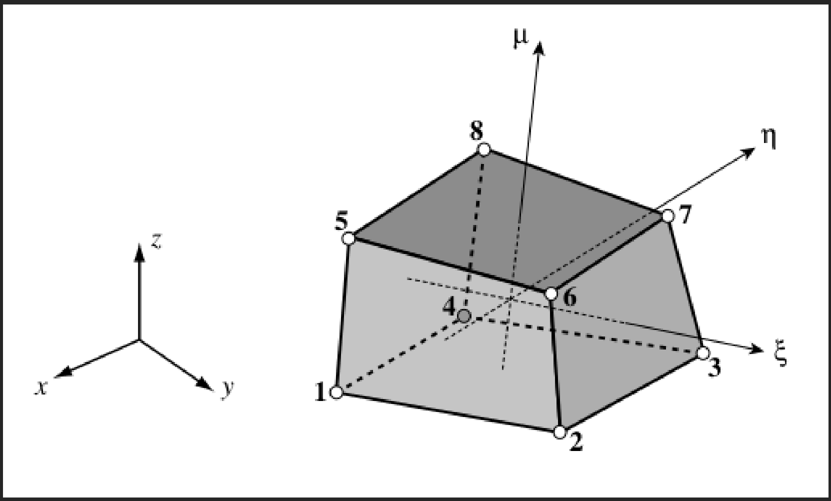


Figure 11.2: The 8-node hexahedron and the natural coordinates η, ξ and μ.

Element Definition

The 8-noded hexahedral element is defined by:

$$\begin{bmatrix} 1 \\ x \\ y \\ z \\ u_x \\ u_y \\ u_z \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & x_7 & x_8 \\ y_1 & y_2 & y_3 & y_4 & y_5 & y_6 & y_7 & y_8 \\ y_1 & y_2 & y_3 & y_4 & y_5 & y_6 & y_7 & y_8 \\ u_{x1} & u_{x2} & u_{x3} & u_{x4} & u_{x5} & u_{x6} & u_{x7} & u_{x8} \\ u_{y1} & u_{y2} & u_{y3} & u_{y4} & u_{y5} & u_{y6} & u_{y7} & u_{y8} \\ u_{z1} & u_{z2} & u_{z3} & u_{z4} & u_{z5} & u_{z6} & u_{z7} & u_{z8} \end{bmatrix} \begin{bmatrix} N_1^e \\ N_2^e \\ N_3^e \\ N_4^e \\ N_5^e \\ N_6^e \\ N_7^e \\ N_8^e \end{bmatrix} \tag{11.122}$$

The hexahedron corners natural coordinates are:

node	ξ	η	μ
1	-1	-1	-1
2	+1	-1	-1
3	+1	+1	-1
4	-1	+1	-1
5	-1	-1	+1
6	+1	-1	+1
7	+1	+1	+1
8	-1	+1	+1

Table 11.5: Hexahedron corners natural coordinates

The shape functions are:

$$\begin{aligned}
N_1^e &= \frac{1}{8}\left(1-\xi\right)\left(1-\eta\right)\left(1-\mu\right) \\
N_2^e &= \frac{1}{8}\left(1+\xi\right)\left(1-\eta\right)\left(1-\mu\right) \\
N_3^e &= \frac{1}{8}\left(1+\xi\right)\left(1+\eta\right)\left(1-\mu\right) \\
N_4^e &= \frac{1}{8}\left(1-\xi\right)\left(1+\eta\right)\left(1-\mu\right) \\
N_5^e &= \frac{1}{8}\left(1-\xi\right)\left(1-\eta\right)\left(1+\mu\right) \\
N_6^e &= \frac{1}{8}\left(1+\xi\right)\left(1-\eta\right)\left(1+\mu\right) \\
N_7^e &= \frac{1}{8}\left(1+\xi\right)\left(1+\eta\right)\left(1+\mu\right) \\
N_8^e &= \frac{1}{8}\left(1-\xi\right)\left(1+\eta\right)\left(1+\mu\right)
\end{aligned} \tag{11.123}$$

The eight formulas can be summarised in a single expression:

$$N_i^e = \frac{1}{8}\left(1+\xi\xi_i\right)\left(1+\eta\eta_i\right)\left(1+\mu\mu_i\right) \tag{11.124}$$

where ξ_i, η_i and μ_i denote the coordinates of the i-th node.

Partial Derivatives

The calculation of the shape functions derivatives with respect to the natural coordinates:

$$
\begin{aligned}
\frac{\partial N_1^e}{\partial \xi} &= -\frac{1}{8}\,(1-\eta)\,(1-\mu) \\
\frac{\partial N_2^e}{\partial \xi} &= \quad \frac{1}{8}\,(1-\eta)\,(1-\mu) \\
\frac{\partial N_3^e}{\partial \xi} &= \quad \frac{1}{8}\,(1+\eta)\,(1-\mu) \\
\frac{\partial N_4^e}{\partial \xi} &= -\frac{1}{8}\,(1+\eta)\,(1-\mu) \\
\frac{\partial N_5^e}{\partial \xi} &= -\frac{1}{8}\,(1-\eta)\,(1+\mu) \\
\frac{\partial N_6^e}{\partial \xi} &= \quad \frac{1}{8}\,(1-\eta)\,(1+\mu) \\
\frac{\partial N_7^e}{\partial \xi} &= \quad \frac{1}{8}\,(1+\eta)\,(1+\mu) \\
\frac{\partial N_8^e}{\partial \xi} &= -\frac{1}{8}\,(1+\eta)\,(1+\mu)
\end{aligned}
\tag{11.125}
$$

$$
\begin{aligned}
\frac{\partial N_1^e}{\partial \eta} &= -\frac{1}{8}(1-\xi)(1-\mu) \\
\frac{\partial N_2^e}{\partial \eta} &= -\frac{1}{8}(1+\xi)(1-\mu) \\
\frac{\partial N_3^e}{\partial \eta} &= \quad \frac{1}{8}(1+\xi)(1-\mu) \\
\frac{\partial N_4^e}{\partial \eta} &= \quad \frac{1}{8}(1-\xi)(1-\mu) \\
\frac{\partial N_5^e}{\partial \eta} &= -\frac{1}{8}(1-\xi)(1+\mu) \\
\frac{\partial N_6^e}{\partial \eta} &= -\frac{1}{8}(1+\xi)(1+\mu) \\
\frac{\partial N_7^e}{\partial \eta} &= \quad \frac{1}{8}(1+\xi)(1+\mu) \\
\frac{\partial N_8^e}{\partial \eta} &= \quad \frac{1}{8}(1-\xi)(1+\mu)
\end{aligned}
\tag{11.126}
$$

$$
\begin{aligned}
\frac{\partial N_1^e}{\partial \mu} &= -\frac{1}{8}(1-\xi)(1-\eta) \\
\frac{\partial N_2^e}{\partial \mu} &= -\frac{1}{8}(1+\xi)(1-\eta) \\
\frac{\partial N_3^e}{\partial \mu} &= -\frac{1}{8}(1+\xi)(1+\eta) \\
\frac{\partial N_4^e}{\partial \mu} &= -\frac{1}{8}(1-\xi)(1+\eta) \\
\frac{\partial N_5^e}{\partial \mu} &= \quad \frac{1}{8}(1-\xi)(1-\eta) \\
\frac{\partial N_6^e}{\partial \mu} &= \quad \frac{1}{8}(1+\xi)(1-\eta) \\
\frac{\partial N_7^e}{\partial \mu} &= \quad \frac{1}{8}(1+\xi)(1+\eta) \\
\frac{\partial N_8^e}{\partial \mu} &= \quad \frac{1}{8}(1-\xi)(1+\eta)
\end{aligned}
\tag{11.127}
$$

Note:
The partial derivatives can be also written as:

$$\begin{aligned}
\frac{\partial N_i^e}{\partial \xi} &= \frac{1}{8}\frac{\xi_i}{|\xi_i|}(1+\eta\eta_i)(1+\mu\mu_i) \\
\frac{\partial N_i^e}{\partial \eta} &= \frac{1}{8}\frac{\eta_i}{|\eta_i|}(1+\xi\xi_i)(1+\mu\mu_i) \\
\frac{\partial N_i^e}{\partial \mu} &= \frac{1}{8}\frac{\mu_i}{|\mu_i|}(1+\xi\xi_i)(1+\eta\eta_i)
\end{aligned} \tag{11.128}$$

The Jacobian:

The derivatives of the shape functions are given by the usual chain rule formulas:

$$\begin{aligned}
\frac{\partial N_i^e}{\partial x} &= \frac{\partial N_i^e}{\partial \xi}\frac{\partial \xi}{\partial x} + \frac{\partial N_i^e}{\partial \eta}\frac{\partial \eta}{\partial x} + \frac{\partial N_i^e}{\partial \mu}\frac{\partial \mu}{\partial x} \\
\frac{\partial N_i^e}{\partial y} &= \frac{\partial N_i^e}{\partial \xi}\frac{\partial \xi}{\partial y} + \frac{\partial N_i^e}{\partial \eta}\frac{\partial \eta}{\partial y} + \frac{\partial N_i^e}{\partial \mu}\frac{\partial \mu}{\partial y} \\
\frac{\partial N_i^e}{\partial z} &= \frac{\partial N_i^e}{\partial \xi}\frac{\partial \xi}{\partial z} + \frac{\partial N_i^e}{\partial \eta}\frac{\partial \eta}{\partial z} + \frac{\partial N_i^e}{\partial \mu}\frac{\partial \mu}{\partial z}
\end{aligned} \tag{11.129}$$

In matrix form:

$$\begin{bmatrix} \frac{\partial N_i^e}{\partial x} \\ \frac{\partial N_i^e}{\partial y} \\ \frac{\partial N_i^e}{\partial z} \end{bmatrix} = \begin{bmatrix} \frac{\partial \xi}{\partial x} & \frac{\partial \eta}{\partial x} & \frac{\partial \mu}{\partial x} \\ \frac{\partial \xi}{\partial y} & \frac{\partial \eta}{\partial y} & \frac{\partial \mu}{\partial y} \\ \frac{\partial \xi}{\partial z} & \frac{\partial \eta}{\partial z} & \frac{\partial \mu}{\partial z} \end{bmatrix} \begin{bmatrix} \frac{\partial N_i^e}{\partial \xi} \\ \frac{\partial N_i^e}{\partial \eta} \\ \frac{\partial N_i^e}{\partial \mu} \end{bmatrix} \tag{11.130}$$

The 3×3 matrix above is $\mathbf{J}^{-1}$, the inverse of:

$$\mathbf{J} = \frac{\partial(x,y,z)}{\partial(\xi,\eta,\mu)} = \begin{bmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial y}{\partial \xi} & \frac{\partial z}{\partial \xi} \\ \frac{\partial x}{\partial \eta} & \frac{\partial y}{\partial \eta} & \frac{\partial z}{\partial \eta} \\ \frac{\partial x}{\partial \mu} & \frac{\partial y}{\partial \mu} & \frac{\partial z}{\partial \mu} \end{bmatrix} \tag{11.131}$$

Matrix $\mathbf{J}$ is called the *Jacobian matrix* of (x, y, z) with respect to (ξ, η, μ). In the finite element literature, matrices $\mathbf{J}$ and $\mathbf{J}^{-1}$ are called simply the *Jacobian* and *inverse Jacobian*, respectively, although such a short name us sometimes ambiguous. The notation

$$\begin{aligned}\mathbf{J} &= \frac{\partial\left(x, y, z\right)}{\partial\left(\xi, \eta, \mu\right)}\\ \mathbf{J}^{-1} &= \frac{\partial\left(\xi, \eta, \mu\right)}{\partial\left(x, y, z\right)}\end{aligned} \tag{11.132}$$

is standard in multivariable calculus and suggests that the Jacobian may be viewed as a generalisation of the ordinary derivativem to which it reduces for a scalar function $\mathbf{x} = x(\xi)$.

Computing the Jacobian Matrix

The isoparametric definitioin of hexahedron element geometry is:

$$\begin{aligned}x &= x_i N_i^e\\ y &= y_i N_i^e\\ z &= z_i N_i^e\end{aligned} \tag{11.133}$$

where the summation convention is understood to apply over $i = 1, 2, \ldots, n$, in which n denotes the number of element nodes.

Note: This for a given hexahedral element gives an $n \times 3$ matrix.

Differentiating these relations with respect to the hexahedron coordinates we construct the matrix $\mathbf{J}$ as follows:

$$\mathbf{J} = \begin{bmatrix} x_i \frac{\partial N_i^e}{\partial \xi} & y_i \frac{\partial N_i^e}{\partial \xi} & z_i \frac{\partial N_i^e}{\partial \xi} \\ x_i \frac{\partial N_i^e}{\partial \eta} & y_i \frac{\partial N_i^e}{\partial \eta} & z_i \frac{\partial N_i^e}{\partial \eta} \\ x_i \frac{\partial N_i^e}{\partial \mu} & y_i \frac{\partial N_i^e}{\partial \mu} & z_i \frac{\partial N_i^e}{\partial \mu} \end{bmatrix} \tag{11.134}$$

or:

$$\mathbf{J}_i = \begin{bmatrix} \frac{\partial N_1^e}{\partial \xi} & \frac{\partial N_2^e}{\partial \xi} & \dots & \frac{\partial N_n^e}{\partial \xi} \\ \frac{\partial N_1^e}{\partial \eta} & \frac{\partial N_2^e}{\partial \eta} & \dots & \frac{\partial N_n^e}{\partial \eta} \\ \frac{\partial N_1^e}{\partial \mu} & \frac{\partial N_2^e}{\partial \mu} & \dots & \frac{\partial N_n^e}{\partial \mu} \end{bmatrix} \begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ \vdots & \vdots & \vdots \\ x_n & y_n & z_n \end{bmatrix} \tag{11.135}$$

Given a point of hexahedron coordinates (ξ, η, μ) the Jacobian $\mathbf{J}$ can be easilly formed using the above formula, and numerically inverted to form $\mathbf{J}^{-1}$.

The Strain-Displacement Matrix

Having obtained the shape function derivatives, the matrix $\mathbf{B}$ for hexahedron element displays the usual structure for 3D elements:

$$\mathbf{B} = \mathbf{D\Phi} = \begin{bmatrix} \frac{\partial}{\partial x} & 0 & 0 \\ 0 & \frac{\partial}{\partial y} & 0 \\ 0 & 0 & \frac{\partial}{\partial z} \\ \frac{\partial}{\partial y} & \frac{\partial}{\partial x} & 0 \\ 0 & \frac{\partial}{\partial z} & \frac{\partial}{\partial y} \\ \frac{\partial}{\partial z} & 0 & \frac{\partial}{\partial x} \end{bmatrix} \begin{bmatrix} \mathbf{q} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{q} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{q} \end{bmatrix} = \begin{bmatrix} \mathbf{q}_x & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{q}_y & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{q}_z \\ \mathbf{q}_y & \mathbf{q}_x & \mathbf{0} \\ \mathbf{0} & \mathbf{q}_z & \mathbf{q}_y \\ \mathbf{q}_z & \mathbf{0} & \mathbf{q}_x \end{bmatrix} \tag{11.136}$$

where:

$$\begin{aligned} \mathbf{q} &= \begin{bmatrix} N_1^e & \dots & N_n^e \end{bmatrix} \\ \mathbf{q}_x &= \begin{bmatrix} \frac{\partial N_1^e}{\partial x} & \dots & \frac{\partial N_n^e}{\partial x} \end{bmatrix} \\ \mathbf{q}_y &= \begin{bmatrix} \frac{\partial N_1^e}{\partial y} & \dots & \frac{\partial N_n^e}{\partial y} \end{bmatrix} \\ \mathbf{q}_z &= \begin{bmatrix} \frac{\partial N_1^e}{\partial z} & \dots & \frac{\partial N_n^e}{\partial z} \end{bmatrix} \end{aligned} \tag{11.137}$$

are row vectors of length n, n being the number of nodes in the element.

Stiffness Matrix Evaluation

The element stiffness Matrix is given by:

$$\mathbf{K}^e = \int_{V^e} \mathbf{B}^T \mathbf{E} \mathbf{B} dV^e \tag{11.138}$$

As in two-dimensional case, this is replaced by a numerical integration formula which now involves a triple loop over conventional Gauss quadrature rules. Assuming that the stress-strain matrix $\mathbf{E}$ is constant over the element,

$$\mathbf{K}^e = \sum_{i=1}^{p_1} \sum_{j=1}^{p_2} \sum_{k=1}^{p_3} w_i w_j w_k \mathbf{B}_{ijk}^T \mathbf{E} \mathbf{B}_{ijk} \mathbf{J}_{ijk} \tag{11.139}$$

Here p_1, p_2 and p_3 are the number of Gauss points in the ξ, η and μ direction, respectively, while $\mathbf{B}_{ijk}$ and $\mathbf{J}_{ijk}$ are abbreviations for:

$$\begin{aligned} \mathbf{B}_{ijk} &= \mathbf{B}\left(\xi_i, \eta_i, \mu_i\right) \\ \mathbf{J}_{ijk} &= det\mathbf{J}\left(\xi_i, \eta_i, \mu_i\right) \end{aligned} \tag{11.140}$$

Usually the number of integration points is taken the same in all directions: $p = p_1 = p_2 = p_3$. The total number of Gauss points is thus p^3. Each point adds at most 6 to the stiffness matrix rank. The minimum rank-sufficient rules for the 8-node and 20-node hexahedra are $p = 2$ and $p = 3$, respectively.

11.5.2 Python implementation

```
#!/usr/bin/python3

import numpy as np
np.set_printoptions(precision=2) # , suppress=True)

```

```
def hex8_stiffness_mass_load(coors: np.ndarray,
                             E: float,
                             nu: float,
                             rho: float,
                             fx: float = 0.,
                             fy: float = 0.,
                             fz: float = 0.):
    """
    In:
        coors - 8 x 3 coordinate matrix
                [[x1,y1,z1], ... , [xn,yn,zn]
        E     - Youngs's Modulus
        nu    - Poisson's Constant
        rho   - Density
        fx    - Volumetric Load in x direction
        fy    - Volumetric Load in y direction
        fz    - Volumetric Load in z direction
    """
    domain = np.array([[-1., -1., -1.],
                       [1., -1., -1.],
                       [1., 1., -1.],
                       [-1., 1., -1.],
                       [-1., -1., 1.],
                       [1., -1., 1.],
                       [1., 1., 1.],
                       [-1., 1., 1.]], dtype=float)
    print('Domain: {0}\n{1}'.format(domain.shape, domain))

    # gaussian interpolation
    gauss_points = 3
    g_points, g_weights = np.polynomial.legendre.leggauss(gauss_points)
    integration_points = []
    interpolation_weights = []
    for i, xi in enumerate(g_points):
        for j, eta in enumerate(g_points):
            for k, mu in enumerate(g_points):
                integration_points.append([xi, eta, mu])
                interpolation_weights.append(g_weights[i] *
                                             g_weights[j] *
                                             g_weights[k])
    integration_points = np.array(integration_points, dtype=float)
```

```
    interpolation_weights = np.array(interpolation_weights, dtype=float)
    print('Gauss Integration '
          '{0}:\n{1}\nWeights:\n{2}'.format(gauss_points,
                                              integration_points,
                                              interpolation_weights))

    full_domain = np.vstack((domain, integration_points))

    # Shape Functions in Natural Coordinates
    xi = integration_points.T[0]
    eta = integration_points.T[1]
    mu = integration_points.T[2]
    psi = np.zeros((8, integration_points.shape[0]), dtype=float)
    for i in range(8):
        psi[i] = 1/8 * (1 + xi * domain[i, 0]) *
                       (1 + eta * domain[i, 1]) *
                       (1 + mu * domain[i, 2])
    psi = psi.T
    print('Shape Functions: {0}\n{1}'.format(psi.shape, psi))

    # Shape Functions in Global Coordinates
    psi_g = psi @ coors
    print('Shape Functions in '
          'Global Coordinates: {0}\n{1}'.format(psi_g.shape, psi_g))

    # Shape Functions Derivatives in Natural Coordinates
    dpsi = 1 / 8 * np.array([[(eta - 1.0) * (1.0 - mu),# 1
                              (xi - 1) * (1 - mu),
                             -(1 - xi) * (1 - eta)],
                             [(1 - eta) * (1 - mu),     # 2
                              (-1 - xi) * (1 - mu),
                             -(1 + xi) * (1 - eta)],
                             [(1 + eta) * (1 - mu),     # 3
                              (1 + xi) * (1 - mu),
                             -(1 + xi) * (1 + eta)],
                             [(-1.0 - eta) * (1 - mu), # 4
                              (1 - xi) * (1 - mu),
                             -(1 - xi) * (1 + eta)],
                             [(1 - eta) * (-1 - mu),    # 5
                             -(1 - xi) * (1 + mu),
                              (1 - xi) * (1 - eta)],
```

```
                                [(1 - eta) * (1 + mu),    # 6
                                -(1 + xi) * (1 + mu),
                                 (1 + xi) * (1 - eta)],
                                [(1 + eta) * (1 + mu),    # 7
                                 (1 + xi) * (1 + mu),
                                 (1 + xi) * (1 + eta)],
                                [-(1 + eta) * (1 + mu),   # 8
                                 (1 - xi) * (1 + mu),
                                 (1 - xi) * (1 + eta)]])
    dpsi = dpsi.T
    print('Shape Functions Derivatives: '
          '{0}\n{1}'.format(dpsi.shape, dpsi))

    # Jacobian Matrix
    jacobi = dpsi @ coors
    print('Jacobian Matrix: {0}\n{1}'.format(jacobi.shape, jacobi))

    # Jacobian Determinants
    d_jacobi = np.linalg.det(jacobi)
    print('Determinant of Jacobian: '
          '{0}\n{1}'.format(d_jacobi.shape, d_jacobi))

    # Inverse Jacobian
    i_jacobi = np.linalg.inv(jacobi)
    print('Inverse Jacobian Matrix: '
          '{0}\n{1}'.format(i_jacobi.shape, i_jacobi))

    # Shape Function Derivatives in Global Coordinates
    dpsi_g = i_jacobi @ dpsi
    print('Shape Function Derivatives in Global Coordinates: '
          '{0}\n{1}'.format(dpsi_g.shape, dpsi_g))

    # Material Stiffness Matrix
    C = E / ((1.0 + nu) * (1.0 - 2.0 * nu)) *
        np.array([[1.0 - nu, nu, nu, 0.0, 0.0, 0.0],
                  [nu, 1.0 - nu, nu, 0.0, 0.0, 0.0],
                  [nu, nu, 1.0 - nu, 0.0, 0.0, 0.0],
                  [0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0, 0.0],
                  [0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0],
                  [0.0, 0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0]],
                  dtype=float)
```

```
    print('Material Stiffness Matrix: {0}\n{1}'.format(C.shape, C))

    # Create Element Stiffness and Mass Matrix
    Ke = np.zeros((domain.size, domain.size), dtype=float)
    Me = np.zeros((domain.size, domain.size), dtype=float)
    Fe = np.zeros((domain.size, 1), dtype=float)
    # o = np.zeros(domain.shape[0], dtype=float)

    # iterate over Gauss points of domain, * means unpack values
    for i in range(integration_points.shape[0]):
        # this is smart but arranges dofs by component, not by node
        # component-wise dof ordering (x1, .. , y1, .. y4, z1, .. , z4)
        # B = np.array([
        #   [*dpsi_g[i, 0, :], *o, *o],
        #   [*o, *dpsi_g[i, 1, :], *o],
        #   [*o, *o, *dpsi_g[i, 2, :]],
        #   [*dpsi_g[i, 2, :], *o, *dpsi_g[i, 0, :]],
        #   [*o, *dpsi_g[i, 2, :], *dpsi_g[i, 1, :]],
        #   [*dpsi_g[i, 1, :], *dpsi_g[i, 0, :], *o]], dtype=float)

        # dofs ordered by node
        # node wise dof ordering (x1, y1, z1, .. , x4, y4, z4)
        dpx = dpsi_g[i,0,:]
        dpy = dpsi_g[i,1,:]
        dpz = dpsi_g[i,2,:]
        # B = np.array(
        #
        ↪  [[dpx[0],0,0,dpx[1],0,0,dpx[2],0,0,dpx[3],0,0,dpx[4],0,0,dpx[5],0,0,dpx[6],0,0,dpx[7]
        #
        ↪  [0,dpy[0],0,0,dpy[1],0,0,dpy[2],0,0,dpy[3],0,0,dpy[4],0,0,dpy[5],0,0,dpy[6],0,0,dpy[7
        #
        ↪  [0,0,dpz[0],0,0,dpz[1],0,0,dpz[2],0,0,dpz[3],0,0,dpz[4],0,0,dpz[5],0,0,dpz[6],0,0,dpz
        #
        ↪  [dpy[0],dpx[0],0,dpy[1],dpx[1],0,dpy[2],dpx[2],0,dpy[3],dpx[3],0,dpy[4],dpx[4],0,dpy[
        #
        ↪  [0,dpz[0],dpy[0],0,dpz[1],dpy[1],0,dpz[2],dpy[2],0,dpz[3],dpy[3],0,dpz[4],dpy[4],0,dp
        #
        ↪  [dpz[0],0,dpx[0],dpz[1],0,dpx[1],dpz[2],0,dpx[2],dpz[3],0,dpx[3],dpz[4],0,dpx[4],dpz[
        #      dtype=_FLOAT)
        B = np.array([np.column_stack((dpx,   o,   o)).flatten(),
                      np.column_stack((  o, dpy,   o)).flatten(),
```

```
                      np.column_stack((  o,   o, dpz)).flatten(),
                      np.column_stack((dpy, dpx,   o)).flatten(),
                      np.column_stack((  o, dpz, dpy)).flatten(),
                      np.column_stack((dpz,   o, dpx)).flatten()], dtype=float)

        # this is smart but arranges dofs by component, not by node
        # component-wise dof ordering (x1, .. , y1, .. y4, z1, .. , z4)
        # N = np.array([
        #   [*psi[i], *o, *o],
        #   [*o, *psi[i], *o],
        #   [*o, *o, *psi[i]]],
        #   dtype=float)

        # node wise dof ordering (x1, y1, z1, .. , x4, y4, z4)
        # N = np.array([[p[0],0,0,p[1],0,0,p[2],0,0,p[3],0,0,p[4],0,0,p[5],0,0,p[6],0,0,p[7],0,0],
        #   [0,p[0],0,0,p[1],0,0,p[2],0,0,p[3],0,0,p[4],0,0,p[5],0,0,p[6],0,0,p[7],0],
        #   [0,0,p[0],0,0,p[1],0,0,p[2],0,0,p[3],0,0,p[4],0,0,p[5],0,0,p[6],0,0,p[7]]],
        #               dtype=_FLOAT)
        N = np.array([np.column_stack((p, o, o)).flatten(),
                      np.column_stack((o, p, o)).flatten(),
                      np.column_stack((p, o, o)).flatten()], dtype=float)

        F = np.array([[fx], [fy], [fz]], dtype=float)

        Ke += (B.T @ C @ B) * d_jacobi[i] * interpolation_weights[i]
        Me += rho * (N.T @ N) * d_jacobi[i] * interpolation_weights[i]
        Fe += (N.T @ F) * d_jacobi[i] * interpolation_weights[i]

    print('Element Stiffness Matrix: {0}\n{1}'.format(Ke.shape, Ke))
    print('Element Mass Matrix: {0}\n{1}'.format(Me.shape, Me))
    print('Element Volume Force Vector: {0}\n{1}'.format(Fe.shape, Fe))
```

And to get stresses:

```
#!/usr/bin/python3
```

```python

import numpy as np
np.set_printoptions(precision=2) # , suppress=True)

def hex8_stresses(coors: np.ndarray,
                  disp: np.ndarray,
                  E: float,
                  nu: float):
    """
    In:
        coors - 8 x 3 coordinate matrix
                [[x1,y1,z1], ... , [xn,yn,zn]
        disp  - 8 x 3 coordinate matrix
                [[u1,v1,w1], ... , [un,vn,wn]
        E     - Youngs's Modulus
        nu    - Poisson's Constant
    """
    domain = np.array([[-1., -1., -1.],
                       [1., -1., -1.],
                       [1., 1., -1.],
                       [-1., 1., -1.],
                       [-1., -1., 1.],
                       [1., -1., 1.],
                       [1., 1., 1.],
                       [-1., 1., 1.]], dtype=float)
    print('Domain: {0}\n{1}'.format(domain.shape, domain))

    # gaussian interpolation
    gauss_points = 3
    g_points, g_weights = np.polynomial.legendre.leggauss(gauss_points)
    integration_points = []
    interpolation_weights = []
    for i, xi in enumerate(g_points):
        for j, eta in enumerate(g_points):
            for k, mu in enumerate(g_points):
                integration_points.append([xi, eta, mu])
                interpolation_weights.append(g_weights[i] *
                                             g_weights[j] *
                                             g_weights[k])
    integration_points = np.array(integration_points, dtype=float)
    interpolation_weights = np.array(interpolation_weights, dtype=float)
```

```
    print('Gauss Integration '
          '{0}:\n{1}\nWeights:\n{2}'.format(gauss_points,
                                            integration_points,
                                            interpolation_weights))

    full_domain = np.vstack((domain, integration_points))

    # Shape Functions in Natural Coordinates
    xi = integration_points.T[0]
    eta = integration_points.T[1]
    mu = integration_points.T[2]
    psi = np.zeros((8, integration_points.shape[0]), dtype=float)
    for i in range(8):
        psi[i] = 1/8 * (1 + xi * domain[i, 0]) *
                       (1 + eta * domain[i, 1]) *
                       (1 + mu * domain[i, 2])
    psi = psi.T
    print('Shape Functions: {0}\n{1}'.format(psi.shape, psi))

    # Shape Functions in Global Coordinates
    psi_g = psi @ coors
    print('Shape Functions in '
          'Global Coordinates: {0}\n{1}'.format(psi_g.shape, psi_g))

    # Shape Functions Derivatives in Natural Coordinates
    dpsi = 1 / 8 * np.array([[(eta - 1.0) * (1.0 - mu),# 1
                              (xi - 1) * (1 - mu),
                             -(1 - xi) * (1 - eta)],
                             [(1 - eta) * (1 - mu),    # 2
                              (-1 - xi) * (1 - mu),
                             -(1 + xi) * (1 - eta)],
                             [(1 + eta) * (1 - mu),    # 3
                              (1 + xi) * (1 - mu),
                             -(1 + xi) * (1 + eta)],
                             [(-1.0 - eta) * (1 - mu), # 4
                              (1 - xi) * (1 - mu),
                             -(1 - xi) * (1 + eta)],
                             [(1 - eta) * (-1 - mu),   # 5
                             -(1 - xi) * (1 + mu),
                              (1 - xi) * (1 - eta)],
                             [(1 - eta) * (1 + mu),    # 6
```

```
                                -(1 + xi) * (1 + mu),
                                 (1 + xi) * (1 - eta)],
                                [(1 + eta) * (1 + mu),    # 7
                                 (1 + xi) * (1 + mu),
                                 (1 + xi) * (1 + eta)],
                                [-(1 + eta) * (1 + mu),   # 8
                                 (1 - xi) * (1 + mu),
                                 (1 - xi) * (1 + eta)]])
    dpsi = dpsi.T
    print('Shape Functions Derivatives: '
          '{0}\n{1}'.format(dpsi.shape, dpsi))

    # Jacobian Matrix
    jacobi = dpsi @ coors
    print('Jacobian Matrix: {0}\n{1}'.format(jacobi.shape, jacobi))

    # Jacobian Determinants
    d_jacobi = np.linalg.det(jacobi)
    print('Determinant of Jacobian: '
          '{0}\n{1}'.format(d_jacobi.shape, d_jacobi))

    # Inverse Jacobian
    i_jacobi = np.linalg.inv(jacobi)
    print('Inverse Jacobian Matrix: '
          '{0}\n{1}'.format(i_jacobi.shape, i_jacobi))

    # Shape Function Derivatives in Global Coordinates
    dpsi_g = i_jacobi @ dpsi
    print('Shape Function Derivatives in Global Coordinates: '
          '{0}\n{1}'.format(dpsi_g.shape, dpsi_g))

    # Material Stiffness Matrix
    C = E / ((1.0 + nu) * (1.0 - 2.0 * nu)) *
        np.array([[1.0 - nu, nu, nu, 0.0, 0.0, 0.0],
                  [nu, 1.0 - nu, nu, 0.0, 0.0, 0.0],
                  [nu, nu, 1.0 - nu, 0.0, 0.0, 0.0],
                  [0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0, 0.0],
                  [0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0],
                  [0.0, 0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0]],
                  dtype=float)
    print('Material Stiffness Matrix: {0}\n{1}'.format(C.shape, C))
```

```

    du = disp.T @ np.transpose(dpsi_g, axes=[0, 2, 1])

    exx = du[:, 0, 0]
    eyy = du[:, 1, 1]
    ezz = du[:, 2, 2]
    exy = du[:, 0, 1] + du[:, 1, 0]
    eyz = du[:, 1, 2] + du[:, 2, 1]
    exz = du[:, 0, 2] + du[:, 2, 0]
    epsilons = np.array([exx, eyy, ezz, exy, eyz, exz])

    sigmas = (C @ epsilons).T
    epsilons = epsilons.T
    print('Element Strains: {0}\n{1}'.format(epsilons.shape, epsilons))
    print('Element Stresses: {0}\n{1}'.format(stresses.shape, stresses))
```

11.6 TET4 Element

11.6.1 Element Formulation

For a linear tetrahedron no numerical integration is needed (there can be only one integration point, shape functions derivatives are equal to 1).

The linear tetrahedron is not often used for stress analysis because of its poor performance. Its main value in structural and solid mechanics is educational: it serves as a vehicle to introduce the basic steps of formulation of 3D solid elements, particularly with regards to the use of natural coordinate systems and node numbering conventions. It should be noted that 3D visualisation is notoriously more difficult than 2D, so it is needed to procede more slowly here.

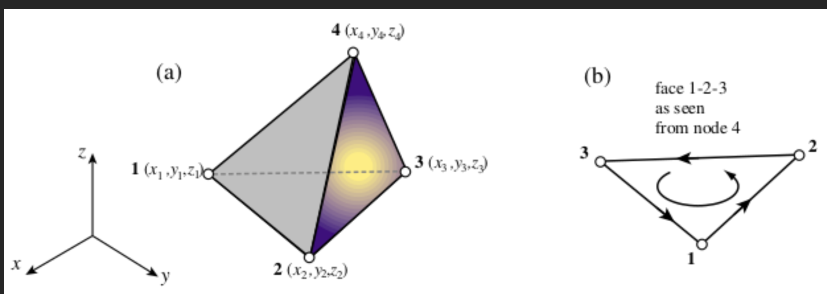


Figure 11.3: (a)The linear tetrahedron element TET4, (b) Node numbering convention.

Tetrahedron Geometry

The tetrahedron geometry is fully defined by giving the location of the four corner nodes with respect to Right-Hand Coordinate Sysetm notation (x, y, z):

11.6.2 Python Implementation

```
#!/usr/bin/python3

import numpy as np
np.set_printoptions(precision=2) # , suppress=True)

def tet4_stiffness_mass_load(coors: np.ndarray,
                             E: float,
                             nu: float,
                             rho: float,
                             fx: float = 0.,
                             fy: float = 0.,
                             fz: float = 0.,
                             gauss_points = 4):
    domain = np.array([[1., 0., 0., 0.],
                       [0., 1., 0., 0.],
                       [0., 0., 1., 0.],
                       [0., 0., 0., 1.]], dtype=float)
    print('Domain: {0}\n{1}'.format(domain.shape, domain))

    # gaussian interpolation
    integration_points = None
    integration_weights = None
    if gauss_points == 1:
        integration_points = np.array([1/4, 1/4, 1/4, 1/4],
                                      dtype=float).reshape(1, 4)
        integration_weights = np.array([1 * 1 * 1 * 1],
                                       dtype=float) * 1/6

    elif gauss_points == 4:
        a = 0.58541020
        b = 0.13819660
        integration_points = np.array([[a, b, b, b],
                                       [b, a, b, b],
                                       [b, b, a, b],
                                       [b, b, b, a]], dtype=float)
        integration_weights = np.array([1/4, 1/4, 1/4, 1/4],
                                       dtype=float) * 1/6
```

```python

    elif gauss_points == 5:
        integration_points = np.array([[1/4, 1/4, 1/4, 1/4],
                                       [1/2, 1/6, 1/6, 1/6],
                                       [1/6, 1/2, 1/6, 1/6],
                                       [1/6, 1/6, 1/2, 1/6],
                                       [1/6, 1/6, 1/6, 1/2]], dtype=float)
        integration_weights = np.array([-4/5, 9/20, 9/20, 9/20, 9/20],
                                       dtype=float) * 1/6

    print('Gauss Integration '
          '{0}:\n{1}\nWeights:\n{2}'.format(gauss_points,
                                            integration_points,
                                            integration_weights))

    full_domain = np.vstack((domain, integration_points))

    # Shape Functions in Natural Coordinates
    xi = integration_points.T[0]
    eta = integration_points.T[1]
    mu = integration_points.T[2]
    # zeta is not used to enforce
    # (1 - xi - eta - mu - zeta) = 0
    # zeta = integration_points.T[3]
    psi = np.zeros((4, integration_points.shape[0]), dtype=float)
    # another way of writing that psi = [xi, eta, mu, zeta]
    # for i in range(4):
    #     psi[i] = (1 + xi * domain[i, 0]) * (1 + eta * domain[i, 1]) * (1 + mu * domain[i, 2]) * (1 + zeta * domain[i, 3])
    psi[0] = xi
    psi[1] = eta
    psi[2] = mu
    psi[3] = 1 - xi - eta - mu  # = zeta
    psi = psi.T
    print('Shape Functions: {0}\n{1}'.format(psi.shape, psi))

    # Shape Functions in Global Coordinates
    psi_g = psi @ coors
    print('Shape Functions in Global Coordinates: '
          '{0}\n{1}'.format(psi_g.shape, psi_g))

```

```
    # Shape Functions Derivatives in Natural Coordinates
    dpsi = np.zeros((integration_points.shape[0], 4, 3), dtype=float)
    dpsi[:,0,0] = 1
    dpsi[:,1,1] = 1
    dpsi[:,2,2] = 1
    dpsi[:,3,:] = -1
    dpsi = dpsi.transpose((0, 2, 1))
    print('Shape Functions Derivatives: '
          '{0}\n{1}'.format(dpsi.shape, dpsi))

    # Jacobian Matrix
    jacobi = dpsi @ coors
    print('Jacobian Matrix: {0}\n{1}'.format(jacobi.shape, jacobi))

    # Jacobian Determinants
    d_jacobi = np.linalg.det(jacobi)
    print('Determinant of Jacobian: '
          '{0}\n{1}'.format(d_jacobi.shape, d_jacobi))

    # Inverse Jacobian
    i_jacobi = np.linalg.inv(jacobi)
    print('Inverse Jacobian Matrix: '
          '{0}\n{1}'.format(i_jacobi.shape, i_jacobi))

    # Shape Function Derivatives in Global Coordinates
    dpsi_g = i_jacobi @ dpsi
    print('Shape Function Derivatives in Global Coordinates: '
          '{0}\n{1}'.format(dpsi_g.shape, dpsi_g))

    # Material Stiffness Matrix
    C = E / ((1.0 + nu) * (1.0 - 2.0 * nu)) *
        np.array([[1.0 - nu, nu, nu, 0.0, 0.0, 0.0],
                  [nu, 1.0 - nu, nu, 0.0, 0.0, 0.0],
                  [nu, nu, 1.0 - nu, 0.0, 0.0, 0.0],
                  [0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0, 0.0],
                  [0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0],
                  [0.0, 0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0]],
                  dtype=float)
    print('Material Stiffness Matrix: {0}\n{1}'.format(C.shape, C))

    # Create Element Stiffness and Mass Matrix
```

```
    Ke = np.zeros((domain.shape[0] * 3, domain.shape[0] * 3), dtype=float)
    Me = np.zeros((domain.shape[0] * 3, domain.shape[0] * 3), dtype=float)
    Fe = np.zeros((domain.shape[0] * 3, 1), dtype=float)
    o = np.zeros(domain.shape[0], dtype=float)

    # iterate over Gauss points of domain, * means unpack values
    for i in range(integration_points.shape[0]):
        # component-wise dof ordering (x1, .. , y1, .. y4, z1, .. , z4)
        # B = np.array([[*dpsi_g[i, 0, :], *o, *o],
        #               [*o, *dpsi_g[i, 1, :], *o],
        #               [*o, *o, *dpsi_g[i, 2, :]],
        #               [*dpsi_g[i, 2, :], *o, *dpsi_g[i, 0, :]],
        #               [*o, *dpsi_g[i, 2, :], *dpsi_g[i, 1, :]],
        #               [*dpsi_g[i, 1, :], *dpsi_g[i, 0, :], *o]],
        #               dtype=float)

        # node wise dof ordering (x1, y1, z1, .. , x4, y4, z4)
        dpx = dpsi_g[i,0,:]
        dpy = dpsi_g[i,1,:]
        dpz = dpsi_g[i,2,:]
        B = np.array(
            [[dpx[0],0,0,dpx[1],0,0,dpx[2],0,0,dpx[3],0,0],
             [0,dpy[0],0,0,dpy[1],0,0,dpy[2],0,0,dpy[3],0],
             [0,0,dpz[0],0,0,dpz[1],0,0,dpz[2],0,0,dpz[3]],
             [dpy[0],dpx[0],0,dpy[1],dpx[1],0,dpy[2],dpx[2],0,dpy[3],dpx[3],0],
             [0,dpz[0],dpy[0],0,dpz[1],dpy[1],0,dpz[2],dpy[2],0,dpz[3],dpy[3]],
             [dpz[0],0,dpx[0],dpz[1],0,dpx[1],dpz[2],0,dpx[2],dpz[3],0,dpx[3]]],
             dtype=float)

        # component-wise dof ordering (x1, .. , y1, .. y4, z1, .. , z4)
        # N = np.array([[*psi[i], *o, *o],
        #               [*o, *psi[i], *o],
        #               [*o, *o, *psi[i]]], dtype=float)

        # node wise dof ordering (x1, y1, z1, .. , x4, y4, z4)
        N = np.array([[psi[i,0],0,0,psi[i,1],0,0,psi[i,2],0,0,psi[i,3],0,0],
                      [0,psi[i,0],0,0,psi[i,1],0,0,psi[i,2],0,0,psi[i,3],0],
                      [0,0,psi[i,0],0,0,psi[i,1],0,0,psi[i,2],0,0,psi[i,3]],
                      dtype=float)

        F = np.array([[fx], [fy], [fz]], dtype=float)
```

```
            print((B.T @ C @ B).shape)
            print(Ke.shape)
            Ke += (B.T @ C @ B) * d_jacobi[i] * integration_weights[i]
            Me += rho * (N.T @ N) * d_jacobi[i] * integration_weights[i]
            Fe += (N.T @ F) * d_jacobi[i] * integration_weights[i]

    print('Element Stiffness Matrix: {0}\n{1}'.format(Ke.shape, Ke))
    print('Element Mass Matrix: {0}\n{1}'.format(Me.shape, Me))
    print('Element Volume Force Vector: {0}\n{1}'.format(Fe.shape, Fe))
```

And to get stresses:

```
#!/usr/bin/python3

import numpy as np
np.set_printoptions(precision=2) # , suppress=True)

def tet4_stresses(coors: np.ndarray,
                  disp: np.ndarray,
                  E: float,
                  nu: float):
    """
    In:
        coors - 4 x 3 coordinate matrix
                [[x1,y1,z1], ... , [xn,yn,zn]
        disp  - 4 x 3 coordinate matrix
                [[u1,v1,w1], ... , [un,vn,wn]
        E     - Youngs's Modulus
        nu    - Poisson's Constant
    """
    domain = np.array([[1., 0., 0., 0.],
                       [0., 1., 0., 0.],
                       [0., 0., 1., 0.],
                       [0., 0., 0., 1.]], dtype=float)
    print('Domain: {0}\n{1}'.format(domain.shape, domain))

    # gaussian interpolation
```

```
    integration_points = None
    integration_weights = None
    if gauss_points == 1:
        integration_points = np.array([1/4, 1/4, 1/4, 1/4],
                                      dtype=float).reshape(1, 4)
        integration_weights = np.array([1 * 1 * 1 * 1],
                                       dtype=float) * 1/6

    elif gauss_points == 4:
        a = 0.58541020
        b = 0.13819660
        integration_points = np.array([[a, b, b, b],
                                       [b, a, b, b],
                                       [b, b, a, b],
                                       [b, b, b, a]], dtype=float)
        integration_weights = np.array([1/4, 1/4, 1/4, 1/4],
                                       dtype=float) * 1/6

    elif gauss_points == 5:
        integration_points = np.array([[1/4, 1/4, 1/4, 1/4],
                                       [1/2, 1/6, 1/6, 1/6],
                                       [1/6, 1/2, 1/6, 1/6],
                                       [1/6, 1/6, 1/2, 1/6],
                                       [1/6, 1/6, 1/6, 1/2]], dtype=float)
        integration_weights = np.array([-4/5, 9/20, 9/20, 9/20, 9/20],
                                       dtype=float) * 1/6

    print('Gauss Integration '
          '{0}:\n{1}\nWeights:\n{2}'.format(gauss_points,
                                            integration_points,
                                            integration_weights))

    full_domain = np.vstack((domain, integration_points))

    # Shape Functions in Natural Coordinates
    xi = integration_points.T[0]
    eta = integration_points.T[1]
    mu = integration_points.T[2]
    # zeta is not used to enforce
    # (1 - xi - eta - mu - zeta) = 0
    # zeta = integration_points.T[3]
```

```
psi = np.zeros((4, integration_points.shape[0]), dtype=float)
# another way of writing that psi = [xi, eta, mu, zeta]
# for i in range(4):
#     psi[i] = (1 + xi * domain[i, 0]) * (1 + eta * domain[i, 1]) * (1 + mu *
↪   domain[i, 2]) * (1 + zeta * domain[i, 3])
psi[0] = xi
psi[1] = eta
psi[2] = mu
psi[3] = 1 - xi - eta - mu  # = zeta
psi = psi.T
print('Shape Functions: {0}\n{1}'.format(psi.shape, psi))

# Shape Functions in Global Coordinates
psi_g = psi @ coors
print('Shape Functions in Global Coordinates: '
      '{0}\n{1}'.format(psi_g.shape, psi_g))

# Shape Functions Derivatives in Natural Coordinates
dpsi = np.zeros((integration_points.shape[0], 4, 3), dtype=float)
dpsi[:,0,0] = 1
dpsi[:,1,1] = 1
dpsi[:,2,2] = 1
dpsi[:,3,:] = -1
dpsi = dpsi.transpose((0, 2, 1))
print('Shape Functions Derivatives: '
      '{0}\n{1}'.format(dpsi.shape, dpsi))

# Jacobian Matrix
jacobi = dpsi @ coors
print('Jacobian Matrix: {0}\n{1}'.format(jacobi.shape, jacobi))

# Jacobian Determinants
d_jacobi = np.linalg.det(jacobi)
print('Determinant of Jacobian: '
      '{0}\n{1}'.format(d_jacobi.shape, d_jacobi))

# Inverse Jacobian
i_jacobi = np.linalg.inv(jacobi)
print('Inverse Jacobian Matrix: '
      '{0}\n{1}'.format(i_jacobi.shape, i_jacobi))
```

```python
    # Shape Function Derivatives in Global Coordinates
    dpsi_g = i_jacobi @ dpsi
    print('Shape Function Derivatives in Global Coordinates: '
          '{0}\n{1}'.format(dpsi_g.shape, dpsi_g))

    # Material Stiffness Matrix
    C = E / ((1.0 + nu) * (1.0 - 2.0 * nu)) *
        np.array([[1.0 - nu, nu, nu, 0.0, 0.0, 0.0],
                  [nu, 1.0 - nu, nu, 0.0, 0.0, 0.0],
                  [nu, nu, 1.0 - nu, 0.0, 0.0, 0.0],
                  [0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0, 0.0],
                  [0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0],
                  [0.0, 0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0]],
                  dtype=float)
    print('Material Stiffness Matrix: {0}\n{1}'.format(C.shape, C))
    du = disp.T @ np.transpose(dpsi_g, axes=[0, 2, 1])

    exx = du[:, 0, 0]
    eyy = du[:, 1, 1]
    ezz = du[:, 2, 2]
    exy = du[:, 0, 1] + du[:, 1, 0]
    eyz = du[:, 1, 2] + du[:, 2, 1]
    exz = du[:, 0, 2] + du[:, 2, 0]
    epsilons = np.array([exx, eyy, ezz, exy, eyz, exz])

    sigmas = (C @ epsilons).T
    epsilons = epsilons.T
    print('Element Strains: {0}\n{1}'.format(epsilons.shape, epsilons))
    print('Element Stresses: {0}\n{1}'.format(stresses.shape, stresses))
```

Chapter 12

Numerics

12.1 Numerical Spaces

The vectors used in numerical methods do not always consist of a finite list of numbers. In the finite element method, for example, the unknown displacement is first considered as a function over the body and only later approximated by a finite set of nodal values. A *space* specifies which objects are admitted as vectors, which operations can be performed on them and how their size or mutual distance is measured.

The spaces introduced below form a useful hierarchy. A **Banach space** is complete with respect to a norm. A **Hilbert space** is a Banach space whose norm is generated by an inner product. A **Sobolev space** is a function space in which the function and a prescribed number of its weak derivatives are integrable. For the exponent $p = 2$, a Sobolev space is also a Hilbert space. A **Krylov space**, on the other hand, is a finite-dimensional subspace generated from a matrix and a starting vector during an iterative calculation.

12.1.1 Hilbert Space

A **Hilbert space** is an inner product space that is complete with respect to the norm induced by its inner product. The definition contains two distinct ideas:

1. The *inner product* $\langle u, v \rangle$ measures both the length of vectors and the angle between them.
2. *Completeness* means that a converging sequence of vectors does not leave the space. More precisely, every Cauchy sequence in the space has a limit that also belongs to the space.

The inner product induces the norm:

$$\|u\| = \sqrt{\langle u, u \rangle}. \tag{12.1}$$

The familiar Euclidean space $\mathbb{R}^n$ with

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^T \mathbf{v} \tag{12.2}$$

is a finite-dimensional Hilbert space. In numerical mechanics the more important examples are often spaces of functions. The space $L^2(\Omega)$ consists of functions whose square is integrable over the domain Ω. Its inner product is:

$$\langle u, v \rangle_{L^2(\Omega)} = \int_\Omega u(\mathbf{x}) v(\mathbf{x}) \, \mathrm{d}\Omega, \tag{12.3}$$

and the corresponding norm is:

$$\|u\|_{L^2(\Omega)} = \sqrt{\int_\Omega |u(\mathbf{x})|^2 \, \mathrm{d}\Omega}. \tag{12.4}$$

Completeness is important because numerical methods construct sequences of approximations. If these approximations converge in the chosen norm, completeness guarantees that their limit is still an admissible member of the space.

Without this property a method could converge towards an object that is not contained in the space in which the problem was formulated.

The inner product also makes the geometric concepts used in finite-dimensional linear algebra available for functions. Two functions are orthogonal if their inner product is zero, projection means finding the nearest member of a subspace, and an orthonormal basis can be constructed in the same sense as for vectors in $\mathbb{R}^n$. Modal expansion, Fourier series and the Galerkin method all use this Hilbert-space geometry.

12.1.2 Sobolev Space

Classical derivatives require a function to be smooth at every point. Finite element functions are usually only piecewise smooth: their value may be continuous between elements while their derivative changes abruptly at an element boundary. **Sobolev spaces** provide a setting in which such functions can still be differentiated in a weaker, integral sense.

The Sobolev space $W^{k,p}(\Omega)$ contains functions for which the function itself and all weak derivatives up to order k belong to $L^p(\Omega)$:

$$\boxed{W^{k,p}(\Omega) = \{u \in L^p(\Omega)\ ;\ D^\alpha u \in L^p(\Omega) \quad \text{for all } |\alpha| \leq k\}\,. \qquad (12.5)}$$

Here α is a multi-index and $D^\alpha u$ denotes the corresponding weak derivative. A weak derivative is defined through integration by parts. A function v is the weak derivative of u in the x_i direction if:

$$\int_\Omega u\,\frac{\partial \varphi}{\partial x_i}\,\mathrm{d}\Omega = -\int_\Omega v\,\varphi\,\mathrm{d}\Omega \tag{12.6}$$

for every sufficiently smooth test function φ that vanishes at the boundary of its support. The derivative is therefore identified by how the function acts inside an integral, not by requiring a pointwise derivative to exist everywhere.

For $1 \le p < \infty$, a commonly used norm on $W^{k,p}(\Omega)$ is:

$$\|u\|_{W^{k,p}(\Omega)} = \left(\sum_{|\alpha| \le k} \|D^\alpha u\|_{L^p(\Omega)}^p \right)^{1/p}. \tag{12.7}$$

For $p = 2$ the notation

$$H^k(\Omega) = W^{k,2}(\Omega) \tag{12.8}$$

is commonly used. The space $H^k(\Omega)$ is a Hilbert space with the inner product:

$$\langle u, v \rangle_{H^k(\Omega)} = \sum_{|\alpha| \le k} \int_\Omega D^\alpha u \, D^\alpha v \, \mathrm{d}\Omega. \tag{12.9}$$

For the standard displacement formulation of linear elasticity the strain contains first derivatives of displacement. Finite strain energy therefore requires the displacement field to have square-integrable first weak derivatives. The natural solution space is consequently based on $H^1(\Omega)$. This does not mean that the displacement derivative must be continuous between elements. It means that the displacement and its first weak derivatives must be square-integrable. Conventional displacement finite elements satisfy this requirement by maintaining displacement continuity while permitting jumps in strain and stress across element boundaries.

Boundary conditions further restrict the space. If a displacement is prescribed on the part Γ_u of the boundary, the admissible trial functions belong to an affine subset of $H^1(\Omega)$, whereas the corresponding test functions vanish on Γ_u. This distinction between trial and test spaces is the functional-space counterpart of inserting prescribed nodal displacements into a finite-dimensional system of equations.

12.1.3 Banach Space

A **Banach space** is a normed vector space that is complete with respect to its norm. A norm assigns a non-negative size $\|u\|$ to every vector and satisfies:

$$\begin{aligned} \|u\| \geq & \quad 0, \qquad \|u\| = 0 \Leftrightarrow u = 0, \\ \|\alpha u\| = & \quad |\alpha|\|u\|, \\ \|u+v\| \leq & \quad \|u\| + \|v\|. \end{aligned} \tag{12.10}$$

Completeness has the same meaning as for Hilbert spaces: every Cauchy sequence has a limit in the space. The difference is that a Banach space needs only a norm; it does not need an inner product. Consequently, notions depending on angles and orthogonality are not automatically available.

Every Hilbert space is a Banach space because its inner product induces a norm through Equation (12.1). The converse is not generally true. For example, $L^p(\Omega)$ is a Banach space for $1 \leq p \leq \infty$, but it is a Hilbert space only for $p = 2$. The L^1 and L^∞ norms measure the size of a function but cannot be generated by an inner product.

This distinction matters in numerical methods. Problems posed in Hilbert spaces can use orthogonal projection, energy minimisation and conjugacy directly. Problems posed in general Banach spaces may still be well defined and complete, but their analysis and numerical algorithms cannot rely on Euclidean geometry without introducing additional structure.

12.1.4 Krylov Space

A **Krylov space**, more commonly called a **Krylov subspace**, is a finite-dimensional space generated by repeatedly applying a matrix $\mathbf{A}$ to a starting vector $\mathbf{b}$. The Krylov subspace of order m is:

$$\boxed{\mathcal{K}_m(\mathbf{A}, \mathbf{b}) = \operatorname{span}\left\{\mathbf{b}, \mathbf{A}\mathbf{b}, \mathbf{A}^2\mathbf{b}, \ldots, \mathbf{A}^{m-1}\mathbf{b}\right\}.} \tag{12.11}$$

The vectors in Equation (12.11) are not usually formed by explicitly computing powers of $\mathbf{A}$. Instead, one matrix-vector product is performed at each iteration:

$$\mathbf{v}_{j+1} = \mathbf{A}\mathbf{v}_j. \tag{12.12}$$

The newly generated vector is then normally orthogonalised against the existing basis. The Arnoldi algorithm performs this operation for a general matrix, while the Lanczos algorithm exploits symmetry to obtain a shorter recurrence.

For the linear system

$$\mathbf{A}\mathbf{x} = \mathbf{b}, \tag{12.13}$$

an iterative Krylov method searches for an approximation of the form:

$$\mathbf{x}_m = \mathbf{x}_0 + \mathbf{z}_m, \qquad \mathbf{z}_m \in \mathcal{K}_m(\mathbf{A}, \mathbf{r}_0), \tag{12.14}$$

where $\mathbf{r}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0$ is the initial residual. Different methods choose $\mathbf{z}_m$ by imposing different conditions on the new residual. The conjugate-gradient method, GMRES and MINRES therefore use related approximation spaces but different projection conditions.

Krylov spaces are also used for eigenvalue calculations. The large matrix is projected onto the much smaller subspace, and eigenvalues of the projected matrix approximate selected eigenvalues of the original problem. This is particularly useful in finite element analysis because a sparse matrix-vector product can be computed without forming dense matrix powers or a complete factorisation.

The dimension of $\mathcal{K}_m$ can be smaller than m if a newly generated vector is linearly dependent on the previous vectors. In exact arithmetic this means that the subspace has become invariant under $\mathbf{A}$. In finite precision, loss of orthogonality and nearly dependent vectors must be handled carefully, which connects Krylov methods directly to the orthogonalisation algorithms in the following section.

12.2 Orthogonalisation

12.2.1 Gram-Schmidt process

In linear algebra the **Gram-Schmidt algorithm** is a way of finding two or more vectors perpendicular to each other. It is a method of constructing an orhtonormal basis form a set of vectors in an inner product space, most commonly the Euclidean space $\mathbb{R}^n$ equipped with a standard inner product.

The Gram-Schmidt process takes a dinite, *linearly independent* set of vectors $\mathbf{S} = \{\mathbf{v}_1, \ldots, \mathbf{v}_k\}$ for $k \leq n$ and generates an **orthogonal** set $\mathbf{S}' = \{\mathbf{u}_1, \ldots, \mathbf{u}_n\}$ that spans the same k-dimensional subspace of $\mathbb{R}^n$ as $\mathbf{S}$.

The application of the Gram-Schmidt process to the column vectors of a full column rank matrix yields the **QR decomposition**.

The vector projection of a vector $\mathbf{v}$ onto $\mathbf{u}$ is defined as:

$$proj_{\mathbf{u}}(\mathbf{v}) = \frac{\langle \mathbf{v}, \mathbf{u} \rangle}{\langle \mathbf{u}, \mathbf{u} \rangle} \mathbf{u} \tag{12.15}$$

where $\langle \mathbf{v}, \mathbf{u} \rangle$ denotes the **dot product** of the vectors $\mathbf{u}$ and $\mathbf{v}$. This means that the $proj_{\mathbf{u}}(\mathbf{v})$ is the orthogonal projection of $\mathbf{v}$ onto the line spanned by $\mathbf{u}$. It can be also thought of as the part of the vector $\mathbf{v}$ that is common with vector $\mathbf{u}$. If the two vectors are perpendicular, then the dot product is equal 0.

By subtracting the $proj_{\mathbf{u}}(\mathbf{v})$ from vector $\mathbf{v}$ we obtain a new vector that is perpendicular to the vector $\mathbf{u}$.

Given k nonzero linearly independent vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k$ the **Gram-Schmidt** process defines the vectors $\mathbf{u}_1, \ldots, \mathbf{u}_k$ as follows:

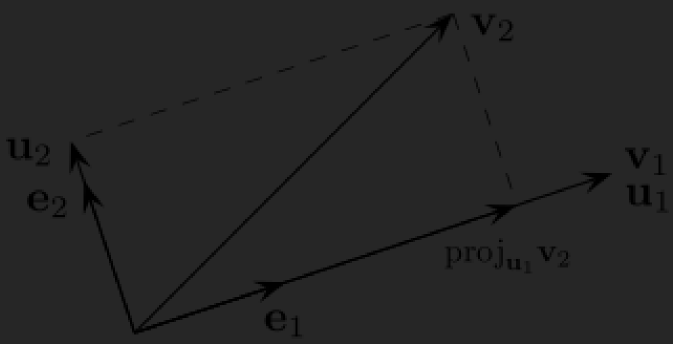


Figure 12.1: first two steps of the Gram-Schmidt process

$$
\begin{aligned}
\mathbf{u}_1 &= \mathbf{v}_1 & \mathbf{e}_1 &= \frac{\mathbf{u}_1}{\|\mathbf{u}_1\|} \\
\mathbf{u}_2 &= \mathbf{v}_2 - proj_{\mathbf{u}_1}(\mathbf{v}_2) & \mathbf{e}_2 &= \frac{\mathbf{u}_2}{\|\mathbf{u}_2\|} \\
\mathbf{u}_3 &= \mathbf{v}_3 - proj_{\mathbf{u}_1}(\mathbf{v}_3) - proj_{\mathbf{u}_2}(\mathbf{v}_3) & \mathbf{e}_3 &= \frac{\mathbf{u}_3}{\|\mathbf{u}_3\|} \\
&\vdots & &\vdots \\
\mathbf{u}_i &= \mathbf{v}_i - \sum_{j=1}^{i-1} proj_{\mathbf{u}_j}(\mathbf{v}_i) & \mathbf{e}_i &= \frac{\mathbf{u}_i}{\|\mathbf{u}_i\|}
\end{aligned}
\tag{12.16}
$$

The sequence $\mathbf{u}_1, \ldots, \mathbf{u}_k$ is the required system of orthogonal vectors, and the normalised vectors $\mathbf{e}_1, \ldots, \mathbf{e}_k$ form an **orthonormal set**. The calculation of $\mathbf{u}_1, \ldots, \mathbf{u}_k$ is known as **Gram-Schmidt orthogonalisation** and the calculation of sequence $\mathbf{e}_1, \ldots, \mathbf{e}_k$ is called **Gram-Schmidt orthonormalisation**.

Python implementation:

```
import numpy as np

```

```python
def GramSchmidtRows(A: np.ndarray, normalise: bool = False):
    """Perform the Gram-Schmidt algorithm on the rows of matrix A so that all
    rows are orthogonalised to the preceding rows"""

    # copy the matrix
    G = A.copy()

    # row to orthogonalise
    for j in range(1, G.shape[0]):
        # the predecessors
        for i in range(j):
            # unit vector representation of preceding row
            unit_row = G[i,:] / np.linalg.norm(G[i,:])
            # projection of current row onto the preceding rows
            G[j,:] -= unit_row * np.dot(G[j,:], unit_row)
        if normalise:
            G[j,:] /= np.linalg.norm(G[j,:])

    return G

def GramSchmidtCols(A: np.ndarray, normalise : bool = False):
    """Perform the Gram-Schmidt algorithm on the columns of matrix A so that all
    columns are orthogonalised to the preceding rows"""

    # copy the matrix
    G = A.copy()

    # column to orthogonalise
    for j in range(1, G.shape[1]):
        # the predecessors
        for i in range(j):
            # unit vector representation of preceding column
            unit_col = G[:,i] / np.linalg.norm(G[:,i])
            # projection of current column onto the preceding columns
            G[:,j] -= unit_col * np.dot(G[:,j], unit_col)
        if normalise:
            G[j,:] /= np.linalg.norm(G[j,:])

    return G
```

12.2.2 Modified Gram-Schmidt process

The **Gram-Schmidt orthogonalisation** is prone to **rounding errors** and is therefore considered **numerically unstable**. For the process as described above it is considered particularly bad.

The **Gram-Schmidt process** can be stabilised by a small modification, sometimes referred to as **modified Gram-Schmidt process** or MGS. This approach gives the same result as the original formula in exact arithmetic and introdices smaller errors in finite-precision arithmethic.

Instead of computing the $\mathbf{u}_i$ as:

$$\mathbf{u}_i = \mathbf{v}_i - \sum_{j=1}^{i-1} proj_{\mathbf{u}_j}(\mathbf{v}_i) \tag{12.17}$$

it is computed as:

$$\begin{aligned}
\mathbf{u}_i^{(1)} &= \mathbf{v}_i - proj_{\mathbf{u}_1}(\mathbf{v}_i) \\
\mathbf{u}_i^{(2)} &= \mathbf{u}_i^{(1)} - proj_{\mathbf{u}_2}(\mathbf{u}_i^{(1)}) \\
&\vdots \\
\mathbf{u}_i^{(i-1)} &= \mathbf{u}_i^{(i-2)} - proj_{\mathbf{u}_{i-1}}(\mathbf{u}_i^{(i-2)}) \\
\mathbf{e}_i &= \frac{\mathbf{u}_i^{(i-1)}}{\|\mathbf{u}_i^{(i-1)}\|}
\end{aligned} \tag{12.18}$$

Another description would be that the orthogonalisation is performed as a multistep process where in first step all vectors but first are made orthogonal to the first vectors, in the second step all vectors but the first two are made orthogonal to the second vector (already orthogonal to 1st one), and so on untill all are processed.

Python implementation:

```python
import numpy as np

def ModifiedGramSchmidtRows(A: np.ndarray, normalise: bool = False):
    """Perform the Modified Gram-Schmidt algorithm on the rows of matrix A
    so that all rows are orthogonalised to the preceding rows"""

    # copy the matrix
    G = A.copy()

    # vector to orthogonalise against
    for i in range(G.shape[0]-1):
        # simplify dot product
        unit_row = G[i,:] / np.linalg.norm(G[i,:])
        if normalise:
            G[i,:] = unit_row
        # all later vectors
        for j in range(i+1, G.shape[0]):
            # subtract the projection
            G[j,:] -= unit_row * np.dot(G[j,:], unit_row)

    return G

def ModifiedGramSchmidtCols(A: np.ndarray, normalise: bool = False):
    """Perform the Modified Gram-Schmidt algorithm on the columns of matrix A
    so that all columns are orthogonalised to the preceding rows"""

    # copy the matrix
    G = A.copy()

    # vector to orthogonalise against
    for i in range(G.shape[1]-1):
        # simplify dot product
        unit_col = G[:,i] / np.linalg.norm(G[:,i])
        if normalise:
            G[:,i] = unit_col
        # all later vectors
        for j in range(i+1, G.shape[1]):
            # subtract the projection
            G[:,j] -= unit_col * np.dot(G[:,j], unit_col)

```

```python
    return G
```

12.3 Factorisation

12.3.1 Cholesky Factorisation

Cholesky Factorisation is a matrix operation such that a matrix $\mathbf{A}$ is decomposed to a multiplication of a lower triangular martrix $\mathbf{L}$ and its conjugate transpose $\mathbf{L}^T$.

The **Cholesky Factorisation** is applicable to a **Hermitian**, **positive definite** matrices.

$$\mathbf{A} = \mathbf{L}\mathbf{L}^* \tag{12.19}$$

Every Hermitian positive definite matrix (and thus every real-valued symmetric positive definite matrix) has a unique Cholesky factorisation.

If $\mathbf{A}$ can be written as $\mathbf{L}\mathbf{L}^*$ for some invertible $\mathbf{L}$, lower triangular or otherwise, then $\mathbf{A}$ is Hermitian and positive definite.

When $\mathbf{A}$ is a real matrix (hence symmetric and positive definite), the factorisation may be written:

$$\mathbf{A} = \mathbf{L}\mathbf{L}^T \tag{12.20}$$

A Python example of *Cholesky-Banachiewicz* factorisation algorithm:

```python
import numpy as np

def Cholesky_Banachiewicz(A: np.ndarray) -> np.ndarray:
    """Perform the Choleski factorisation of matrix A such that:
```

```
    A = L @ L.T

    where:
        L is lower triangular matrix

    Args:
        A (np.ndarray): a real, symmetric, positive-definite matrix

    Returns:
        (np.ndarray): the result of the decomposition stored as a lower triangular
        matrix.
    """
    L = np.zeros(A.shape, dtype=A.dtype)
    for i in range(A.shape[0]):
        for j in range(i+1):
            sum = 0
            for k in range(j):
                sum += L[i,k] * L[j,k]

            if i == j:
                L[i,j] = np.sqrt(A[i,i] - sum)
            else:
                L[i,j] = 1. / L[j,j] * (A[i,j] - sum)

    return L
```

And the solution of matrix equation:

$$
\begin{aligned}
\mathbf{A}\mathbf{x} &= \mathbf{b} \\
\mathbf{A} &= \mathbf{L}\mathbf{L}^T \\
\mathbf{L}\mathbf{L}^T\mathbf{x} &= \mathbf{b} \quad , \textit{substitute} \quad \mathbf{y} = \mathbf{L}^T\mathbf{x} \\
\mathbf{L}\mathbf{y} &= \mathbf{b} \quad , \textit{ solve using forward substitution} \\
\mathbf{L}^T\mathbf{x} &= \mathbf{y} \quad , \textit{ solve using backward substitution}
\end{aligned}
\tag{12.21}
$$

the $\mathbf{b}$ can be either a vector or a matrix.

Python example:

```python
import numpy as np

def Solve_Triangular(A: np.ndarray, b: np.ndarray, kind="lower") -> np.ndarray:
    """Solve Ax = b using either forward or backward substitution. For lower
    triangular matrix the forward substitution is used, for upper triangular
    matrix the backward substitution is used.
    """
    b = b.copy()
    flatten = False
    if len(b.shape) == 1:
        b = b.reshape(-1,1)
        flatten = True

    m = A.shape[0]
    n = b.shape[1]
    x = np.zeros(b.shape, dtype=b.dtype)

    if kind == "lower":
        for i in range(m):                  # rows of A
            x[i,:] = b[i,:]                 # perform operations on whole row of b
            for j in range(i):              # columns of A from start to diagonal
                x[i,:] -= A[i,j] * x[j,:] # perform operations on whole row of b
            x[i,:] /= A[i,i]                # divide by the element on the diagonal

    elif kind == "upper":
        for i in reversed(range(m)):        # rows of A from end
            x[i,:] = b[i,:]                 # perform operations on whole row of b
            for j in range(i+1, m):         # columns of A from diagonal up to end
                x[i,:] -= A[i,j] * x[j,:] # perform operations on whole row of b
            x[i,:] /= A[i,i]                # divide by the element on the diagonal

    else:
        raise NotImplementedError(f"Substitution for other than 'lower' or
        ↪ 'upper' triangular not implemented: '{kind:s}'.")

    if flatten:
        return x.flatten()
```

```
    else:
        return x


def Cholesky_Solve(L: np.ndarray, b: np.ndarray) -> np.ndarray:
    """Solve the matrix equation using results of Cholesky decomposition

        Ax = b
        A = L @ L.T
        L @ L.T @ x = b
        L @ z = b
        L.T @ x = z

    Args:
        L (np.ndarray): a lower triangular matrix from Cholesky decomposition
        b (np.ndarray): right hand side of the equation

    Returns:
        (np.ndarray): the solution on the left hand side
    """
    z = Solve_Triangular(L,   b, "lower")
    x = Solve_Triangular(L.T, z, "upper")
    return x

```

12.3.2 LU Factorisation

The **LU factorisation** writes a square matrix as a product of a lower triangular matrix and an upper triangular matrix:

$$\mathbf{A} = \mathbf{L}\mathbf{U} \tag{12.22}$$

The factorisation is obtained by applying Gaussian elimination to $\mathbf{A}$. The multipliers used to remove the entries below a pivot are stored in $\mathbf{L}$, while the resulting upper triangular matrix is $\mathbf{U}$. Using a unit diagonal in $\mathbf{L}$ removes the otherwise arbitrary scaling between the two factors and gives the **Doolittle form**:

$$L_{ii} = 1 \tag{12.23}$$

Once the factors are available, the equation $\mathbf{Ax} = \mathbf{b}$ is solved in two triangular steps:

$$\begin{aligned} \mathbf{LUx} &= \mathbf{b} \\ \mathbf{Ly} &= \mathbf{b}, \quad \textit{solve by forward substitution} \\ \mathbf{Ux} &= \mathbf{y}, \quad \textit{solve by backward substitution} \end{aligned} \tag{12.24}$$

The factorisation is more expensive than one triangular solution, but it may be reused for many right hand sides. This is important in finite element analysis, where the same tangent matrix may be used for several load vectors or for several operations within one iteration.

LU Factorisation without pivoting

For the Doolittle form, comparison of the entries in $\mathbf{A} = \mathbf{LU}$ gives:

$$\begin{aligned} U_{kj} &= A_{kj} - \sum_{s=1}^{k-1} L_{ks} U_{sj}, \qquad j = k, \ldots, n \\ L_{ik} &= \frac{1}{U_{kk}} \left(A_{ik} - \sum_{s=1}^{k-1} L_{is} U_{sk} \right), \qquad i = k+1, \ldots, n \end{aligned} \tag{12.25}$$

The first equation forms row k of $\mathbf{U}$. The second equation then forms column k of $\mathbf{L}$. The calculation requires every pivot U_{kk} to be nonzero. A nonsingular matrix can nevertheless produce a zero pivot. For example:

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \tag{12.26}$$

is nonsingular, but elimination cannot start without exchanging rows. Even when a pivot is merely very small, division by it can amplify rounding errors. Factorisation without pivoting is therefore used only when the matrix structure ensures safe pivots, or when this has been verified separately.

Python implementation:

```python
import numpy as np


def LU_No_Pivoting(A: np.ndarray,
                   tolerance: float = 1.0e-14):
    """Factorise a square matrix A = L @ U without pivoting.

    L has a unit diagonal. No NumPy linear algebra factorisation or
    solution routine is used.
    """
    A = np.asarray(A)
    n = A.shape[0]

    if A.shape != (n, n):
        raise ValueError("A must be square")

    L = np.eye(n, dtype=A.dtype)
    U = np.zeros_like(A)

    for k in range(n):
        # Form row k of U.
        for j in range(k, n):
            total = 0.0
            for s in range(k):
                total += L[k, s] * U[s, j]
            U[k, j] = A[k, j] - total

        if abs(U[k, k]) <= tolerance:
            raise ZeroDivisionError(
                "zero or numerically small pivot; pivoting is required"
            )

```

```python
        # Form column k of L.
        for i in range(k + 1, n):
            total = 0.0
            for s in range(k):
                total += L[i, s] * U[s, k]
            L[i, k] = (A[i, k] - total) / U[k, k]

    return L, U


def LU_Solve(L: np.ndarray,
             U: np.ndarray,
             b: np.ndarray) -> np.ndarray:
    """Solve L @ U @ x = b using the triangular solver defined above."""
    y = Solve_Triangular(L, b, kind="lower")
    x = Solve_Triangular(U, y, kind="upper")
    return x
```

LU Factorisation with row pivoting

Partial pivoting exchanges rows before each elimination step. In column k, the row containing the largest available absolute entry is selected:

$$p = \underset{i=k,\dots,n}{\operatorname{argmax}} |U_{ik}| \tag{12.27}$$

The corresponding rows are exchanged and the factorisation becomes:

$$\mathbf{PA} = \mathbf{LU} \tag{12.28}$$

where $\mathbf{P}$ is a permutation matrix. A permutation matrix only changes the order of equations. It is orthogonal, so that:

$$\mathbf{P}^{-1} = \mathbf{P}^{T} \tag{12.29}$$

The row exchange avoids division by zero whenever a usable pivot exists in the current column. Selecting the largest candidate also limits the elimination multipliers:

$$|L_{ik}| = |\frac{U_{ik}}{U_{kk}}| \leq 1 \tag{12.30}$$

Partial pivoting is the usual general-purpose form of LU factorisation. To solve $\mathbf{Ax} = \mathbf{b}$, the right hand side must be permuted in the same way:

$$\begin{aligned} \mathbf{PAx} &= \mathbf{Pb} \\ \mathbf{LUx} &= \mathbf{Pb} \end{aligned} \tag{12.31}$$

Python implementation:

```python
import numpy as np


def LU_Partial_Pivoting(A: np.ndarray,
                        tolerance: float = 1.0e-14):
    """Factorise P @ A = L @ U using partial row pivoting."""
    A = np.array(A, copy=True)
    n = A.shape[0]

    if A.shape != (n, n):
        raise ValueError("A must be square")

    U = A.copy()
    L = np.eye(n, dtype=A.dtype)
    P = np.eye(n, dtype=A.dtype)

    for k in range(n - 1):
        # Find the largest available entry in column k.
        pivot = k
        for i in range(k + 1, n):
```

```
            if abs(U[i, k]) > abs(U[pivot, k]):
                pivot = i

        if abs(U[pivot, k]) <= tolerance:
            raise ZeroDivisionError("matrix is singular to working precision")

        if pivot != k:
            U[[k, pivot], :] = U[[pivot, k], :]
            P[[k, pivot], :] = P[[pivot, k], :]

            # The already computed part of L belongs to the exchanged rows.
            if k > 0:
                L[[k, pivot], :k] = L[[pivot, k], :k]

        # Gaussian elimination below the pivot.
        for i in range(k + 1, n):
            L[i, k] = U[i, k] / U[k, k]
            U[i, k:] -= L[i, k] * U[k, k:]
            U[i, k] = 0.0

    if abs(U[-1, -1]) <= tolerance:
        raise ZeroDivisionError("matrix is singular to working precision")

    return P, L, U


def LU_Partial_Solve(P: np.ndarray,
                     L: np.ndarray,
                     U: np.ndarray,
                     b: np.ndarray) -> np.ndarray:
    """Solve A @ x = b from P @ A = L @ U."""
    y = Solve_Triangular(L, P @ b, kind="lower")
    x = Solve_Triangular(U, y, kind="upper")
    return x
```

LU Factorisation with complete pivoting

Complete pivoting searches the whole remaining submatrix rather than only the current column:

$$(p, q) = \operatorname*{argmax}_{i,j=k,\dots,n} |U_{ij}| \tag{12.32}$$

Both rows and columns are exchanged. The resulting factorisation is:

$$\mathbf{PAQ} = \mathbf{LU} \tag{12.33}$$

The row permutation $\mathbf{P}$ reorders equations, while the column permutation $\mathbf{Q}$ reorders unknowns. Complete pivoting usually produces smaller element growth than partial pivoting, but searching the complete trailing submatrix is more expensive and column permutations complicate bookkeeping. It is therefore used when additional robustness is more important than the extra work.

For a linear system:

$$\begin{aligned} \mathbf{PAQ} &= \mathbf{LU} \\ \mathbf{LUQ}^T\mathbf{x} &= \mathbf{Pb} \end{aligned} \tag{12.34}$$

After the triangular solves, the original ordering is restored by $\mathbf{x} = \mathbf{Qy}$.

Python implementation:

```python
import numpy as np


def LU_Complete_Pivoting(A: np.ndarray,
                         tolerance: float = 1.0e-14):
    """Factorise P @ A @ Q = L @ U using complete pivoting."""
```

```
    A = np.array(A, copy=True)
    n = A.shape[0]

    if A.shape != (n, n):
        raise ValueError("A must be square")

    U = A.copy()
    L = np.eye(n, dtype=A.dtype)
    P = np.eye(n, dtype=A.dtype)
    Q = np.eye(n, dtype=A.dtype)

    for k in range(n - 1):
        pivot_row = k
        pivot_col = k
        pivot_value = abs(U[k, k])

        for i in range(k, n):
            for j in range(k, n):
                if abs(U[i, j]) > pivot_value:
                    pivot_row = i
                    pivot_col = j
                    pivot_value = abs(U[i, j])

        if pivot_value <= tolerance:
            raise ZeroDivisionError("matrix is singular to working precision")

        if pivot_row != k:
            U[[k, pivot_row], :] = U[[pivot_row, k], :]
            P[[k, pivot_row], :] = P[[pivot_row, k], :]
            if k > 0:
                L[[k, pivot_row], :k] = L[[pivot_row, k], :k]

        if pivot_col != k:
            U[:, [k, pivot_col]] = U[:, [pivot_col, k]]
            Q[:, [k, pivot_col]] = Q[:, [pivot_col, k]]

        for i in range(k + 1, n):
            L[i, k] = U[i, k] / U[k, k]
            U[i, k:] -= L[i, k] * U[k, k:]
            U[i, k] = 0.0
```

```python
    if abs(U[-1, -1]) <= tolerance:
        raise ZeroDivisionError("matrix is singular to working precision")

    return P, L, U, Q


def LU_Complete_Solve(P: np.ndarray,
                      L: np.ndarray,
                      U: np.ndarray,
                      Q: np.ndarray,
                      b: np.ndarray) -> np.ndarray:
    """Solve A @ x = b from P @ A @ Q = L @ U."""
    z = Solve_Triangular(L, P @ b, kind="lower")
    y = Solve_Triangular(U, z, kind="upper")
    x = Q @ y
    return x
```

12.3.3 QR Factorisation

The **QR factorisation** writes an $m \times n$ matrix with $m \geq n$ as:

$$\mathbf{A} = \mathbf{QR} \tag{12.35}$$

where the columns of $\mathbf{Q}$ are orthonormal and $\mathbf{R}$ is upper triangular:

$$\mathbf{Q}^*\mathbf{Q} = \mathbf{I} \tag{12.36}$$

In the **thin QR factorisation**, $\mathbf{Q}$ has size $m \times n$ and $\mathbf{R}$ has size $n \times n$. In the full factorisation, $\mathbf{Q}$ is $m \times m$ and $\mathbf{R}$ is $m \times n$.

Because multiplication by an orthogonal matrix does not change the Euclidean norm, QR factorisation transforms a least-squares problem without squaring its condition number:

$$\begin{aligned}\underset{\mathbf{x}}{\text{minimise}}\|\mathbf{A}\mathbf{x}-\mathbf{b}\|_2 &= \underset{\mathbf{x}}{\text{minimise}}\|\mathbf{Q}^*\mathbf{A}\mathbf{x}-\mathbf{Q}^*\mathbf{b}\|_2 \\ &= \underset{\mathbf{x}}{\text{minimise}}\|\mathbf{R}\mathbf{x}-\mathbf{Q}^*\mathbf{b}\|_2\end{aligned} \tag{12.37}$$

This is numerically preferable to forming the normal equations $\mathbf{A}^*\mathbf{A}\mathbf{x} = \mathbf{A}^*\mathbf{b}$.

QR Factorisation using Gram-Schmidt algorithm

Gram-Schmidt orthogonalisation of the columns of $\mathbf{A}$ gives $\mathbf{Q}$. Let $\mathbf{a}_j$ be column j of $\mathbf{A}$. Then:

$$\begin{aligned}R_{ij} &= \mathbf{q}_i^*\mathbf{a}_j, \qquad i < j \\ \mathbf{v}_j &= \mathbf{a}_j - \sum_{i=1}^{j-1} R_{ij}\mathbf{q}_i \\ R_{jj} &= \|\mathbf{v}_j\|_2 \\ \mathbf{q}_j &= \frac{\mathbf{v}_j}{R_{jj}}\end{aligned} \tag{12.38}$$

The coefficients removed from column j form column j of $\mathbf{R}$. If $R_{jj} = 0$, the new column is linearly dependent on the previous columns and a full-rank thin QR factorisation does not exist.

The following code uses the modified ordering of the Gram-Schmidt operations, which is less sensitive to rounding errors than evaluating all projections from the original column at once.

```python
import numpy as np


def Vector_Norm(x: np.ndarray) -> float:
    """Euclidean norm without using numpy.linalg."""
```

```python
    value = np.dot(np.conjugate(x), x)
    return float(np.sqrt(np.real(value)))


def QR_Gram_Schmidt(A: np.ndarray,
                    tolerance: float = 1.0e-14):
    """Compute the thin QR factorisation A = Q @ R."""
    A = np.asarray(A)
    m, n = A.shape

    if m < n:
        raise ValueError("the example assumes m >= n")

    Q = np.zeros((m, n), dtype=A.dtype)
    R = np.zeros((n, n), dtype=A.dtype)

    for j in range(n):
        v = A[:, j].copy()

        for i in range(j):
            R[i, j] = np.dot(np.conjugate(Q[:, i]), v)
            v -= R[i, j] * Q[:, i]

        R[j, j] = Vector_Norm(v)
        if R[j, j] <= tolerance:
            raise ValueError("columns of A are linearly dependent")

        Q[:, j] = v / R[j, j]

    return Q, R
```

QR Factorisation using Givens rotations

A **Givens rotation** acts only in a plane spanned by two coordinate axes. For real numbers a and b, choose:

$$c = \frac{a}{r}, \qquad s = \frac{b}{r}, \qquad r = \sqrt{a^2 + b^2} \tag{12.39}$$

and define:

$$\mathbf{G} = \begin{bmatrix} c & s \\ -s & c \end{bmatrix} \tag{12.40}$$

Then:

$$\mathbf{G} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} r \\ 0 \end{bmatrix} \tag{12.41}$$

Embedding this 2×2 block into an identity matrix allows one entry below the diagonal of $\mathbf{A}$ to be removed without changing the other rows. A sequence of rotations gives:

$$\mathbf{G}_p \cdots \mathbf{G}_2 \mathbf{G}_1 \mathbf{A} = \mathbf{R} \tag{12.42}$$

and therefore:

$$\mathbf{A} = \mathbf{QR}, \qquad \mathbf{Q} = \mathbf{G}_1^T \mathbf{G}_2^T \cdots \mathbf{G}_p^T \tag{12.43}$$

Givens rotations are especially useful when the matrix is sparse or when only a small number of new entries must be eliminated, because each rotation modifies only two rows.

Python implementation:

```
import numpy as np


def Givens_Parameters(a: float, b: float):
    """Return c and s so [[c,s],[-s,c]] @ [a,b] = [r,0]."""
    r = float(np.hypot(abs(a), abs(b)))
    if r == 0.0:
        return 1.0, 0.0
```

```
    return a / r, b / r


def QR_Givens(A: np.ndarray,
              tolerance: float = 1.0e-14):
    """Compute A = Q @ R using real Givens rotations."""
    R = np.array(A, dtype=float, copy=True)
    m, n = R.shape
    Q = np.eye(m)

    for j in range(min(m, n)):
        # Work upwards so each rotation removes one entry in column j.
        for i in reversed(range(j + 1, m)):
            if abs(R[i, j]) <= tolerance:
                continue

            c, s = Givens_Parameters(R[i - 1, j], R[i, j])

            row_1 = R[i - 1, j:].copy()
            row_2 = R[i, j:].copy()
            R[i - 1, j:] = c * row_1 + s * row_2
            R[i, j:] = -s * row_1 + c * row_2

            # Q <- Q @ G.T, applied only to two columns.
            col_1 = Q[:, i - 1].copy()
            col_2 = Q[:, i].copy()
            Q[:, i - 1] = c * col_1 + s * col_2
            Q[:, i] = -s * col_1 + c * col_2

    R[abs(R) < tolerance] = 0.0
    return Q, R
```

QR Factorisation using Householder reflections

A **Householder reflection** maps a vector onto a coordinate axis in one operation. For a unit vector $\mathbf{v}$:

$$\mathbf{H} = \mathbf{I} - 2\mathbf{v}\mathbf{v}^* \tag{12.44}$$

The matrix is Hermitian and unitary:

$$\mathbf{H}^* = \mathbf{H}, \qquad \mathbf{H}^*\mathbf{H} = \mathbf{I} \tag{12.45}$$

For a column segment $\mathbf{x}$, the vector $\mathbf{v}$ is selected so that:

$$\mathbf{H}\mathbf{x} = -\operatorname{sign}(x_1)\|\mathbf{x}\|_2\mathbf{e}_1 \tag{12.46}$$

The sign is chosen to avoid subtracting nearly equal numbers while constructing $\mathbf{v}$. Each reflection removes all entries below one diagonal position. A matrix with n columns therefore needs at most n reflections. Compared with Gram-Schmidt, Householder QR usually gives better numerical orthogonality and is the standard dense-matrix method.

Python implementation:

```python
import numpy as np


def Householder_Vector(x: np.ndarray,
                       tolerance: float = 1.0e-14):
    """Return a unit vector v defining H = I - 2 v v*."""
    x = np.asarray(x)
    norm_x = Vector_Norm(x)

    if norm_x <= tolerance:
        return np.zeros_like(x), False

    phase = 1.0 if x[0] == 0.0 else x[0] / abs(x[0])
    v = x.copy()
    v[0] += phase * norm_x

    norm_v = Vector_Norm(v)
    if norm_v <= tolerance:
        return np.zeros_like(x), False

```

```
    return v / norm_v, True


def QR_Householder(A: np.ndarray,
                   tolerance: float = 1.0e-14):
    """Compute the full factorisation A = Q @ R."""
    R = np.array(A, copy=True)
    m, n = R.shape
    Q = np.eye(m, dtype=R.dtype)

    for k in range(min(m, n)):
        v, active = Householder_Vector(R[k:, k], tolerance)
        if not active:
            continue

        # R <- H @ R, applied only to the active trailing block.
        projection = np.dot(np.conjugate(v), R[k:, k:])
        R[k:, k:] -= 2.0 * np.outer(v, projection)

        # Q <- Q @ H. H is Hermitian, hence H* = H.
        projection = np.dot(Q[:, k:], v)
        Q[:, k:] -= 2.0 * np.outer(projection, np.conjugate(v))

    R[abs(R) < tolerance] = 0.0
    return Q, R
```

QR Factorisation using Householder reflections with column pivoting

If columns of $\mathbf{A}$ are nearly linearly dependent, a small diagonal value in $\mathbf{R}$ may occur before the independent directions have been processed. **Column-pivoted QR** chooses at every step the remaining column with the largest residual norm. The factorisation is written:

$$\mathbf{AP} = \mathbf{QR} \tag{12.47}$$

where $\mathbf{P}$ is a column permutation matrix. The diagonal entries of $\mathbf{R}$ then tend to decrease in magnitude. This reveals the numerical rank:

$$|R_{11}| \geq |R_{22}| \geq \dots \tag{12.48}$$

The relation is not guaranteed to be exact for every matrix, but the diagonal usually separates well-resolved and nearly dependent directions. This makes pivoted QR useful for rank detection, least-squares problems and selecting independent interface or measurement coordinates.

Python implementation:

```python
import numpy as np


def QR_Householder_Pivoting(A: np.ndarray,
                            tolerance: float = 1.0e-14):
    """Compute A @ P = Q @ R using Householder column pivoting."""
    R = np.array(A, copy=True)
    m, n = R.shape
    Q = np.eye(m, dtype=R.dtype)
    P = np.eye(n, dtype=R.dtype)

    for k in range(min(m, n)):
        # Recompute the residual column norms for clarity.
        pivot = k
        largest_norm_squared = -1.0
        for j in range(k, n):
            value = np.dot(np.conjugate(R[k:, j]), R[k:, j])
            norm_squared = float(np.real(value))
            if norm_squared > largest_norm_squared:
                largest_norm_squared = norm_squared
                pivot = j

        if pivot != k:
            R[:, [k, pivot]] = R[:, [pivot, k]]
            P[:, [k, pivot]] = P[:, [pivot, k]]

        v, active = Householder_Vector(R[k:, k], tolerance)
        if not active:
            continue
```

```python

        projection = np.dot(np.conjugate(v), R[k:, k:])
        R[k:, k:] -= 2.0 * np.outer(v, projection)

        projection = np.dot(Q[:, k:], v)
        Q[:, k:] -= 2.0 * np.outer(projection, np.conjugate(v))

    R[abs(R) < tolerance] = 0.0
    return Q, R, P
```

12.3.4 Conversion to Hessenberg form using Givens rotations

The QR factorisation applies orthogonal transformations from the left and changes $\mathbf{A}$ into an upper triangular matrix. An eigenvalue problem requires a **similarity transformation**, because similarity preserves eigenvalues:

$$\mathbf{H} = \mathbf{Q}^*\mathbf{A}\mathbf{Q} \tag{12.49}$$

An **upper Hessenberg matrix** has zeros below its first subdiagonal:

$$H_{ij} = 0 \qquad \text{for} \quad i > j + 1 \tag{12.50}$$

A Givens rotation is first applied from the left to remove one entry below the first subdiagonal and then from the right with its transpose. The right multiplication is essential: without it the operation would not be a similarity transformation. Repeating this process gives a Hessenberg matrix with the same eigenvalues as $\mathbf{A}$. If $\mathbf{A}$ is symmetric, symmetry is preserved and the Hessenberg matrix is tridiagonal.

Python implementation:

```python
import numpy as np

```

```python

def Hessenberg_Givens(A: np.ndarray,
                      tolerance: float = 1.0e-14):
    """Compute H = Q.T @ A @ Q using real Givens rotations."""
    H = np.array(A, dtype=float, copy=True)
    n = H.shape[0]

    if H.shape != (n, n):
        raise ValueError("A must be square")

    Q = np.eye(n)

    for k in range(n - 2):
        for i in reversed(range(k + 2, n)):
            if abs(H[i, k]) <= tolerance:
                continue

            c, s = Givens_Parameters(H[i - 1, k], H[i, k])

            # H <- G @ H: rotate rows i-1 and i.
            row_1 = H[i - 1, :].copy()
            row_2 = H[i, :].copy()
            H[i - 1, :] = c * row_1 + s * row_2
            H[i, :] = -s * row_1 + c * row_2

            # H <- H @ G.T: rotate the matching columns.
            col_1 = H[:, i - 1].copy()
            col_2 = H[:, i].copy()
            H[:, i - 1] = c * col_1 + s * col_2
            H[:, i] = -s * col_1 + c * col_2

            # Q <- Q @ G.T.
            col_1 = Q[:, i - 1].copy()
            col_2 = Q[:, i].copy()
            Q[:, i - 1] = c * col_1 + s * col_2
            Q[:, i] = -s * col_1 + c * col_2

    H[abs(H) < tolerance] = 0.0
    return Q, H
```

12.3.5 Conversion to Hessenberg form using Householder reflections

A Householder reflection can remove a complete column segment in one operation. At step k, a reflector is constructed from:

$$\mathbf{x} = \mathbf{A}_{k+1:n,k} \tag{12.51}$$

and embedded into the identity matrix:

$$\widehat{\mathbf{H}}_k = \begin{bmatrix} \mathbf{I}_k & \mathbf{0} \\ \mathbf{0} & \mathbf{H}_k \end{bmatrix} \tag{12.52}$$

The similarity update is:

$$\mathbf{A}_{k+1} = \widehat{\mathbf{H}}_k^* \mathbf{A}_k \widehat{\mathbf{H}}_k \tag{12.53}$$

Since a Householder reflector is Hermitian, its conjugate transpose is the reflector itself. For dense matrices this method requires fewer transformations than the Givens method and is normally preferred. Again, a symmetric matrix is reduced to symmetric tridiagonal form.

Python implementation:

```python
import numpy as np


def Hessenberg_Householder(A: np.ndarray,
                           tolerance: float = 1.0e-14):
    """Compute H = Q* @ A @ Q by Householder similarity transforms."""
    H = np.array(A, copy=True)
    n = H.shape[0]

    if H.shape != (n, n):
```

```python
            raise ValueError("A must be square")

    Q = np.eye(n, dtype=H.dtype)

    for k in range(n - 2):
        v, active = Householder_Vector(H[k + 1:, k], tolerance)
        if not active:
            continue

        # Left multiplication by the embedded reflector.
        projection = np.dot(np.conjugate(v), H[k + 1:, :])
        H[k + 1:, :] -= 2.0 * np.outer(v, projection)

        # Right multiplication by the same reflector.
        projection = np.dot(H[:, k + 1:], v)
        H[:, k + 1:] -= 2.0 * np.outer(
            projection, np.conjugate(v)
        )

        # Accumulate Q.
        projection = np.dot(Q[:, k + 1:], v)
        Q[:, k + 1:] -= 2.0 * np.outer(
            projection, np.conjugate(v)
        )

    H[abs(H) < tolerance] = 0.0
    return Q, H
```

12.3.6 Jacobi similarity rotations

A **Jacobi rotation** is a symmetric plane rotation used on both sides of a real symmetric matrix:

$$\mathbf{A}_{new} = \mathbf{J}^T \mathbf{A} \mathbf{J} \tag{12.54}$$

For selected indices p and q, the nontrivial part of $\mathbf{J}$ is:

$$\begin{bmatrix} c & -s \\ s & c \end{bmatrix} \tag{12.55}$$

The angle can be selected so that A_{pq} and A_{qp} become zero. Unlike Householder Hessenberg reduction, however, a later Jacobi rotation can recreate entries that an earlier rotation removed. Jacobi rotations are therefore not a finite general-purpose Hessenberg-reduction algorithm. Their natural use is to reduce a symmetric matrix directly towards diagonal form, as described in the next subsection.

A single Jacobi similarity step is useful for understanding the operation:

```python
import numpy as np


def Jacobi_Rotation(A: np.ndarray, p: int, q: int):
    """Apply one Jacobi similarity rotation to a real symmetric matrix."""
    A = np.array(A, dtype=float, copy=True)
    n = A.shape[0]

    if A.shape != (n, n):
        raise ValueError("A must be square")
    if p == q:
        return A, np.eye(n)

    theta = 0.5 * np.arctan2(
        2.0 * A[p, q], A[p, p] - A[q, q]
    )
    c = float(np.cos(theta))
    s = float(np.sin(theta))

    J = np.eye(n)
    J[p, p] = c
    J[q, q] = c
    J[p, q] = -s
    J[q, p] = s

    return J.T @ A @ J, J
```

12.3.7 Jacobi rotations to diagonal matrix

For a real symmetric matrix, repeated Jacobi rotations reduce the off-diagonal entries while preserving symmetry and eigenvalues. When the process converges:

$$\mathbf{D} = \mathbf{V}^T \mathbf{A} \mathbf{V} \tag{12.56}$$

where $\mathbf{D}$ is diagonal and the columns of $\mathbf{V}$ are orthonormal eigenvectors. The diagonal entries of $\mathbf{D}$ are the eigenvalues.

A simple strategy selects the largest off-diagonal entry in every iteration:

$$(p, q) = \underset{p<q}{\operatorname{argmax}} |A_{pq}| \tag{12.57}$$

The rotation angle is obtained from:

$$\tan(2\theta) = \frac{2A_{pq}}{A_{pp} - A_{qq}} \tag{12.58}$$

Using atan2 avoids an explicit division and preserves the correct quadrant. The method is easy to understand and produces accurate eigenvectors, but each rotation changes rows and columns of a dense matrix. It is therefore mainly useful for small or medium symmetric problems and as a reference implementation.

Python implementation:

```python
import numpy as np


def Jacobi_Eigenvalue(A: np.ndarray,
                      tolerance: float = 1.0e-12,
                      max_iterations: int = 10000):
    """Diagonalise a real symmetric matrix with Jacobi rotations."""
    D = np.array(A, dtype=float, copy=True)
```

```
    n = D.shape[0]

    if D.shape != (n, n):
        raise ValueError("A must be square")

    V = np.eye(n)

    for iteration in range(max_iterations):
        # Locate the largest off-diagonal entry.
        p = 0
        q = 1
        largest = 0.0
        for i in range(n - 1):
            for j in range(i + 1, n):
                if abs(D[i, j]) > largest:
                    largest = abs(D[i, j])
                    p = i
                    q = j

        if largest <= tolerance:
            break

        theta = 0.5 * np.arctan2(
            2.0 * D[p, q], D[p, p] - D[q, q]
        )
        c = float(np.cos(theta))
        s = float(np.sin(theta))

        J = np.eye(n)
        J[p, p] = c
        J[q, q] = c
        J[p, q] = -s
        J[q, p] = s

        D = J.T @ D @ J
        V = V @ J

        # Remove round-off remaining in the selected location.
        D[p, q] = 0.0
        D[q, p] = 0.0
    else:
```

```
        raise RuntimeError("Jacobi iteration did not converge")

    eigenvalues = np.diag(D).copy()
    return eigenvalues, V
```

12.3.8 Arnoldi algorithm

The **Arnoldi algorithm** constructs an orthonormal basis of a Krylov subspace for a general square matrix:

$$\mathcal{K}_m(\mathbf{A}, \mathbf{b}) = \operatorname{span}\left\{\mathbf{b}, \mathbf{A}\mathbf{b}, \ldots, \mathbf{A}^{m-1}\mathbf{b}\right\} \tag{12.59}$$

Directly forming the powers of $\mathbf{A}$ would rapidly produce nearly dependent vectors. Arnoldi instead multiplies the newest basis vector by $\mathbf{A}$ and orthogonalises the result against all preceding vectors. This gives:

$$\mathbf{A}\mathbf{V}_m = \mathbf{V}_{m+1}\overline{\mathbf{H}}_m \tag{12.60}$$

where $\mathbf{V}_{m+1}$ has orthonormal columns and $\overline{\mathbf{H}}_m$ is upper Hessenberg. The projected matrix:

$$\mathbf{H}_m = \mathbf{V}_m^* \mathbf{A}\mathbf{V}_m \tag{12.61}$$

is small. Its eigenvalues are called **Ritz values** and approximate selected eigenvalues of $\mathbf{A}$. The same relation is the basis of GMRES for linear systems.

If the new vector has zero norm, the Krylov subspace has become invariant under $\mathbf{A}$. This is called a **happy breakdown**: no further basis vectors are required in exact arithmetic.

Python implementation:

```
import numpy as np

```

```

def Arnoldi(A: np.ndarray,
            b: np.ndarray,
            steps: int,
            tolerance: float = 1.0e-14):
    """Construct an Arnoldi basis and upper Hessenberg projection."""
    A = np.asarray(A)
    b = np.asarray(b)
    n = A.shape[0]
    m = min(steps, n)
    dtype = np.result_type(A, b)

    V = np.zeros((n, m + 1), dtype=dtype)
    H = np.zeros((m + 1, m), dtype=dtype)

    beta = Vector_Norm(b)
    if beta <= tolerance:
        raise ValueError("the starting vector must be nonzero")

    V[:, 0] = b / beta
    used_steps = m

    for j in range(m):
        w = A @ V[:, j]

        # Modified Gram-Schmidt orthogonalisation.
        for i in range(j + 1):
            H[i, j] = np.dot(np.conjugate(V[:, i]), w)
            w -= H[i, j] * V[:, i]

        H[j + 1, j] = Vector_Norm(w)

        if H[j + 1, j] <= tolerance:
            used_steps = j + 1
            break

        V[:, j + 1] = w / H[j + 1, j]

    return (
        V[:, :used_steps + 1],
        H[:used_steps + 1, :used_steps]
```

```
    )
```

12.3.9 Lanczos algorithm

For a Hermitian matrix, the Arnoldi projected matrix is also Hermitian. A matrix that is both Hermitian and upper Hessenberg must be tridiagonal. This removes the need to orthogonalise against every preceding vector and gives the **Lanczos three-term recurrence**:

$$\begin{aligned} \mathbf{w}_j &= \mathbf{A}\mathbf{v}_j - \beta_{j-1}\mathbf{v}_{j-1} \\ \alpha_j &= \mathbf{v}_j^* \mathbf{w}_j \\ \mathbf{w}_j &\leftarrow \mathbf{w}_j - \alpha_j \mathbf{v}_j \\ \beta_j &= \|\mathbf{w}_j\|_2 \\ \mathbf{v}_{j+1} &= \frac{\mathbf{w}_j}{\beta_j} \end{aligned} \tag{12.62}$$

The resulting relation is:

$$\mathbf{A}\mathbf{V}_m = \mathbf{V}_m \mathbf{T}_m + \beta_m \mathbf{v}_{m+1} \mathbf{e}_m^T \tag{12.63}$$

where $\mathbf{T}_m$ is real symmetric tridiagonal for a real symmetric $\mathbf{A}$. Lanczos is well suited to large sparse structural eigenvalue problems because it requires only matrix-vector products and stores a limited number of vectors.

In exact arithmetic, the three-term recurrence preserves orthogonality. In finite precision, converged directions may reappear and produce duplicate Ritz values. The example therefore includes optional full reorthogonalisation. This increases the cost but makes the educational implementation reliable.

Python implementation:

```python
import numpy as np

```

```python

def Lanczos(A: np.ndarray,
            b: np.ndarray,
            steps: int,
            tolerance: float = 1.0e-14,
            reorthogonalise: bool = True):
    """Construct a Lanczos basis and symmetric tridiagonal projection."""
    A = np.asarray(A)
    b = np.asarray(b)
    n = A.shape[0]
    m = min(steps, n)
    dtype = np.result_type(A, b)

    V = np.zeros((n, m), dtype=dtype)
    alpha = np.zeros(m)
    beta = np.zeros(max(m - 1, 0))

    norm_b = Vector_Norm(b)
    if norm_b <= tolerance:
        raise ValueError("the starting vector must be nonzero")

    v = b / norm_b
    v_previous = np.zeros_like(v)
    beta_previous = 0.0
    used_steps = m

    for j in range(m):
        V[:, j] = v
        w = A @ v - beta_previous * v_previous

        alpha[j] = float(np.real(np.dot(np.conjugate(v), w)))
        w -= alpha[j] * v

        if reorthogonalise:
            for i in range(j + 1):
                coefficient = np.dot(np.conjugate(V[:, i]), w)
                w -= coefficient * V[:, i]

        if j == m - 1:
            break

```

```
        beta[j] = Vector_Norm(w)
        if beta[j] <= tolerance:
            used_steps = j + 1
            break

        v_previous = v
        v = w / beta[j]
        beta_previous = beta[j]

    T = np.diag(alpha[:used_steps])
    if used_steps > 1:
        T += np.diag(beta[:used_steps - 1], 1)
        T += np.diag(beta[:used_steps - 1], -1)

    return V[:, :used_steps], T
```

12.3.10 Schur algorithm

Every square complex matrix has a **Schur decomposition**:

$$\mathbf{A} = \mathbf{Z}\mathbf{T}\mathbf{Z}^* \tag{12.64}$$

where $\mathbf{Z}$ is unitary and $\mathbf{T}$ is upper triangular. The eigenvalues of $\mathbf{A}$ appear on the diagonal of $\mathbf{T}$. For real arithmetic, a real matrix is generally reduced to **real Schur form**, which is upper quasi-triangular and may contain 2×2 diagonal blocks representing complex-conjugate eigenvalue pairs.

The QR algorithm first reduces $\mathbf{A}$ to Hessenberg form and then repeatedly performs shifted QR steps:

$$\begin{aligned} \mathbf{H}_k - \mu_k \mathbf{I} &= \mathbf{Q}_k \mathbf{R}_k \\ \mathbf{H}_{k+1} &= \mathbf{R}_k \mathbf{Q}_k + \mu_k \mathbf{I} \end{aligned} \tag{12.65}$$

Since:

$$\mathbf{H}_{k+1} = \mathbf{Q}_k^* \mathbf{H}_k \mathbf{Q}_k \tag{12.66}$$

all iterates are similar and have the same eigenvalues. The shift μ_k accelerates convergence. When a subdiagonal entry becomes small, the corresponding eigenvalue is **deflated** and the iteration continues on the remaining leading block.

The following educational implementation converts the matrix to complex arithmetic. It therefore converges to triangular complex Schur form even when the original real matrix has complex eigenvalues. Production eigensolvers use more sophisticated double shifts, aggressive deflation and carefully blocked operations.

```python
import numpy as np


def Schur_QR(A: np.ndarray,
             tolerance: float = 1.0e-10,
             max_iterations: int = 5000):
    """Compute a simple complex Schur form using shifted QR iteration.

    This example reuses Hessenberg_Householder and QR_Householder.
    """
    A = np.array(A, dtype=complex, copy=True)
    n = A.shape[0]

    if A.shape != (n, n):
        raise ValueError("A must be square")

    Z, H = Hessenberg_Householder(A, tolerance * 0.01)
    active = n
    iterations = 0

    while active > 1:
        scale = (
            abs(H[active - 2, active - 2])
            + abs(H[active - 1, active - 1])
```

```
            + 1.0
        )

        if abs(H[active - 1, active - 2]) <= tolerance * scale:
            H[active - 1, active - 2] = 0.0
            active -= 1
            continue

        if iterations >= max_iterations:
            raise RuntimeError("shifted QR iteration did not converge")

        # A simple Rayleigh shift from the last diagonal entry.
        shift = H[active - 1, active - 1]
        shifted = (
            H[:active, :active]
            - shift * np.eye(active, dtype=complex)
        )
        Q_step, R_step = QR_Householder(
            shifted, tolerance * 0.01
        )

        # Apply the block similarity transformation to the whole matrix.
        H[:active, :] = np.conjugate(Q_step).T @ H[:active, :]
        H[:, :active] = H[:, :active] @ Q_step
        Z[:, :active] = Z[:, :active] @ Q_step

        iterations += 1

    H[abs(H) < tolerance] = 0.0
    return Z, H
```

12.3.11 Singular Value Decomposition (SVD)

For any real or complex $m \times n$ matrix, the **singular value decomposition** is:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^* \tag{12.67}$$

The columns of $\mathbf{U}$ and $\mathbf{V}$ are orthonormal. The diagonal entries of $\mathbf{\Sigma}$ are nonnegative singular values:

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq 0 \tag{12.68}$$

The right singular vectors are eigenvectors of $\mathbf{A}^*\mathbf{A}$, while the left singular vectors are eigenvectors of $\mathbf{A}\mathbf{A}^*$:

$$\begin{aligned} \mathbf{A}^*\mathbf{A}\mathbf{v}_i &= \sigma_i^2 \mathbf{v}_i \\ \mathbf{A}\mathbf{A}^*\mathbf{u}_i &= \sigma_i^2 \mathbf{u}_i \\ \mathbf{A}\mathbf{v}_i &= \sigma_i \mathbf{u}_i \end{aligned} \tag{12.69}$$

The SVD exposes matrix rank, null spaces and conditioning. It also gives the best rank-r approximation in the Euclidean and Frobenius norms:

$$\mathbf{A}_r = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^* \tag{12.70}$$

The algorithms below form $\mathbf{A}^*\mathbf{A}$ implicitly through repeated matrix-vector or matrix-block products. This is useful for explaining the relation to eigenvalue iteration, but it squares the condition number and is not the preferred production SVD algorithm. Robust libraries use bidiagonal reduction followed by specialised iterations.

SVD using Power Iteration

Power iteration applied to $\mathbf{A}^*\mathbf{A}$ converges to the dominant right singular vector:

$$\mathbf{v}^{(k+1)} = \frac{\mathbf{A}^*\mathbf{A}\mathbf{v}^{(k)}}{\|\mathbf{A}^*\mathbf{A}\mathbf{v}^{(k)}\|_2} \tag{12.71}$$

After convergence:

$$\begin{aligned}\sigma &= \|\mathbf{A}\mathbf{v}\|_2 \\ \mathbf{u} &= \frac{\mathbf{A}\mathbf{v}}{\sigma}\end{aligned} \tag{12.72}$$

The rank-one part is then removed:

$$\mathbf{A} \leftarrow \mathbf{A} - \sigma \mathbf{u}\mathbf{v}^* \tag{12.73}$$

and the iteration is repeated for the next singular triplet. This **deflation** is simple but accumulated rounding errors can reduce orthogonality, especially for clustered or repeated singular values.

Python implementation:

```python
import numpy as np


def SVD_Power(A: np.ndarray,
              rank: int = None,
              tolerance: float = 1.0e-10,
              max_iterations: int = 10000):
    """Approximate a thin SVD by power iteration and rank-one deflation."""
    B = np.array(A, dtype=float, copy=True)
    m, n = B.shape
    maximum_rank = min(m, n)
    r = maximum_rank if rank is None else min(rank, maximum_rank)

    U = np.zeros((m, r))
    singular_values = np.zeros(r)
    V = np.zeros((n, r))
    used_rank = r

    for k in range(r):
        # A deterministic nonzero initial vector.
        v = np.arange(1, n + 1, dtype=float) + k
```

```
        v /= Vector_Norm(v)

        for iteration in range(max_iterations):
            w = B.T @ (B @ v)
            norm_w = Vector_Norm(w)

            if norm_w <= tolerance:
                used_rank = k
                break

            v_new = w / norm_w

            # The sign of a singular vector is arbitrary.
            same_sign = Vector_Norm(v_new - v)
            opposite_sign = Vector_Norm(v_new + v)
            v = v_new

            if min(same_sign, opposite_sign) <= tolerance:
                break
        else:
            raise RuntimeError("power iteration did not converge")

        if used_rank == k:
            break

        Av = B @ v
        sigma = Vector_Norm(Av)
        if sigma <= tolerance:
            used_rank = k
            break

        u = Av / sigma
        U[:, k] = u
        singular_values[k] = sigma
        V[:, k] = v

        # Remove the converged rank-one contribution.
        B -= sigma * np.outer(u, v)

    return (
        U[:, :used_rank],
```

```
        singular_values[:used_rank],
        V[:, :used_rank].T
    )
```

SVD using QR (orthogonal) Iteration

Power iteration finds one direction at a time. **Orthogonal iteration** propagates a complete subspace. Starting with an orthonormal matrix $\mathbf{Q}^{(0)}$, repeat:

$$\begin{aligned} \mathbf{Y}^{(k)} &= \mathbf{A}^*\mathbf{A}\mathbf{Q}^{(k)} \\ \mathbf{Y}^{(k)} &= \mathbf{Q}^{(k+1)}\mathbf{R}^{(k+1)} \end{aligned} \tag{12.74}$$

The QR factorisation prevents all columns from collapsing onto the dominant singular vector. After convergence, each column $\mathbf{q}_i$ approximates a right singular vector. The corresponding left vector and singular value are:

$$\begin{aligned} \sigma_i &= \|\mathbf{A}\mathbf{q}_i\|_2 \\ \mathbf{u}_i &= \frac{\mathbf{A}\mathbf{q}_i}{\sigma_i} \end{aligned} \tag{12.75}$$

Convergence depends on separation between singular values. Repeated singular values define an invariant subspace rather than unique individual vectors; any orthonormal basis of that subspace is valid.

Python implementation:

```
import numpy as np


def SVD_Orthogonal_Iteration(A: np.ndarray,
                             tolerance: float = 1.0e-10,
```

```
                              max_iterations: int = 10000):
    """Approximate a thin SVD using block orthogonal iteration.

    The example reuses QR_Householder and does not call numpy.linalg.
    """
    A = np.array(A, dtype=float, copy=True)
    m, n = A.shape
    r = min(m, n)

    # Construct a deterministic full-rank starting block.
    seed = np.empty((n, r))
    for i in range(n):
        for j in range(r):
            seed[i, j] = 1.0 / (1.0 + i + j)
            if i == j:
                seed[i, j] += 1.0

    Q, unused_R = QR_Householder(seed)
    Q = Q[:, :r]

    for iteration in range(max_iterations):
        Y = A.T @ (A @ Q)
        Q_new, unused_R = QR_Householder(Y)
        Q_new = Q_new[:, :r]

        # Compare columns while accounting for arbitrary signs.
        error = 0.0
        for j in range(r):
            same_sign = Vector_Norm(Q_new[:, j] - Q[:, j])
            opposite_sign = Vector_Norm(Q_new[:, j] + Q[:, j])
            error = max(error, min(same_sign, opposite_sign))

        Q = Q_new
        if error <= tolerance:
            break
    else:
        raise RuntimeError("orthogonal iteration did not converge")

    U = np.zeros((m, r))
    singular_values = np.zeros(r)

```

```
    for j in range(r):
        Aq = A @ Q[:, j]
        singular_values[j] = Vector_Norm(Aq)
        if singular_values[j] > tolerance:
            U[:, j] = Aq / singular_values[j]

    # Sort the singular triplets from largest to smallest.
    order = np.argsort(-singular_values)
    singular_values = singular_values[order]
    U = U[:, order]
    Q = Q[:, order]

    return U, singular_values, Q.T
```

12.3.12 Positive semidefinite matrices

If a Hermitian matrix $\mathbf{A}$ is only positive semidefinite, instead of positive definite, then it still has a decomposition of the form $\mathbf{A} = \mathbf{L}\mathbf{L}^*$ where the diagonal entries of $\mathbf{L}$ are allowed to be zero. The decompostion need not be unique, for example:

$$\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} = \mathbf{L}\mathbf{L}^* \tag{12.76}$$

$$\mathbf{L} = \begin{bmatrix} 0 & 0 \\ \cos\theta & \sin\theta \end{bmatrix} \tag{12.77}$$

However, if the rank of $\mathbf{A}$ is r, then there is a unique lower triangular $\mathbf{L}$ with exactly r positive diagonal elements adn $n - r$ columns containing all zeros.

Alternatively, the decomposition can be made unique when a pivoting choice is fixed. Formally, if $\mathbf{A}$ is an $n \times n$ positive semidefinite matrix of rank r, then there is at least one permutation matrix $\mathbf{P}$ such that $\mathbf{P}\mathbf{A}\mathbf{P}^T$ has a unique decompostion of the form

$$\mathbf{P}\mathbf{A}\mathbf{P}^T = \mathbf{L}\mathbf{L}^* \tag{12.78}$$

with

$$\mathbf{L} = \begin{bmatrix} \mathbf{L}_1 & \mathbf{0} \\ \mathbf{L}_2 & \mathbf{0} \end{bmatrix} \tag{12.79}$$

where $\mathbf{L}_1$ is a $r \times r$ lower triangular matrix with positive diagonal.

Example

$$\begin{bmatrix} 4 & 12 & -16 \\ 12 & 37 & -43 \\ -16 & -43 & 98 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 \\ 6 & 1 & 0 \\ -8 & 5 & 3 \end{bmatrix} \begin{bmatrix} 2 & 6 & -8 \\ 0 & 1 & 5 \\ 0 & 0 & 3 \end{bmatrix} \tag{12.80}$$

12.3.13 Applications

Solution of a system of linear equations

The Cholesky decomposition is mainly used for the numerical solution of **linear equations** $\mathbf{Ax} = \mathbf{b}$. If $\mathbf{A}$ is symmetric and positive definite, then we can solve $\mathbf{Ax} = \mathbf{b}$ by first computing the Cholesky decomposition $\mathbf{A} = \mathbf{LL}^*$, then solving $\mathbf{Ly} = \mathbf{b}$ for $\mathbf{y}$ by forward substitution and finally solving $\mathbf{L}^*\mathbf{x} = \mathbf{y}$ for $\mathbf{x}$ by back substitution.

$$\begin{aligned} \mathbf{Ax} &= \mathbf{b} \\ \mathbf{LL}^*\mathbf{x} &= \mathbf{b}, \ \textit{substitute } \mathbf{L}^*\mathbf{x} = \mathbf{y} \\ \mathbf{Ly} &= \mathbf{b}, \ \textit{then solve} \\ \mathbf{L}^*\mathbf{x} &= \mathbf{y} \end{aligned} \tag{12.81}$$

For linear systems that can be put into symmetric form, the Cholesky decomposition is the method choice, for superior efficiency and numerical stability. Compared to the **LU** decomposition, it is roughly twice as efficient.

12.3.14 Matrix Inversion

A non-Hermitian matrix **B** can be inverted using the following identity, where $\mathbf{BB}^*$ will always be Hermitian:

$$\mathbf{B}^{-1} = \mathbf{B}^*(\mathbf{BB}^*)^{-1} \tag{12.82}$$

The above stated matrix relation is also called the **pseudoinverse** of a matrix. The following equality holds:

$$\mathbf{B}^*(\mathbf{BB}^*)^{-1} = (\mathbf{B}^*\mathbf{B})^{-1}\mathbf{B}^* \tag{12.83}$$

The best way to compute the **pseudoinverse** is to use **singular value decomposition** (**SVD**). With **A** being $\mathbf{A} = \mathbf{USV}^*$, where **U** and **V** (both $n \times n$) are orthogonal and **S** ($m \times n$) is **diagonal** with real, non-negative *singular values* $\sigma_{i,1} = 1, \ldots, n$. We find that

$$\mathbf{A}^* = \mathbf{V}(\mathbf{S}^*\mathbf{S})^{-1}\mathbf{S}^*\mathbf{U}^* \tag{12.84}$$

If the rank r of **A** is smaller than n, the inverse of $\mathbf{S}^*\mathbf{S}$ does not exist, and one uses only the first r singular values. **S** then becomes an $r \times r$ matrix and **U**, **V** shrink accordingly.

12.4 Linear Regression

12.4.1 Derivation

Linear Regression is an approach for predicting a response using a **single feature**. In linear regression we assume that two variables i. e. independent and dependent are linearly related.

Generalrly we define **x** as a **feature vector**:

$$\mathbf{x} = [x_1, x_2, ..., x_n] \tag{12.85}$$

and $\mathbf{y}$ as **response vector**:

$$\mathbf{y} = [y_1, y_2, ..., y_n] \tag{12.86}$$

then a predicted response:

$$h(x_i) = \beta_0 + \beta_1 \times x_i \tag{12.87}$$

where:
x_i represents the i-th observation
β_1 represents the slope, and
β_0 represents the intercept of the regression line.

To predict a new value y_i, we need to consider:

$$y_i = \beta_0 + \beta_1 \times x_i + \epsilon_i = h(x_i) + \epsilon_i \tag{12.88}$$

where:
ϵ_i is the **error** of the estimate.

The error function is therefore obtained:

$$\epsilon_i = y_i - h(x_i) \tag{12.89}$$

The β_0 and β_1 coefficients are obtained by minimizing the residual **error** of the estimate using e.g. the **least square method**:

$$\begin{aligned} J(\beta_0, \beta_1) &= \frac{1}{2n} \sum_{i=1}^{n} \epsilon_i^2 \\ &= \frac{1}{2n} \sum_{i=1}^{n} (y_i - h(x_i))^2 \\ &= \frac{1}{2n} \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 \times x_i)^2 \end{aligned} \tag{12.90}$$

To find the coefficients we can derviate the cost function, first with regards to β_0:

$$\frac{\partial J}{\partial \beta_0} \left[\frac{1}{2n} \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 \times x_i)^2 \right] = 0 \tag{12.91}$$

where using the chain rule we can write:

$$
\begin{aligned}
u &= y_i - \beta_0 - \beta_1 \times x_i \\
J &= \frac{1}{2n} \sum_{i=1}^{n} u^2 \\
\frac{\partial J}{\partial \beta_0} &= \frac{\partial J}{\partial u} \times \frac{\partial u}{\partial \beta_0} \\
\frac{\partial J}{\partial u} &= 2u \\
\frac{\partial u}{\partial \beta_0} &= -1 \\
\frac{\partial J}{\partial \beta_0} &= \frac{1}{2n} \times (-2) \times \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 \times x_i) \\
0 &= \frac{1}{n} \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 \times x_i) \\
0 &= \frac{1}{n} \sum_{i=1}^{n} y_i - \frac{1}{n} \sum_{i=1}^{n} \beta_0 - \frac{1}{n} \beta_1 \sum_{i=1}^{n} x_i \\
0 &= \bar{y}_i - \beta_0 - \beta_1 \bar{x}_i \\
\beta_0 &= \bar{y} - \beta_1 \bar{x}
\end{aligned}
\tag{12.92}
$$

To find β_1:

$$
\frac{\partial J}{\partial \beta_1} \left[\frac{1}{2n} \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 \times x_i)^2 \right] = 0 \tag{12.93}
$$

$$
\begin{aligned}
u &= y_i - \beta_0 - \beta_1 \times x_i \\
J &= \frac{1}{2n} \sum_{i=1}^{n} u^2 \\
\frac{\partial J}{\partial \beta_1} &= \frac{\partial J}{\partial u} \times \frac{\partial u}{\partial \beta_1} \\
\frac{\partial J}{\partial u} &= 2u \\
\frac{\partial u}{\partial \beta_1} &= -x \\
\frac{\partial J}{\partial \beta_1} &= \frac{1}{2n} \times (-2) \sum_{i=1}^{n} x_i \times (y_i - \beta_0 - \beta_1 x_i) \\
0 &= \frac{1}{n} \sum_{i=1}^{n} x_i \left(y_i - \beta_0 - \beta_1 x_i\right) \\
0 &= \frac{1}{n} \sum_{i=1}^{n} \left(x_i y_i - \beta_0 x_i - \beta_1 x_i^2\right)
\end{aligned}
\tag{12.94}
$$

Substituting the value of β_0:

$$
\begin{aligned}
\beta_0 &= \bar{y} - \beta_1 \bar{x} \\
0 &= \frac{1}{n} \sum_{i=1}^{n} \left(x_i y_i - \left(\bar{y} - \beta_1 \bar{x}\right) x_i - \beta_1 x_i^2\right) \\
0 &= \frac{1}{n} \sum_{i=1}^{n} \left(x_i y_i - \bar{y} x_i + \beta_1 \bar{x} x_i - \beta_1 x_i^2\right) \\
0 &= \frac{1}{n} \left(\sum_{i=1}^{n} \left(x_i y_i - \bar{y} x_i\right) - \beta_1 \sum_{i=1}^{n} \left(x_i^2 - \bar{x} x_i\right) \right) \\
\beta_1 &= \frac{\frac{1}{n} \sum_{i=1}^{n} \left(x_i y_i - \bar{y} x_i\right)}{\frac{1}{n} \sum_{i=1}^{n} \left(x_i^2 - \bar{x} x_i\right)}
\end{aligned}
\tag{12.95}
$$

Simplifying the formula using:

$$
\begin{aligned}
\bar{y} &= \frac{1}{n}\sum_{i=1}^{n} y_i \\
\bar{x} &= \frac{1}{n}\sum_{i=1}^{n} x_i
\end{aligned} \tag{12.96}
$$

we get:

$$
\begin{aligned}
\beta_1 &= \frac{\frac{1}{n}\sum_{i=1}^{n} x_i y_i - \bar{x}\bar{y}}{\frac{1}{n}\sum_{i=1}^{n} x_i^2 - \bar{x}\bar{x}} \\
\beta_1 &= \frac{\sum_{i=1}^{n} x_i y_i - n\bar{x}\bar{y}}{\sum_{i=1}^{n} x_i^2 - n\bar{x}^2}
\end{aligned} \tag{12.97}
$$

12.4.2 Result

So, to recapitulate, to get a linear regression coefficients for a dataset:

$$
\begin{aligned}
\mathbf{x} &= [x_1, x_2, ..., x_n] \\
\mathbf{y} &= [y_1, y_2, ..., y_n]
\end{aligned} \tag{12.98}
$$

we may use equations:

$$
\begin{aligned}
\beta_1 &= \frac{\sum_{i=1}^{n} x_i y_i - n\bar{x}\bar{y}}{\sum_{i=1}^{n} x_i^2 - n\bar{x}^2} \\
\beta_0 &= \bar{y} - \beta_1\bar{x}
\end{aligned} \tag{12.99}
$$

with an error:

$$
\epsilon_i = y_i - \beta_0 - \beta_1 x_i \tag{12.100}
$$

12.5 Interpolation and Mapping of Values

Values often have to be transferred from one set of points or one mesh to another. Examples are the transfer of pressure from a fluid mesh to a structural mesh, the evaluation of measured data at finite element nodes, the mapping of temperatures between analyses and the comparison of results from two different discretisations.

Let the source data be known at positions $\mathbf{x}_i$:

$$u_i = u(\mathbf{x}_i), \qquad i = 1, \ldots, n_s \tag{12.101}$$

and let $\mathbf{x}$ be a target position. The mapping operation constructs an estimate:

$$\widehat{u}(\mathbf{x}) \tag{12.102}$$

from the available source values. In many methods this estimate is a weighted sum:

$$\widehat{u}(\mathbf{x}) = \sum_{i=1}^{n_s} w_i(\mathbf{x}) u_i \tag{12.103}$$

with weights that depend on the geometry and on the selected interpolation method.

A useful first requirement is the reproduction of a constant field. If all source values are equal to c, the mapped value should also be c. This is obtained when:

$$\sum_{i=1}^{n_s} w_i(\mathbf{x}) = 1 \tag{12.104}$$

A stronger requirement is the reproduction of a linear field. If:

$$u(\mathbf{x}) = a_0 + \mathbf{a}^T \mathbf{x} \tag{12.105}$$

then an exactly linearly consistent interpolation should satisfy:

$$\begin{aligned} \sum_i w_i(\mathbf{x}) &= 1 \\ \sum_i w_i(\mathbf{x})\mathbf{x}_i &= \mathbf{x} \end{aligned} \tag{12.106}$$

The first equation reproduces the constant part and the second equation reproduces the linear part.

12.5.1 Interpolation, Approximation and Extrapolation

The words interpolation, approximation and extrapolation describe different situations:

- **Interpolation** evaluates a field between known values. An interpolating method normally reproduces the source values exactly.
- **Approximation** constructs a smooth or otherwise restricted field that need not pass through every source value. This is useful when the data contain noise.
- **Extrapolation** evaluates the field outside the region covered by the source data. The farther the target lies outside this region, the less information is available to determine the value.

For a point cloud, the natural interpolation region is usually its convex hull. For a finite element mesh, it is the union of the source elements. Inside this region, surrounding source values constrain the estimate from several directions. Outside it, all source points may lie on one side of the target and the result depends more strongly on assumptions made by the method.

A second distinction concerns what the mapping should preserve. Pointwise mapping attempts to reproduce the value at each target point. Conservative mapping instead preserves an integral quantity such as total force, heat, mass or flux. The two objectives are not generally equivalent.

12.5.2 Nearest-Neighbour Mapping

The simplest mapping assigns the value of the closest source point. Let:

$$j = \arg\min_i \|\mathbf{x} - \mathbf{x}_i\| \tag{12.107}$$

then:

$$\widehat{u}(\mathbf{x}) = u_j \tag{12.108}$$

The method is piecewise constant. Space is divided into Voronoi cells and every target point in one cell receives the value of the source point belonging to that cell.

Nearest-neighbour mapping is suitable for discrete quantities such as material numbers, region identifiers and contact status. It does not create values that were absent from the source data and therefore cannot overshoot. For a smooth physical field, however, it produces jumps at the Voronoi boundaries and only first-order geometric accuracy.

Python implementation:

```python
import numpy as np


def Nearest_Neighbour(source_points: np.ndarray,
                      source_values: np.ndarray,
                      target_points: np.ndarray) -> np.ndarray:
    """Map values from source points to target points by nearest neighbour."""
    source_points = np.asarray(source_points, dtype=float)
    source_values = np.asarray(source_values)
    target_points = np.asarray(target_points, dtype=float)

    if source_points.ndim != 2 or target_points.ndim != 2:
        raise ValueError("Point arrays must have shape (number, dimension).")
    if source_points.shape[0] != source_values.shape[0]:
```

```
        raise ValueError("Each source point must have one source value.")
    if source_points.shape[1] != target_points.shape[1]:
        raise ValueError("Source and target dimensions must agree.")

    result_shape = (target_points.shape[0],) + source_values.shape[1:]
    result = np.empty(result_shape, dtype=source_values.dtype)

    for t, point in enumerate(target_points):
        best_index = 0
        best_distance_squared = float("inf")

        for i, source_point in enumerate(source_points):
            difference = point - source_point
            distance_squared = np.dot(difference, difference)
            if distance_squared < best_distance_squared:
                best_distance_squared = distance_squared
                best_index = i

        result[t] = source_values[best_index]

    return result


source_points = np.array([[0.0], [1.0], [2.0]])
source_values = np.array([0.0, 10.0, 20.0])
target_points = np.array([[0.2], [0.8], [1.6]])

mapped = Nearest_Neighbour(source_points, source_values, target_points)
print(mapped)  # [ 0. 10. 20.]
```

A direct search compares every target point with every source point and costs $\mathcal{O}(n_s n_t)$. For large point clouds the same rule is normally accelerated by a spatial search structure such as a uniform grid, a k-d tree or a bounding volume hierarchy.

12.5.3 Inverse-Distance Weighting

Nearest-neighbour mapping uses only one source value. A smoother estimate is obtained when several nearby points contribute with weights that decrease with distance. For inverse-distance weighting:

$$\widetilde{w}_i(\mathbf{x}) = \frac{1}{(\|\mathbf{x} - \mathbf{x}_i\| + \epsilon)^p} \tag{12.109}$$

and the normalized weights are:

$$w_i(\mathbf{x}) = \frac{\widetilde{w}_i(\mathbf{x})}{\sum_j \widetilde{w}_j(\mathbf{x})} \tag{12.110}$$

so that:

$$\widehat{u}(\mathbf{x}) = \frac{\sum_i \widetilde{w}_i(\mathbf{x}) u_i}{\sum_i \widetilde{w}_i(\mathbf{x})} \tag{12.111}$$

The exponent p controls how local the interpolation is. A larger exponent gives more importance to the nearest points. The value $p = 2$ is common, but it is not universally optimal.

If the target coincides with a source point, its value should be returned exactly instead of relying on the regularisation ϵ. In practice the sum is also restricted to the k nearest points or to a radius of influence. This reduces cost and prevents distant data from affecting a local estimate.

Inverse-distance weighting reproduces constant fields because its normalized weights sum to one. It does not generally reproduce linear fields exactly. It is also sensitive to nonuniform point density: a dense cluster can receive a large total weight merely because it contains many points.

Python implementation using the k nearest source points:

```
import numpy as np

```

```python

def Inverse_Distance_Weighting(source_points: np.ndarray,
                               source_values: np.ndarray,
                               target_points: np.ndarray,
                               power: float = 2.0,
                               neighbours: int | None = None,
                               tolerance: float = 1.0e-12) -> np.ndarray:
    """Map scalar or component-wise values by inverse-distance weighting."""
    source_points = np.asarray(source_points, dtype=float)
    source_values = np.asarray(source_values, dtype=float)
    target_points = np.asarray(target_points, dtype=float)

    if source_points.shape[0] != source_values.shape[0]:
        raise ValueError("Each source point must have one source value.")
    if power <= 0.0:
        raise ValueError("The distance exponent must be positive.")

    n_source = source_points.shape[0]
    if neighbours is None:
        neighbours = n_source
    neighbours = max(1, min(int(neighbours), n_source))

    result_shape = (target_points.shape[0],) + source_values.shape[1:]
    result = np.empty(result_shape, dtype=float)

    for t, point in enumerate(target_points):
        difference = source_points - point
        distance_squared = np.sum(difference * difference, axis=1)
        closest = int(np.argmin(distance_squared))

        if distance_squared[closest] <= tolerance * tolerance:
            result[t] = source_values[closest]
            continue

        indices = np.argsort(distance_squared)[:neighbours]
        distances = np.sqrt(distance_squared[indices])
        weights = 1.0 / distances**power
        weights /= np.sum(weights)

        values = source_values[indices]
        result[t] = np.tensordot(weights, values, axes=(0, 0))
```

```python

    return result


source_points = np.array([[0.0], [1.0], [2.0]])
source_values = np.array([0.0, 1.0, 4.0])
target_points = np.array([[0.5], [1.5]])

mapped = Inverse_Distance_Weighting(
    source_points,
    source_values,
    target_points,
    power=2.0,
    neighbours=2,
)
print(mapped)
```

12.5.4 One-Dimensional Interpolation

When the source coordinates are ordered along one axis, the interval containing the target can be found directly. Let:

$$x_i \leq x \leq x_{i+1} \tag{12.112}$$

and define the local coordinate:

$$\xi = \frac{x - x_i}{x_{i+1} - x_i} \tag{12.113}$$

For a target inside the interval, $0 \leq \xi \leq 1$. Piecewise linear interpolation is:

$$\widehat{u}(x) = (1 - \xi)u_i + \xi u_{i+1} \tag{12.114}$$

The interpolation is continuous, reproduces every source value and reproduces linear functions exactly. Its derivative is constant within each interval and may jump at a source point.

If the same equation is evaluated with $\xi < 0$ or $\xi > 1$, it becomes linear extrapolation. This preserves the slope of the first or last interval, but the magnitude can grow without bound as the target moves away from the known data.

Python implementation:

```python
import numpy as np


def Piecewise_Linear_Interpolation(x_source: np.ndarray,
                                   u_source: np.ndarray,
                                   x_target: np.ndarray,
                                   extrapolation: str = "linear") -> np.ndarray:
    """Interpolate ordered one-dimensional data component by component."""
    x_source = np.asarray(x_source, dtype=float)
    u_source = np.asarray(u_source, dtype=float)
    x_target = np.asarray(x_target, dtype=float)

    if x_source.ndim != 1 or x_source.size < 2:
        raise ValueError("At least two one-dimensional source coordinates are
        ↪  required.")
    if u_source.shape[0] != x_source.size:
        raise ValueError("Each source coordinate must have one source value.")
    if np.any(x_source[1:] <= x_source[:-1]):
        raise ValueError("Source coordinates must be strictly increasing.")
    if extrapolation not in ("linear", "constant", "error"):
        raise ValueError("Unknown extrapolation rule.")

    result_shape = x_target.shape + u_source.shape[1:]
    result = np.empty(result_shape, dtype=float)

    flat_target = x_target.reshape(-1)
    flat_result = result.reshape((flat_target.size,) + u_source.shape[1:])

    for j, x in enumerate(flat_target):
        if x < x_source[0]:
            if extrapolation == "error":
                raise ValueError("Target lies below the source interval.")
            if extrapolation == "constant":
```

```
                flat_result[j] = u_source[0]
                continue
            i = 0
        elif x > x_source[-1]:
            if extrapolation == "error":
                raise ValueError("Target lies above the source interval.")
            if extrapolation == "constant":
                flat_result[j] = u_source[-1]
                continue
            i = x_source.size - 2
        else:
            i = int(np.searchsorted(x_source, x, side="right") - 1)
            i = min(max(i, 0), x_source.size - 2)

        xi = (x - x_source[i]) / (x_source[i + 1] - x_source[i])
        flat_result[j] = (1.0 - xi) * u_source[i] + xi * u_source[i + 1]

    return result


x_source = np.array([0.0, 1.0, 3.0])
u_source = np.array([0.0, 2.0, 4.0])
x_target = np.array([-0.5, 0.5, 2.0, 4.0])

print(Piecewise_Linear_Interpolation(
    x_source,
    u_source,
    x_target,
    extrapolation="linear",
))
```

Higher-order interpolation uses more than two neighbouring points. A global polynomial through n points may oscillate strongly, especially near the ends of a long interval. Piecewise polynomials avoid this problem.

A cubic spline is piecewise cubic and usually enforces continuity of the first and second derivatives. It is smooth, but it can overshoot between monotone source values. For constitutive curves and other data that must remain monotone, a monotone cubic Hermite method is often safer. Akima interpolation is another local cubic construction that reduces oscillation near abrupt changes of slope.

12.5.5 Interpolation on Structured Grids

For a Cartesian grid, interpolation may be performed successively along each coordinate. In a rectangular cell define:

$$\begin{aligned} \xi &= \frac{x - x_0}{x_1 - x_0} \\ \eta &= \frac{y - y_0}{y_1 - y_0} \end{aligned} \tag{12.115}$$

The four bilinear shape functions are:

$$\begin{aligned} N_{00} &= (1 - \xi)(1 - \eta) \\ N_{10} &= \xi(1 - \eta) \\ N_{11} &= \xi\eta \\ N_{01} &= (1 - \xi)\eta \end{aligned} \tag{12.116}$$

and:

$$\widehat{u}(\xi, \eta) = N_{00}u_{00} + N_{10}u_{10} + N_{11}u_{11} + N_{01}u_{01} \tag{12.117}$$

Bilinear interpolation reproduces functions of the form:

$$u(x, y) = a_0 + a_1x + a_2y + a_3xy \tag{12.118}$$

within an axis-aligned rectangular cell. It is linear along every line parallel to one coordinate axis, but it is not generally linear along an arbitrary diagonal.

For a three-dimensional rectangular cell, trilinear interpolation uses eight corner values:

$$\widehat{u}(\xi, \eta, \zeta) = \sum_{i=0}^{1} \sum_{j=0}^{1} \sum_{k=0}^{1} N_i(\xi) N_j(\eta) N_k(\zeta) u_{ijk} \tag{12.119}$$

where:

$$N_0(s) = 1 - s, \qquad N_1(s) = s \tag{12.120}$$

The same tensor-product idea extends to bicubic and tricubic interpolation, where values and derivatives from neighbouring grid points are used to obtain a smoother field.

12.5.6 Interpolation on Finite Elements

A source finite element mesh already defines an interpolation field. The mapping problem therefore consists of two steps:

1. find the source element containing the target point,
2. evaluate the source element shape functions at that point.

The element search can dominate the total cost. Testing every target against every source element is rarely acceptable for large meshes. Bounding boxes, uniform spatial bins, octrees and bounding volume hierarchies are commonly used to create a short list of candidate elements.

Barycentric Coordinates in a Triangle

For a triangle with nodes $\mathbf{x}_1$, $\mathbf{x}_2$ and $\mathbf{x}_3$, the target point is written as:

$$\mathbf{x} = \lambda_1 \mathbf{x}_1 + \lambda_2 \mathbf{x}_2 + \lambda_3 \mathbf{x}_3 \tag{12.121}$$

with:

$$\lambda_1 + \lambda_2 + \lambda_3 = 1 \tag{12.122}$$

The barycentric coordinates are also the linear triangle shape functions. The mapped value is:

$$\widehat{u}(\mathbf{x}) = \lambda_1 u_1 + \lambda_2 u_2 + \lambda_3 u_3 \tag{12.123}$$

The point lies inside or on the triangle when:

$$0 \leq \lambda_i \leq 1 \tag{12.124}$$

apart from a numerical tolerance. If one coordinate is negative, the point lies outside the edge opposite that node. Evaluating the same equation still gives an affine extrapolation.

For coordinates $\mathbf{x}_i = [x_i, y_i]^T$, solve:

$$\begin{bmatrix} x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \lambda_3 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \tag{12.125}$$

For a triangle it is convenient to evaluate the closed-form determinant expressions directly.

Python implementation:

```python
import numpy as np


def Triangle_Barycentric_Coordinates(vertices: np.ndarray,
                                     point: np.ndarray,
                                     tolerance: float = 1.0e-14) -> np.ndarray:
    """Return the three barycentric coordinates of a two-dimensional point."""
    vertices = np.asarray(vertices, dtype=float)
```

```python
    point = np.asarray(point, dtype=float)

    if vertices.shape != (3, 2) or point.shape != (2,):
        raise ValueError("Expected three 2D vertices and one 2D point.")

    x1, y1 = vertices[0]
    x2, y2 = vertices[1]
    x3, y3 = vertices[2]
    x, y = point

    denominator = (y2 - y3) * (x1 - x3) + (x3 - x2) * (y1 - y3)
    if abs(denominator) <= tolerance:
        raise ValueError("The triangle is degenerate.")

    lambda_1 = ((y2 - y3) * (x - x3)
                + (x3 - x2) * (y - y3)) / denominator
    lambda_2 = ((y3 - y1) * (x - x3)
                + (x1 - x3) * (y - y3)) / denominator
    lambda_3 = 1.0 - lambda_1 - lambda_2

    return np.array([lambda_1, lambda_2, lambda_3])


def Triangle_Interpolation(vertices: np.ndarray,
                           values: np.ndarray,
                           point: np.ndarray,
                           allow_extrapolation: bool = False,
                           tolerance: float = 1.0e-12):
    """Interpolate scalar or component-wise values in a linear triangle."""
    values = np.asarray(values, dtype=float)
    barycentric = Triangle_Barycentric_Coordinates(vertices, point)

    if not allow_extrapolation:
        if np.any(barycentric < -tolerance) or np.any(barycentric > 1.0 +
        ↪  tolerance):
            raise ValueError("The target point lies outside the triangle.")

    return np.tensordot(barycentric, values, axes=(0, 0))


vertices = np.array([[0.0, 0.0],
```

```
                     [2.0, 0.0],
                     [0.0, 1.0]])
values = np.array([1.0, 5.0, 4.0])
point = np.array([0.5, 0.25])

weights = Triangle_Barycentric_Coordinates(vertices, point)
value = Triangle_Interpolation(vertices, values, point)
print(weights, value)
```

The same idea applies to a tetrahedron using four barycentric coordinates. A linear simplex is especially convenient because the natural coordinates are obtained from one linear system and the interpolation reproduces all affine fields exactly.

Isoparametric Shape Functions

For a general finite element, both geometry and field values are interpolated using shape functions in natural coordinates $\boldsymbol{\xi}$:

$$\begin{aligned} \mathbf{x}(\boldsymbol{\xi}) &= \sum_{a=1}^{n_e} N_a(\boldsymbol{\xi})\mathbf{x}_a \\ \widehat{u}(\boldsymbol{\xi}) &= \sum_{a=1}^{n_e} N_a(\boldsymbol{\xi})u_a \end{aligned} \tag{12.126}$$

The target position $\mathbf{x}$ is known, but its natural coordinate $\boldsymbol{\xi}$ is not. It is obtained from the inverse isoparametric mapping:

$$\mathbf{r}(\boldsymbol{\xi}) = \sum_a N_a(\boldsymbol{\xi})\mathbf{x}_a - \mathbf{x} = \mathbf{0} \tag{12.127}$$

For nonlinear geometry interpolation this equation is solved iteratively. A Newton step is:

$$\boldsymbol{\xi}^{(k+1)} = \boldsymbol{\xi}^{(k)} - \mathbf{J}^{-1}(\boldsymbol{\xi}^{(k)})\mathbf{r}(\boldsymbol{\xi}^{(k)}) \tag{12.128}$$

where:

$$\mathbf{J} = \frac{\partial \mathbf{x}}{\partial \boldsymbol{\xi}} \tag{12.129}$$

is the geometric Jacobian. A converged natural coordinate must still be checked against the element domain. Newton convergence alone does not prove that the target lies inside the element.

For a four-node quadrilateral with natural coordinates $-1 \leq \xi, \eta \leq 1$, the shape functions are:

$$\begin{aligned} N_1 &= \frac{1}{4}(1-\xi)(1-\eta) \\ N_2 &= \frac{1}{4}(1+\xi)(1-\eta) \\ N_3 &= \frac{1}{4}(1+\xi)(1+\eta) \\ N_4 &= \frac{1}{4}(1-\xi)(1+\eta) \end{aligned} \tag{12.130}$$

Python implementation including the inverse mapping:

```
import numpy as np


def Quad4_Shape_Functions(xi: float, eta: float):
    N = 0.25 * np.array([
        (1.0 - xi) * (1.0 - eta),
        (1.0 + xi) * (1.0 - eta),
        (1.0 + xi) * (1.0 + eta),
        (1.0 - xi) * (1.0 + eta),
```

```python
    ])

    dN_dxi = 0.25 * np.array([
        -(1.0 - eta),
         (1.0 - eta),
         (1.0 + eta),
        -(1.0 + eta),
    ])

    dN_deta = 0.25 * np.array([
        -(1.0 - xi),
        -(1.0 + xi),
         (1.0 + xi),
         (1.0 - xi),
    ])
    return N, dN_dxi, dN_deta


def Solve_2x2(A: np.ndarray, b: np.ndarray,
              tolerance: float = 1.0e-14) -> np.ndarray:
    determinant = A[0, 0] * A[1, 1] - A[0, 1] * A[1, 0]
    if abs(determinant) <= tolerance:
        raise ValueError("Singular two by two system.")

    return np.array([
        ( b[0] * A[1, 1] - A[0, 1] * b[1]) / determinant,
        (-b[0] * A[1, 0] + A[0, 0] * b[1]) / determinant,
    ])


def Quad4_Natural_Coordinates(vertices: np.ndarray,
                              point: np.ndarray,
                              tolerance: float = 1.0e-12,
                              maximum_iterations: int = 25) -> np.ndarray:
    """Find natural coordinates of a point in a bilinear quadrilateral."""
    vertices = np.asarray(vertices, dtype=float)
    point = np.asarray(point, dtype=float)
    natural = np.zeros(2, dtype=float)

    for _ in range(maximum_iterations):
        xi, eta = natural
```

```
        N, dN_dxi, dN_deta = Quad4_Shape_Functions(xi, eta)

        mapped_point = N @ vertices
        residual = mapped_point - point
        if np.sqrt(np.dot(residual, residual)) <= tolerance:
            return natural

        Jacobian = np.column_stack((dN_dxi @ vertices,
                                    dN_deta @ vertices))
        increment = Solve_2x2(Jacobian, residual)
        natural -= increment

        if np.sqrt(np.dot(increment, increment)) <= tolerance:
            return natural

    raise RuntimeError("Inverse isoparametric mapping did not converge.")


def Quad4_Interpolation(vertices: np.ndarray,
                        values: np.ndarray,
                        point: np.ndarray,
                        allow_extrapolation: bool = False,
                        tolerance: float = 1.0e-10):
    natural = Quad4_Natural_Coordinates(vertices, point)
    xi, eta = natural

    if not allow_extrapolation:
        if abs(xi) > 1.0 + tolerance or abs(eta) > 1.0 + tolerance:
            raise ValueError("The target point lies outside the quadrilateral.")

    N, _, _ = Quad4_Shape_Functions(xi, eta)
    return np.tensordot(N, np.asarray(values, dtype=float), axes=(0, 0))


vertices = np.array([[0.0, 0.0],
                     [2.0, 0.2],
                     [1.8, 1.2],
                     [-0.1, 1.0]])
values = np.array([0.0, 2.0, 4.0, 1.0])
point = np.array([0.9, 0.55])
```

```python
print(Quad4_Natural_Coordinates(vertices, point))
print(Quad4_Interpolation(vertices, values, point))
```

12.5.7 Extrapolation Outside the Source Domain

Extrapolation requires an assumption about the field where no source data exist. The selected rule should therefore be stated explicitly instead of being hidden inside the mapping implementation.

Nearest-Boundary Projection

A target point outside the source domain may be projected onto the closest source boundary point:

$$\mathbf{x}_p = \arg\min_{\mathbf{y}\in\Gamma_s} \|\mathbf{x} - \mathbf{y}\| \tag{12.131}$$

and the field is extended by:

$$\widehat{u}(\mathbf{x}) = u(\mathbf{x}_p) \tag{12.132}$$

The value is constant in the outward normal direction. This rule is bounded and often suitable when a surface result is to be mapped to nearby offset points. It does not continue the normal gradient.

For a polygonal or finite element boundary, the closest point is found by projecting onto candidate edges or faces and clamping the local coordinates to their valid range.

Affine Extrapolation

The shape functions of a linear element can be evaluated outside the element. For a triangle, for example, one or more barycentric coordinates become negative, but:

$$\widehat{u}(\mathbf{x}) = \sum_i \lambda_i(\mathbf{x}) u_i \tag{12.133}$$

still continues the same affine field. This is exact when the true field is linear, but it may produce large values far from the element. A distance limit or a transition to a bounded far-field value is therefore commonly applied.

Artificial Bounding Domain

Unrestricted extrapolation may be replaced by interpolation over an enlarged artificial domain. Construct a box Ω_b around the source data and add support points at its corners. Assign these artificial points a prescribed far-field value, for example the source mean:

$$\bar{u} = \frac{1}{n_s} \sum_{i=1}^{n_s} u_i \tag{12.134}$$

The augmented data set is:

$$\begin{aligned} \mathbf{X}_{aug} &= [\mathbf{X}_{source}; \mathbf{X}_{box}] \\ \mathbf{u}_{aug} &= [\mathbf{u}_{source}; \bar{u}, \ldots, \bar{u}] \end{aligned} \tag{12.135}$$

A target outside the original source hull but inside the box is then inside the hull of the augmented points. A scattered-data interpolant, a triangulation, an RBF or a moving least-squares method can use these added points.

The box does not add measured information. It imposes a boundary assumption. Its size controls how quickly the estimate approaches the assigned far-field value. A close box creates a strong transition; a distant box allows the local source trend to continue farther. Assigning one global mean is bounded and neutral, but a physical far-field value, a local boundary average or zero may be more appropriate for a particular quantity.

Adding only corners is sufficient to enclose the convex hull, but it may not control the field well along a long box edge. Additional points can therefore be placed on the faces. The distribution of artificial points should not be so dense that it overwhelms the real data in methods whose weights depend on point density.

The following example augments a two-dimensional point cloud with mean-valued box corners and uses the inverse-distance weighting routine defined above for the final interpolation:

```python
import numpy as np


def Add_Mean_Valued_Box(source_points: np.ndarray,
                        source_values: np.ndarray,
                        margin: float = 1.0):
    """Add the corners of an expanded 2D box to a scattered data set."""
    source_points = np.asarray(source_points, dtype=float)
    source_values = np.asarray(source_values, dtype=float)

    if source_points.ndim != 2 or source_points.shape[1] != 2:
        raise ValueError("This example expects two-dimensional source points.")
    if margin <= 0.0:
        raise ValueError("The box margin must be positive.")

    minimum = np.min(source_points, axis=0)
    maximum = np.max(source_points, axis=0)
    extent = maximum - minimum

    # A zero extent can occur when all points share one coordinate.
    scale = max(float(np.max(extent)), 1.0)
    padding = margin * np.where(extent > 0.0, extent, scale)
    lower = minimum - padding
    upper = maximum + padding

    corners = np.array([
        [lower[0], lower[1]],
        [upper[0], lower[1]],
        [upper[0], upper[1]],
```

```
        [lower[0], upper[1]],
    ])

    mean_value = np.mean(source_values, axis=0)
    corner_values = np.empty((4,) + source_values.shape[1:], dtype=float)
    corner_values[...] = mean_value

    augmented_points = np.concatenate((source_points, corners), axis=0)
    augmented_values = np.concatenate((source_values, corner_values), axis=0)
    return augmented_points, augmented_values


source_points = np.array([[0.0, 0.0],
                          [1.0, 0.0],
                          [0.0, 1.0],
                          [1.0, 1.0]])
source_values = np.array([0.0, 2.0, 4.0, 6.0])

target_points = np.array([[1.2, 0.5],
                          [2.5, 0.5]])

points, values = Add_Mean_Valued_Box(
    source_points,
    source_values,
    margin=2.0,
)

mapped = Inverse_Distance_Weighting(
    points,
    values,
    target_points,
    power=2.0,
    neighbours=6,
)
print(mapped)
```

The result depends on both the box and the selected interpolant. The method should therefore be validated by varying the box size and checking whether the mapped values in the region of interest remain stable.

12.5.8 Interpolation of Scattered Data

When only scattered source points are available, no element connectivity exists a priori. The interpolation method must construct a neighbourhood or a global basis from the point positions.

Delaunay and Natural-Neighbour Interpolation

A Delaunay triangulation divides the convex hull of a point cloud into simplices. In two dimensions these are triangles; in three dimensions they are tetrahedra. The triangulation is chosen so that no source point lies inside the circumcircle or circumsphere of a simplex.

After the containing simplex has been found, ordinary piecewise linear interpolation uses its barycentric coordinates. The result is continuous and linearly consistent, but its gradient changes between neighbouring simplices.

Natural-neighbour interpolation is based on the dual Voronoi diagram. Insert the target point into the diagram. The area or volume taken from each neighbouring Voronoi cell defines its weight. The weights are local, positive inside the convex hull and sum to one. The method is smoother than piecewise linear Delaunay interpolation and has no arbitrary distance exponent.

Neither method defines a unique value outside the convex hull. Extrapolation must be handled separately, for example by boundary projection or by adding an artificial bounding domain.

Radial Basis Function Interpolation

A radial basis function depends only on the distance from a centre. A global RBF interpolant is:

$$\widehat{u}(\mathbf{x}) = \sum_{i=1}^{n_s} \lambda_i \phi\left(\|\mathbf{x} - \mathbf{x}_i\|\right) + \mathbf{p}^T(\mathbf{x})\mathbf{c} \tag{12.136}$$

The polynomial term supplies the required polynomial consistency. For an affine term in d dimensions:

$$\mathbf{p}(\mathbf{x}) = [1, x_1, \ldots, x_d]^T \tag{12.137}$$

The coefficients follow from:

$$\begin{bmatrix} \boldsymbol{\Phi} & \mathbf{P} \\ \mathbf{P}^T & \mathbf{0} \end{bmatrix} \begin{bmatrix} \boldsymbol{\lambda} \\ \mathbf{c} \end{bmatrix} = \begin{bmatrix} \mathbf{u} \\ \mathbf{0} \end{bmatrix} \tag{12.138}$$

where:

$$\begin{aligned} \Phi_{ij} &= \phi(\|\mathbf{x}_i - \mathbf{x}_j\|) \\ P_{ij} &= p_j(\mathbf{x}_i) \end{aligned} \tag{12.139}$$

Common radial functions include:

$$\begin{aligned} &\phi(r) = r^3 && \text{cubic polyharmonic spline} \\ &\phi(r) = r^2 \ln r && \text{thin-plate spline} \\ &\phi(r) = \exp[-(\epsilon r)^2] && \text{Gaussian} \\ &\phi(r) = \sqrt{1 + (\epsilon r)^2} && \text{multiquadric} \end{aligned} \tag{12.140}$$

Global RBF interpolation is smooth and works in any spatial dimension, but its matrix is dense. The factorisation cost grows approximately with n_s^3 and the storage with n_s^2. Compactly supported radial functions produce sparse matrices and are therefore preferable for large point sets.

The following implementation uses the cubic kernel $\phi(r) = r^3$ and an affine polynomial term. The Gaussian elimination is included so the numerical steps remain visible:

```python
import numpy as np


def Gaussian_Elimination(A: np.ndarray,
                         b: np.ndarray,
                         tolerance: float = 1.0e-14) -> np.ndarray:
    """Solve a dense linear system by elimination with partial pivoting."""
    A = np.asarray(A, dtype=float).copy()
    b = np.asarray(b, dtype=float).copy()

    flatten = b.ndim == 1
    if flatten:
        b = b.reshape(-1, 1)
    if A.ndim != 2 or A.shape[0] != A.shape[1]:
        raise ValueError("The coefficient matrix must be square.")
    if b.shape[0] != A.shape[0]:
        raise ValueError("The right hand side has an incompatible size.")

    n = A.shape[0]
    for k in range(n - 1):
        pivot = k + int(np.argmax(np.abs(A[k:, k])))
        if abs(A[pivot, k]) <= tolerance:
            raise ValueError("The interpolation system is singular.")

        if pivot != k:
            A[[k, pivot], :] = A[[pivot, k], :]
            b[[k, pivot], :] = b[[pivot, k], :]

        for i in range(k + 1, n):
            factor = A[i, k] / A[k, k]
            A[i, k:] -= factor * A[k, k:]
            b[i, :] -= factor * b[k, :]

    if abs(A[-1, -1]) <= tolerance:
        raise ValueError("The interpolation system is singular.")

    x = np.zeros_like(b)
    for i in range(n - 1, -1, -1):
        residual = b[i, :].copy()
        for j in range(i + 1, n):
```

```python
            residual -= A[i, j] * x[j, :]
        x[i, :] = residual / A[i, i]

    return x[:, 0] if flatten else x


def Cubic_RBF_Fit(source_points: np.ndarray,
                  source_values: np.ndarray):
    """Fit a cubic RBF with an affine polynomial term."""
    points = np.asarray(source_points, dtype=float)
    values = np.asarray(source_values, dtype=float)

    if points.ndim != 2 or values.shape[0] != points.shape[0]:
        raise ValueError("Source point and value counts must agree.")

    n, dimension = points.shape
    difference = points[:, None, :] - points[None, :, :]
    radius = np.sqrt(np.sum(difference * difference, axis=2))
    Phi = radius**3
    P = np.column_stack((np.ones(n), points))

    system = np.zeros((n + dimension + 1, n + dimension + 1))
    system[:n, :n] = Phi
    system[:n, n:] = P
    system[n:, :n] = P.T

    right_shape = (n + dimension + 1,) + values.shape[1:]
    right = np.zeros(right_shape, dtype=float)
    right[:n] = values

    coefficients = Gaussian_Elimination(system, right)
    radial_coefficients = coefficients[:n]
    polynomial_coefficients = coefficients[n:]
    return radial_coefficients, polynomial_coefficients


def Cubic_RBF_Evaluate(source_points: np.ndarray,
                       radial_coefficients: np.ndarray,
                       polynomial_coefficients: np.ndarray,
                       target_points: np.ndarray) -> np.ndarray:
    points = np.asarray(source_points, dtype=float)
```

```
    targets = np.asarray(target_points, dtype=float)

    difference = targets[:, None, :] - points[None, :, :]
    radius = np.sqrt(np.sum(difference * difference, axis=2))
    Phi_target = radius**3
    P_target = np.column_stack((np.ones(targets.shape[0]), targets))

    return (np.tensordot(Phi_target, radial_coefficients, axes=(1, 0))
            + np.tensordot(P_target, polynomial_coefficients, axes=(1, 0)))


source_points = np.array([[0.0, 0.0],
                          [1.0, 0.0],
                          [0.0, 1.0],
                          [1.0, 1.0],
                          [0.5, 0.4]])
source_values = np.array([0.0, 1.0, 1.0, 2.0, 0.7])
target_points = np.array([[0.25, 0.25], [0.75, 0.60]])

radial, polynomial = Cubic_RBF_Fit(source_points, source_values)
mapped = Cubic_RBF_Evaluate(
    source_points,
    radial,
    polynomial,
    target_points,
)
print(mapped)
```

The polynomial side conditions are not an optional decoration. They remove nonuniqueness for conditionally positive definite kernels and ensure that an affine source field is reproduced exactly. The source points must also support the selected polynomial basis; for an affine basis in two dimensions they must not all lie on one line.

Moving Least Squares

Moving least squares fits a local polynomial around every target point. Let:

$$u_h(\mathbf{y};\mathbf{x}) = \mathbf{p}^T(\mathbf{y})\mathbf{a}(\mathbf{x}) \tag{12.141}$$

where $\mathbf{x}$ is the target and $\mathbf{y}$ is a source coordinate. The coefficient vector is chosen to minimize:

$$J(\mathbf{a}) = \sum_{i=1}^{n_s} w(\|\mathbf{x}-\mathbf{x}_i\|)\left[\mathbf{p}^T(\mathbf{x}_i)\mathbf{a} - u_i\right]^2 \tag{12.142}$$

Differentiation gives the local normal equations:

$$\mathbf{A}(\mathbf{x})\mathbf{a}(\mathbf{x}) = \mathbf{b}(\mathbf{x}) \tag{12.143}$$

with:

$$\begin{aligned} \mathbf{A}(\mathbf{x}) &= \sum_i w_i \mathbf{p}(\mathbf{x}_i)\mathbf{p}^T(\mathbf{x}_i) \\ \mathbf{b}(\mathbf{x}) &= \sum_i w_i \mathbf{p}(\mathbf{x}_i)u_i \end{aligned} \tag{12.144}$$

and the mapped value is:

$$\widehat{u}(\mathbf{x}) = \mathbf{p}^T(\mathbf{x})\mathbf{a}(\mathbf{x}) \tag{12.145}$$

MLS is an approximation unless special interpolating weights are used. It is well suited to noisy scattered data and can reproduce polynomials up to the order of its basis. A sufficiently rich and well distributed neighbourhood is required. Near a boundary, all points may lie on one side and the moment matrix $\mathbf{A}$ can become poorly conditioned. Artificial support points or a larger neighbourhood can improve the situation, but they also introduce assumptions.

Kriging

Kriging treats the source values as samples of a spatially correlated random field. The estimate is again a weighted sum:

$$\widehat{u}(\mathbf{x}) = \sum_i w_i(\mathbf{x}) u_i \tag{12.146}$$

but the weights are determined from a covariance or variogram model. In ordinary kriging:

$$\begin{bmatrix} \mathbf{C} & \mathbf{1} \\ \mathbf{1}^T & 0 \end{bmatrix} \begin{bmatrix} \mathbf{w} \\ \mu \end{bmatrix} = \begin{bmatrix} \mathbf{c}(\mathbf{x}) \\ 1 \end{bmatrix} \tag{12.147}$$

where:

$$\begin{aligned} C_{ij} &= C(\mathbf{x}_i, \mathbf{x}_j) \\ c_i(\mathbf{x}) &= C(\mathbf{x}_i, \mathbf{x}) \end{aligned} \tag{12.148}$$

The constraint $\mathbf{1}^T\mathbf{w} = 1$ makes the estimate unbiased for an unknown constant mean. Kriging can represent anisotropic correlation, measurement noise and spatial length scales. It also provides an estimate of interpolation uncertainty. These benefits require a justified covariance model and make the method more involved than deterministic mesh-to-mesh interpolation.

12.5.9 Conservative Field Transfer

Pointwise interpolation preserves a field value locally, but it does not normally preserve its integral. Suppose a distributed load p is mapped between two surface meshes. Even if the mapped pressure looks smooth, the total force:

$$\mathbf{F} = \int_\Gamma p\mathbf{n}\,\mathrm{d}\Gamma \tag{12.149}$$

and the total moment:

$$\mathbf{M} = \int_{\Gamma} (\mathbf{x} - \mathbf{x}_0) \times p\mathbf{n} \, \mathrm{d}\Gamma \tag{12.150}$$

may change. This is unacceptable when the mapped field represents a conserved quantity.

L^2 Projection

Let the target field be represented by target shape functions:

$$u_t(\mathbf{x}) = \sum_i N_i^t(\mathbf{x}) u_i^t \tag{12.151}$$

An L^2 projection chooses the target coefficients so that the mapping error is orthogonal to the target approximation space:

$$\int_{\Omega_{overlap}} N_i^t \left(u_t - u_s\right) \, \mathrm{d}\Omega = 0 \tag{12.152}$$

Substitution gives:

$$\mathbf{M}_t \mathbf{u}_t = \mathbf{C}_{ts} \mathbf{u}_s \tag{12.153}$$

where:

$$\begin{aligned} (M_t)_{ij} &= \int_{\Omega_{overlap}} N_i^t N_j^t \, \mathrm{d}\Omega \\ (C_{ts})_{ik} &= \int_{\Omega_{overlap}} N_i^t N_k^s \, \mathrm{d}\Omega \end{aligned} \tag{12.154}$$

If the target shape functions form a partition of unity, the constant function belongs to the target space. Summing the projection equations then shows that the scalar integral of the field is preserved. If the coordinate functions also belong to the target space and the same geometry is used for integration, first moments are preserved as well. For vector tractions on differently approximated surfaces, changed normals and geometry still require a separate force and moment check.

The difficult part is not the final linear system. It is the integration over the geometric intersections of source and target elements. In one dimension, the intersections are intervals. In two dimensions, intersecting surface elements form polygons that must be segmented and integrated. In three dimensions, intersecting volumes are still more complicated.

The following one-dimensional example projects a piecewise linear source field onto a different piecewise linear target mesh. Each overlap interval is split at every source and target node, so both fields are polynomial within each integration segment. It uses the Gaussian elimination routine introduced in the RBF example.

```python
import numpy as np


def Linear_Element_Shape(x: float, x_0: float, x_1: float) -> np.ndarray:
    length = x_1 - x_0
    if length <= 0.0:
        raise ValueError("Element coordinates must be increasing.")
    xi = (x - x_0) / length
    return np.array([1.0 - xi, xi])


def Find_Interval(nodes: np.ndarray, x: float) -> int:
    """Return the element index containing x, including the last end point."""
    if x < nodes[0] or x > nodes[-1]:
        raise ValueError("Point lies outside the mesh interval.")
    index = int(np.searchsorted(nodes, x, side="right") - 1)
    return min(max(index, 0), nodes.size - 2)


def L2_Project_Linear_1D(source_nodes: np.ndarray,
```

```
                         source_values: np.ndarray,
                         target_nodes: np.ndarray) -> np.ndarray:
    """Project a linear source field onto a linear target mesh."""
    source_nodes = np.asarray(source_nodes, dtype=float)
    source_values = np.asarray(source_values, dtype=float)
    target_nodes = np.asarray(target_nodes, dtype=float)

    if source_values.shape[0] != source_nodes.size:
        raise ValueError("Each source node must have one source value.")
    if np.any(source_nodes[1:] <= source_nodes[:-1]):
        raise ValueError("Source nodes must be strictly increasing.")
    if np.any(target_nodes[1:] <= target_nodes[:-1]):
        raise ValueError("Target nodes must be strictly increasing.")
    if target_nodes[0] < source_nodes[0] or target_nodes[-1] > source_nodes[-1]:
        raise ValueError("This example requires the target mesh inside the source interval.")

    n_target = target_nodes.size
    mass = np.zeros((n_target, n_target), dtype=float)
    right_shape = (n_target,) + source_values.shape[1:]
    right = np.zeros(right_shape, dtype=float)

    # Split the integration domain at every source and target node.
    break_points = np.unique(np.concatenate((source_nodes, target_nodes)))
    lower = target_nodes[0]
    upper = target_nodes[-1]
    break_points = break_points[(break_points >= lower) & (break_points <= upper)]

    gauss_coordinate = 1.0 / np.sqrt(3.0)
    gauss_points = (-gauss_coordinate, gauss_coordinate)

    for a, b in zip(break_points[:-1], break_points[1:]):
        if b <= a:
            continue

        middle = 0.5 * (a + b)
        source_element = Find_Interval(source_nodes, middle)
        target_element = Find_Interval(target_nodes, middle)

        source_x0 = source_nodes[source_element]
```

```
        source_x1 = source_nodes[source_element + 1]
        target_x0 = target_nodes[target_element]
        target_x1 = target_nodes[target_element + 1]

        for gauss in gauss_points:
            x = 0.5 * (a + b) + 0.5 * (b - a) * gauss
            weight = 0.5 * (b - a)

            Ns = Linear_Element_Shape(x, source_x0, source_x1)
            Nt = Linear_Element_Shape(x, target_x0, target_x1)
            source_value = np.tensordot(
                Ns,
                source_values[source_element:source_element + 2],
                axes=(0, 0),
            )

            target_indices = (target_element, target_element + 1)
            for i_local, i_global in enumerate(target_indices):
                right[i_global] += weight * Nt[i_local] * source_value
                for j_local, j_global in enumerate(target_indices):
                    mass[i_global, j_global] += (
                        weight * Nt[i_local] * Nt[j_local]
                    )

    return Gaussian_Elimination(mass, right)


source_nodes = np.array([0.0, 0.4, 1.0, 1.7, 2.0])
source_values = np.array([0.0, 1.0, 0.2, 1.4, 1.0])
target_nodes = np.array([0.0, 0.7, 1.2, 2.0])

target_values = L2_Project_Linear_1D(
    source_nodes,
    source_values,
    target_nodes,
)
print(target_values)
```

The consistent target mass matrix gives the true L^2 projection. A lumped matrix produces a cheaper local approximation, but changes the projection. When transferring loads, a dual formulation or a mapping of nodal resultants may be used so that the virtual work is preserved.

Mortar Mapping

Mortar methods impose compatibility weakly between nonmatching meshes. Let $\mathbf{u}_s$ and $\mathbf{u}_t$ be source and target traces on an interface and let $\boldsymbol{\lambda}$ be a multiplier field. The weak compatibility condition is:

$$\int_\Gamma \boldsymbol{\lambda}^T (\mathbf{u}_t - \mathbf{u}_s)\, \mathrm{d}\Gamma = 0 \tag{12.155}$$

After discretisation this gives coupling matrices built from products of source, target and multiplier shape functions over their overlap. Mortar mappings are used for nonconforming mesh coupling, contact, domain decomposition and fluid-structure interaction.

The multiplier space and numerical integration must be chosen carefully. A poor choice can create rank deficiency or oscillatory transfer. The additional complexity is justified when consistency and conservation across nonmatching meshes are more important than a simple pointwise value.

12.5.10 Mapping Vector and Tensor Quantities

For a vector field expressed in one common Cartesian coordinate system, scalar interpolation weights may be applied component by component:

$$\widehat{\mathbf{v}}(\mathbf{x}) = \sum_i w_i(\mathbf{x}) \mathbf{v}_i \tag{12.156}$$

If source and target values use different local bases, the components must first be transformed to a common frame. Let $\mathbf{R}_s$ and $\mathbf{R}_t$ contain the source and target basis vectors. Then:

$$
\begin{aligned}
\mathbf{v}_{global} &= \mathbf{R}_s \mathbf{v}_{source} \\
\mathbf{v}_{target} &= \mathbf{R}_t^T \mathbf{v}_{global}
\end{aligned}
\tag{12.157}
$$

A second-order tensor transforms as:

$$
\mathbf{T}_{target} = \mathbf{R}_t^T \mathbf{R}_s \mathbf{T}_{source} \mathbf{R}_s^T \mathbf{R}_t \tag{12.158}
$$

Interpolating tensor components in inconsistent local frames can create a physically meaningless result even when each scalar interpolation is numerically correct.

Rotations require additional care. Rotation vectors are only locally additive. For large rotations, directly averaging Euler angles or rotation matrices does not generally produce a valid or path-independent rotation. Unit quaternions can be interpolated with normalized linear interpolation for nearby rotations or spherical interpolation for two rotations. The sign ambiguity of a quaternion must be resolved before interpolation because $\mathbf{q}$ and $-\mathbf{q}$ represent the same orientation.

12.5.11 Accuracy, Robustness and Validation

A mapping algorithm should be tested with fields whose exact behaviour is known. Useful checks are:

- **Source reproduction:** an interpolating method should return u_i when evaluated at $\mathbf{x}_i$.
- **Partition of unity:** mapping a constant field should return the same constant everywhere in the interpolation domain.
- **Linear consistency:** map an affine field and compare with its exact value at the targets.
- **Conservation:** compare integrated force, moment, mass, heat or flux before and after transfer.

- **Bounds:** check whether a method creates overshoot or values outside physically admissible limits.
- **Extrapolation distance:** report how far every extrapolated target lies from the source domain.
- **Neighbour quality:** detect nearly collinear, coplanar or otherwise insufficient point configurations.
- **Mesh refinement:** repeat the transfer with finer source and target discretisations and check convergence.

For source values u_i^{exact} and mapped values $\widehat{u}_i$, common error measures are:

$$\begin{aligned} e_{max} &= \max_i |\widehat{u}_i - u_i^{exact}| \\ e_{RMS} &= \sqrt{\frac{1}{n_t}\sum_{i=1}^{n_t}(\widehat{u}_i - u_i^{exact})^2} \\ e_{rel} &= \frac{\sqrt{\sum_i(\widehat{u}_i - u_i^{exact})^2}}{\sqrt{\sum_i(u_i^{exact})^2}} \end{aligned} \tag{12.159}$$

No single interpolation method is best for every quantity. A practical choice starts from the information available and the property that must be preserved:

- use nearest neighbour for discrete labels,
- use element shape functions when source connectivity is known,
- use IDW, RBF or MLS for scattered continuous data,
- state and limit every extrapolation rule,
- use an L^2 or mortar projection when integral conservation is the primary requirement.

12.6 Modal Assurance Criterion and Similarity Metrics

A comparison of two numerical or experimental models rarely reduces to checking whether individual numbers are equal. Eigenvectors may have different scaling, a reversed sign or a constant complex phase, even when they describe the same physical deformation. Closely spaced modes may also rotate within their common modal subspace, so that two valid bases look different when compared vector by vector.

Similarity measures remove selected ambiguities and reduce the comparison to one or more dimensionless values. The result is useful only when the removed ambiguity is physically irrelevant. A scale-invariant criterion, for example, cannot reveal an amplitude error. Therefore modal correlation should normally be accompanied by natural-frequency differences, scale factors and direct residual measures.

Let $\boldsymbol{\phi}_r$ denote mode r from a reference model and $\boldsymbol{\psi}_s$ mode s from a second model. Both vectors must first be expressed at the same physical coordinates, in the same order and in the same coordinate systems. A correlation value calculated from mismatched degrees of freedom has no physical meaning, even when the resulting number is close to one.

12.6.1 Complex Inner Product and Normalised Correlation

For real mode shapes, the ordinary dot product is sufficient. Experimental modes, damped-system modes and frequency-response functions may be complex. The corresponding inner product uses the conjugate transpose:

$$\langle \boldsymbol{x}, \boldsymbol{y} \rangle = \boldsymbol{x}^H \boldsymbol{y}. \tag{12.160}$$

A positive-definite weighting matrix $\mathbf{W}$ may be introduced:

$$\langle \boldsymbol{x}, \boldsymbol{y} \rangle_W = \boldsymbol{x}^H \mathbf{W} \boldsymbol{y}. \tag{12.161}$$

With $\mathbf{W} = \mathbf{I}$, every retained coordinate receives the same algebraic weight. A diagonal matrix may represent coordinate importance or quadrature weights. A mass matrix gives the inner product used by structural modes. The weighting matrix must be the same for both vectors and must not create a negative squared norm.

The normalised complex correlation coefficient is:

$$\boxed{\rho(\boldsymbol{x}, \boldsymbol{y}) = \frac{\boldsymbol{x}^H \mathbf{W} \boldsymbol{y}}{\sqrt{(\boldsymbol{x}^H \mathbf{W} \boldsymbol{x})\,(\boldsymbol{y}^H \mathbf{W} \boldsymbol{y})}}. \qquad (12.162)}$$

The magnitude satisfies $0 \leq |\rho| \leq 1$. For real vectors it is the cosine of the angle between the two vectors. The argument of ρ gives the constant phase rotation that best aligns the vectors. Multiplying either vector by a non-zero scalar changes this phase but not $|\rho|$.

12.6.2 Modal Assurance Criterion

The **Modal Assurance Criterion** (MAC) is the squared magnitude of the normalised correlation coefficient:

$$\boxed{\mathrm{MAC}(\boldsymbol{\phi}_r, \boldsymbol{\psi}_s) = \frac{|\boldsymbol{\phi}_r^H \mathbf{W} \boldsymbol{\psi}_s|^2}{(\boldsymbol{\phi}_r^H \mathbf{W} \boldsymbol{\phi}_r)\,(\boldsymbol{\psi}_s^H \mathbf{W} \boldsymbol{\psi}_s)} = |\rho_{rs}|^2. \qquad (12.163)}$$

Geometric Interpretation for Real Vectors

For two non-zero real vectors in two or three dimensions, the ordinary dot product has the familiar geometric interpretation

$$\boldsymbol{x}^T \boldsymbol{y} = \|\boldsymbol{x}\|_2 \|\boldsymbol{y}\|_2 \cos\alpha, \tag{12.164}$$

where $\alpha \in [0, \pi]$ is the angle between the oriented vectors. With the unweighted Euclidean inner product, substitution into Equation 12.163 gives

$$\boxed{\mathrm{MAC}(\boldsymbol{x}, \boldsymbol{y}) = \frac{(\boldsymbol{x}^T \boldsymbol{y})^2}{(\boldsymbol{x}^T \boldsymbol{x})(\boldsymbol{y}^T \boldsymbol{y})} = \cos^2 \alpha. \qquad (12.165)}$$

The same relation is valid in any finite dimension, but in two and three dimensions it can be visualised directly as the angle between arrows. For example, vectors separated by 60° have

$$\mathrm{MAC} = \cos^2 60^\circ = 0.25. \tag{12.166}$$

Squaring the cosine removes its sign. Parallel vectors ($\alpha = 0$) and antiparallel vectors ($\alpha = \pi$) therefore both have a MAC of one. Likewise, angles α and $\pi - \alpha$ give the same MAC. This is not a defect for mode-shape comparison because the sign of a real eigenvector is arbitrary, but it means that MAC does not preserve the orientation of an ordinary geometric vector.

When the vectors are treated as unoriented lines, the relevant acute angle is

$$\boxed{\beta = \arccos\left(\sqrt{\mathrm{MAC}}\right), \qquad 0 \le \beta \le \frac{\pi}{2}. \qquad (12.167)}$$

Thus, MAC $= 0.81$ corresponds to an unoriented angle $\beta = \arccos(0.9) \approx 25.84^\circ$. The full oriented angle α cannot be recovered from MAC alone; the sign of the normalised dot product is also required.

For a positive-definite weighting matrix $\mathbf{W}$, the same interpretation holds in the weighted geometry:

$$\cos\alpha_W = \frac{\boldsymbol{x}^T\mathbf{W}\boldsymbol{y}}{\sqrt{(\boldsymbol{x}^T\mathbf{W}\boldsymbol{x})(\boldsymbol{y}^T\mathbf{W}\boldsymbol{y})}}, \qquad \mathrm{MAC}_W = \cos^2\alpha_W. \tag{12.168}$$

The angle α_W is generally not the visible Euclidean angle between the two arrows. The weighting changes the geometry by stretching and rotating the coordinate space. A mass-weighted MAC therefore measures the angle induced by the mass matrix.

```python
import numpy as np


def Angle_And_MAC(vector_1, vector_2, tolerance=1.0e-14):
    vector_1 = np.asarray(vector_1, dtype=float).reshape(-1)
    vector_2 = np.asarray(vector_2, dtype=float).reshape(-1)

    if vector_1.size not in (2, 3) or vector_2.size != vector_1.size:
        raise ValueError("Use two vectors of equal length in 2D or 3D.")

    norm_1 = np.sqrt(np.dot(vector_1, vector_1))
    norm_2 = np.sqrt(np.dot(vector_2, vector_2))

    if norm_1 <= tolerance or norm_2 <= tolerance:
        raise ValueError("The angle is undefined for a zero-length vector.")

    cosine = np.dot(vector_1, vector_2) / (norm_1 * norm_2)
    cosine = min(1.0, max(-1.0, float(cosine)))

    oriented_angle = np.arccos(cosine)
    mac = cosine**2
    unoriented_angle = np.arccos(abs(cosine))

    return np.degrees(oriented_angle), mac, np.degrees(unoriented_angle)


x_2d = np.array([1.0, 0.0])
```

```
y_2d = np.array([0.5, np.sqrt(3.0) / 2.0])
print(Angle_And_MAC(x_2d, y_2d))
# (60.0 degrees, 0.25, 60.0 degrees)

x_3d = np.array([1.0, 0.0, 0.0])
y_3d = np.array([-1.0, -1.0, -1.0])
print(Angle_And_MAC(x_3d, y_3d))
# (125.264 degrees, 1/3, 54.736 degrees)
```

The MAC lies between zero and one. Its limiting values mean:

- $\mathrm{MAC}_{rs} = 1$: the two vectors are linearly dependent,
- $\mathrm{MAC}_{rs} = 0$: the two vectors are orthogonal in the selected inner product.

For a real mode, a sign reversal does not change the MAC:

$$\mathrm{MAC}(\boldsymbol{\phi}, -\boldsymbol{\phi}) = 1. \tag{12.169}$$

For a complex mode, neither does a constant phase rotation:

$$\mathrm{MAC}(\boldsymbol{\phi}, e^{\mathrm{i}\theta}\boldsymbol{\phi}) = 1. \tag{12.170}$$

This invariance is desirable because eigenvectors have arbitrary scale and phase. It also explains what MAC does *not* test. A MAC of one does not show that two modes have the same natural frequency, modal mass or physical amplitude. It only shows that the vectors have the same direction in the selected vector space.

There is no universal value that separates a correct mode pair from an incorrect one. The interpretation depends on modal density, the number and placement of observed coordinates, measurement noise and the intended use of the model. A reduced set of coordinates can make physically distinct modes look nearly parallel. A high MAC must therefore be read together with the available spatial information and with the off-diagonal terms of the complete MAC matrix.

```python
import numpy as np


def Apply_Weight(vector, weight=None):
    vector = np.asarray(vector)

    if weight is None:
        return vector

    weight = np.asarray(weight)

    if weight.ndim == 1:
        if weight.size != vector.shape[0]:
            raise ValueError("The weight vector has the wrong length.")
        return weight * vector

    if weight.ndim == 2:
        if weight.shape != (vector.shape[0], vector.shape[0]):
            raise ValueError("The weight matrix has the wrong shape.")
        return weight @ vector

    raise ValueError("Weight must be None, a vector or a square matrix.")


def Weighted_Inner_Product(left, right, weight=None):
    left = np.asarray(left).reshape(-1)
    right = np.asarray(right).reshape(-1)

    if left.size != right.size:
        raise ValueError("The vectors must contain the same coordinates.")

    return np.vdot(left, Apply_Weight(right, weight))


def MAC(reference, comparison, weight=None, tolerance=1.0e-14):
    reference = np.asarray(reference).reshape(-1)
    comparison = np.asarray(comparison).reshape(-1)

    cross = Weighted_Inner_Product(reference, comparison, weight)
    norm_reference = Weighted_Inner_Product(reference, reference, weight).real
```

```
    norm_comparison = Weighted_Inner_Product(comparison, comparison, weight).real

    if norm_reference <= tolerance or norm_comparison <= tolerance:
        raise ValueError("MAC is undefined for a zero-length vector.")

    value = abs(cross)**2 / (norm_reference * norm_comparison)

    # Remove only round-off excursions outside the theoretical interval.
    return float(min(1.0, max(0.0, value.real)))


phi = np.array([1.0, 2.0, -1.0, 0.5])
psi = -3.0 * phi
chi = np.array([2.0, -1.0, 0.0, 0.0])

print(MAC(phi, psi))  # 1.0: different scale and sign, same shape
print(MAC(phi, chi))  # smaller: a genuinely different vector direction
```

12.6.3 Direction- and Magnitude-Sensitive Vector Measures

MAC is useful when the scale and sign or phase of a vector are arbitrary. It is not a complete comparison of two vectors. In particular,

$$\mathrm{MAC}(\boldsymbol{x}, \boldsymbol{x}) = \mathrm{MAC}(\boldsymbol{x}, -\boldsymbol{x}) = \mathrm{MAC}(\boldsymbol{x}, 10\boldsymbol{x}) = 1. \tag{12.171}$$

The three comparisons have the same vector line, but they do not have the same orientation or magnitude. When these quantities are physically meaningful, MAC should be accompanied by measures that retain them.

Signed Cosine Similarity

For non-zero real vectors, the signed cosine similarity is

$$\boxed{c(\boldsymbol{x}, \boldsymbol{y}) = \frac{\boldsymbol{x}^T \boldsymbol{y}}{\|\boldsymbol{x}\|_2 \|\boldsymbol{y}\|_2} = \cos\alpha.} \tag{12.172}$$

Unlike MAC, the signed cosine is not squared. It therefore distinguishes parallel and antiparallel vectors:

$$c(\boldsymbol{x}, \boldsymbol{x}) = 1, \qquad c(\boldsymbol{x}, -\boldsymbol{x}) = -1. \tag{12.173}$$

The measure retains direction but still ignores magnitude. Multiplication of one vector by a positive scalar does not change c. For a negative scalar, only the sign changes. The relation to MAC is

$$\mathrm{MAC}(\boldsymbol{x}, \boldsymbol{y}) = c(\boldsymbol{x}, \boldsymbol{y})^2. \tag{12.174}$$

Consequently, MAC alone cannot determine whether the signed cosine is $+\sqrt{\mathrm{MAC}}$ or $-\sqrt{\mathrm{MAC}}$.

Norm Ratio and Direct Relative Error

The ratio of the vector magnitudes is

$$r = \frac{\|\boldsymbol{y}\|_2}{\|\boldsymbol{x}\|_2}. \tag{12.175}$$

The quantity r describes scale but contains no directional information. A simple magnitude-only error is

$$e_{size} = |r - 1|. \tag{12.176}$$

Direction and magnitude are both retained by the direct relative vector error:

$$e_{rel,x} = \frac{\|\boldsymbol{y}-\boldsymbol{x}\|_2}{\|\boldsymbol{x}\|_2}. \tag{12.177}$$

The subscript x indicates that $\boldsymbol{x}$ is the reference. Interchanging the vectors generally changes the result. Expanding the squared numerator gives

$$\|\boldsymbol{y}-\boldsymbol{x}\|_2^2 = \|\boldsymbol{x}\|_2^2 + \|\boldsymbol{y}\|_2^2 - 2\boldsymbol{x}^T\boldsymbol{y}, \tag{12.178}$$
$$= \|\boldsymbol{x}\|_2^2\left(1+r^2-2r\cos\alpha\right). \tag{12.179}$$

Therefore,

$$e_{rel,x}^2 = 1+r^2-2r\cos\alpha = 1+r^2-2rc. \tag{12.180}$$

This equation separates the two sources of disagreement. The norm ratio r contains the scale difference, while the signed cosine c contains the orientation difference. Replacing c by MAC is insufficient because the sign of c has been lost.

For vectors of equal magnitude, $r = 1$, and

$$e_{rel,x} = \sqrt{2-2\cos\alpha} = 2\sin\left(\frac{\alpha}{2}\right). \tag{12.181}$$

Thus equal and parallel vectors give zero error, equal orthogonal vectors give $\sqrt{2}$, and equal antiparallel vectors give an error of two. MAC gives one for both the first and the last case.

The same definition can use a positive-definite weighting matrix:

$$e_{rel,x,W} = \frac{\|\boldsymbol{y} - \boldsymbol{x}\|_W}{\|\boldsymbol{x}\|_W}, \qquad \|\boldsymbol{z}\|_W = \sqrt{\boldsymbol{z}^H \mathbf{W} \boldsymbol{z}}. \tag{12.182}$$

Unlike MAC, the direct complex-vector error is sensitive to a constant phase rotation. This is appropriate when both vectors share a physical phase reference. If their global phase is arbitrary, the phase should first be aligned, for example by the modal scale factor introduced later.

Symmetric Normalised Difference

A reference-independent alternative normalises the difference by the sum of both magnitudes:

$$\boxed{e_{sym} = \frac{\|\boldsymbol{x} - \boldsymbol{y}\|_2}{\|\boldsymbol{x}\|_2 + \|\boldsymbol{y}\|_2}.} \tag{12.183}$$

The triangle inequality gives

$$0 \le e_{sym} \le 1. \tag{12.184}$$

The value is zero only for identical vectors. It reaches one for antiparallel vectors and also when one vector is zero while the other is not. In terms of r and the signed cosine,

$$e_{sym}^2 = \frac{1 + r^2 - 2rc}{(1+r)^2}. \tag{12.185}$$

A bounded similarity score may be defined as

$$S_{sym} = 1 - e_{sym}. \tag{12.186}$$

This score is convenient for ranking, but the error e_{sym} is usually easier to interpret because it remains a normalised distance. Neither quantity should be called MAC because it tests different properties.

For a unit reference vector, several limiting cases are:

comparison	c	r	MAC	$e_{rel,x}$	e_{sym}
$\boldsymbol{y} = \boldsymbol{x}$	1	1	1	0	0
$\boldsymbol{y} = 2\boldsymbol{x}$	1	2	1	1	1/3
$\boldsymbol{y} = -\boldsymbol{x}$	-1	1	1	2	1
$\boldsymbol{y} \perp \boldsymbol{x}$, $\|\boldsymbol{y}\| = 1$	0	1	0	$\sqrt{2}$	$1/\sqrt{2}$

The table shows why no single number can simultaneously replace MAC, signed correlation and scale comparison. A useful report may contain the MAC, signed cosine, norm ratio and a direct error. Together they state whether two vectors have the same line, orientation, magnitude and actual coordinates.

For eigenvectors, a direct magnitude comparison is meaningful only after a common normalisation has been imposed. Examples are mass normalisation or unit displacement at the same reference coordinate. Without a common normalisation, the norm ratio merely compares arbitrary eigenvector scaling. For physical displacement vectors, forces or frequency-response functions with a common calibration, the direct errors retain information that MAC intentionally removes.

```python
import numpy as np


def Euclidean_Norm(vector):
    vector = np.asarray(vector, dtype=float).reshape(-1)
    return np.sqrt(np.dot(vector, vector))


def Direction_And_Size_Metrics(reference, comparison,
                               tolerance=1.0e-14):
```

```
    reference = np.asarray(reference, dtype=float).reshape(-1)
    comparison = np.asarray(comparison, dtype=float).reshape(-1)

    if reference.size != comparison.size:
        raise ValueError("The vectors must have the same length.")

    norm_reference = Euclidean_Norm(reference)
    norm_comparison = Euclidean_Norm(comparison)

    if norm_reference <= tolerance or norm_comparison <= tolerance:
        raise ValueError("Direction is undefined for a zero vector.")

    signed_cosine = (
        np.dot(reference, comparison)
        / (norm_reference * norm_comparison)
    )
    signed_cosine = min(1.0, max(-1.0, float(signed_cosine)))

    difference = comparison - reference
    norm_difference = Euclidean_Norm(difference)
    norm_ratio = norm_comparison / norm_reference

    return {
        "signed_cosine": signed_cosine,
        "mac": signed_cosine**2,
        "norm_ratio": norm_ratio,
        "size_error": abs(norm_ratio - 1.0),
        "relative_error": norm_difference / norm_reference,
        "symmetric_error": (
            norm_difference / (norm_reference + norm_comparison)
        ),
    }


x = np.array([1.0, 0.0])
comparisons = {
    "same": np.array([1.0, 0.0]),
    "twice": np.array([2.0, 0.0]),
    "opposite": np.array([-1.0, 0.0]),
    "orthogonal": np.array([0.0, 1.0]),
}
```

```python

for name, y in comparisons.items():
    print(name, Direction_And_Size_Metrics(x, y))
```

12.6.4 AutoMAC and CrossMAC Matrices

A modal model normally contains several vectors. Arranging the modes as columns,

$$\boldsymbol{\Phi} = [\boldsymbol{\phi}_1 \ \boldsymbol{\phi}_2 \ \cdots \ \boldsymbol{\phi}_{n_r}], \qquad \boldsymbol{\Psi} = [\boldsymbol{\psi}_1 \ \boldsymbol{\psi}_2 \ \cdots \ \boldsymbol{\psi}_{n_s}], \tag{12.187}$$

allows every pair to be compared. The **CrossMAC** matrix contains:

$$(\mathbf{C}_{MAC})_{rs} = \mathrm{MAC}(\boldsymbol{\phi}_r, \boldsymbol{\psi}_s). \tag{12.188}$$

Rows correspond to reference modes and columns to comparison modes. A clear one-to-one modal correspondence appears as one dominant value in each matched row and column. If two comparison modes both correlate strongly with one reference mode, the causes may include closely spaced modes, an exchanged mode order, insufficient coordinates or an actual modelling difference.

An **AutoMAC** matrix compares a set with itself:

$$(\mathbf{A}_{MAC})_{rs} = \mathrm{MAC}(\boldsymbol{\phi}_r, \boldsymbol{\phi}_s). \tag{12.189}$$

Its diagonal is one. Large off-diagonal terms mean that the retained coordinates do not distinguish the corresponding modes well. Exact modes of a symmetric undamped model are orthogonal with respect to the mass matrix, not necessarily with respect to the unweighted Euclidean product. Consequently, an unweighted AutoMAC may have non-zero off-diagonal terms even for a mathematically correct modal basis.

```python
import numpy as np


def MAC_Matrix(reference_modes, comparison_modes=None, weight=None):
    reference_modes = np.asarray(reference_modes)

    if reference_modes.ndim != 2:
        raise ValueError("Modes must be columns of a two-dimensional array.")

    if comparison_modes is None:
        comparison_modes = reference_modes
    else:
        comparison_modes = np.asarray(comparison_modes)

    if comparison_modes.ndim != 2:
        raise ValueError("Modes must be columns of a two-dimensional array.")

    if reference_modes.shape[0] != comparison_modes.shape[0]:
        raise ValueError("Both mode sets must contain the same coordinates.")

    result = np.empty((reference_modes.shape[1], comparison_modes.shape[1]))

    for r in range(reference_modes.shape[1]):
        for s in range(comparison_modes.shape[1]):
            result[r, s] = MAC(
                reference_modes[:, r],
                comparison_modes[:, s],
                weight,
            )

    return result


reference = np.array([
    [ 1.0,  0.0,  1.0],
    [ 0.0,  1.0,  1.0],
    [-1.0,  0.0,  1.0],
    [ 0.0, -1.0,  1.0],
])
```

```
# The first two modes are exchanged and slightly perturbed.
comparison = np.array([
    [ 0.05,  1.00,  1.00],
    [ 1.00,  0.05,  0.95],
    [-0.05, -1.00,  1.05],
    [-1.00, -0.05,  1.00],
])

print(MAC_Matrix(reference, comparison))
```

12.6.5 Weighted MAC and Cross-Orthogonality

When complete finite element mode shapes and a compatible mass matrix are available, the natural structural inner product is obtained with $\mathbf{W} = \mathbf{M}$. The mass-weighted MAC becomes:

$$\mathrm{MAC}_M(\boldsymbol{\phi}_r, \boldsymbol{\psi}_s) = \frac{|\boldsymbol{\phi}_r^H \mathbf{M} \boldsymbol{\psi}_s|^2}{(\boldsymbol{\phi}_r^H \mathbf{M} \boldsymbol{\phi}_r)(\boldsymbol{\psi}_s^H \mathbf{M} \boldsymbol{\psi}_s)}. \tag{12.190}$$

A full mass matrix cannot normally be applied directly to a sparse set of test coordinates. Extracting only the measured rows and columns of $\mathbf{M}$ does not in general preserve the inner product of the omitted coordinates. In that case an identity or carefully defined diagonal weighting is more transparent than a matrix described incorrectly as the physical mass matrix.

Before taking the absolute square, the normalised weighted products form a **cross-orthogonality matrix**:

$$C_{rs} = \frac{\boldsymbol{\phi}_r^H \mathbf{W} \boldsymbol{\psi}_s}{\sqrt{(\boldsymbol{\phi}_r^H \mathbf{W} \boldsymbol{\phi}_r)(\boldsymbol{\psi}_s^H \mathbf{W} \boldsymbol{\psi}_s)}}. \tag{12.191}$$

Thus:

$$\mathrm{MAC}_{rs} = |C_{rs}|^2. \tag{12.192}$$

The MAC discards the sign or phase of C_{rs}, while the cross-orthogonality matrix retains it. If two real mass-normalised modal bases have the same order and orientation, $\mathbf{C}$ is close to the identity. A diagonal entry of -1 means that the shape is identical but its arbitrary sign is reversed.

```python
import numpy as np


def Normalise_Mode(mode, weight=None, tolerance=1.0e-14):
    mode = np.asarray(mode)
    norm_squared = Weighted_Inner_Product(mode, mode, weight).real

    if norm_squared <= tolerance:
        raise ValueError("A zero mode cannot be normalised.")

    return mode / np.sqrt(norm_squared)


def Cross_Orthogonality(reference_modes, comparison_modes, weight=None):
    reference_modes = np.asarray(reference_modes)
    comparison_modes = np.asarray(comparison_modes)

    if reference_modes.shape[0] != comparison_modes.shape[0]:
        raise ValueError("Both sets must contain the same coordinates.")

    reference_normalised = np.column_stack([
        Normalise_Mode(reference_modes[:, i], weight)
        for i in range(reference_modes.shape[1])
    ])
    comparison_normalised = np.column_stack([
        Normalise_Mode(comparison_modes[:, i], weight)
        for i in range(comparison_modes.shape[1])
    ])

    weighted_comparison = np.column_stack([
        Apply_Weight(comparison_normalised[:, i], weight)
        for i in range(comparison_normalised.shape[1])
    ])

```

```
    return reference_normalised.conj().T @ weighted_comparison


mass = np.diag([2.0, 1.0, 2.0, 1.0])
cross_orthogonality = Cross_Orthogonality(reference, comparison, mass)
weighted_mac = abs(cross_orthogonality)**2

print(cross_orthogonality)
print(weighted_mac)
```

12.6.6 Modal Scale Factor and Scaled Residual

MAC intentionally ignores scale. To compare amplitudes, choose a reference mode $\boldsymbol{\phi}$ and scale the comparison mode $\boldsymbol{\psi}$ by a complex factor a. The factor that minimises the weighted least-squares residual

$$J(a) = \|\boldsymbol{\phi} - a\boldsymbol{\psi}\|_W^2 \tag{12.193}$$

is the **Modal Scale Factor** (MSF):

$$\boxed{a_{MSF} = \frac{\boldsymbol{\psi}^H \mathbf{W} \boldsymbol{\phi}}{\boldsymbol{\psi}^H \mathbf{W} \boldsymbol{\psi}}.} \tag{12.194}$$

The scaled comparison vector is:

$$\widehat{\boldsymbol{\psi}} = a_{MSF} \boldsymbol{\psi}. \tag{12.195}$$

For real modes, the sign of a_{MSF} aligns the arbitrary modal sign. For complex modes, its argument aligns the constant phase and its magnitude aligns the scale. The remaining relative error is:

$$e_{mode} = \frac{\|\boldsymbol{\phi} - a_{MSF}\boldsymbol{\psi}\|_W}{\|\boldsymbol{\phi}\|_W}. \tag{12.196}$$

With the same inner product and the optimum complex scale factor, Equations (12.163) and (12.196) are related by:

$$e_{mode} = \sqrt{1 - \mathrm{MAC}(\boldsymbol{\phi}, \boldsymbol{\psi})}. \tag{12.197}$$

The relation is useful for interpretation: MAC close to one corresponds to a small angular residual. It also means that this particular scaled residual does not provide information independent of MAC. The scale factor itself does.

```python
import numpy as np


def Modal_Scale_Factor(reference, comparison, weight=None,
                       tolerance=1.0e-14):
    denominator = Weighted_Inner_Product(comparison, comparison, weight)

    if denominator.real <= tolerance:
        raise ValueError("The comparison mode has zero length.")

    numerator = Weighted_Inner_Product(comparison, reference, weight)
    return numerator / denominator


def Scaled_Modal_Error(reference, comparison, weight=None):
    reference = np.asarray(reference)
    comparison = np.asarray(comparison)

    scale = Modal_Scale_Factor(reference, comparison, weight)
    residual = reference - scale * comparison

    residual_norm = Weighted_Inner_Product(residual, residual, weight).real
    reference_norm = Weighted_Inner_Product(reference, reference, weight).real

```

```python
    return scale, np.sqrt(residual_norm / reference_norm)


reference_mode = np.array([1.0, 2.0, -1.0, 0.5])
comparison_mode = 2.0j * reference_mode + np.array([0.00, 0.02, 0.00, -0.01])

scale, error = Scaled_Modal_Error(reference_mode, comparison_mode)
mac = MAC(reference_mode, comparison_mode)

print(scale)
print(error)
print(np.sqrt(1.0 - mac))
```

12.6.7 Coordinate Modal Assurance Criterion

MAC indicates whether a pair of complete mode vectors agrees, but it does not show where the disagreement occurs. The **Coordinate Modal Assurance Criterion** (COMAC) reverses the comparison. First, m corresponding mode pairs are selected and scaled consistently. Then the values at one coordinate j are compared over all paired modes:

$$\boxed{\mathrm{COMAC}_j = \frac{\left|\sum_{r=1}^{m} \overline{\phi_{jr}}\psi_{jr}\right|^2}{\left(\sum_{r=1}^{m} |\phi_{jr}|^2\right)\left(\sum_{r=1}^{m} |\psi_{jr}|^2\right)}. \qquad (12.198)}$$

COMAC is therefore a MAC performed along the modal direction of the two mode shape matrices at a fixed physical coordinate. A value near one means that this coordinate behaves consistently over the selected mode pairs. A low value marks a coordinate that contributes strongly to the disagreement.

COMAC requires more preparation than MAC:

1. the two mode sets must first be paired,
2. arbitrary sign or phase differences must be aligned,
3. the modes should be normalised or otherwise scaled consistently,
4. coordinates with negligible amplitude over all selected modes should not be interpreted as reliable localisation information.

A low COMAC value is not automatically evidence of a local modelling defect. It may also be caused by a sensor orientation error, coordinate mapping error, measurement noise, a poorly paired mode or a node located close to modal lines.

```python
import numpy as np


def Align_Mode_Set(reference_modes, comparison_modes, weight=None):
    reference_modes = np.asarray(reference_modes)
    comparison_modes = np.asarray(comparison_modes)

    if reference_modes.shape != comparison_modes.shape:
        raise ValueError("COMAC requires already paired mode sets.")

    aligned = comparison_modes.astype(
        np.result_type(reference_modes, comparison_modes, complex),
        copy=True,
    )

    for mode in range(reference_modes.shape[1]):
        scale = Modal_Scale_Factor(
            reference_modes[:, mode],
            aligned[:, mode],
            weight,
        )
        aligned[:, mode] *= scale

    return aligned
```

```python
def COMAC(reference_modes, comparison_modes, align=True,
          tolerance=1.0e-14):
    reference_modes = np.asarray(reference_modes)
    comparison_modes = np.asarray(comparison_modes)

    if reference_modes.shape != comparison_modes.shape:
        raise ValueError("The paired mode matrices must have the same shape.")

    if align:
        comparison_modes = Align_Mode_Set(reference_modes, comparison_modes)

    numerator = abs(
        np.sum(np.conj(reference_modes) * comparison_modes, axis=1)
    )**2
    denominator = (
        np.sum(abs(reference_modes)**2, axis=1)
        * np.sum(abs(comparison_modes)**2, axis=1)
    )

    result = np.full(reference_modes.shape[0], np.nan, dtype=float)
    valid = denominator > tolerance
    result[valid] = numerator[valid] / denominator[valid]
    return np.clip(result, 0.0, 1.0)


paired_reference = reference[:, :2]
paired_comparison = comparison[:, [1, 0]]

# Introduce a local discrepancy at coordinate 2.
paired_comparison = paired_comparison.copy()
paired_comparison[2, :] += np.array([0.45, -0.35])

print(COMAC(paired_reference, paired_comparison))
```

12.6.8 Repeated Modes and Subspace Similarity

Individual vectors are not always the correct objects to compare. Consider a repeated eigenvalue. Any linearly independent rotation of its modal basis spans the same physical eigenspace. Two solvers may therefore return:

$$\mathbf{\Psi} = \mathbf{\Phi}\mathbf{R}, \tag{12.199}$$

where $\mathbf{R}$ is an orthogonal or unitary matrix. Individual diagonal MAC values can then be low even though both bases describe exactly the same subspace.

Let $\boldsymbol{Q}_\Phi$ and $\boldsymbol{Q}_\Psi$ contain weighted-orthonormal bases of two subspaces with the same dimension p:

$$\boldsymbol{Q}_\Phi^H \mathbf{W} \boldsymbol{Q}_\Phi = \mathbf{I}, \qquad \boldsymbol{Q}_\Psi^H \mathbf{W} \boldsymbol{Q}_\Psi = \mathbf{I}. \tag{12.200}$$

The singular values of $\boldsymbol{Q}_\Phi^H \mathbf{W} \boldsymbol{Q}_\Psi$ are the cosines of the principal angles between the subspaces. A simple basis-invariant mean similarity is:

$$\boxed{S_{sub} = \frac{1}{p} \|\boldsymbol{Q}_\Phi^H \mathbf{W} \boldsymbol{Q}_\Psi\|_F^2. \tag{12.201}}$$

The value is one when the subspaces are identical and zero when they are orthogonal. Unlike a diagonal MAC average, S_{sub} does not depend on the particular basis chosen inside either subspace. For unequal subspace dimensions, normalising by the smaller dimension answers whether the smaller subspace is contained in the larger one.

```python
import numpy as np


def Weighted_Gram_Schmidt(vectors, weight=None, tolerance=1.0e-12):
    vectors = np.asarray(vectors)
    basis = []

```

```python
    for column in range(vectors.shape[1]):
        vector = vectors[:, column].astype(
            np.result_type(vectors, complex),
            copy=True,
        )

        # Two passes reduce the loss of orthogonality.
        for _ in range(2):
            for q in basis:
                vector -= q * Weighted_Inner_Product(q, vector, weight)

        norm_squared = Weighted_Inner_Product(vector, vector, weight).real
        if norm_squared > tolerance**2:
            basis.append(vector / np.sqrt(norm_squared))

    if not basis:
        raise ValueError("The vectors do not span a non-zero subspace.")

    return np.column_stack(basis)


def Subspace_Similarity(first, second, weight=None):
    q_first = Weighted_Gram_Schmidt(first, weight)
    q_second = Weighted_Gram_Schmidt(second, weight)

    weighted_second = np.column_stack([
        Apply_Weight(q_second[:, i], weight)
        for i in range(q_second.shape[1])
    ])
    cross = q_first.conj().T @ weighted_second

    dimension = min(q_first.shape[1], q_second.shape[1])
    return float(np.sum(abs(cross)**2).real / dimension)


subspace_a = np.array([
    [1.0, 0.0],
    [0.0, 1.0],
    [0.0, 0.0],
])

```

```python
angle = 0.73
rotation = np.array([
    [np.cos(angle), -np.sin(angle)],
    [np.sin(angle),  np.cos(angle)],
])
subspace_b = subspace_a @ rotation

print(MAC_Matrix(subspace_a, subspace_b))
print(Subspace_Similarity(subspace_a, subspace_b))  # exactly the same plane
```

12.6.9 Natural-Frequency Agreement and Mode Pairing

MAC compares shapes and is independent of eigenvalues. For a candidate pair, the signed relative natural-frequency difference may be written:

$$e_{f,rs} = \frac{f_s^{comparison} - f_r^{reference}}{f_r^{reference}}. \tag{12.202}$$

The absolute percentage error is $100|e_{f,rs}|$. Frequency agreement alone is not sufficient for pairing because different modes may cross or lie close together. MAC alone is also insufficient because spatially undersampled modes may look similar. A practical pairing procedure considers both.

One possible dimensionless assignment cost is:

$$c_{rs} = w_{MAC}\left(1 - \mathrm{MAC}_{rs}\right) + w_f e_{f,rs}^2, \tag{12.203}$$

where the weights state the relative importance assigned by the analyst. This is not a universal physical law; it is a transparent optimisation criterion. The complete set of pairs should be selected as a global assignment problem. Choosing the largest entry independently in every MAC row can assign one comparison mode to several reference modes and may miss the best overall pairing.

After pairing, report at least the mode numbers, both frequencies, relative frequency error, MAC and modal scale factor. For repeated or strongly coupled modes, report subspace similarity in place of forcing an arbitrary one-to-one vector pairing.

12.6.10 Frequency Response Assurance Criterion

Mode-shape correlation requires modal parameter extraction. The measured and calculated frequency-response functions can instead be compared directly. Let $\boldsymbol{h}_A$ and $\boldsymbol{h}_B$ contain complex FRF samples from the same frequencies and channels. The **Frequency Response Assurance Criterion** (FRAC) has the same algebraic form as MAC:

$$\boxed{\mathrm{FRAC}(\boldsymbol{h}_A, \boldsymbol{h}_B) = \frac{|\boldsymbol{h}_A^H \mathbf{W}_f \boldsymbol{h}_B|^2}{(\boldsymbol{h}_A^H \mathbf{W}_f \boldsymbol{h}_A)(\boldsymbol{h}_B^H \mathbf{W}_f \boldsymbol{h}_B)}. \qquad (12.204)}$$

The vectors may be constructed in different ways:

- for one input-output pair, stack the FRF values over a selected frequency range,
- at one frequency, stack all measured response channels,
- for a global comparison, stack several channels and frequency lines into one vector.

The construction must be stated because it determines what a single FRAC value means. A global value can hide a narrow frequency range with poor correlation. A frequency-by-frequency or moving-window FRAC reveals where the disagreement occurs.

FRAC is invariant to one constant complex scale factor over the selected vector. Thus two FRFs that differ only by a constant gain and phase shift have FRAC one. The corresponding complex scale factor is:

$$a_H = \frac{\boldsymbol{h}_B^H \mathbf{W}_f \boldsymbol{h}_A}{\boldsymbol{h}_B^H \mathbf{W}_f \boldsymbol{h}_B}. \qquad (12.205)$$

The unsquared normalised correlation coefficient from Equation (12.162) is also useful for FRFs. Its magnitude is $\sqrt{\mathrm{FRAC}}$, while its argument gives the best constant phase offset between the two selected FRF vectors.

Frequency weights can prevent a densely sampled or very large-amplitude region from dominating the comparison. For non-uniform frequency spacing, quadrature weights represent the integration intervals. Logarithmic amplitude comparison may be useful when anti-resonances and low-amplitude regions are important, but it answers a different question from complex FRAC and should be reported separately.

```python
import numpy as np


def Complex_Correlation(reference, comparison, weight=None,
                        tolerance=1.0e-14):
    reference = np.asarray(reference).reshape(-1)
    comparison = np.asarray(comparison).reshape(-1)

    cross = Weighted_Inner_Product(reference, comparison, weight)
    norm_reference = Weighted_Inner_Product(reference, reference, weight).real
    norm_comparison = Weighted_Inner_Product(comparison, comparison, weight).real

    if norm_reference <= tolerance or norm_comparison <= tolerance:
        raise ValueError("Correlation is undefined for a zero vector.")

    return cross / np.sqrt(norm_reference * norm_comparison)


def FRAC(reference_frf, comparison_frf, weight=None):
    correlation = Complex_Correlation(reference_frf, comparison_frf, weight)
    return float(abs(correlation)**2)


def Moving_FRAC(reference_frf, comparison_frf, half_window):
    reference_frf = np.asarray(reference_frf).reshape(-1)
    comparison_frf = np.asarray(comparison_frf).reshape(-1)

    if reference_frf.size != comparison_frf.size:
```

```python
        raise ValueError("The FRFs must use the same frequency lines.")

    result = np.empty(reference_frf.size)

    for centre in range(reference_frf.size):
        start = max(0, centre - half_window)
        stop = min(reference_frf.size, centre + half_window + 1)
        result[centre] = FRAC(
            reference_frf[start:stop],
            comparison_frf[start:stop],
        )

    return result


frequency = np.linspace(0.0, 200.0, 401)
reference_frf = 1.0 / (100.0**2 - frequency**2 + 2.0j * 0.02 * 100.0 * frequency)
comparison_frf = 1.0 / (103.0**2 - frequency**2 + 2.0j * 0.025 * 103.0 * frequency)

correlation = Complex_Correlation(reference_frf, comparison_frf)
print(FRAC(reference_frf, comparison_frf))
print(abs(correlation), np.angle(correlation))
local_frac = Moving_FRAC(reference_frf, comparison_frf, half_window=10)
worst = np.argmin(local_frac)
print(frequency[worst], local_frac[worst])
```

12.6.11 Direct Error Measures

Correlation criteria measure alignment but intentionally discard some information. Direct errors should therefore accompany them. For complex vectors sampled with non-negative weights w_i, the relative RMS error is:

$$e_{RMS,rel} = \sqrt{\frac{\sum_i w_i |x_i - y_i|^2}{\sum_i w_i |x_i|^2}}. \tag{12.206}$$

Unlike MAC or FRAC, this measure detects amplitude differences. It is not symmetric: the first vector is the reference. A symmetric normalised error can be defined by dividing by the mean energy of both vectors:

$$e_{sym} = \sqrt{\frac{2\sum_i w_i|x_i - y_i|^2}{\sum_i w_i|x_i|^2 + \sum_i w_i|y_i|^2}}. \tag{12.207}$$

For FRFs it is often useful to separate magnitude and phase. A logarithmic magnitude difference is:

$$e_{dB,i} = 20\log_{10}\frac{|y_i|}{|x_i|}, \tag{12.208}$$

while the wrapped phase difference is:

$$\Delta\varphi_i = \operatorname{atan2}\left(\sin(\varphi_{y,i} - \varphi_{x,i}), \cos(\varphi_{y,i} - \varphi_{x,i})\right). \tag{12.209}$$

Phase is poorly defined where both magnitudes are close to zero. A phase-error summary should therefore use an amplitude threshold or physically justified weights rather than treating every frequency line equally.

```python
import numpy as np


def Relative_RMS_Error(reference, comparison, weight=None,
                       tolerance=1.0e-14):
    reference = np.asarray(reference).reshape(-1)
    comparison = np.asarray(comparison).reshape(-1)

    if reference.size != comparison.size:
        raise ValueError("Both vectors must have the same length.")

    residual = reference - comparison
    numerator = Weighted_Inner_Product(residual, residual, weight).real
    denominator = Weighted_Inner_Product(reference, reference, weight).real

```

```python
    if denominator <= tolerance:
        raise ValueError("The reference vector has zero energy.")

    return np.sqrt(numerator / denominator)


def Symmetric_Normalised_Error(first, second, weight=None,
                               tolerance=1.0e-14):
    first = np.asarray(first).reshape(-1)
    second = np.asarray(second).reshape(-1)
    residual = first - second

    numerator = 2.0 * Weighted_Inner_Product(residual, residual, weight).real
    denominator = (
        Weighted_Inner_Product(first, first, weight).real
        + Weighted_Inner_Product(second, second, weight).real
    )

    if denominator <= tolerance:
        raise ValueError("Both vectors have zero energy.")

    return np.sqrt(numerator / denominator)


def Magnitude_And_Phase_Error(reference, comparison, minimum_magnitude=0.0):
    reference = np.asarray(reference).reshape(-1)
    comparison = np.asarray(comparison).reshape(-1)

    magnitude_reference = abs(reference)
    magnitude_comparison = abs(comparison)
    valid = (
        (magnitude_reference > minimum_magnitude)
        & (magnitude_comparison > minimum_magnitude)
    )

    magnitude_db = np.full(reference.size, np.nan)
    phase = np.full(reference.size, np.nan)

    magnitude_db[valid] = 20.0 * np.log10(
        magnitude_comparison[valid] / magnitude_reference[valid]
    )
```

```python

    phase_difference = np.angle(comparison[valid]) - np.angle(reference[valid])
    phase[valid] = np.arctan2(
        np.sin(phase_difference),
        np.cos(phase_difference),
    )

    return magnitude_db, phase


print(Relative_RMS_Error(reference_frf, comparison_frf))
print(Symmetric_Normalised_Error(reference_frf, comparison_frf))
```

12.6.12 Choosing and Reporting Similarity Measures

The metrics answer different questions:

- **MAC** asks whether two mode vectors have the same shape after arbitrary scaling and constant phase are ignored.
- **AutoMAC** asks whether the retained coordinates distinguish the modes within one set.
- **Cross-orthogonality** retains the sign or phase of normalised weighted modal products.
- **MSF** gives the scale and constant phase required to align one vector with another.
- **COMAC** localises coordinates that disagree over an already paired and consistently scaled group of modes.
- **Subspace similarity** compares repeated or closely spaced modal groups without depending on the arbitrary basis inside them.
- **FRAC** applies the same normalised complex correlation idea to frequency-response data.
- **Direct RMS, magnitude and phase errors** retain amplitude and local error information that correlation criteria discard.

A defensible modal comparison should document:

1. the common coordinates and their transformations,
2. omitted and interpolated degrees of freedom,
3. the weighting inner product,
4. the mode-pairing rule,
5. natural-frequency errors,
6. MAC or subspace similarity,
7. modal scale factors and direct residuals,
8. the frequency and coordinate ranges used for every FRF metric.

No scalar criterion can prove that two models are physically equivalent. A similarity number is a compressed diagnostic. Its value is understood only through the data included in the comparison and through the information that the normalisation deliberately removes.

12.7 Fast Fourier Transform (FFT)

Many measured and simulated quantities are functions of time. A displacement, force or acceleration history may be difficult to interpret directly when several oscillations are superposed. The Fourier transform rewrites the same signal as a combination of harmonic functions and thereby shows which frequencies are present.

The **Discrete Fourier Transform** (DFT) is the transform applied to a finite sequence of sampled values. The **Fast Fourier Transform** (FFT) is not a different transform. It is an efficient algorithm for evaluating the DFT. This distinction is important: the sampling, scaling and leakage properties belong to the DFT, while the FFT only reduces the amount of work required to calculate it.

12.7.1 Discrete Fourier Transform

Let a continuous signal $x(t)$ be sampled with a constant sampling interval Δt. A record containing N samples is written as:

$$
\begin{aligned}
x_n &= x(n\Delta t), \qquad n = 0, 1, \ldots, N-1 \\
f_s &= \frac{1}{\Delta t} \\
T &= N\Delta t
\end{aligned}
\tag{12.210}
$$

where f_s is the sampling frequency and T is the length of the sampled record. The DFT coefficients are:

$$
X_k = \sum_{n=0}^{N-1} x_n \exp\left(-\mathrm{i}\, 2\pi \frac{kn}{N}\right), \qquad k = 0, 1, \ldots, N-1 \tag{12.211}
$$

The inverse transform reconstructs the samples:

$$
x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k \exp\left(+\mathrm{i}\, 2\pi \frac{kn}{N}\right) \tag{12.212}
$$

This book uses the common convention in which the forward transform is unscaled and the factor $1/N$ is applied to the inverse transform. Other software may distribute the scaling differently, but the product of the forward and inverse scaling factors must equal $1/N$.

Each coefficient X_k is the projection of the sampled signal onto a complex harmonic basis vector. Its associated frequency is:

$$
f_k = \frac{k f_s}{N} = \frac{k}{T} \tag{12.213}
$$

and therefore the frequency spacing is:

$$\Delta f = \frac{f_s}{N} = \frac{1}{T} \tag{12.214}$$

A longer record improves the spacing of the frequency bins. Increasing only the sampling frequency while keeping the record length unchanged does not improve Δf; it merely increases the highest representable frequency and the number of samples.

12.7.2 Frequencies of the DFT Bins

The DFT returns an array of N complex coefficients. The array index k is not itself a frequency; it must be converted using the sampling frequency and the number of samples. For the first part of the array, the relation is:

$$f_k = k\Delta f = \frac{k f_s}{N} \tag{12.215}$$

The DFT output is ordered with the zero-frequency bin first, followed by the positive-frequency bins and then the negative-frequency bins. A signed bin index may be defined as:

$$\widetilde{k} = \begin{cases} k, & 0 \le k \le \lfloor N/2 \rfloor \\ k - N, & \lfloor N/2 \rfloor < k < N \end{cases} \tag{12.216}$$

and the corresponding signed frequency is:

$$f_k = \widetilde{k}\frac{f_s}{N} \tag{12.217}$$

For an even number of samples, the bin $k = N/2$ lies at the Nyquist frequency $f_s/2$. The sampled values cannot distinguish $+f_s/2$ from $-f_s/2$, so either sign may be used when labelling the full spectrum. This book labels that bin as positive. For an odd number of samples, no bin lies exactly at the Nyquist frequency.

For example, let $N = 8$ and $f_s = 800\,\text{Hz}$. The bin spacing is:

$$\Delta f = \frac{800}{8} = 100\,\text{Hz} \tag{12.218}$$

and the full DFT array corresponds to:

$$[0,\ 100,\ 200,\ 300,\ 400,\ -300,\ -200,\ -100]\ \text{Hz} \tag{12.219}$$

For a real-valued signal, only the non-negative part is normally retained. Its frequency array contains $\lfloor N/2 \rfloor + 1$ bins:

$$f_k = \frac{k f_s}{N}, \qquad k = 0, 1, \ldots, \lfloor N/2 \rfloor \tag{12.220}$$

The following function constructs either frequency array directly from the sampling information:

```python
import numpy as np


def DFT_Frequencies(n_samples: int,
                    sample_rate: float,
                    one_sided: bool = False) -> np.ndarray:
    """Return the frequencies associated with DFT array indices."""
    if n_samples < 1:
        raise ValueError("n_samples must be positive")
    if sample_rate <= 0.0:
        raise ValueError("sample_rate must be positive")

    if one_sided:
        n_bins = n_samples // 2 + 1
        return np.arange(n_bins) * sample_rate / n_samples

    k = np.arange(n_samples)
    signed_k = np.where(k <= n_samples // 2, k, k - n_samples)
```

```
    return signed_k * sample_rate / n_samples


full_frequency = DFT_Frequencies(8, 800.0)
one_sided_frequency = DFT_Frequencies(8, 800.0, one_sided=True)

print(full_frequency)
# [   0.  100.  200.  300.  400. -300. -200. -100.]

print(one_sided_frequency)
# [  0. 100. 200. 300. 400.]
```

When a spectral peak occurs at array index k, its frequency is therefore obtained either from kf_s/N or by indexing the corresponding frequency array. Keeping the coefficient and frequency arrays in the same order avoids mistakes when plotting or selecting spectral components.

The direct evaluation of the DFT computes N sums containing N terms. Its operation count is therefore proportional to:

$$\mathcal{O}(N^2) \tag{12.221}$$

This is acceptable for short sequences and useful as a reference implementation, but it becomes expensive for long measurement records.

12.7.3 Radix-2 Cooley–Tukey Algorithm

The FFT obtains the same coefficients by reusing repeated calculations. For a radix-2 transform, the number of samples is a power of two:

$$N = 2^p \tag{12.222}$$

Introduce the twiddle factor:

$$W_N = \exp\left(-\mathrm{i}\frac{2\pi}{N}\right) \tag{12.223}$$

so that the DFT can be written:

$$X_k = \sum_{n=0}^{N-1} x_n W_N^{kn} \tag{12.224}$$

The sequence is separated into samples with even and odd indices:

$$\begin{aligned} X_k &= \sum_{m=0}^{N/2-1} x_{2m} W_N^{2mk} + \sum_{m=0}^{N/2-1} x_{2m+1} W_N^{(2m+1)k} \\ &= \sum_{m=0}^{N/2-1} x_{2m} W_{N/2}^{mk} + W_N^k \sum_{m=0}^{N/2-1} x_{2m+1} W_{N/2}^{mk} \end{aligned} \tag{12.225}$$

The two sums are DFTs of length $N/2$. Denote them by E_k and O_k. The first half of the result is then:

$$X_k = E_k + W_N^k O_k, \qquad k = 0, 1, \ldots, N/2 - 1 \tag{12.226}$$

Using $W_N^{k+N/2} = -W_N^k$, the second half follows without another pair of transforms:

$$X_{k+N/2} = E_k - W_N^k O_k \tag{12.227}$$

The combination:

$$\begin{aligned} X_k &= E_k + W_N^k O_k \\ X_{k+N/2} &= E_k - W_N^k O_k \end{aligned} \tag{12.228}$$

is commonly called an FFT **butterfly**. The same split is applied recursively until transforms of length one remain. The work satisfies:

$$C(N) = 2C\left(\frac{N}{2}\right) + \mathcal{O}(N) \tag{12.229}$$

and therefore:

$$C(N) = \mathcal{O}(N \log_2 N) \tag{12.230}$$

For example, a direct DFT of $N = 2^{20}$ samples requires on the order of 2^{40} complex operations, whereas a radix-2 FFT requires on the order of $20 \cdot 2^{20}$. The exact operation count depends on the implementation, but the difference in scaling is the reason why frequency-domain processing of long records is practical.

The recursive formulation is particularly clear and is used in the example below. Production FFT libraries generally use iterative, in-place and cache-aware implementations and support lengths that are not powers of two by combining several radices or by using algorithms such as Bluestein's method.

12.7.4 Negative Frequencies and Real Signals

The indices above $N/2$ represent negative frequencies. A convenient signed interpretation is:

$$f_k = \begin{cases} \dfrac{k f_s}{N}, & 0 \le k \le N/2 \\ \dfrac{(k-N) f_s}{N}, & N/2 < k < N \end{cases} \tag{12.231}$$

For a real-valued time signal, the DFT has conjugate symmetry:

$$X_{N-k} = X_k^* \tag{12.232}$$

The negative-frequency half therefore contains no independent information. A one-sided spectrum keeps the bins from zero to the Nyquist frequency:

$$f_{\mathrm{Nyquist}} = \frac{f_s}{2} \tag{12.233}$$

The zero-frequency coefficient is the sum of the samples. Its amplitude after division by N is the mean value of the signal. The Nyquist bin, present for an even number of samples, is also unpaired. All other positive-frequency bins have matching negative-frequency bins.

12.7.5 Amplitude and Phase Scaling

For a real signal and the transform convention used above, a one-sided peak amplitude spectrum is obtained from:

$$A_k = \frac{2|X_k|}{N}, \qquad 0 < k < N/2 \tag{12.234}$$

The factor two collects the equal positive- and negative-frequency contributions into the retained positive-frequency bin. It must not be applied to the direct-current or Nyquist bins:

$$\begin{aligned} A_0 &= \frac{|X_0|}{N} \\ A_{N/2} &= \frac{|X_{N/2}|}{N} \end{aligned} \tag{12.235}$$

The phase is:

$$\varphi_k = \operatorname{atan2}\left(\operatorname{Im}(X_k), \operatorname{Re}(X_k)\right) \tag{12.236}$$

Amplitude conventions must always be stated. Peak amplitude, root-mean-square amplitude, power spectrum and power spectral density are different quantities. For a sinusoidal component, its root-mean-square amplitude is its peak amplitude divided by $\sqrt{2}$. Power spectral density scaling is discussed in the following spectral-density section.

12.7.6 Sampling, Aliasing and Leakage

Sampling cannot distinguish frequencies that differ by an integer multiple of the sampling frequency. A component above the Nyquist frequency is therefore folded into the represented frequency range. This is **aliasing**. It cannot be removed after sampling. The analogue signal must be sufficiently band-limited, usually by an anti-aliasing filter, before it is sampled.

The DFT also treats the finite record as one period of an infinitely repeated signal. If the values at the two ends do not join continuously, the repeated signal contains a discontinuity. Energy from a harmonic component then spreads into neighbouring frequency bins. This is **spectral leakage**.

Leakage is absent for a harmonic whose record contains an integer number of periods, because its frequency lies exactly on a DFT bin:

$$f = k\Delta f \tag{12.237}$$

For general measurement records this condition is rarely satisfied. A window reduces the discontinuity by gradually reducing the samples near the ends of the record. For example, the Hann window is:

$$w_n = \frac{1}{2}\left(1 - \cos\left(\frac{2\pi n}{N-1}\right)\right) \tag{12.238}$$

and the transformed samples become:

$$x_n^{(w)} = w_n x_n \tag{12.239}$$

A window reduces distant side lobes but broadens the main spectral peak. It also changes the measured amplitude. For deterministic harmonic amplitudes, a simple correction uses the **coherent gain**:

$$G_c = \frac{1}{N} \sum_{n=0}^{N-1} w_n \tag{12.240}$$

so that the amplitude spectrum is divided by G_c. Different windows trade peak width against side-lobe suppression; no window is best for every purpose.

Appending zeros increases the number of displayed frequency samples and makes the plotted spectrum smoother, but it does not improve the physical frequency resolution $1/T$. Only a longer measured record does that. Zero-padding can, however, make interpolation of an isolated spectral peak more convenient.

12.7.7 Python Implementation

The following direct DFT implements the definition literally. It is useful for checking the FFT on short sequences and for showing where the quadratic cost comes from.

```python
import numpy as np


def DFT(x: np.ndarray) -> np.ndarray:
    """Evaluate the discrete Fourier transform directly.

    The implementation uses NumPy arrays for storage and complex arithmetic.
    """
    x = np.asarray(x, dtype=complex).reshape(-1)
    n_samples = x.size
    X = np.zeros(n_samples, dtype=complex)

    for k in range(n_samples):
        value = 0.0j
        for n in range(n_samples):
            angle = -2.0 * np.pi * k * n / n_samples
            value += x[n] * np.exp(1.0j * angle)
        X[k] = value
```

```python

    return X
```

A recursive radix-2 Cooley–Tukey implementation follows directly from the even–odd derivation. The input length is checked explicitly because silently padding or truncating the data would change the transform being calculated.

```python
import numpy as np


def FFT_Radix2(x: np.ndarray) -> np.ndarray:
    """Evaluate a radix-2 FFT.

    Args:
        x: one-dimensional real or complex sequence. Its length must be a
           positive power of two.

    Returns:
        The unscaled forward DFT of x.
    """
    x = np.asarray(x, dtype=complex).reshape(-1)
    n_samples = x.size

    if n_samples == 0:
        raise ValueError("x must contain at least one sample")
    if n_samples & (n_samples - 1):
        raise ValueError("radix-2 FFT requires a power-of-two length")
    if n_samples == 1:
        return x.copy()

    even = FFT_Radix2(x[0::2])
    odd = FFT_Radix2(x[1::2])

    k = np.arange(n_samples // 2)
    twiddle = np.exp(-2.0j * np.pi * k / n_samples) * odd

    X = np.empty(n_samples, dtype=complex)
    X[:n_samples // 2] = even + twiddle
```

```python
    X[n_samples // 2:] = even - twiddle
    return X


def IFFT_Radix2(X: np.ndarray) -> np.ndarray:
    """Evaluate the inverse transform using conjugation of the forward FFT."""
    X = np.asarray(X, dtype=complex).reshape(-1)
    if X.size == 0:
        raise ValueError("X must contain at least one coefficient")

    return np.conjugate(FFT_Radix2(np.conjugate(X))) / X.size
```

The inverse relation follows from changing the sign in the complex exponential. Conjugation allows the same forward implementation to be reused:

$$\mathrm{IDFT}(\mathbf{X}) = \frac{1}{N}\left[\mathrm{DFT}(\mathbf{X}^*)\right]^* \tag{12.241}$$

The following helper forms a one-sided peak-amplitude spectrum of a real signal. It optionally removes the mean and applies a Hann window. The frequency array is constructed from the sample rate and record length using the bin relation derived above.

```python
import numpy as np


def One_Sided_Spectrum(x: np.ndarray,
                       sample_rate: float,
                       remove_mean: bool = True,
                       hann_window: bool = True):
    """Return frequency, peak amplitude and phase for a real signal.

    The sequence length must be a power of two because FFT_Radix2 is used.
    Amplitudes are corrected for the coherent gain of the selected window.
    """
    x = np.asarray(x, dtype=float).reshape(-1).copy()
    n_samples = x.size
```

```python

    if n_samples < 2:
        raise ValueError("at least two samples are required")
    if n_samples & (n_samples - 1):
        raise ValueError("the sequence length must be a power of two")
    if hann_window and n_samples < 4:
        raise ValueError("the Hann window requires at least four samples")
    if sample_rate <= 0.0:
        raise ValueError("sample_rate must be positive")

    if remove_mean:
        x -= np.sum(x) / n_samples

    if hann_window:
        n = np.arange(n_samples)
        window = 0.5 - 0.5 * np.cos(2.0 * np.pi * n / (n_samples - 1))
    else:
        window = np.ones(n_samples)

    coherent_gain = np.sum(window) / n_samples
    X = FFT_Radix2(x * window)

    n_positive = n_samples // 2 + 1
    X_positive = X[:n_positive]
    frequency = DFT_Frequencies(
        n_samples,
        sample_rate,
        one_sided=True,
    )

    amplitude = np.abs(X_positive) / (n_samples * coherent_gain)
    amplitude[1:-1] *= 2.0
    phase = np.angle(X_positive)

    return frequency, amplitude, phase
```

For example, a signal containing components at 50 Hz and 120 Hz may be transformed as follows:

```python
import numpy as np


sample_rate = 1024.0
n_samples = 1024
time = np.arange(n_samples) / sample_rate

signal = (3.0 * np.sin(2.0 * np.pi * 50.0 * time)
          + 0.8 * np.cos(2.0 * np.pi * 120.0 * time))

frequency, amplitude, phase = One_Sided_Spectrum(
    signal,
    sample_rate,
    remove_mean=True,
    hann_window=False,
)

# The largest bins occur at 50 Hz and 120 Hz with peak amplitudes 3.0 and 0.8.
indices = np.argsort(amplitude)[-2:]
for index in indices[np.argsort(frequency[indices])]:
    print(f"{frequency[index]:7.1f} Hz  {amplitude[index]:7.3f}")
```

Here the record length is one second, so $\Delta f = 1\,\mathrm{Hz}$, and both harmonics coincide exactly with DFT bins. No window is required for this constructed example. A general measured signal should not be expected to have this property.

12.7.8 Relation to Campbell Diagrams and Order Analysis

A single FFT describes one time record under approximately stationary operating conditions. A Campbell diagram repeats this local spectral calculation over a run-up, coast-down or sequence of operating points and retains rotational speed as a second coordinate. Order analysis then expresses frequency relative to the shaft rotational frequency. The following section develops both representations and explains when ordinary time blocks must be replaced by angular resampling.

12.8 Campbell Diagram and Order Analysis

Rotating machines rarely operate at only one constant speed. During a run-up or coast-down, excitation frequencies linked to the rotation move through the frequency range and may cross structural resonances. A single spectrum cannot show this development because it contains no operating-speed coordinate. Campbell diagrams and order maps retain that additional coordinate.

Two different plots are commonly called a **Campbell diagram**:

1. A *modal Campbell diagram* plots calculated natural frequencies against rotational speed. Gyroscopic systems may show separate forward- and backward-whirl branches.
2. A *response Campbell diagram* plots a measured or simulated spectral amplitude against frequency and rotational speed. It is a field, usually displayed as colour or contours.

Both plots use the same geometrical idea. Structural frequencies form branches $f_j(n_{\mathrm{rpm}})$, while rotational excitations form **order lines**. Their intersections identify speeds at which a rotating excitation can excite a structural mode.

12.8.1 Rotational Frequency and Orders

For a shaft speed n_{rpm} in revolutions per minute, the rotational frequency is:

$$f_{\mathrm{rot}} = \frac{n_{\mathrm{rpm}}}{60}. \tag{12.242}$$

An excitation at o cycles per revolution has frequency:

$$\boxed{f_o(n_{\mathrm{rpm}}) = o f_{\mathrm{rot}} = o\frac{n_{\mathrm{rpm}}}{60}.} \tag{12.243}$$

The dimensionless quantity o is the **order**. The first order, written 1x, repeats once per revolution. The second order repeats twice per revolution. A rotor with z equally spaced blades produces a blade-passing component at order z, and a gear with z teeth produces its mesh frequency at order z relative to that gear shaft.

A constant order is a straight line through the origin in a frequency–speed Campbell diagram. A speed-independent structural resonance is approximately horizontal. If a natural frequency f_n is independent of speed, its intersection with order o occurs at the critical speed:

$$n_{\text{critical}} = \frac{60 f_n}{o}. \tag{12.244}$$

For a rotating structure, the natural frequency may itself depend on speed. Centrifugal stiffening, stress softening and gyroscopic terms can curve or split its branch. The critical speed must then satisfy:

$$f_j(n_{\text{critical}}) = o \frac{n_{\text{critical}}}{60}. \tag{12.245}$$

An intersection is a possible resonance condition, not a prediction of response amplitude by itself. The actual response also depends on excitation strength, damping, mode shape, measurement position and the time spent near the crossing.

12.8.2 Response Campbell Diagram from Block Spectra

Let $x(t)$ be a measured response and let $n_{\text{rpm}}(t)$ be the corresponding shaft speed. The signal is divided into overlapping blocks. For block b, a windowed FFT gives a local complex spectrum:

$$X_b(f_k) = \text{DFT}\left\{w_m x_{b,m}\right\}. \tag{12.246}$$

Each block is assigned a representative speed, commonly the mean speed in the block:

$$\bar{n}_b = \frac{1}{N_b} \sum_{m=0}^{N_b-1} n_{b,m}. \tag{12.247}$$

The spectra are then arranged into the field:

$$A(\bar{n}_b, f_k) = |X_b(f_k)|. \tag{12.248}$$

Frequency is one axis, speed is the other axis, and amplitude is represented by colour or contours. The complex spectrum may also be retained so that phase and complex order tracks can be extracted later.

The speed slices should be sorted before constructing the plotted field. If several blocks have the same representative speed, their complex spectra may be averaged. Averaging magnitudes alone discards phase and generally gives a different result. A uniformly spaced speed axis can be produced afterwards by interpolating between the sorted slices. This *rebinning* improves the regularity of the displayed field but does not create additional physical resolution.

Run-up and coast-down data should normally be kept separate. Sorting both into one speed axis and averaging duplicate speeds can hide hysteresis, thermal drift, clearance changes or nonlinear jump phenomena that depend on sweep direction.

12.8.3 Frequency and Speed Resolution

For a block with N_b samples and sample rate f_s, the block duration and frequency spacing are:

$$T_b = \frac{N_b}{f_s}, \qquad \Delta f = \frac{f_s}{N_b} = \frac{1}{T_b}. \tag{12.249}$$

A longer block improves frequency resolution but includes a larger speed change. If the speed changes from $n_{\min,b}$ to $n_{\max,b}$ inside the block, the speed spread is:

$$\Delta n_b = n_{\max,b} - n_{\min,b}. \tag{12.250}$$

Order o consequently moves through approximately:

$$\Delta f_{o,b} \approx o \frac{\Delta n_b}{60} \tag{12.251}$$

within the block. When $\Delta f_{o,b}$ is comparable to or larger than a frequency bin, the order ridge is smeared even if the FFT itself is calculated correctly. Block length is therefore a compromise between frequency resolution and speed stationarity.

Overlap creates more speed slices and a smoother-looking diagram, but it does not change Δf. Strong overlap also means that neighbouring slices contain much of the same data and are not statistically independent.

The amplitude convention must be stated. The implementations below return a one-sided *peak-amplitude* spectrum corrected for the coherent gain of the Hann window. RMS amplitude would be smaller by $\sqrt{2}$ for a sinusoid. A decibel plot additionally requires a clearly stated reference value.

12.8.4 Python Implementation of a Campbell Field

The following helper calculates the one-sided complex amplitude of one block. It uses the radix-2 FFT introduced in the previous section. Keeping the complex values permits subsequent amplitude, phase and order calculations.

```
import numpy as np


def One_Sided_Complex_Amplitude(x: np.ndarray,
                                sample_rate: float,
                                remove_mean: bool = True,
                                hann_window: bool = True):
    """Return frequency and one-sided complex peak amplitude."""
```

```
    x = np.asarray(x, dtype=float).reshape(-1).copy()
    n_samples = x.size

    if n_samples < 2:
        raise ValueError("at least two samples are required")
    if n_samples & (n_samples - 1):
        raise ValueError("the block length must be a power of two")
    if sample_rate <= 0.0:
        raise ValueError("sample_rate must be positive")

    if remove_mean:
        x -= np.sum(x) / n_samples

    if hann_window:
        index = np.arange(n_samples)
        window = 0.5 - 0.5 * np.cos(
            2.0 * np.pi * index / (n_samples - 1)
        )
    else:
        window = np.ones(n_samples)

    coherent_gain = np.sum(window) / n_samples
    spectrum = FFT_Radix2(x * window)

    n_positive = n_samples // 2 + 1
    complex_amplitude = (
        spectrum[:n_positive] / (n_samples * coherent_gain)
    )

    # Interior positive-frequency bins represent both members of a
    # conjugate pair. DC and the Nyquist bin occur only once.
    complex_amplitude[1:-1] *= 2.0

    frequency = DFT_Frequencies(
        n_samples,
        sample_rate,
        one_sided=True,
    )
    return frequency, complex_amplitude
```

Before optional rebinning, the slices are sorted and duplicate speed values are averaged. A positive tolerance may be used to combine nearly equal speed values.

```python
import numpy as np


def Sort_And_Average_Speed_Slices(speed_rpm: np.ndarray,
                                  field: np.ndarray,
                                  speed_spread: np.ndarray,
                                  tolerance_rpm: float = 0.0):
    """Sort slices by speed and average duplicate or near-duplicate rows."""
    speed_rpm = np.asarray(speed_rpm, dtype=float).reshape(-1)
    field = np.asarray(field, dtype=complex)
    speed_spread = np.asarray(speed_spread, dtype=float).reshape(-1)

    if field.shape[0] != speed_rpm.size:
        raise ValueError("one field row is required for each speed")
    if speed_spread.size != speed_rpm.size:
        raise ValueError("one speed spread is required for each speed")
    if tolerance_rpm < 0.0:
        raise ValueError("tolerance_rpm must be non-negative")

    permutation = np.argsort(speed_rpm, kind="stable")
    speed_sorted = speed_rpm[permutation]
    field_sorted = field[permutation]
    spread_sorted = speed_spread[permutation]

    grouped_speed = []
    grouped_field = []
    grouped_spread = []

    first = 0
    while first < speed_sorted.size:
        last = first + 1
        while (last < speed_sorted.size
               and speed_sorted[last] - speed_sorted[first]
               <= tolerance_rpm):
            last += 1
```

```

        grouped_speed.append(np.mean(speed_sorted[first:last]))
        grouped_field.append(np.mean(field_sorted[first:last], axis=0))
        grouped_spread.append(np.max(spread_sorted[first:last]))
        first = last

    return (
        np.asarray(grouped_speed),
        np.asarray(grouped_field),
        np.asarray(grouped_spread),
    )


def Interpolate_Field_Rows(x: np.ndarray,
                           field: np.ndarray,
                           x_new: np.ndarray) -> np.ndarray:
    """Linearly interpolate every field column along its first axis."""
    x = np.asarray(x, dtype=float).reshape(-1)
    field = np.asarray(field)
    x_new = np.asarray(x_new, dtype=float).reshape(-1)

    if x.size < 2:
        raise ValueError("at least two source rows are required")
    if field.shape[0] != x.size:
        raise ValueError("field.shape[0] must equal x.size")
    if np.any(np.diff(x) <= 0.0):
        raise ValueError("x must be strictly increasing")
    if x_new[0] < x[0] or x_new[-1] > x[-1]:
        raise ValueError("x_new must remain inside the source interval")

    result = np.empty((x_new.size,) + field.shape[1:], dtype=field.dtype)

    for row, value in enumerate(x_new):
        right = np.searchsorted(x, value, side="right")
        if right == 0:
            result[row] = field[0]
        elif right == x.size:
            result[row] = field[-1]
        else:
            left = right - 1
            fraction = (value - x[left]) / (x[right] - x[left])
```

```python
            result[row] = (
                (1.0 - fraction) * field[left]
                + fraction * field[right]
            )

    return result
```

The Campbell construction collects the local spectra, records the speed spread of every block, sorts the slices and optionally rebins them to a uniform speed axis. The argument `resample=N` requests N uniformly spaced speed bins.

```python
import numpy as np


def Campbell_Diagram(signal: np.ndarray,
                      speed_rpm: np.ndarray,
                      sample_rate: float,
                      block_size: int,
                      overlap: float = 0.5,
                      duplicate_tolerance_rpm: float = 0.0,
                      resample: int | None = None):
    """Construct a complex response Campbell field."""
    signal = np.asarray(signal, dtype=float).reshape(-1)
    speed_rpm = np.asarray(speed_rpm, dtype=float).reshape(-1)

    if signal.size != speed_rpm.size:
        raise ValueError("signal and speed_rpm must have equal length")
    if block_size < 2 or block_size > signal.size:
        raise ValueError("invalid block_size")
    if block_size & (block_size - 1):
        raise ValueError("block_size must be a power of two")
    if not 0.0 <= overlap < 1.0:
        raise ValueError("overlap must lie in [0, 1)")

    hop_size = max(1, int(round(block_size * (1.0 - overlap))))
    starts = range(0, signal.size - block_size + 1, hop_size)

    block_speeds = []
```

```python
    speed_spreads = []
    spectra = []
    frequency = None

    for start in starts:
        stop = start + block_size
        block_speed = speed_rpm[start:stop]

        frequency, complex_amplitude = One_Sided_Complex_Amplitude(
            signal[start:stop],
            sample_rate,
            remove_mean=True,
            hann_window=True,
        )

        block_speeds.append(np.mean(block_speed))
        speed_spreads.append(np.max(block_speed) - np.min(block_speed))
        spectra.append(complex_amplitude)

    block_speeds, spectra, speed_spreads = (
        Sort_And_Average_Speed_Slices(
            np.asarray(block_speeds),
            np.asarray(spectra),
            np.asarray(speed_spreads),
            tolerance_rpm=duplicate_tolerance_rpm,
        )
    )

    if resample is not None:
        if resample < 2:
            raise ValueError("resample must be at least two")
        speed_uniform = np.linspace(
            block_speeds[0],
            block_speeds[-1],
            resample,
        )
        spectra = Interpolate_Field_Rows(
            block_speeds,
            spectra,
            speed_uniform,
        )
```

```
        speed_spreads = Interpolate_Field_Rows(
            block_speeds,
            speed_spreads[:, None],
            speed_uniform,
        )[:, 0]
        block_speeds = speed_uniform

    return {
        "speed_rpm": block_speeds,
        "frequency_hz": frequency,
        "complex_amplitude": spectra,
        "amplitude": np.abs(spectra),
        "speed_spread_rpm": speed_spreads,
    }
```

The returned speed-spread array is a useful diagnostic. A highly smeared region can be identified before interpreting broad spectral ridges as damping or as multiple physical components.

12.8.5 Example of a Response Campbell Diagram

The following synthetic run-up contains a first-order component, a second-order component amplified near a 100 Hz structural resonance, a sixth order and a stationary 140 Hz tone. The rotational angle is obtained by trapezoidal integration of the speed.

```
import numpy as np
import matplotlib.pyplot as plt


def Rotational_Angle(time: np.ndarray,
                     speed_rpm: np.ndarray) -> np.ndarray:
    """Integrate rotational speed to angle in radians."""
    time = np.asarray(time, dtype=float).reshape(-1)
    speed_rpm = np.asarray(speed_rpm, dtype=float).reshape(-1)

```

```
    if time.size != speed_rpm.size:
        raise ValueError("time and speed_rpm must have equal length")
    if time.size < 2 or np.any(np.diff(time) <= 0.0):
        raise ValueError("time must be strictly increasing")

    angular_speed = 2.0 * np.pi * speed_rpm / 60.0
    angle = np.zeros(time.size)
    angle[1:] = np.cumsum(
        0.5
        * (angular_speed[1:] + angular_speed[:-1])
        * np.diff(time)
    )
    return angle


sample_rate = 2048.0
duration = 8.0
time = np.arange(int(sample_rate * duration)) / sample_rate
speed_rpm = 600.0 + 3600.0 * time / duration
angle = Rotational_Angle(time, speed_rpm)

second_order_frequency = 2.0 * speed_rpm / 60.0
natural_frequency = 100.0
damping_ratio = 0.04
frequency_ratio = second_order_frequency / natural_frequency
resonance = 1.0 / np.sqrt(
    (1.0 - frequency_ratio**2)**2
    + (2.0 * damping_ratio * frequency_ratio)**2
)
resonance /= np.max(resonance)

signal = (
    0.45 * np.sin(angle)
    + 1.20 * resonance * np.sin(2.0 * angle)
    + 0.20 * np.sin(6.0 * angle)
    + 0.12 * np.sin(2.0 * np.pi * 140.0 * time)
)

campbell = Campbell_Diagram(
    signal,
    speed_rpm,
```

```
    sample_rate,
    block_size=1024,
    overlap=0.75,
    resample=120,
)

reference = np.max(campbell["amplitude"])
level_db = 20.0 * np.log10(
    np.maximum(campbell["amplitude"] / reference, 1.0e-6)
)

plt.figure()
plt.pcolormesh(
    campbell["speed_rpm"],
    campbell["frequency_hz"],
    level_db.T,
    shading="auto",
)
for order in (1.0, 2.0, 6.0):
    plt.plot(
        campbell["speed_rpm"],
        order * campbell["speed_rpm"] / 60.0,
        linewidth=1.0,
    )
plt.xlabel("Rotational speed [rpm]")
plt.ylabel("Frequency [Hz]")
plt.ylim(0.0, 180.0)
plt.colorbar(label="Relative amplitude [dB]")
plt.tight_layout()
plt.show()
```

In the frequency–speed plane, the three shaft-synchronous components follow straight order lines. The fixed 140 Hz component remains horizontal. The second-order ridge becomes strongest near:

$$n_{\text{critical}} = \frac{60 \cdot 100}{2} = 3000\,\text{rpm}. \tag{12.252}$$

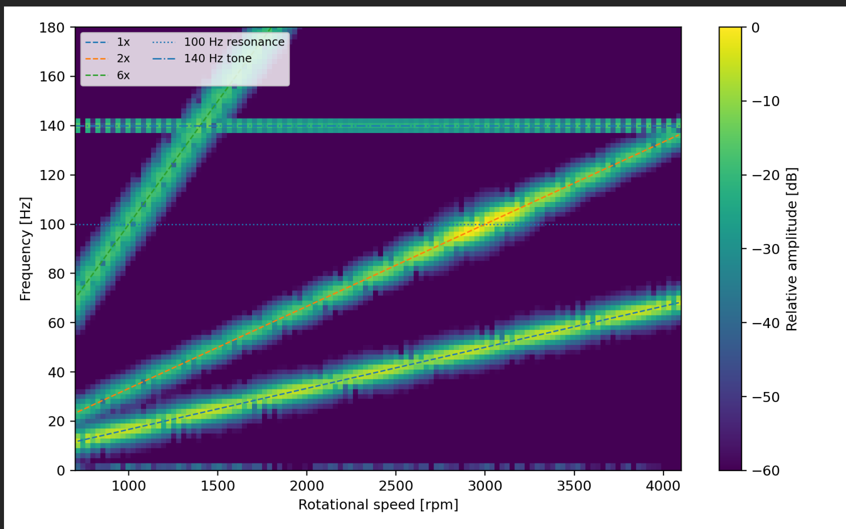


Figure 12.2: Synthetic response Campbell diagram. The first, second and sixth orders rise linearly with speed. The second order is amplified where it crosses the 100 Hz resonance, while the stationary 140 Hz tone remains horizontal.

12.8.6 Waterfall Representation of the Response Campbell Field

The response Campbell field in Equation (12.248) may also be displayed as a **waterfall diagram**. No new transform is performed. Each row of the Campbell field is drawn as an individual spectrum at its corresponding speed:

$$\mathcal{W}_b = \{(f_k, \bar{n}_b, A(\bar{n}_b, f_k)) \mid k = 0, \ldots, N_f - 1\}\,. \tag{12.253}$$

The frequency axis remains horizontal, the spectra are separated along the speed axis, and amplitude is shown by the height of each curve. A waterfall is therefore a three-dimensional perspective view of the same data that a response Campbell diagram displays by colour or contours. A time waterfall can be formed in exactly the same manner by replacing speed with block-centre time.

The two views emphasise different properties. A colour map makes ridge geometry, order lines and weak components easier to compare quantitatively. A waterfall makes the individual spectra and the growth and decay of resonant peaks more intuitive. Perspective, curve overlap and the selected viewing angle can, however, hide weaker components behind stronger curves. Only every few speed slices are therefore commonly drawn.

When amplitudes are converted to decibels, every slice should use the same global reference. Normalising each spectrum separately would make every speed slice appear equally strong and would remove the amplitude increase at a resonance. The chosen decibel floor also affects the visual appearance but not the underlying Campbell field.

```python
import numpy as np
import matplotlib.pyplot as plt


def Plot_Campbell_Waterfall(campbell: dict,
                            maximum_frequency: float | None = None,
                            stride: int = 5,
                            decibel: bool = True,
                            floor_db: float = -60.0):
    """Plot selected speed slices of a response Campbell field."""
    speed_rpm = np.asarray(campbell["speed_rpm"], dtype=float)
    frequency = np.asarray(campbell["frequency_hz"], dtype=float)
    amplitude = np.asarray(campbell["amplitude"], dtype=float)

    if amplitude.shape != (speed_rpm.size, frequency.size):
        raise ValueError("the Campbell field dimensions are inconsistent")
    if stride < 1:
        raise ValueError("stride must be positive")

    if maximum_frequency is None:
        maximum_frequency = frequency[-1]
    frequency_mask = frequency <= maximum_frequency
    frequency_plot = frequency[frequency_mask]
    amplitude_plot = amplitude[:, frequency_mask]

    if decibel:
```

```
        reference = np.max(amplitude_plot)
        if reference <= 0.0:
            raise ValueError("the Campbell field contains no positive amplitude")
        minimum_ratio = 10.0**(floor_db / 20.0)
        ordinate = 20.0 * np.log10(
            np.maximum(amplitude_plot / reference, minimum_ratio)
        )
        amplitude_label = "Relative amplitude [dB]"
    else:
        ordinate = amplitude_plot
        amplitude_label = "Peak amplitude"

    figure = plt.figure()
    axis = figure.add_subplot(111, projection="3d")

    rows = list(range(0, speed_rpm.size, stride))
    if rows[-1] != speed_rpm.size - 1:
        rows.append(speed_rpm.size - 1)

    for row in rows:
        speed_line = np.full(frequency_plot.size, speed_rpm[row])
        axis.plot(
            frequency_plot,
            speed_line,
            ordinate[row],
            linewidth=0.8,
        )

    axis.set_xlabel("Frequency [Hz]")
    axis.set_ylabel("Rotational speed [rpm]")
    axis.set_zlabel(amplitude_label)
    figure.tight_layout()
    return figure, axis


Plot_Campbell_Waterfall(
    campbell,
    maximum_frequency=180.0,
    stride=6,
    decibel=True,
)
```

```
plt.show()
```

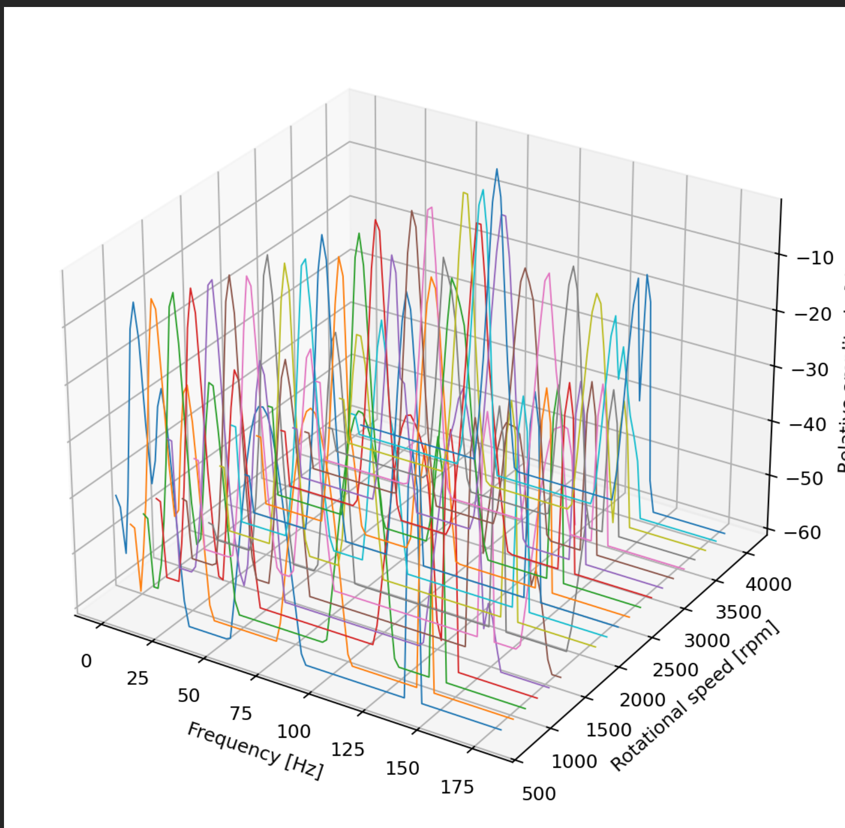


Figure 12.3: Waterfall representation of the synthetic response Campbell field. Each line is one spectrum at one rotational speed. The order-related peaks move in frequency, while the stationary 140 Hz component remains at the same frequency.

The response Campbell colour map in Figure 12.2 and the waterfall in Figure 12.3 contain the same spectral slices. They should not be confused with rainflow cycle counting, which starts from a load or stress history and answers a fatigue question rather than a frequency-content question.

12.8.7 Order Map Obtained from a Campbell Field

The order corresponding to frequency f at speed n_{rpm} is:

$$o = \frac{f}{f_{\mathrm{rot}}} = \frac{60f}{n_{\mathrm{rpm}}}. \tag{12.254}$$

A Campbell field can therefore be sampled along the frequency curves $f = on_{\mathrm{rpm}}/60$ to form an order map:

$$A_o(n_{\mathrm{rpm}}, o) = A\left(n_{\mathrm{rpm}}, o\frac{n_{\mathrm{rpm}}}{60}\right). \tag{12.255}$$

Because the required frequency usually lies between FFT bins, interpolation is necessary. Interpolating the complex spectrum retains amplitude and phase more consistently than separately interpolating magnitude and wrapped phase.

This conversion changes coordinates but does not remove spectral smearing that already occurred inside the time blocks. It is suitable when speed is nearly constant within each block. Angular resampling, introduced later, is preferable for rapid speed changes.

```python
import numpy as np


def Interpolate_Complex_Spectrum(frequency: np.ndarray,
                                 spectrum: np.ndarray,
                                 target_frequency: np.ndarray) -> np.ndarray:
    """Interpolate a complex spectrum at requested frequencies."""
    frequency = np.asarray(frequency, dtype=float).reshape(-1)
    spectrum = np.asarray(spectrum, dtype=complex).reshape(-1)
    target_frequency = np.asarray(target_frequency, dtype=float).reshape(-1)

```

```python
    if spectrum.size != frequency.size:
        raise ValueError("spectrum and frequency must have equal length")
    if np.any(np.diff(frequency) <= 0.0):
        raise ValueError("frequency must be strictly increasing")

    result = np.full(target_frequency.size, np.nan + 1.0j * np.nan)

    for index, value in enumerate(target_frequency):
        if value < frequency[0] or value > frequency[-1]:
            continue

        right = np.searchsorted(frequency, value, side="right")
        if right == 0:
            result[index] = spectrum[0]
        elif right == frequency.size:
            result[index] = spectrum[-1]
        else:
            left = right - 1
            fraction = (
                (value - frequency[left])
                / (frequency[right] - frequency[left])
            )
            result[index] = (
                (1.0 - fraction) * spectrum[left]
                + fraction * spectrum[right]
            )

    return result


def Campbell_To_Order_Map(campbell: dict,
                          n_order_bins: int = 401,
                          maximum_order: float | None = None):
    """Interpolate a complex Campbell field onto a common order axis."""
    speed_rpm = np.asarray(campbell["speed_rpm"], dtype=float)
    frequency = np.asarray(campbell["frequency_hz"], dtype=float)
    field = np.asarray(campbell["complex_amplitude"], dtype=complex)

    if np.any(speed_rpm <= 0.0):
        raise ValueError("order conversion requires positive speed")
    if n_order_bins < 2:
```

```
        raise ValueError("n_order_bins must be at least two")

    common_limit = frequency[-1] / np.max(speed_rpm / 60.0)
    if maximum_order is None:
        maximum_order = common_limit
    if maximum_order <= 0.0 or maximum_order > common_limit:
        raise ValueError("maximum_order exceeds the common frequency range")

    order = np.linspace(0.0, maximum_order, n_order_bins)
    order_field = np.empty((speed_rpm.size, order.size), dtype=complex)

    for row, speed in enumerate(speed_rpm):
        target_frequency = order * speed / 60.0
        order_field[row] = Interpolate_Complex_Spectrum(
            frequency,
            field[row],
            target_frequency,
        )

    return {
        "speed_rpm": speed_rpm,
        "order": order,
        "complex_amplitude": order_field,
        "amplitude": np.abs(order_field),
    }
```

In an order map, shaft-synchronous excitations become approximately horizontal. A speed-independent structural resonance follows:

$$o_n(n_{\mathrm{rpm}}) = \frac{60 f_n}{n_{\mathrm{rpm}}}, \tag{12.256}$$

and therefore curves downward as speed increases. This is the complementary view of the frequency–speed Campbell diagram: orders are horizontal and fixed frequencies are curved.

```
import numpy as np
import matplotlib.pyplot as plt
```

```python


order_map = Campbell_To_Order_Map(
    campbell,
    n_order_bins=481,
    maximum_order=8.0,
)

reference = np.nanmax(order_map["amplitude"])
level_db = 20.0 * np.log10(
    np.maximum(order_map["amplitude"] / reference, 1.0e-6)
)

plt.figure()
plt.pcolormesh(
    order_map["speed_rpm"],
    order_map["order"],
    level_db.T,
    shading="auto",
)
plt.xlabel("Rotational speed [rpm]")
plt.ylabel("Order [-]")
plt.ylim(0.0, 8.0)
plt.colorbar(label="Relative amplitude [dB]")
plt.tight_layout()
plt.show()
```

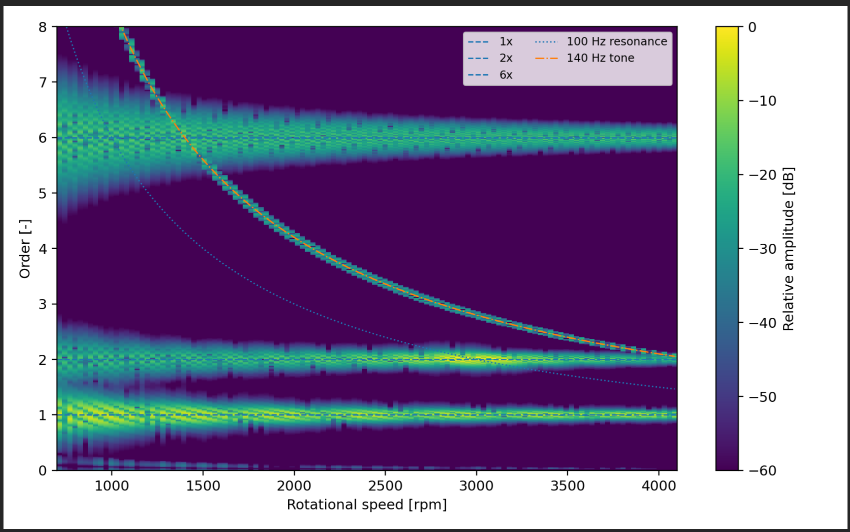


Figure 12.4: Order map obtained from the synthetic Campbell field. The first, second and sixth orders are horizontal. The stationary 140 Hz component curves towards lower order as speed increases.

12.8.8 Extraction of Individual Order Tracks

Often the full field is less useful than the amplitude and phase of selected orders. For each speed slice, the complex spectrum is interpolated at the order frequency from Equation (12.243). The resulting complex track contains both quantities:

$$a_o(n) = |X_o(n)|, \qquad \varphi_o(n) = \arg X_o(n). \tag{12.257}$$

```python
import numpy as np


def Extract_Order_Tracks(campbell: dict,
                         orders: np.ndarray) -> dict:
    """Extract complex order tracks from a Campbell field."""
```

```
    speed_rpm = np.asarray(campbell["speed_rpm"], dtype=float)
    frequency = np.asarray(campbell["frequency_hz"], dtype=float)
    field = np.asarray(campbell["complex_amplitude"], dtype=complex)
    orders = np.asarray(orders, dtype=float).reshape(-1)

    tracks = np.empty((speed_rpm.size, orders.size), dtype=complex)

    for row, speed in enumerate(speed_rpm):
        target_frequency = orders * speed / 60.0
        tracks[row] = Interpolate_Complex_Spectrum(
            frequency,
            field[row],
            target_frequency,
        )

    return {
        "speed_rpm": speed_rpm,
        "order": orders,
        "complex_amplitude": tracks,
        "amplitude": np.abs(tracks),
        "phase": np.angle(tracks),
    }


tracks = Extract_Order_Tracks(
    campbell,
    orders=np.array([1.0, 2.0, 6.0]),
)
```

Phase is meaningful only relative to a consistent rotational reference. Speed obtained from a slowly varying analogue channel may be sufficient for amplitude tracking but not for absolute order phase. A once-per-revolution tachometer or encoder reference is required when phase relative to the shaft is important.

12.8.9 Angular Resampling for Order Tracking

During a rapid speed sweep, equal time increments do not correspond to equal shaft-angle increments. A time-domain sinusoid of constant order is therefore a chirp. A block FFT assumes approximately stationary frequency and smears the component if its frequency changes appreciably inside the block.

Order tracking removes this motion by changing the independent variable from time to rotational angle. The angle is obtained from the rotational speed:

$$\theta(t) = \theta_0 + \int_{t_0}^{t} 2\pi \frac{n_{\mathrm{rpm}}(\tau)}{60}\,\mathrm{d}\tau. \tag{12.258}$$

The response is then interpolated at uniformly spaced angles:

$$\theta_m = \theta_0 + m\Delta\theta, \qquad x_m^{(\theta)} = x\left(t(\theta_m)\right). \tag{12.259}$$

A component at order o becomes an ordinary sinusoid in angle:

$$x^{(\theta)}(\theta) = A\cos(o\theta + \varphi). \tag{12.260}$$

The following implementation performs trapezoidal angle integration and linear interpolation onto a uniform angular grid. It requires monotonically increasing angle. Data containing a reversal or a prolonged zero-speed interval should be split into monotonic rotational segments.

```python
import numpy as np


def Linear_Interpolate_Monotone(x: np.ndarray,
                                y: np.ndarray,
                                x_new: np.ndarray) -> np.ndarray:
    """Linear interpolation for a strictly increasing source coordinate."""
    x = np.asarray(x, dtype=float).reshape(-1)
    y = np.asarray(y).reshape(-1)
    x_new = np.asarray(x_new, dtype=float).reshape(-1)

    if y.size != x.size:
        raise ValueError("x and y must have equal length")
```

```python
    if np.any(np.diff(x) <= 0.0):
        raise ValueError("x must be strictly increasing")
    if x_new[0] < x[0] or x_new[-1] > x[-1]:
        raise ValueError("x_new must remain inside the source interval")

    result = np.empty(x_new.size, dtype=np.result_type(y, float))

    for index, value in enumerate(x_new):
        right = np.searchsorted(x, value, side="right")
        if right == 0:
            result[index] = y[0]
        elif right == x.size:
            result[index] = y[-1]
        else:
            left = right - 1
            fraction = (value - x[left]) / (x[right] - x[left])
            result[index] = (
                (1.0 - fraction) * y[left]
                + fraction * y[right]
            )

    return result


def Angular_Resample(time: np.ndarray,
                     signal: np.ndarray,
                     speed_rpm: np.ndarray,
                     samples_per_revolution: int):
    """Resample a time signal at uniform shaft-angle increments."""
    time = np.asarray(time, dtype=float).reshape(-1)
    signal = np.asarray(signal, dtype=float).reshape(-1)
    speed_rpm = np.asarray(speed_rpm, dtype=float).reshape(-1)

    if signal.size != time.size or speed_rpm.size != time.size:
        raise ValueError("time, signal and speed_rpm must have equal length")
    if samples_per_revolution < 2:
        raise ValueError("samples_per_revolution must be at least two")

    angle = Rotational_Angle(time, speed_rpm)
    if np.any(np.diff(angle) <= 0.0):
        raise ValueError("rotational angle must be strictly increasing")
```

```python

    angle_step = 2.0 * np.pi / samples_per_revolution
    first_index = int(np.ceil(angle[0] / angle_step))
    last_index = int(np.floor(angle[-1] / angle_step))
    angle_uniform = (
        np.arange(first_index, last_index + 1) * angle_step
    )

    signal_angle = Linear_Interpolate_Monotone(
        angle,
        signal,
        angle_uniform,
    )
    speed_angle = Linear_Interpolate_Monotone(
        angle,
        speed_rpm,
        angle_uniform,
    )

    return angle_uniform, signal_angle, speed_angle
```

If the angular signal has M samples per revolution and a block spans R revolutions, then:

$$N_b = MR, \qquad \Delta o = \frac{1}{R}, \qquad o_{\text{Nyquist}} = \frac{M}{2}. \tag{12.261}$$

Thus the number of revolutions per block controls order resolution, while the number of samples per revolution controls the highest representable order. This is the angular-domain counterpart of time-record duration and sample rate.

```python
import numpy as np


def Angular_Order_Map(time: np.ndarray,
                      signal: np.ndarray,
                      speed_rpm: np.ndarray,
```

```
                        samples_per_revolution: int,
                        revolutions_per_block: int,
                        overlap: float = 0.5,
                        duplicate_tolerance_rpm: float = 0.0,
                        resample: int | None = None):
    """Construct an order map after uniform-angle resampling."""
    _, signal_angle, speed_angle = Angular_Resample(
        time,
        signal,
        speed_rpm,
        samples_per_revolution,
    )

    block_size = samples_per_revolution * revolutions_per_block
    if block_size < 2 or block_size & (block_size - 1):
        raise ValueError(
            "samples_per_revolution * revolutions_per_block "
            "must be a power of two"
        )
    if block_size > signal_angle.size:
        raise ValueError("the angular record is shorter than one block")
    if not 0.0 <= overlap < 1.0:
        raise ValueError("overlap must lie in [0, 1)")

    hop_size = max(1, int(round(block_size * (1.0 - overlap))))
    starts = range(0, signal_angle.size - block_size + 1, hop_size)

    index = np.arange(block_size)
    window = 0.5 - 0.5 * np.cos(
        2.0 * np.pi * index / (block_size - 1)
    )
    coherent_gain = np.sum(window) / block_size

    order = (
        np.arange(block_size // 2 + 1)
        / revolutions_per_block
    )
    block_speeds = []
    speed_spreads = []
    spectra = []

```

```
    for start in starts:
        stop = start + block_size
        block = signal_angle[start:stop].copy()
        block -= np.mean(block)

        spectrum = FFT_Radix2(block * window)
        complex_amplitude = (
            spectrum[:block_size // 2 + 1]
            / (block_size * coherent_gain)
        )
        complex_amplitude[1:-1] *= 2.0

        local_speed = speed_angle[start:stop]
        block_speeds.append(np.mean(local_speed))
        speed_spreads.append(np.max(local_speed) - np.min(local_speed))
        spectra.append(complex_amplitude)

    block_speeds, spectra, speed_spreads = (
        Sort_And_Average_Speed_Slices(
            np.asarray(block_speeds),
            np.asarray(spectra),
            np.asarray(speed_spreads),
            tolerance_rpm=duplicate_tolerance_rpm,
        )
    )

    if resample is not None:
        if resample < 2:
            raise ValueError("resample must be at least two")
        speed_uniform = np.linspace(
            block_speeds[0],
            block_speeds[-1],
            resample,
        )
        spectra = Interpolate_Field_Rows(
            block_speeds,
            spectra,
            speed_uniform,
        )
        speed_spreads = Interpolate_Field_Rows(
            block_speeds,
```

```
            speed_spreads[:, None],
            speed_uniform,
        )[:, 0]
        block_speeds = speed_uniform

    return {
        "speed_rpm": block_speeds,
        "order": order,
        "complex_amplitude": spectra,
        "amplitude": np.abs(spectra),
        "speed_spread_rpm": speed_spreads,
    }
```

For the synthetic run-up, choosing 64 samples per revolution and 16 revolutions per block gives:

$$N_b = 1024, \qquad \Delta o = 0.0625, \qquad o_{\mathrm{Nyquist}} = 32. \tag{12.262}$$

The first, second and sixth orders then remain at fixed order positions despite the changing speed.

```
order_map_angular = Angular_Order_Map(
    time,
    signal,
    speed_rpm,
    samples_per_revolution=64,
    revolutions_per_block=16,
    overlap=0.75,
    resample=120,
)
```

12.8.10 Tachometer Processing and Practical Limitations

Angular resampling is only as accurate as the estimated angle. Integrating a noisy speed signal accumulates phase error. A once-per-revolution tachometer provides discrete angular reference events and prevents long-term drift. An encoder with several pulses per revolution provides more local information, but missing or false pulses must be detected before interpolation.

A practical procedure is:

1. detect and validate tachometer edges;
2. assign a known angular increment to consecutive edges;
3. interpolate angle monotonically between the edges;
4. resample the response at uniform angular increments;
5. apply an angular-domain window and FFT;
6. associate each block with its mean speed and speed spread.

The original time sampling must be high enough for the largest frequency present at the highest speed. Angular resampling cannot recover a component that was already aliased in the time-domain acquisition. Conversely, at low speed a fixed number of angular samples per revolution may correspond to a low effective time sample rate. Anti-alias filtering and the selected speed range must therefore be considered together.

If two shafts rotate at different speeds, the stated order always refers to one chosen reference shaft. A component that is synchronous with another shaft may appear as a non-integer or speed-dependent order. Gear ratios, slip and variable transmission ratios should be applied explicitly rather than assuming that every ridge belongs to the reference shaft.

12.8.11 Interpretation and Validation

A Campbell or order map should be checked against the underlying time and speed data. Useful validation steps include:

1. Verify the FFT frequency bins, amplitude convention and window gain on a harmonic of known amplitude.

2. Plot the speed spread of every block. Broad ridges in blocks with large Δn_b may be processing smear rather than physical damping.

3. Overlay expected order lines and known structural frequencies. A ridge should follow the appropriate geometry in both frequency and order views.

4. Extract selected complex order tracks and compare them with direct narrow-band or synchronous calculations.

5. Process run-up and coast-down separately before deciding whether they can be combined.

6. Confirm that speed sorting, duplicate averaging and optional uniform speed rebinning do not alter integrated amplitudes unexpectedly.

The frequency–speed Campbell diagram and the speed–order map are not competing representations. They emphasize different structures. A constant order is easy to identify in the order map, while a speed-independent resonance is easy to identify in the frequency Campbell diagram. Interpreting both views together is usually more reliable than relying on either one alone.

12.9 Rainflow Cycle Counting

A **rainflow analysis** is not another representation of a Campbell field. Campbell and waterfall plots describe how spectral content varies with speed or time. Rainflow counting operates directly on a scalar load, strain or stress history and reduces it to fatigue cycles.

Method	**Coordinates or output**	**Main purpose**
Response Campbell	speed, frequency and amplitude	rotating-vibration diagnosis
Waterfall	stacked spectra versus speed or time	perspective view of spectra
Rainflow counting	range, mean and cycle count	fatigue-cycle description

The conventional rainflow result does not retain frequency, phase or the exact time ordering of the cycles. If cycle counts must also be associated with speed, temperature or another operating state, the record must be divided or each counted cycle must be linked back to the relevant interval before this information is discarded.

12.9.1 Ranges, Means and Half Cycles

Let $s(t)$ be a scalar stress history. Consecutive equal values and samples that are not local extrema are first removed. The remaining reversal sequence is:

$$r_0, r_1, \ldots, r_{N_r-1}. \tag{12.263}$$

A counted excursion between reversal values r_a and r_b is described by its range, amplitude and mean:

$$\Delta s = |r_b - r_a|, \qquad s_a = \frac{\Delta s}{2}, \qquad s_m = \frac{r_a + r_b}{2}. \tag{12.264}$$

A closed interior cycle has count $c = 1$. An unclosed excursion at the beginning or end of a finite record has count $c = 1/2$. Two identical half cycles therefore contribute the same total count as one full cycle. Whether residual half cycles should be retained, repeated to close a periodic record or treated by another record-specific convention must be stated.

Rainflow counting is often explained by imagining rain flowing down a pagoda roof formed by the rotated stress history. Numerically, a stack of reversals is more direct. When a new reversal arrives, the two most recent ranges are compared:

$$R_{\text{old}} = |r_{j-1} - r_{j-2}|, \qquad R_{\text{new}} = |r_j - r_{j-1}|. \tag{12.265}$$

If $R_{\text{new}} < R_{\text{old}}$, the older excursion remains open. If $R_{\text{new}} \geq R_{\text{old}}$, the older excursion is closed and counted. An excursion touching the beginning of the current stack is counted as a half cycle; an enclosed excursion is counted as a full cycle. After all reversals have been processed, the adjacent residual excursions are counted as half cycles.

12.9.2 Python Implementation of Rainflow Counting

The following implementation returns one row per counted cycle. The columns are cycle range, cycle mean and count. It deliberately retains half cycles rather than silently forcing the record to be periodic.

```python
import numpy as np


def Reversal_Points(signal: np.ndarray) -> np.ndarray:
    """Return endpoints and local extrema after removing plateaus."""
    values = np.asarray(signal, dtype=float).reshape(-1)
    if values.size < 2:
        raise ValueError("at least two samples are required")

    compact = [values[0]]
    for value in values[1:]:
        if value != compact[-1]:
            compact.append(value)

    compact = np.asarray(compact)
    if compact.size == 1:
        return compact.copy()

    reversals = [compact[0]]
    for index in range(1, compact.size - 1):
        incoming = compact[index] - compact[index - 1]
        outgoing = compact[index + 1] - compact[index]
        if incoming * outgoing < 0.0:
            reversals.append(compact[index])
    reversals.append(compact[-1])
    return np.asarray(reversals)


def Rainflow_Cycles(signal: np.ndarray) -> np.ndarray:
    """Return rows [range, mean, count] using a reversal stack."""
    reversals = Reversal_Points(signal)
    stack = []
    cycles = []
```

```python

    for value in reversals:
        stack.append(float(value))

        while len(stack) >= 3:
            old_range = abs(stack[-2] - stack[-3])
            new_range = abs(stack[-1] - stack[-2])

            if new_range < old_range:
                break

            cycle_mean = 0.5 * (stack[-3] + stack[-2])

            if len(stack) == 3:
                if old_range > 0.0:
                    cycles.append([old_range, cycle_mean, 0.5])
                stack.pop(0)
            else:
                if old_range > 0.0:
                    cycles.append([old_range, cycle_mean, 1.0])
                last = stack.pop()
                stack.pop()
                stack.pop()
                stack.append(last)

    for first, second in zip(stack[:-1], stack[1:]):
        cycle_range = abs(second - first)
        if cycle_range > 0.0:
            cycles.append([
                cycle_range,
                0.5 * (first + second),
                0.5,
            ])

    if not cycles:
        return np.empty((0, 3), dtype=float)
    return np.asarray(cycles, dtype=float)


triangular_history = np.array([0.0, 10.0, 0.0])
print(Rainflow_Cycles(triangular_history))
```

```
# Two boundary half cycles of range 10; their total count is one.
```

A constant or monotonic record contains no closed interior cycles. Its total change appears only as a residual half cycle. This is appropriate because a single ramp has not completed a load reversal.

12.9.3 Rainflow Matrix

Individual cycles are commonly collected into a two-dimensional histogram. Let ρ_i be cycle-range bin edges and μ_j mean-value bin edges. The rainflow-matrix entry is:

$$N_{ij} = \sum_c c_c \, \mathbf{1}(\rho_i \le \Delta s_c < \rho_{i+1}) \, \mathbf{1}(\mu_j \le s_{m,c} < \mu_{j+1}) \,, \tag{12.266}$$

where c_c is 1 for a full cycle and 1/2 for a half cycle. Summing the matrix gives the total number of equivalent full cycles, including the fractional contribution of residual excursions.

```python
import numpy as np


def Rainflow_Matrix(cycles: np.ndarray,
                    range_edges: np.ndarray,
                    mean_edges: np.ndarray) -> np.ndarray:
    """Bin rows [range, mean, count] into a rainflow matrix."""
    cycles = np.asarray(cycles, dtype=float)
    range_edges = np.asarray(range_edges, dtype=float).reshape(-1)
    mean_edges = np.asarray(mean_edges, dtype=float).reshape(-1)

    if cycles.ndim != 2 or cycles.shape[1] != 3:
        raise ValueError("cycles must contain [range, mean, count] rows")
    if np.any(np.diff(range_edges) <= 0.0):
        raise ValueError("range_edges must be strictly increasing")
```

```python
    if np.any(np.diff(mean_edges) <= 0.0):
        raise ValueError("mean_edges must be strictly increasing")

    matrix = np.zeros((range_edges.size - 1, mean_edges.size - 1))

    for cycle_range, cycle_mean, count in cycles:
        range_bin = np.searchsorted(
            range_edges,
            cycle_range,
            side="right",
        ) - 1
        mean_bin = np.searchsorted(
            mean_edges,
            cycle_mean,
            side="right",
        ) - 1

        if cycle_range == range_edges[-1]:
            range_bin = matrix.shape[0] - 1
        if cycle_mean == mean_edges[-1]:
            mean_bin = matrix.shape[1] - 1

        if (0 <= range_bin < matrix.shape[0]
                and 0 <= mean_bin < matrix.shape[1]):
            matrix[range_bin, mean_bin] += count

    return matrix
```

The example below contains oscillations with changing amplitude and mean value. Its time history and resulting rainflow matrix are shown together. The matrix contains cycle counts, not spectral amplitude.

```python
import numpy as np
import matplotlib.pyplot as plt


time_rf = np.linspace(0.0, 20.0, 4001)
envelope = 65.0 + 20.0 * np.sin(2.0 * np.pi * 0.08 * time_rf)
```

```
mean_stress = 25.0 * np.sin(2.0 * np.pi * 0.03 * time_rf)
stress = (
    mean_stress
    + envelope * np.sin(2.0 * np.pi * 1.4 * time_rf)
    + 18.0 * np.sin(2.0 * np.pi * 0.37 * time_rf)
)

cycles = Rainflow_Cycles(stress)
range_edges = np.linspace(0.0, 1.001 * np.max(cycles[:, 0]), 17)
mean_edges = np.linspace(
    np.min(cycles[:, 1]) - 1.0e-9,
    np.max(cycles[:, 1]) + 1.0e-9,
    17,
)
counts = Rainflow_Matrix(cycles, range_edges, mean_edges)

figure, axes = plt.subplots(2, 1)
axes[0].plot(time_rf, stress)
axes[0].set_xlabel("Time [s]")
axes[0].set_ylabel("Stress")

mesh = axes[1].pcolormesh(
    mean_edges,
    range_edges,
    counts,
    shading="flat",
)
axes[1].set_xlabel("Cycle mean")
axes[1].set_ylabel("Cycle range")
figure.colorbar(mesh, ax=axes[1], label="Cycle count")
figure.tight_layout()
plt.show()
```

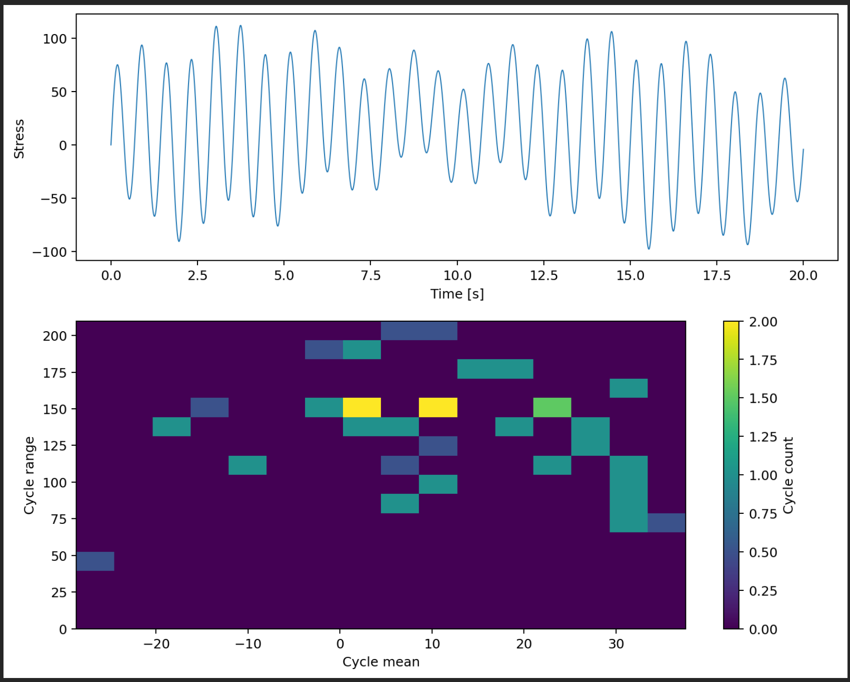


Figure 12.5: Variable stress history and its rainflow matrix. Each matrix cell accumulates full and half cycles having a similar range and mean value. The result contains no frequency or rotational-speed coordinate.

12.9.4 Relation to Fatigue Damage

Rainflow counting supplies the cycle ranges, means and counts required by a fatigue model; it does not calculate damage by itself. For a stress-life curve that assigns an allowable life N_c to each corrected cycle amplitude, linear Palmgren–Miner accumulation is:

$$D = \sum_c \frac{c_c}{N_c}. \tag{12.267}$$

Before looking up N_c, a mean-stress correction may be required because two cycles with the same range but different means need not cause the same damage. Material data, surface condition, stress concentration, multiaxial state and the chosen fatigue model remain separate modelling inputs. The rainflow matrix is therefore an economical description of the load history, not a complete fatigue assessment.

12.10 Power Spectral Density, Cross Spectral Density and Coherence

12.11 Sorting

12.12 Reverse Cuthill-McKee Algorithm for rearranging Sparse matrix

12.13 Topology Optimization

12.14 Generalised Eigenvalue Problem

Chapter 13

Miscellaneous

13.1 Bolt

13.1.1 Bolt pretension

The relation between the **tightening moment** and bolt **pretension force** can be characterised by the following equation:

$$M_A = F_V \left[\frac{d_2}{2} \tan\left(\varphi^\circ + \rho'\right) + \mu_K \frac{d_{K,R}}{2} \right] \tag{13.1}$$

with:

$$d_{K,R} = \frac{d_k + d_i}{2} \tag{13.2}$$

$$\varphi^\circ = \arctan\left(\frac{P}{d_2 * \pi} \right) \tag{13.3}$$

$$\rho' = \arctan\left(\frac{\mu_G}{\cos\frac{\beta}{2}}\right) \tag{13.4}$$

at standard thread angle of $\beta = 60°$:

$$\rho' = \arctan\left(\mu_G * 1.155\right) \tag{13.5}$$

where:
M_A ... Tightening Moment [Nmm]
F_V ... Pretension Force [N]
d_2 ... Bolt thread midiameter ((core + outer diameter) / 2) [mm]
ρ' ... Bolt thread friction angle [°]
$\varphi°$ ... Bolt thread angle of rising (pitch) [°]
P ... Bolt thread pitch [mm]
μ_G ... Thread friction coefficient [-]
μ_K ... Friction coefficient of bolt head [-]
β ... Thread angle (angle between 'teeth') [°]
$d_{K,R}$ ... Middle diameter of friction surface under bolt head [mm]
d_K ... Outer diameter of surface under bolt head [mm]
d_i ... Diameter of the hole in the material [mm]